

Context Management in Large Language Models

A Survey of Human-Inspired
Memory Architectures

J.C. Vaught

January 2026

A comprehensive literature review examining context window limitations, memory architectures, and human-inspired approaches to persistent memory in large language models. This survey spans attention mechanisms, positional encoding strategies, key-value cache optimization, context compression, retrieval-augmented generation, hierarchical memory systems grounded in cognitive science, hallucination phenomena arising from context mismanagement, multi-turn dialogue coherence, and evaluation methodologies for long-context capabilities.

Abstract

Large language models have demonstrated remarkable capabilities across diverse natural language processing tasks, yet they remain fundamentally constrained by finite context windows that limit the amount of information available during inference. This survey provides a comprehensive examination of context management strategies, organized around ten interconnected themes. We begin with the foundational attention mechanism and its quadratic computational complexity, then trace the evolution of positional encoding methods from absolute embeddings through rotary position encodings and their scaling variants that have progressively extended effective context lengths from thousands to millions of tokens. We analyze sparse and efficient attention architectures, including Longformer, BigBird, and state-space models such as Mamba, that achieve sub-quadratic complexity while preserving modeling quality. The survey examines key-value cache optimization techniques encompassing eviction policies, quantization schemes, and dynamic memory management systems such as PagedAttention and vAttention. Context compression methods, including prompt compression frameworks like LLMingua and learned compression via gist tokens and in-context autoencoders, are analyzed for their trade-offs between information preservation and computational efficiency. We provide an extensive treatment of memory-augmented architectures spanning classical neural memory networks, retrieval-augmented generation and its modern variants including Self-RAG, CRAG, and GraphRAG, and operating-system-inspired designs such as MemGPT. The core of this survey is dedicated to hierarchical, human-inspired memory architectures, where we draw systematic parallels between cognitive science constructs including the Atkinson-Shiffrin model, Baddeley’s working memory, Tulving’s memory taxonomy, Ebbinghaus’s forgetting curve, and McClelland’s complementary learning systems, and their computational analogues in systems such as Generative Agents, MemoryBank, the Cognitive Architectures for Language Agents framework, the Self-Controlled Memory framework, and the Hierarchical Memory Transformer. We examine how hallucination phenomena arise from context management failures, including the lost-in-the-middle effect and positional biases. Multi-turn dialogue presents unique challenges for memory persistence, and we survey approaches from conversation summarization to episodic memory buffers. Finally, we critically assess evaluation methodologies including Needle-in-a-Haystack, RULER, LongBench, InfiniteBench, HELMET, and domain-specific benchmarks, identifying gaps between benchmark performance and real-world long-context utilization. Throughout, we argue that human cognitive memory architectures provide essential design principles for building language model systems capable of coherent, persistent, and scalable context management.

1 Introduction

The transformer architecture has fundamentally transformed natural language processing, establishing itself as the dominant paradigm for sequence modeling and the foundation for virtually all modern large language models. Central to its success is the self-attention mechanism, which enables direct pairwise interactions between all positions in a sequence without the sequential bottlenecks inherent in recurrent architectures. This architectural innovation has enabled the remarkable scaling of language models from millions to hundreds of billions of parameters, yielding systems capable of impressive feats of text generation, question answering, reasoning, and code synthesis.

However, the very mechanism that grants transformers their expressive power introduces a fundamental computational constraint: quadratic scaling with respect to sequence length. Processing a context of length n requires $O(n^2)$ attention computations, meaning that doubling the context window quadruples the computational cost. For the million-token contexts now emerging in frontier models, this quadratic dependency poses severe challenges for both training and inference, motivating extensive research into techniques that extend effective context while managing computational demands.

The challenge of context management extends beyond raw computational complexity to encompass questions of effective utilization, information retention, and coherent integration of information distributed across extended sequences. Empirical research has revealed that models often fail to effectively utilize information positioned in the middle of long contexts, exhibiting a U-shaped performance curve where information at the beginning and end of sequences is more accessible than middle-positioned content. Furthermore, attention patterns in trained models exhibit systematic biases including primacy effects favoring early tokens, recency effects privileging recent tokens, and attention sink phenomena where initial tokens receive disproportionate attention regardless of semantic relevance.

These phenomena suggest that extending context windows requires more than architectural modifications to reduce computational complexity. Effective long-context processing demands sophisticated memory management strategies that selectively retain important information, compress or summarize less critical content, and maintain coherent representations across extended interactions. This recognition has motivated research drawing inspiration from human cognitive architectures, which successfully manage effectively unbounded information streams through multi-store memory systems, hierarchical organization, selective consolidation, and intelligent forgetting.

This survey provides a comprehensive examination of context management in large language models, organized around ten interconnected themes. Following this introduction, Section 2 examines transformer attention mechanisms and positional encoding schemes that determine sequence length capabilities. Section 3 surveys context window extension techniques including RoPE-based scaling methods and continual pre-training strategies. Section 4 analyzes sparse and efficient attention mechanisms that achieve sub-quadratic complexity. Section 5 investigates key-value cache optimiza-

tion through eviction policies, quantization, and architectural modifications. Section 6 examines context compression techniques from hard prompt pruning to learned soft compression. Section 7 provides extensive treatment of memory-augmented and retrieval-augmented generation architectures. Section 8 constitutes the theoretical core, examining hierarchical human-inspired memory systems and their computational implementations. Section 9 analyzes hallucination phenomena arising from context management failures. Section 10 addresses multi-turn conversation management. Section 11 critically assesses evaluation methodologies for long-context capabilities. We conclude in Section 12 with synthesis and future directions.

2 Transformer Attention and Context Fundamentals

The transformer architecture has fundamentally reshaped natural language processing, establishing itself as the dominant paradigm for sequence modeling. At its core lies the attention mechanism, which serves as the computational substrate upon which all context management strategies ultimately depend. This section provides a comprehensive analysis of self-attention mechanisms, multi-head attention architectures, the evolution of positional encoding methods, and the empirical phenomena that reveal fundamental limitations in how transformers access and utilize contextual information across extended sequences.

2.1 The Self-Attention Mechanism

The transformer architecture introduced by [1] represents one of the most influential contributions to machine learning in the 21st century, accumulating over 173,000 citations as of 2025. The architecture dispenses with recurrence and convolutions entirely, relying instead on attention mechanisms to model dependencies between sequence elements. The core innovation computes attention as a function of query (Q), key (K), and value (V) matrices derived from input representations:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (1)$$

where d_k is the dimension of the key vectors and the scaling factor $1/\sqrt{d_k}$ prevents the dot products from growing large in magnitude, which would push the softmax function into regions with extremely small gradients. The query matrix $Q = XW_Q$, key matrix $K = XW_K$, and value matrix $V = XW_V$ are obtained by multiplying the input representation X with learned projection matrices W_Q , W_K , and W_V respectively.

The self-attention operation establishes $O(1)$ path length between any two positions in the sequence, representing a dramatic improvement over the $O(n)$ path length required in recurrent architectures. This constant-time access enables full parallelization during training, as every position can attend to every other position simultaneously without sequential dependencies. However, this computational advantage comes at a significant cost. The self-attention operation requires computing and storing

the $n \times n$ attention matrix, resulting in $O(n^2)$ time and memory complexity. [1] demonstrated that the total computation for a single self-attention layer requires $O(n^2 \cdot d)$ floating-point operations and $O(n^2 + n \cdot d)$ memory, where d is the model dimension and n is the sequence length. For a model with L layers, this becomes $O(L \cdot n^2 \cdot d)$, making each additional token in the context disproportionately expensive.

Recent theoretical work has established the fundamental nature of this quadratic bottleneck. [1] proved that the time complexity of self-attention is necessarily quadratic in the input length, demonstrating that $O(n^2 d)$ complexity holds even for approximate attention mechanisms unless the Strong Exponential Time Hypothesis is proven false. This formal analysis confirms that the quadratic scaling is not merely an implementation artifact but rather an inherent property of pairwise token attention. The practical implications are severe: when sequence length increases by a factor of 10, computational requirements increase by a factor of 100, making naive scaling to very long sequences prohibitively expensive on current hardware.

Interestingly, while time complexity appears fundamentally bounded, memory complexity can be partially decoupled from the quadratic constraint. Recent work has demonstrated that $O(n^2)$ memory is not strictly necessary even though time complexity remains quadratic, opening avenues for memory-efficient implementations that maintain exact attention computation. The softmax function in the attention computation introduces a further subtlety: it produces a probability distribution over positions, meaning that as context length grows, the attention mass is distributed over more positions. This attention dilution effect means that individual tokens receive less focused attention in longer contexts, potentially reducing the model’s ability to strongly attend to any single relevant position regardless of its semantic importance.

2.2 Multi-Head Attention Analysis

The multi-head attention mechanism distributes the model’s attention capacity across h independent heads, each operating on a $d_k = d_{\text{model}}/h$ dimensional subspace. [1] formalized this as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2)$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. The motivation for multiple heads stems from the hypothesis that different heads can learn to attend to different aspects of the input, capturing diverse relationships within a single representation subspace.

Empirical analysis has revealed substantial functional specialization across attention heads. Research categorizes heads into distinct types: positional heads that focus on local position patterns and attend primarily to specific relative positions, syntactic heads that capture grammatical relationships and dependency structures, semantic heads that attend to semantically related words regardless of position, and rare or specialized heads with task-specific or infrequent firing patterns. This specialization is not uniform across layers. Early layers tend to exhibit more balanced head

usage during preprocessing of input representations, while middle layers show mixed utilization patterns with increasing specialization, and late layers are often dominated by one or two heads responsible for final output generation.

The discovery of head specialization has profound implications for efficiency. Multiple studies have demonstrated that extensive redundancy exists in multi-head attention mechanisms, with experiments showing that 50-90 percent of heads can be removed with minimal performance degradation. Not all heads contribute equally to final model performance, and some heads appear to learn nearly identical attention patterns. This redundancy has motivated the development of more efficient architectures. [2] introduced Grouped Query Attention, which shares attention heads across multiple queries, dramatically reducing the key-value cache requirements while maintaining performance. Their work demonstrated that full head diversity is not necessary for strong results, as grouped representations can capture sufficient variation in attention patterns.

Recent investigations into how transformers utilize multi-head attention for in-context learning reveal sophisticated algorithmic patterns. Analysis shows that first-layer heads preprocess examples, extracting relevant features and relationships, while subsequent layers use a single dominant head for iterative optimization, refining predictions through repeated attention to critical positions. This finding suggests that transformers implement a preprocess-then-optimize algorithm using selective head utilization, rather than uniformly distributing computation across all heads throughout the network depth.

2.3 Positional Encoding: Evolution and Variants

Because the self-attention operation is inherently permutation-invariant, positional information must be explicitly injected into the model through positional encodings. The evolution of positional encoding methods has been a critical enabler of context window expansion and remains an active area of research.

2.3.1 Sinusoidal and Learned Encodings

[1] proposed sinusoidal positional encodings using sine and cosine functions of different frequencies, designed to allow the model to extrapolate to sequence lengths not seen during training. The encoding for position pos and dimension i is:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (3)$$

The sinusoidal formulation was chosen for its mathematical properties: the functions provide a unique encoding for each position while maintaining consistent spacing across different positions, and the dot product between positional encodings at different positions depends only on their relative distance, enabling the model to learn to attend by relative position. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$, providing multiple scales of positional information

simultaneously.

An alternative approach uses learned positional embeddings, where position vectors are treated as trainable parameters initialized randomly and optimized during training. This method was adopted by GPT-2 and GPT-3, offering the potential for task-specific optimization of positional representations. With sufficient training data, learned embeddings often achieve better in-distribution performance compared to fixed sinusoidal encodings. However, learned positional embeddings suffer from a fundamental limitation: they cannot generalize beyond the maximum training sequence length, as positions beyond the training range have no corresponding learned parameters. This constraint became increasingly problematic as researchers sought to extend context windows beyond training length.

2.3.2 Relative Positional Encodings

[3] introduced a paradigm shift with relative positional representations that encode pairwise distances between tokens rather than absolute positions. Their approach extends self-attention to efficiently consider relative positions or distances between sequence elements, modifying the attention mechanism to incorporate edge representations between nodes in the fully-connected input graph. This modification achieved significant improvements: +1.3 BLEU on WMT 2014 English-German translation and +0.3 BLEU on English-French, establishing that relative position information is more beneficial than absolute position alone. Notably, combining relative and absolute encodings yielded no further improvement, suggesting that relative position captures the essential positional information for many natural language tasks.

The key advantage of relative positional encoding is its ability to generalize to sequences longer than those seen during training. By encoding relative distances rather than absolute positions, the model learns translation-invariant patterns that apply regardless of sequence length. This property proved foundational for subsequent long-context extensions, as models could handle unseen positions by computing familiar relative distances.

2.3.3 Transformer-XL and Segment Recurrence

[4] extended relative positional encoding in Transformer-XL, introducing a segment-level recurrence mechanism with a novel relative positional encoding scheme compatible with recurrence. The architecture addresses the context fragmentation problem inherent in vanilla transformers, which process fixed-length segments independently, preventing information flow across segment boundaries. Transformer-XL maintains hidden states from previous segments during processing, enabling context to extend beyond current segment boundaries while avoiding temporal confusion about which segment generated which representation.

The performance improvements demonstrated by Transformer-XL were substantial. The model learned dependencies 80 percent longer than recurrent neural networks and 291-447 percent longer

than vanilla transformers, achieving up to 1,800 times faster evaluation speed than baseline transformers. The relative positional encoding scheme replaces absolute positions with relative encodings that remain consistent across segments, addressing the temporal confusion that would otherwise arise from reusing the same absolute position indices in different segments.

2.3.4 Rotary Position Embedding (RoPE)

[5] proposed Rotary Position Embedding, which has become the dominant positional encoding method in modern large language models including the LLaMA family. RoPE represents an elegant unification of absolute and relative positional encoding approaches: it encodes absolute position with rotation matrices while incorporating explicit relative position dependency in the self-attention formulation. The method rotates pairs of features by angles that depend on the token’s absolute position:

$$\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]\} \quad (4)$$

where the rotation parameters form a geometric sequence similar to sinusoidal encodings but applied through rotation transformations rather than additive position embeddings.

The geometric intuition behind RoPE is compelling. Query and key vectors are rotated by angles proportional to their token positions, such that the dot product between a query at position m and a key at position n depends only on the relative distance $m - n$. This relative dependency emerges naturally from the rotation geometry: the angle between two rotated vectors depends on the difference in their rotation angles. RoPE exhibits several desirable properties: flexibility of sequence length allowing application to varying context sizes, decaying inter-token dependency with increasing relative distances mimicking linguistic locality, and compatibility with linear self-attention variants enabling efficient approximations. The method has been integrated into HuggingFace transformers and serves as the foundation for most modern LLM architectures.

2.3.5 Attention with Linear Biases (ALiBi)

[6] proposed a fundamentally different approach with Attention with Linear Biases, which eliminates positional embeddings entirely. Instead of adding position information to input representations, ALiBi biases the query-key attention scores with a penalty proportional to the distance between tokens:

$$\text{softmax}\left(q_i K^\top + m \cdot [-(i-1), \dots, -2, -1, 0]\right) \quad (5)$$

where m is a head-specific slope fixed before training and never updated. Each attention head receives a different slope, allowing different heads to implement different locality biases. The bias grows linearly as the distance between query and key tokens increases, implementing an inductive bias toward recency similar to discounting factors in reinforcement learning.

The crucial advantage of ALiBi is its extrapolation capability, enabling models to perform inference on sequences longer than those seen during training. The bias formula applies uniformly to any

token distance, whether seen during training or not, providing consistent positional information at arbitrary sequence lengths. This property motivated the paper’s title, "Train Short, Test Long." A 1.3 billion parameter model trained on 1,024-token sequences achieved the same perplexity on 2,048-token sequences as sinusoidal models trained on the full 2,048-token length, demonstrating effective 2x extrapolation. Moreover, ALiBi training provided 11 percent faster training and 11 percent lower memory usage compared to sinusoidal encodings, due to the elimination of positional embedding parameters and computations. ALiBi is designed specifically for autoregressive decoder-only scenarios with causal attention masks and has been adopted in models such as BLOOM and MPT, outperforming multiple strong baseline position methods on the WikiText-103 language modeling benchmark.

2.4 Context Window Scaling: Historical Evolution

The progression of context window sizes in frontier large language models reflects the convergence of architectural innovations, engineering advances in efficient attention implementation, and scaled computational infrastructure. The timeline demonstrates remarkable progress, with context capabilities increasing roughly 1,000-fold over five years.

GPT-2, released by OpenAI in February 2019, established a baseline with a 1,024-token context window, representing approximately 512 words or about one to two pages of text. This relatively small window reflected both the quadratic attention complexity constraints and the limited computational resources available for training at the time. The fixed-length constraint meant that processing longer documents required sliding window approaches or truncation, fundamentally limiting the model’s ability to maintain coherent long-range dependencies.

GPT-3, released in June 2020, doubled the context to 2,048 tokens, accommodating roughly 1,000 words or three to four pages. While still limited by modern standards, this expansion was considered state-of-the-art for large-scale language models at the time. The 2K context window became the standard for commercial deployments and research applications throughout 2020 and 2021.

ChatGPT, based on GPT-3.5-Turbo and released in November 2022, featured a 4,096-token context window, approximately 2,000 words or six to eight pages. This further doubling addressed user feedback about context limitations in conversational interactions and document analysis tasks. The 4K window became representative of the industry standard for accessible commercial language models, enabling processing of substantial documents but still requiring chunking strategies for longer content.

GPT-4, released in March 2023, marked a significant inflection point with two variants: a standard version with 8,192 tokens (8K) and an extended version with 32,768 tokens (32K). The 32K variant, offering approximately 24,000 words or 50-70 pages, enabled processing of technical papers, book chapters, and substantial codebases within a single context. This 4x to 16x expansion over GPT-3 demonstrated the exponential growth trajectory of context capabilities. GPT-4 Turbo, released

in November 2023, achieved 128,000 tokens (128K), approximately 100,000 words or 300+ pages. This represented a 4x improvement over the GPT-4 32K variant and a 64x improvement over the original GPT-3 baseline, enabling processing of entire books, large codebases, and extensive document collections in a single request.

Anthropic’s Claude models pursued aggressive context expansion as a core differentiating feature. In May 2023, Claude expanded from 9,000 to 100,000 tokens, approximately 75,000 words, achieving industry-leading context capacity at the time. Claude 2.1, released in November 2023, reached 200,000 tokens, approximately 150,000 words or 500+ pages, doubling the context from Claude 2.0 and enabling absorption of technical manuals, financial filings, and multiple books simultaneously. The Claude 3 family, launched in March 2024, featured a standard 200,000-token window with extended capability of 1 million tokens for select customers, establishing Claude as a leader in very long context processing.

Google’s Gemini 1.5, released in February 2024, achieved a breakthrough with a 1 million token context window, the longest of any large-scale foundation model at launch. This unprecedented capacity enables processing of 1 hour of video, 11 hours of audio, 30,000+ lines of code, or 700,000+ words in a single context. The model demonstrated 99 percent accuracy on Needle-in-Haystack evaluation at the full 1 million token length, showing that the extended context was not merely nominal but functionally effective. Research testing demonstrated successful processing of up to 10 million tokens in experimental settings, suggesting that further scaling remains feasible.

Table 1: Evolution of context window sizes in major language models.

Model	Date	Context (tokens)
GPT-2	February 2019	1,024
GPT-3	June 2020	2,048
ChatGPT (GPT-3.5)	November 2022	4,096
GPT-4	March 2023	8,192 / 32,768
GPT-4 Turbo	November 2023	128,000
Claude 2.1	November 2023	200,000
Claude 3	March 2024	200,000 / 1,000,000
Gemini 1.5 Pro	February 2024	1,000,000

This roughly 1,000-fold increase over five years was enabled by the convergence of multiple innovations. Positional encoding advances including RoPE and ALiBi enabled effective position extrapolation beyond training lengths. Efficient attention implementations, particularly FlashAttention, reduced the memory footprint and computational requirements of exact attention. Distributed computing techniques such as Ring Attention enabled context to be distributed across multiple devices, overcoming single-device memory constraints. Continual pre-training strategies allowed models ini-

tially trained on shorter contexts to be adapted to longer sequences through position interpolation and targeted fine-tuning. By 2025-2026, 128K to 200K context windows have become standard in production large language models, with leading research systems pushing toward multi-million token contexts.

2.5 The “Lost in the Middle” Phenomenon

Despite the impressive expansion of context windows, a fundamental question remained: do models effectively utilize information throughout their entire context, or does longer context simply provide a larger buffer that remains partially unused? [7] provided a rigorous answer through controlled experiments with multi-document question answering and key-value retrieval tasks. Their methodology varied the position of relevant information within the input context while keeping context length and content constant, isolating positional effects from content and length effects.

The findings revealed a striking pattern: performance was highest when relevant information appeared at the very beginning or end of the input context and degraded substantially when the information was positioned in the middle. In some cases, middle-positioned information retrieval fell below closed-book baseline performance, meaning the model performed worse than if it had never seen the information at all. This U-shaped performance curve was observed across multiple models including GPT-3.5-Turbo, demonstrating that the limitation was not specific to a particular architecture but rather a general property of transformer-based language models.

Critically, the phenomenon persisted even in models explicitly designed for long-context processing. Models with 100,000-token context windows showed the same positional bias, failing to robustly utilize information from the middle of their available context. The authors characterized this as a fundamental architectural limitation rather than a training artifact, as models with extensive long-context training still exhibited the U-shaped performance pattern. The implication is profound: having a large context window does not guarantee effective use of all information within it, and the position of critical information within the context significantly affects whether the model can successfully retrieve and utilize it.

Several hypotheses have been proposed to explain this phenomenon. Attention patterns may concentrate on early tokens regardless of relevance, creating a recency and primacy bias similar to human memory. Training data may exhibit positional biases, with important information more commonly appearing at the beginning or end of documents, causing models to learn positional heuristics. The attention dilution effect means that with longer contexts, attention mass is spread more thinly, potentially reducing the salience of any individual middle position. Whatever the underlying mechanism, the Lost in the Middle phenomenon demonstrates that effective context utilization requires more than capacity expansion alone, motivating the development of more sophisticated context management strategies including retrieval augmentation, attention pattern modifications, and training procedures that explicitly address positional biases.

2.6 The Attention Sink Phenomenon

[8] discovered another fundamental phenomenon that constrains how transformers process long sequences: attention sinks, where initial tokens in a sequence receive disproportionately strong attention scores even when they carry no special semantic significance. Through systematic analysis of attention patterns in models including LLaMA-2, MPT, Falcon, and Pythia, the researchers found that beginning-of-sequence tokens consistently attract large attention mass across layers and heads, regardless of the semantic content of those tokens.

The attention sink phenomenon arises from the mathematical structure of softmax attention. The softmax operation enforces normalization, ensuring that attention weights sum to one across all positions. When processing a sequence, there are frequent cases where no token is strongly relevant to the current query. However, the normalization constraint requires attention mass to be allocated somewhere. Rather than distributing attention uniformly or allocating it to the most relevant available token, models learn to dump this excess attention onto initial tokens, which effectively serve as numerical sinks to satisfy the normalization requirement without strongly influencing the output through the value vectors.

The discovery of attention sinks led to the development of StreamingLLM, a framework that enables large language models with finite attention windows to process sequences of arbitrary length without fine-tuning. The key insight is that maintaining the key-value states of a small number of initial sink tokens, combined with a sliding window of recent tokens, largely recovers the performance of full attention over the entire history. StreamingLLM achieved stable language modeling with up to 4 million tokens across multiple model families, demonstrating that the attention sink mechanism, while initially appearing to be a limitation, can actually be leveraged for efficient streaming inference.

The performance gains from StreamingLLM were substantial. Compared to sliding window recomputation baselines, where the model must recompute attention over the full window at each step, StreamingLLM achieved up to 22.2x speedup in streaming settings while maintaining comparable perplexity. The framework requires no fine-tuning or architectural modifications, operating purely through strategic management of the key-value cache to retain sink tokens and recent context while discarding middle positions.

The authors recommended adding a dedicated placeholder token as an attention sink during pre-training to improve streaming deployment. By explicitly designating a token whose sole purpose is to absorb excess attention, models can more cleanly separate the attention sink function from semantic processing, potentially reducing interference between the two roles. This design choice represents a principled approach to acknowledging and accommodating the mathematical constraints of softmax attention rather than attempting to eliminate them.

Recent empirical investigations have provided deeper insights into when and why attention sinks emerge. Analysis shows that the phenomenon is strongest in deeper layers and becomes more pronounced as training progresses, suggesting that attention sinks emerge as a learned optimization

strategy rather than an initialization artifact. Specific tokens and layer combinations show consistent sink behavior across different inputs, indicating that the sink pattern is stable and reproducible. The interaction between attention sinks and the Lost in the Middle phenomenon is synergistic: models that allocate substantial attention to initial sink tokens have correspondingly less attention available for middle positions, exacerbating the difficulty of accessing mid-context information.

2.7 Efficient Attention Mechanisms

The quadratic complexity barrier motivated extensive research into efficient attention mechanisms that reduce computational and memory requirements while preserving the modeling capabilities of full attention. These approaches fall into several categories, each making different trade-offs between efficiency, exactness, and implementation complexity.

Sparse attention methods replace the dense $n \times n$ attention matrix with structured sparsity patterns, reducing complexity by computing attention only over a subset of position pairs. Longformer, developed by [9], combines local windowed attention using dilated sliding windows for better long-range coverage with global attention where task-motivated tokens attend to all positions. This hybrid approach scales linearly with sequence length rather than quadratically, achieving state-of-the-art results on character-level language modeling benchmarks and consistently outperforming RoBERTa on long document tasks after pretraining and fine-tuning. BigBird, introduced by [10], implements a more complex sparse attention pattern with three components: global tokens that attend to all parts of the sequence, local tokens where all tokens attend to neighboring positions, and random tokens where each token attends to a fixed number of randomly selected positions. The random attention component introduces long-range connections between distant tokens inspired by graph sparsification methods, allowing discovery of unexpected long-distance dependencies. BigBird proves to be a universal approximator of sequence functions and is Turing complete, preserving the expressiveness of full attention while reducing quadratic dependency to linear complexity and scaling to eight times longer sequences than full attention.

Linear attention mechanisms approximate the attention computation through algebraic transformations that avoid materializing the full attention matrix. These approaches typically involve kernel approximations of the softmax function, allowing attention to be computed in $O(n \cdot d^2)$ time rather than $O(n^2 \cdot d)$. Linformer demonstrates that self-attention can be approximated by a low-rank matrix, reducing complexity from $O(n^2)$ to $O(n)$ in both time and space through projection of keys and values to a lower-dimensional space. The method achieves 1.5x faster inference and 1.7x larger batch sizes compared to standard transformers while maintaining comparable performance on downstream tasks.

IO-aware attention represents a different optimization approach focused on the memory hierarchy of modern GPUs. [11] introduced FlashAttention, an exact attention algorithm that accounts for reads and writes between GPU memory levels. Rather than changing the attention computation itself, FlashAttention optimizes the order and granularity of operations to minimize expensive trans-

fers between high-bandwidth memory and fast on-chip SRAM. The algorithm uses tiling to process blocks of the attention matrix in fast memory, preventing materialization of the full $N \times N$ attention matrix in slow GPU high-bandwidth memory. FlashAttention requires $O(N^2 d^2 M^{-1})$ memory accesses where M is the SRAM size, compared to $\Omega(Nd + N^2)$ for standard attention. Critically, FlashAttention computes exact attention without approximation, achieving up to 7.6x speedup on GPT-2 while maintaining numerical precision. The memory usage scales linearly in sequence length rather than quadratically, enabling longer sequences on the same hardware and establishing FlashAttention as a foundational component of modern efficient transformer implementations.

2.8 Summary

The transformer’s self-attention mechanism provides powerful sequence modeling capabilities through constant-time access between any two positions and full parallelization during training. However, the quadratic computational complexity in sequence length represents a fundamental bottleneck that limits context capacity and motivates extensive research into efficient alternatives. Theoretical analysis confirms that this quadratic scaling is not an implementation artifact but an inherent property of pairwise token attention that persists even with approximations.

Multi-head attention distributes modeling capacity across independent heads that learn functional specialization, with different heads capturing positional patterns, syntactic relationships, and semantic dependencies. Research reveals substantial redundancy, with 50-90 percent of heads removable with minimal performance loss, motivating efficient architectures such as Grouped Query Attention that share representations across queries while maintaining modeling power.

Positional encoding has evolved from fixed sinusoidal functions through learned embeddings to relative encodings, rotary position embeddings, and linear biases. Each innovation has enabled progressively longer and more flexible context handling, with RoPE emerging as the dominant method in modern large language models due to its elegant unification of absolute and relative position information. ALiBi demonstrates that effective positional encoding can be achieved without any learned position parameters, instead using simple bias terms that enable training on short sequences with inference on arbitrarily long sequences.

Context window sizes have grown roughly 1,000-fold from 1,024 tokens in GPT-2 to 1 million or more tokens in Gemini 1.5 and Claude 3, enabled by advances in positional encoding, efficient attention implementations, distributed computing, and continual pre-training strategies. This dramatic expansion has transformed the capabilities of language models, enabling processing of books, large codebases, and hours of multimedia content in single contexts.

Despite remarkable progress in scaling context windows, the Lost in the Middle phenomenon reveals that effective context utilization requires more than capacity expansion. Models consistently fail to robustly access information positioned in the middle of long contexts, showing U-shaped performance curves with degraded retrieval for middle-positioned information even when using con-

texts well within their nominal capacity. This limitation appears to be architectural rather than training-related, persisting even in models explicitly designed for long-context processing.

The attention sink phenomenon demonstrates that initial tokens receive disproportionate attention regardless of semantic relevance, serving as numerical anchors for softmax normalization. While initially appearing problematic, this behavior can be leveraged through frameworks like StreamingLLM to enable efficient processing of arbitrarily long sequences by maintaining sink tokens and recent context while discarding middle history. The 22.2x speedup achieved through attention sink awareness demonstrates that understanding and accommodating architectural constraints can yield practical benefits for deployment.

Efficient attention mechanisms including sparse attention patterns, linear attention approximations, and IO-aware implementations provide complementary approaches to overcoming quadratic complexity. Sparse methods like Longformer and BigBird reduce computation through structured sparsity while maintaining theoretical expressiveness. Linear attention methods achieve computational speedups through kernel approximations and algebraic transformations. FlashAttention demonstrates that exact attention can be substantially accelerated through memory-aware implementations that optimize for modern GPU memory hierarchies without changing the underlying computation.

The convergence of these innovations establishes the foundation for effective long-context processing, but significant challenges remain. Future research must address not only context capacity but also context utilization, developing architectures and training procedures that enable uniform access to information regardless of position while maintaining computational efficiency at million-token scales and beyond.

3 Context Window Extension Techniques

The fundamental challenge facing transformer-based large language models in processing extended sequences stems from the quadratic computational complexity of self-attention mechanisms. Standard attention computation scales as $O(n^2)$ in both time and memory with respect to sequence length n , imposing severe practical limitations on the maximum context window that can be processed during both training and inference. When sequence length increases by a factor of ten, computational requirements increase by a factor of one hundred. This quadratic bottleneck has motivated extensive research into techniques that extend the effective context window of pre-trained language models beyond their original training limits, with contemporary approaches achieving context windows extending from the early 2-4K token limitations to 128K tokens in production systems and beyond 2M tokens in research demonstrations.

This section examines the major categories of context window extension techniques, beginning with modifications to rotary position embeddings, proceeding through alternative positional encoding schemes, examining architectural and system-level innovations, exploring memory-augmented ap-

proaches, and concluding with continual pre-training strategies that have become standard practice in modern long-context language model development.

3.1 RoPE-Based Scaling Methods

Rotary Position Embedding has emerged as the dominant positional encoding method in modern large language models, adopted in flagship architectures including LLaMA, Mistral, Gemma, and GPT-J. RoPE encodes absolute position information while enabling models to learn relative position dependencies by representing positions through rotation matrices operating in 2D planes across the embedding dimensions [5]. The mathematical elegance of RoPE, which applies position-dependent rotations to query and key vectors, enables the dot product between vectors to naturally encode relative positional information through the geometric properties of rotation. However, despite these theoretical advantages, RoPE-based models exhibit severe degradation when processing sequences longer than those encountered during training, a phenomenon termed the position extrapolation problem. The core challenge arises because RoPE’s frequency components are optimized for the positional range seen during pre-training, and extending beyond this range introduces out-of-distribution rotational angles that cause attention score instabilities and catastrophic performance collapse.

3.1.1 Position Interpolation

[12] introduced Position Interpolation as a solution to the extrapolation problem by proposing a conceptually simple but theoretically grounded approach. Rather than extrapolating to unseen positional indices beyond the training range, Position Interpolation linearly down-scales input position indices to compress them into the original context window size. Formally, if a model was pre-trained on positions $[0, L]$ and must process a sequence of length sL where $s > 1$ is the scaling factor, Position Interpolation applies the transformation $p' = p/s$ to map position p to its interpolated value p' . This transformation ensures that the maximum position index matches the pre-training limit, keeping all positional encodings within the distribution seen during training.

The theoretical foundation for Position Interpolation rests on the observation that interpolation is fundamentally more stable than extrapolation in the frequency domain. [12] demonstrated that the theoretical upper bound on interpolation error is approximately 600 times smaller than the corresponding extrapolation error bound, providing mathematical justification for why interpolation enables reliable context extension with minimal fine-tuning. In practice, Position Interpolation enabled LLaMA models to extend from their original 2K token context to 32,768 tokens after fine-tuning for fewer than 1,000 training steps on long sequences. This remarkable efficiency stemmed from the fact that the model’s attention patterns, learned during pre-training, remained valid under interpolation because no out-of-distribution positional encodings were introduced.

However, Position Interpolation suffers from a critical limitation that becomes apparent when examining its effect on RoPE’s frequency components. By uniformly scaling all positions by the same

factor $1/s$, Position Interpolation effectively compresses the entire positional space, which in the frequency domain corresponds to removing high-frequency components of the positional encoding. This uniform compression makes adjacent tokens appear perceptually closer in the model’s positional space, potentially conflating tokens that should be distinguished. The degradation worsens as the scaling factor increases, with empirical results showing that Position Interpolation typically achieves reliable extensions of $2\text{-}4\times$ the original context length before significant performance degradation occurs.

3.1.2 NTK-Aware Scaling

The limitations of Position Interpolation motivated the development of frequency-aware scaling methods that treat different frequency components of RoPE with varying degrees of interpolation. NTK-Aware scaling, proposed by community researcher bloc97 through GitHub and Reddit discussions, represents a breakthrough in context extension efficiency by recognizing that different frequency dimensions of RoPE embeddings serve different representational purposes and should be scaled accordingly. The method derives its name from the Neural Tangent Kernel perspective on how neural networks generalize, though the practical implementation focuses on dimension-dependent frequency scaling.

The core innovation of NTK-Aware scaling lies in its modification of the RoPE base frequency. While Position Interpolation scales positions uniformly, NTK-Aware scaling instead modifies the base frequency θ used in RoPE’s sinusoidal functions according to $\theta_{new} = \theta_{original} \cdot \alpha^{d/(d-2)}$, where α is the desired extension factor and d is the dimensionality. This modification distributes interpolation pressure across frequency dimensions in a non-uniform manner. High-frequency dimensions, which encode fine-grained local positional distinctions between nearby tokens, receive less interpolation pressure and thus preserve their ability to distinguish adjacent positions. Low-frequency dimensions, which encode coarse-grained global structure and long-range positional relationships, receive more interpolation pressure to accommodate the extended context.

This frequency-aware approach addresses Position Interpolation’s fundamental weakness by preventing the loss of high-frequency information that is critical for local coherence. The practical impact is substantial: NTK-Aware scaling enables $4\text{-}8\times$ context extension without any fine-tuning whatsoever, compared to Position Interpolation’s maximum $2\text{-}4\times$ extension even with fine-tuning. Perhaps most remarkably, NTK-Aware scaling requires modifying only three lines of code in the RoPE implementation, making it trivially easy to integrate into existing models. Empirical evaluations demonstrated that models extended with NTK-Aware scaling maintain substantially lower perplexity across extended context lengths compared to Position Interpolation, validating the importance of frequency-aware scaling strategies.

The community further refined NTK-Aware scaling through the NTK-by-parts extension, which applies different interpolation strategies to different parts of the frequency spectrum rather than using a single smooth function across all frequencies. This refinement addressed catastrophic failures

observed with pure NTK-Aware scaling at certain specific context lengths, discovering that different frequency ranges benefit from distinct scaling approaches. The success of NTK-Aware scaling in enabling zero-shot context extension without fine-tuning demonstrated that the position extrapolation problem could be largely solved through careful frequency domain manipulation alone.

3.1.3 YaRN

Building upon the insights from NTK-Aware scaling, [13] introduced YaRN (Yet another RoPE extensioN method), a compute-efficient approach that combined sophisticated frequency-aware scaling with attention temperature adjustments. YaRN represents the culmination of community-driven research on RoPE scaling, incorporating lessons learned from Position Interpolation, NTK-Aware scaling, and their variants into a unified framework that achieves state-of-the-art efficiency for context extension.

YaRN’s fundamental innovation lies in its three-way classification of RoPE frequency dimensions into distinct groups that receive tailored treatment. High-frequency dimensions, which encode local positional patterns and fine-grained distinctions, are not interpolated at all, preserving the model’s ability to maintain local coherence and distinguish between adjacent tokens. Low-frequency dimensions, which encode global structural information and long-range dependencies, receive full interpolation to accommodate the extended context range. Mid-range frequencies, which operate at intermediate scales, follow NTK-Aware scaling rules that smoothly transition between the high and low-frequency behaviors. This tripartite classification enables YaRN to preserve both local and global positional information while extending context windows.

In addition to frequency-aware scaling, YaRN introduces attention temperature scaling to address the phenomenon of attention entropy collapse during context extension. As context length increases, attention distributions can become overly concentrated or overly diffuse, degrading the model’s ability to focus on relevant information. YaRN’s temperature adjustment counteracts this effect by modulating attention sharpness as a function of context length, maintaining stable attention patterns across varying sequence lengths.

The computational efficiency of YaRN represents perhaps its most significant practical contribution. [13] demonstrated that YaRN requires $10\times$ fewer tokens and $2.5\times$ fewer training steps compared to previous context extension methods. In concrete terms, YaRN extended LLaMA 2 models (both 7B and 13B parameter variants) to 128K token context windows after training for merely 400 steps with batch size 64 on the PG19 dataset. This corresponded to training on less than 0.1% of the original pre-training data, yet achieved state-of-the-art performance on long-context benchmarks. Furthermore, YaRN demonstrated extrapolation capability beyond the fine-tuning dataset’s context length, indicating that the method instilled robust long-context understanding rather than simply memorizing longer training sequences.

The practical impact of YaRN has been substantial, with the method being adopted in numerous

production language models including Qwen, DeepSeek, and various open-source LLaMA derivatives. Its combination of effectiveness, efficiency, and relative simplicity of implementation has made it the de facto standard for RoPE-based context extension in the research community and industry applications.

3.1.4 LongRoPE and Advanced Extensions

[14] pushed the boundaries of RoPE-based context extension to unprecedented lengths, demonstrating the feasibility of extending context windows to 2,048K tokens (over 2 million tokens) through three complementary innovations. LongRoPE’s first contribution involves exploiting non-uniformities in positional interpolation requirements through evolutionary search. Rather than applying uniform interpolation rules across all positional dimensions and all regions of the extended context, LongRoPE identifies that different frequency components and different positional ranges benefit from different interpolation strategies. The method employs evolutionary search guided by perplexity on long-context validation data to discover optimal non-uniform interpolation schedules, providing superior initialization for subsequent fine-tuning. In scenarios without fine-tuning, this non-uniform interpolation alone enables $8\times$ context extension.

LongRoPE’s second innovation introduces a progressive extension strategy that stages the context expansion process into multiple phases. Rather than directly extending from the base context length to the target 2M token context in a single step, LongRoPE first fine-tunes the model to an intermediate length of 256K tokens. The model is then subjected to a second round of positional interpolation and fine-tuning to reach the final 2,048K token context window. This staged approach allows the model to gradually adapt to longer contexts, mitigating the distribution shift that occurs when abruptly extending context by orders of magnitude. The progressive strategy also enables more efficient exploration of the extended positional space during fine-tuning, as the model can leverage intermediate-length representations when learning to process even longer sequences.

The third key innovation of LongRoPE addresses a pervasive problem across context extension methods: degradation of performance on the original short context lengths. When models are extended to very long contexts through interpolation and fine-tuning, they often suffer reduced performance on the shorter sequences they were originally trained on. This phenomenon, sometimes termed "short-context forgetting," occurs because the fine-tuning process optimizes for long-context performance and may shift the model’s representations in ways that harm short-context processing. LongRoPE introduces a short context recovery mechanism that readjusts the model on 8K length sequences after long-context extension, explicitly recovering performance at the original context scale. This recovery phase is relatively brief compared to the long-context fine-tuning but proves essential for maintaining balanced performance across all sequence lengths.

LongRoPE was successfully integrated into Microsoft’s Phi-3 production model, demonstrating its practical viability beyond research demonstrations. The method achieves its impressive 2M token context window with remarkably modest fine-tuning requirements, requiring at most 1K fine-tuning

steps at training lengths within 256K. Extensive experiments on LLaMA-2 and Mistral models across diverse tasks validated LongRoPE’s effectiveness while confirming that it maintains compatibility with the original transformer architecture, requiring only minimal modifications to the positional embedding mechanism.

The evolution of RoPE scaling methods continued with recent work including LongRoPE2, which further refined the approach through evolutionary search guided by what the authors term "needle-driven" perplexity. This variant uses targeted perplexity measurements focused on specific retrieval tasks to guide the search for optimal interpolation parameters. LongRoPE2 also introduced mixed context window training, where models are trained on a mixture of sequences of varying lengths rather than exclusively on long sequences. This mixed training approach addresses short-context degradation more directly by ensuring continued exposure to short sequences throughout the long-context adaptation process. Additional innovations included fine-tuning model weights to adopt rescaled RoPE specifically for long sequences while preserving short-context performance, achieving what the authors characterized as near-lossless scaling across the full range of context lengths from short to ultra-long.

Beyond the mainstream RoPE scaling methods, recent research has explored more fundamental modifications to rotary embeddings. ComRoPE [15], published at CVPR 2025 despite being positioned primarily for vision tasks, generalized RoPE through trainable commuting angle matrices. This work provided theoretical analysis demonstrating that pairwise commutativity of rotation matrices is essential for positional robustness and scalability, offering new mathematical insights into why RoPE functions effectively and how its parameters can be learned rather than fixed. Such theoretical advances continue to deepen understanding of the mathematical foundations underlying practical context extension methods.

3.2 Alternative Position Encoding Methods

While RoPE-based scaling methods have dominated recent context extension research, alternative approaches to position encoding offer fundamentally different perspectives on the extrapolation problem. The most prominent alternative, ALiBi (Attention with Linear Biases), eschews positional embeddings entirely in favor of a recency-biased attention mechanism.

3.2.1 ALiBi and Linear Bias Approaches

[6] introduced ALiBi with the explicit goal of enabling the "train short, test long" paradigm, where models trained on relatively short sequences could extrapolate to substantially longer sequences at test time without fine-tuning. ALiBi eliminates positional embeddings entirely, neither adding position information to input embeddings at the model’s input nor incorporating learned positional parameters anywhere in the architecture. Instead, ALiBi modifies the attention mechanism itself by applying a bias directly to the attention scores after the query-key dot product.

Formally, ALiBi modifies the attention computation as $\text{softmax}(\mathbf{q}_i \mathbf{K}^\top + m \cdot [-(i-1), \dots, -2, -1, 0])$, where m is a head-specific scalar slope that is fixed before training and remains constant during inference. The bias term creates a linear penalty proportional to the distance between query position i and each key position, with the penalty increasing linearly for more distant tokens. Each attention head receives a different slope m , with slopes typically following a geometric progression (e.g., $\{1/2, 1/4, 1/8, 1/16, \dots\}$) to enable different heads to specialize in different distance ranges.

The design of ALiBi encodes an inductive bias toward recency, explicitly penalizing attention to distant tokens regardless of their semantic relevance. This recency bias reflects an assumption that more recent tokens are generally more relevant for predicting or processing the current token, an assumption that holds reasonably well for many natural language processing tasks. The linear distance penalty creates a smooth degradation of attention weights as a function of distance, rather than requiring the model to learn distance-dependent attention patterns from the positional encodings.

The extrapolation capabilities of ALiBi proved remarkable in empirical evaluations. [6] demonstrated that a 1.3B parameter model trained on sequences of 1,024 tokens could extrapolate to 2,048-token sequences, achieving perplexity equivalent to sinusoidal position embedding models that had been trained directly on 2,048-token sequences. Beyond mere extrapolation to $2\times$ the training length, ALiBi models maintained reasonable performance on sequences $5\text{-}10\times$ longer than their training length, a degree of extrapolation that proved impossible for standard positional encodings. Furthermore, ALiBi provided practical training benefits, achieving 11% faster training speed and 11% reduced memory usage compared to models using learned positional embeddings, because the bias computation is simpler than embedding lookups and additions.

However, ALiBi’s design involves tradeoffs that limit its applicability to certain scenarios. The recency bias, while beneficial for many language modeling tasks, may not be optimal for tasks requiring equal attention to distant context regardless of recency. Retrieval tasks, where relevant information may appear anywhere in a long context with equal probability, potentially suffer under ALiBi’s distance penalty. In contrast, RoPE’s learned rotational encodings do not impose an explicit recency bias, instead allowing the model to learn task-appropriate attention patterns. This distinction explains why RoPE has been preferred for general-purpose large language models despite ALiBi’s superior zero-shot extrapolation capability.

Theoretical analysis comparing interpolation and extrapolation approaches has illuminated the fundamental tradeoffs involved. Interpolation methods like Position Interpolation and YaRN require selecting a target context length and a corresponding scaling factor during the adaptation process. This fixed scaling factor introduces a tradeoff: extending context capability at long lengths necessarily degrades performance at the original short lengths due to the compression of positional information. Extrapolation methods like ALiBi naturally handle arbitrary lengths without modification, but require learned parameters (the head-specific slopes) that must be appropriate for the range of lengths encountered during training and the range needed at inference. Dynamic scaling approaches, discussed earlier in the context of NTK-Aware scaling, attempt to bridge these

approaches by adapting interpolation factors dynamically based on the actual sequence length at inference time.

3.2.2 Advanced Positional Encoding Analysis

Recent research has applied sophisticated analytical frameworks to understand positional encoding behavior during context extension. Distributional analysis of RoPE reveals that existing extension methods can introduce out-of-distribution rotary angles even when using interpolation, because the interpolation may be applied suboptimally across different frequency dimensions [16]. This distributional perspective motivated approaches that selectively apply interpolation or extrapolation on a per-dimension basis depending on measured disturbance scores that quantify how far each dimension’s angles deviate from their training distribution. Such distributional minimization approaches demonstrated 4.33% improvement over standard Position Interpolation when extending LLaMA2-7B to 16K tokens, illustrating the value of fine-grained per-dimension analysis.

Other work has explored decomposition of positional vectors into frequency-dependent components, enabling independent decisions about interpolation versus extrapolation for each frequency band. This component-wise analysis provides a principled framework for understanding why certain frequency bands benefit from interpolation while others prefer extrapolation or no modification at all. The resulting flexibility in positional encoding design has advanced the theoretical understanding of context extension while also yielding practical methods that outperform simpler uniform approaches.

3.3 Architectural and System-Level Approaches

While positional encoding modifications address context extension through representational changes, architectural and system-level innovations approach the problem through fundamental reconsideration of attention computation itself. These methods recognize that position encoding, while important, represents only one aspect of the context extension challenge, and that substantial gains can be achieved through memory-efficient attention computation, distributed processing, and hardware-aware optimization.

3.3.1 Ring Attention and Distributed Context Processing

[17] introduced Ring Attention as a fundamentally different approach to enabling ultra-long contexts, one that sidesteps position encoding limitations entirely by instead distributing attention computation across multiple devices. Ring Attention leverages blockwise computation of self-attention and feedforward layers, decomposing the attention mechanism to operate on sequence blocks rather than requiring the entire sequence to reside in a single device’s memory. The devices are arranged in a conceptual ring topology, with each device holding a portion of the sequence and computing attention for its local blocks.

The key innovation of Ring Attention lies in its overlapping of communication and computation. During the inner loop of attention computation, each device computes attention between its local

queries and its current key-value blocks. Simultaneously, it sends its key-value blocks to the next device in the ring topology and receives key-value blocks from the previous device. By carefully orchestrating the timing of computation and communication, Ring Attention ensures that computation on one block of key-value pairs proceeds in parallel with the transfer of the next block. As long as the computation time for processing a block exceeds the communication time for transferring a block, the overlapping results in zero added overhead compared to standard attention that processes the entire sequence on a single device.

The scalability implications of Ring Attention are profound. The approach enables sequences up to device-count times longer than can be processed with memory-efficient transformers on a single device, effectively multiplying the maximum context length by the number of available devices. [17] demonstrated training on sequences exceeding 100 million tokens without any approximations to the attention mechanism, representing sequences $500\times$ longer than prior state-of-the-art memory-efficient methods. The approach maintains exact attention computation rather than resorting to sparse or approximate attention, preserving the full representational capacity of the transformer architecture.

Ring Attention found practical application in the Large World Model (LWM), which employed the technique for million-length vision-language training. The ability to process such extreme context lengths enabled new applications in domains requiring dense, long-range modeling, such as video understanding, long document processing, and extended dialogue modeling. The distributed nature of Ring Attention also provides natural parallelism for training, though it requires careful implementation to achieve the overlapping communication and computation that eliminates overhead.

Distributed extensions of memory-efficient attention continue to advance the field. DistFlashAttention builds upon FlashAttention (discussed below) by extending its memory-efficient computation to distributed settings. DistFlashAttention introduces token-level workload balancing to ensure even distribution of computation across devices, overlaps key-value communication with attention computation similar to Ring Attention, and incorporates rematerialization-aware gradient checkpointing to minimize memory usage during backpropagation. These innovations enabled DistFlashAttention to process sequences $8\times$ longer than Ring Attention alone, achieving $4.45\text{--}5.64\times$ speedup over Ring Attention on benchmark tasks. The method successfully trained LLaMA-7B with sequences up to 512K tokens, demonstrating the feasibility of distributed training at extreme context lengths.

3.3.2 FlashAttention: Memory-Efficient Exact Attention

While Ring Attention addressed context scaling through distribution across devices, FlashAttention tackled the problem through within-device memory optimization. [11] observed that the primary bottleneck for attention computation on modern GPUs is not the raw number of floating-point operations but rather the memory movement between different levels of the memory hierarchy. GPUs feature fast on-chip SRAM with limited capacity and slower high-bandwidth memory (HBM) with substantially larger capacity. Standard attention implementations materialize the full $N \times N$

attention matrix in HBM, requiring repeated transfers between HBM and SRAM that dominate the total computation time.

FlashAttention eliminates this memory bottleneck through an IO-aware algorithm that uses tiling to restructure attention computation. Rather than computing the full attention matrix and then applying softmax and value multiplication, FlashAttention divides the query, key, and value matrices into blocks that fit in SRAM, computes attention for each block, and incrementally combines the results. The key algorithmic innovation involves maintaining running statistics for the softmax normalization, enabling correct computation of attention outputs without ever materializing the full attention matrix. This restructuring reduces the number of memory reads and writes between HBM and SRAM from $O(N^2)$ to $O(N^2 d^2 M^{-1})$, where d is the head dimension and M is SRAM size, yielding substantial reductions for typical parameters.

The practical impact of FlashAttention on training and inference proved substantial. On GPT-2, FlashAttention achieved $7.6\times$ speedup compared to standard PyTorch attention implementations. For BERT-large with 512-token sequences, FlashAttention provided 15% speedup over the MLPerf 1.1 training record, which represented highly optimized baseline implementations. Beyond speed improvements, FlashAttention’s memory efficiency enabled training with longer sequences and larger batch sizes. On GPT-2, FlashAttention enabled training sequences $4\times$ longer than was possible with standard attention implementations for a given memory budget, translating to substantially reduced perplexity (0.7 improvement) due to the benefits of longer training contexts.

The evolution of FlashAttention through subsequent versions demonstrated continued optimization potential. FlashAttention-2 [18] achieved an additional $2\times$ speedup over the original FlashAttention through algorithmic improvements in parallelism and work partitioning, reaching 50-73% of the theoretical maximum FLOPs per second achievable on A100 GPUs. This represented exceptional hardware utilization, approaching the limits of what is theoretically possible given memory bandwidth constraints. FlashAttention-3, optimized for Hopper-generation GPUs (H100), introduced asynchronous execution to better overlap different operations and added support for low-precision computation (FP8) to further accelerate processing while maintaining acceptable accuracy.

Hardware-specific optimizations have extended beyond FlashAttention’s core algorithm. S2-Attention introduced heterogeneous context sharding across attention heads, where different heads attend to different subsets of the context tokens while collectively the full context is covered. This approach exploits the observation that attention heads in transformers often specialize in different types of patterns, and not every head needs access to every token. By sharding context across heads, S2-Attention reduced memory bandwidth requirements and achieved $8.79\times$ to $25.3\times$ wall-clock speedup compared to FlashAttention-2 while maintaining full attention performance. The method demonstrated perfect retrieval accuracy on 128K context benchmarks, confirming that strategic context sharding does not sacrifice representational capacity when designed appropriately.

Inference-time efficiency has motivated additional attention innovations. Recycled Attention ad-

dresses the memory burden of the key-value cache that must be maintained during autoregressive generation. As generation proceeds, the KV cache grows linearly with the number of generated tokens, and attention computation time grows quadratically. Recycled Attention alternates between full attention steps that attend to all past tokens and recycled attention steps that attend only to a reduced cache of important tokens. This alternating pattern balances computational efficiency with semantic preservation, maintaining high-quality generation while dramatically reducing the memory footprint and attention computation cost for long-context generation.

3.4 Memory-Augmented and Streaming Methods

An alternative paradigm for handling long contexts involves augmenting transformer models with explicit memory mechanisms that operate outside the standard attention framework. These memory-augmented approaches often draw inspiration from cognitive science and neural memory systems, introducing specialized components designed specifically for long-term information retention.

3.4.1 StreamingLLM and Attention Sinks

[8] discovered a surprising phenomenon in pre-trained large language models that enabled infinite-length sequence processing without fine-tuning. Their analysis revealed that language models exhibit what they termed "attention sinks," where initial tokens in a sequence receive disproportionately high attention weights regardless of their semantic content. This phenomenon occurs because the softmax attention mechanism requires weights to sum to one, and when there is no particularly relevant token to attend to, models learn to allocate attention to consistent sink tokens rather than distributing it diffusely across all tokens.

StreamingLLM exploits this attention sink phenomenon by retaining the initial tokens (typically 4-16 tokens) in the key-value cache alongside recent context maintained through a sliding window. By keeping these sink tokens permanently in the cache, StreamingLLM enables models to maintain stable attention distributions even as the context extends far beyond the training length. The model can attend to the sink tokens when needed to "dump" attention weights, preventing the attention distribution instabilities that would otherwise occur in infinite-length streaming scenarios.

The practical performance of StreamingLLM proved remarkable across multiple model families. StreamingLLM enabled LLaMA-2, MPT, Falcon, and Pythia models to process sequences exceeding 4 million tokens with stable performance, despite these models having been trained on contexts orders of magnitude shorter. Compared to naive sliding window approaches that recompute attention for the entire window at each step, StreamingLLM achieved 22.2× speedup in streaming settings by maintaining the KV cache for recent tokens and only computing attention for new tokens. This efficiency gain made real-time processing of effectively infinite-length streams computationally feasible.

The generality of the attention sink phenomenon across diverse model families and architectures

suggests it represents a fundamental aspect of how softmax attention handles ambiguous attention scenarios. The discovery has influenced subsequent research on efficient attention mechanisms and memory-augmented models, highlighting the importance of understanding emergent behaviors in trained models rather than relying solely on architectural design principles.

3.4.2 Landmark Attention and Block-Level Retrieval

[19] introduced Landmark Attention as an approach to enable random-access to arbitrarily long contexts through learned retrieval of relevant blocks. The method divides input sequences into blocks and associates each block with a landmark token that serves as a compressed representation of the block’s content. During attention computation, the model first attends to the landmark tokens to identify which blocks are relevant to the current query, then attends to the full content of the selected blocks.

This two-stage attention process enables efficient processing of very long contexts by avoiding the need to compute attention over every token in the full context. Instead, the model performs coarse-grained retrieval at the block level through landmark attention, followed by fine-grained attention within selected blocks. The landmark tokens are trained through the standard language modeling objective, learning to encode information that enables effective block selection. This training approach differs from retrieval-augmented methods that use separate retrieval systems, instead integrating retrieval directly into the attention mechanism.

Fine-tuning LLaMA-7B with Landmark Attention extended context to 32K tokens, matching the context capability of models like GPT-4 despite LLaMA’s original 2K context limit. The method maintained accuracy on retrieval tasks while providing computational efficiency through block-level routing. The approach demonstrated that learned attention-based retrieval could provide an alternative to both full dense attention and external retrieval systems, with the routing decisions emerging from the model’s training rather than being hard-coded.

3.4.3 Recurrent Memory Transformers

The Recurrent Memory Transformer (RMT) introduced by [20] combined transformer attention with recurrent connections across segments through special memory tokens. RMT processes long sequences by dividing them into segments, processing each segment with a transformer, and passing memory tokens from one segment to the next to maintain information flow. These memory tokens function as a learned recurrent state that aggregates information across segment boundaries, enabling the model to maintain long-term dependencies despite processing sequences in bounded-length chunks.

The key advantage of RMT lies in its linear scaling of computation with sequence length, achieved by maintaining constant segment length while increasing the number of segments for longer sequences. Memory tokens provide additional capacity beyond the segment length for cross-segment information

aggregation, with the number and dimensionality of memory tokens serving as hyperparameters that control the information bottleneck between segments. Empirical evaluations demonstrated that RMT outperformed Transformer-XL on tasks requiring long-term dependencies, validating the effectiveness of learned memory tokens for sequential information propagation.

Extensions to the RMT framework have explored more sophisticated memory mechanisms. The Associative Recurrent Memory Transformer (ARMT) augmented RMT with associative memory mechanisms inspired by computational neuroscience, enabling more flexible information storage and retrieval patterns within the memory tokens. These associative mechanisms allow memories to be stored and retrieved based on content similarity rather than solely through sequential propagation, potentially enabling better handling of long-range dependencies with non-sequential structure.

3.4.4 External Memory Augmentation

LongMem introduced external memory modules through a decoupled architecture that kept the original language model frozen while adding a side network for memory retrieval and integration [21]. This design preserved the pre-trained model’s capabilities while augmenting it with long-term memory spanning up to 65K tokens. The frozen backbone model served as a memory encoder, processing text to create memory representations, while an adaptive residual side network learned to retrieve and integrate relevant memories for the current context.

The decoupled design offered several advantages. By freezing the backbone model, LongMem avoided catastrophic forgetting that can occur when fine-tuning pre-trained models on long-context data. The side network could be trained specifically for the memory task without interfering with the model’s existing capabilities. The external memory enabled caching of many-shot demonstrations for few-shot learning, allowing the model to access extensive in-context examples without exceeding its base context window.

Self-Extend [22] proposed a related but distinct approach that required no training whatsoever. Self-Extend employed bi-level attention where grouped attention with floor-based position remapping captured dependencies between distant tokens, while neighbor attention maintained fine-grained local relationships. This simple modification, requiring only four lines of code changes, achieved performance matching or exceeding fine-tuning methods on various long-context benchmarks. The success of Self-Extend demonstrated that context extension can sometimes be achieved through clever inference-time modifications to position encoding without the need for extensive retraining.

3.5 Continual Pre-training for Context Extension

While architectural and algorithmic innovations provide the mechanisms for processing long contexts, effective deployment of long-context capabilities in practice has increasingly relied on continual pre-training strategies. Most large language models undergo initial pre-training on large-scale corpora with relatively modest context windows, typically 2-4K tokens, due to the computational

expense of training with longer sequences. Following this base pre-training, a separate context extension phase through continual pre-training on long-context data has emerged as standard practice for developing production long-context models.

3.5.1 Data Construction and Training Strategies

Research into optimal continual pre-training strategies has revealed that data composition critically influences long-context capabilities. Code repositories provide an excellent source of naturally long, highly structured data where dependencies span many lines. Books offer long-form narrative coherence that exercises different aspects of long-range understanding than code. Perhaps counter-intuitively, including high-quality short data in the continual pre-training mixture proves essential for maintaining short-context performance, preventing the degradation that occurs when models are exclusively trained on long sequences.

Evidence suggests that training with sequence lengths exceeding the target evaluation length improves long-context performance, possibly because longer training sequences provide more opportunities to learn long-range dependencies and develop robust positional understanding. Practical training configurations vary by model scale. For 7B and 13B parameter models, training with 32,768-token sequences has proven effective, while larger 34B and 70B parameter models often use 16,384-token sequences due to memory constraints. Typical continual pre-training regimes process 400B additional tokens at extended sequence lengths, representing a substantial but manageable fraction of the original pre-training data volume.

Recent open-source frontier models including Llama-3.1 and Jamba have employed continual pre-training strategies that combine long-context continued training with subsequent supervised fine-tuning on instruction-following data. An interesting finding from these efforts indicates that using predominantly short instruction datasets during the supervised fine-tuning phase can yield strong performance on long-context tasks, provided the model has received adequate long-context continual pre-training. This observation suggests that long-context understanding and instruction-following capabilities can be developed somewhat independently, with continual pre-training establishing the long-context capacity and supervised fine-tuning adapting it to instruction-following without requiring extensive long-form instruction data.

Code Llama exemplifies the continual pre-training approach, having been developed through continued training of LLaMA 2 checkpoints on an additional 400B tokens formed into long training sequences. The domain specificity of code, with its structured syntax and long-range dependencies between definitions and uses, makes it particularly amenable to long-context modeling. LongLLaMA-Code, a variant of Code Llama, incorporated Focused Transformer (FoT) methods during fine-tuning to further extend context handling, demonstrating that domain-specific continual pre-training can be combined with architectural modifications for enhanced long-context capability.

3.5.2 LongAlign and Comprehensive Alignment Recipes

LongAlign established the first complete recipe for long-context alignment, addressing both the continual pre-training phase and the subsequent supervised fine-tuning required to align long-context models with user intentions [23]. The continual pre-training component involved expanding the RoPE base frequency by $200\times$ (from 10K to 2M), conducting continual training on sequences up to 64K tokens for a total of 10B tokens, and developing loss-weighting strategies for packed training with varied lengths. These techniques enabled efficient utilization of training data spanning diverse lengths while maintaining balanced learning across different sequence lengths.

LongAlign’s key contribution included the LongAlign-10k dataset, containing 10,000 long instruction examples ranging from 8K to 64K tokens in length. This dataset was constructed using collected long sequences from nine diverse sources and applying the Self-Instruct approach with a long-context LLM (Claude 2.1) to generate instruction-following examples. The availability of this dataset addressed a critical gap in long-context alignment, as prior work lacked sufficient long-form instruction data for effective supervised fine-tuning.

Training efficiency improvements constituted another important aspect of LongAlign. Packing with loss weighting enabled processing multiple sequences of varying lengths within a single training batch, with loss weights ensuring that shorter sequences did not dominate the training signal. Sorted batching grouped similar-length sequences together to minimize padding waste and improve computational efficiency. These techniques significantly reduced the computational cost of long-context fine-tuning while improving final model quality.

LongAlign introduced LongBench-Chat as an evaluation benchmark specifically designed for assessing instruction-following capability on queries requiring 10K-100K token contexts. This benchmark filled a gap in evaluation methodology, as prior benchmarks focused primarily on retrieval tasks or short-context instruction following. The comprehensive nature of LongAlign, spanning continual pre-training, dataset construction, training efficiency, and evaluation methodology, established it as a foundational reference for practitioners developing long-context models. The work demonstrated conclusively that effective long-context capability requires coordinated attention to both continual pre-training for context extension and proper alignment through supervised fine-tuning with long instruction data.

3.5.3 Efficient Extension Methods

Recent work has focused on reducing the computational cost of continual pre-training for context extension. E²-LLM (Efficient and Extreme Length Extension) achieved dramatic reductions in training costs through a dual RoPE augmentation strategy that trained exclusively on short sequences while still enabling extreme length extension [24]. By scaling and adjusting position indices across samples during training on 4K sequences, E²-LLM eliminated the need to collect and train on long-context data entirely. This approach reduced continual pre-training costs by orders of mag-

nitude while supporting variable context windows at evaluation time, demonstrating that careful manipulation of position encodings during training can substitute for actual long-sequence training in some scenarios.

CLEX (Continuous Length Extrapolation) modeled context extension dynamics through ordinary differential equations over length scaling factors [25]. This formulation enabled smooth extrapolation to $4\text{--}8\times$ the training context length without performance degradation, with a 4K-trained model achieving competitive performance at 32K on the LongBench benchmark. CLEX required minimal additional training or inference latency and integrated seamlessly into RoPE-equipped models including LLaMA and GPT-NeoX. The ODE-based perspective provided theoretical grounding for understanding length extrapolation as a continuous process rather than a discrete scaling operation.

Data engineering for context extension has received substantial attention as researchers recognized that data quality and diversity often matter more than sheer quantity. Comprehensive studies of data engineering for 128K context scaling have examined training data construction, curriculum design, and evaluation methodology. These studies demonstrated that strategic data selection, incorporating diverse long-context sources and maintaining data quality standards, outperforms naive approaches that simply concatenate documents to create long training sequences.

3.5.4 Training Curriculum and Scheduling

SkyLadder introduced context window scheduling as a training-time optimization that progressively increases sequence length across training phases [26]. Rather than training with a fixed maximum sequence length throughout the entire training run, SkyLadder begins with shorter sequences and gradually increases the context window as training progresses. This curriculum-based approach reduces total compute requirements because earlier training phases with shorter sequences are substantially cheaper, while still enabling the model to develop long-context capabilities through exposure to progressively longer sequences in later phases. Empirical results demonstrated that SkyLadder reduced pretraining compute while simultaneously improving final model quality, indicating that the curriculum learning provided better optimization trajectories than static long-context training.

Recent research has also examined the interaction between context length during supervised fine-tuning and resulting model behavior. Counterintuitively, supervised fine-tuning at extended context lengths was found to improve short-context performance in certain scenarios, contrary to the expectation that long-context fine-tuning would exclusively benefit long-context tasks. This bidirectional relationship between short and long-context capabilities during training suggests complex interactions in how models represent and utilize positional information, informing the design of more efficient multi-scale context training procedures.

Synthetic data generation for continued pre-training has emerged as a practical approach for resource-constrained settings where collecting natural long-context corpora proves difficult. Synthetic continued pre-training explores generation of training data specifically designed for length extension,

reducing dependency on naturally occurring long sequences. While synthetic data cannot fully replace diverse natural data for general long-context capability, targeted synthetic generation can provide cost-effective training data for specific long-context tasks or domains where natural data is scarce.

3.6 Emerging Techniques and Current Frontiers

Beyond the established categories of context extension methods, emerging research explores novel approaches that combine insights from multiple paradigms or introduce entirely new perspectives on the problem.

3.6.1 Adaptive and Learned Approaches

Core Context Aware Attention (CCA-Attention) introduced globality-pooling attention where tokens communicate through a reduced-size set of core tokens, reducing computational complexity while preserving semantic understanding [27]. This approach balances efficiency gains with performance preservation by maintaining dense attention within local regions while using pooled global tokens for long-range communication. Similar ideas appear in tokenwise attention scaling, which proposes dynamic scaling of attention mechanisms on a per-token basis to maintain stability across varying context lengths, and context-aware routing mechanisms that use learned routing to distribute attention computation intelligently across the context.

TokenSelect applied learned scoring to identify important tokens for retention in the KV cache during inference, enabling efficient processing of ultra-long sequences without approximating full attention [28]. By selectively retaining only tokens deemed important by a learned scoring function, TokenSelect reduced memory requirements and inference latency while maintaining high task performance. This selective retention approach proved particularly effective for retrieval tasks where relevant information concentrates in small portions of the full context.

Extending LLM context with adaptive grouped attention proposed grouping tokens for attention computation with group sizes that adapt based on position and importance [29]. This enabled smooth context scaling with minimal training overhead through learned grouping strategies that automatically balanced computational efficiency and representational capacity. The adaptive nature allowed the model to allocate more attention computation to important regions while processing less critical regions more efficiently.

3.6.2 Training Efficiency and Sample Efficiency

Research has demonstrated that context extension can be achieved with remarkably limited training data when data is carefully selected. Work showing context window extension with merely 100 training samples illustrated that sample-efficient training is possible through optimal selection of training instances and careful optimization strategies. This dramatic reduction in data require-

ments enables rapid prototyping and experimentation with context extension techniques without the computational resources required for large-scale continual pre-training.

Partial context training explored the counterintuitive finding that efficient long-context training can be achieved through partial context masking during training, where the model is exposed to incomplete contexts rather than always receiving full context. This approach reduced memory requirements during training while paradoxically improving generalization to full contexts at inference time, possibly because partial contexts forced the model to develop more robust representations that could handle missing information.

Segmented base adjustment for RoPE proposed segment-wise adjustment of RoPE base frequencies rather than global scaling, enabling more flexible adaptation to different context regions [30]. By dividing input sequences into segments and applying optimized base adjustments per segment, this approach improved context extension efficiency and performance compared to uniform scaling approaches.

3.6.3 Theoretical Foundations and Analysis

Recent theoretical work has explored probabilistic frameworks for understanding context extension. Bayesian Attention proposed treating position uncertainty explicitly through Bayesian inference, providing a principled framework for length extrapolation [31]. This probabilistic perspective offered theoretical grounding for understanding when and why certain extrapolation strategies succeed, potentially guiding the development of more robust extension methods.

Meta-analyses examining the fundamental requirements for context scaling have argued that extending context requires rethinking attention mechanisms themselves, not merely adjusting position encodings. These analyses advocate for integrated approaches that combine position scaling, attention approximation, and memory strategies, recognizing that optimal long-context performance emerges from coordinated advances across multiple components of the architecture rather than optimization of any single component in isolation.

Work on sparse attention tradeoffs has provided decision frameworks for choosing between sparse and dense attention based on task requirements, sequence length, and hardware constraints. Comprehensive surveys of sparse attention patterns, including fixed patterns, learned patterns, and dynamic routing approaches, have organized the design space and provided guidance on when different approximation strategies prove effective.

3.7 Summary and Current State

Context window extension techniques have evolved rapidly from simple position interpolation approaches to sophisticated integrated systems combining frequency-aware scaling, memory-efficient computation, distributed processing, and targeted continual pre-training. The field has progressed from early 2K-4K token limitations to contemporary production deployments at 128K tokens and

research demonstrations exceeding 2 million tokens. This thousand-fold increase in practical context capacity has been achieved through complementary advances across multiple fronts.

RoPE-based scaling methods, progressing from Position Interpolation through NTK-Aware scaling and YaRN to LongRoPE, have established that careful frequency-domain manipulation enables dramatic context extension with minimal fine-tuning. The key insight underlying these methods is that different frequency components of positional embeddings serve different representational purposes and must be scaled accordingly, with high frequencies preserving local coherence and low frequencies maintaining global structure. Alternative position encoding approaches like ALiBi have demonstrated that strong length extrapolation is possible without traditional positional embeddings, though with different tradeoffs regarding recency bias and task generality.

Architectural and system-level innovations, particularly FlashAttention and Ring Attention, have made long-context processing computationally feasible by addressing memory bandwidth bottlenecks and enabling distributed computation. These advances demonstrate that the quadratic complexity of attention, while fundamental to the algorithm, need not prevent processing of very long sequences when memory hierarchy and device communication are carefully optimized. The progression from FlashAttention through FlashAttention-2 and FlashAttention-3, each achieving substantial additional speedups through deeper hardware-software co-design, illustrates the continuing potential for system-level optimization.

Memory-augmented approaches and streaming methods have shown that explicit memory mechanisms, attention sinks, and block-level retrieval can complement standard attention for handling ultra-long contexts. The discovery of attention sinks in pre-trained models highlights the importance of understanding emergent behaviors in trained systems, while learned memory mechanisms like those in RMT demonstrate that combining transformer attention with recurrent state propagation can achieve linear scaling in computation.

Continual pre-training has emerged as essential practice for deploying long-context capabilities, with research establishing optimal data compositions, training curricula, and alignment strategies. The recognition that long-context capability requires both architectural support for processing long sequences and training procedures that expose models to long-range dependencies reflects a mature understanding of the multifaceted nature of context extension. Methods like LongAlign that provide comprehensive recipes spanning continual pre-training, dataset construction, and evaluation methodology have accelerated practical deployment by providing validated frameworks rather than requiring practitioners to discover effective strategies through trial and error.

Current challenges include maintaining short-context performance while extending long-context capability, a persistent issue addressed through progressive extension strategies, short-context recovery phases, and mixed-length training curricula. Computational constraints remain significant despite algorithmic advances, motivating continued work on sparse attention, efficient training methods, and inference-time optimization. Evaluation methodology has evolved to assess not merely retrieval

capability but genuine long-context understanding, reasoning, and instruction following, with benchmarks like LongBench-Chat providing more comprehensive assessment of long-context competence.

The convergence of these techniques in modern production systems, which typically combine RoPE-based scaling with FlashAttention optimization and continual pre-training on long-context data, demonstrates that effective context extension requires coordinated advances across position encoding, attention computation, system optimization, and training methodology. No single technique proves sufficient in isolation; rather, the combination of complementary approaches yields practical long-context language models. As research continues to push toward million-token contexts and beyond, the integration of insights from theoretical analysis, empirical optimization, and system design promises further advances in humanity’s ability to equip artificial intelligence systems with extended memory and reasoning capabilities over long sequences.

4 Sparse and Efficient Attention Mechanisms

The quadratic complexity of standard self-attention mechanisms represents a fundamental bottleneck for processing long sequences in transformer-based language models. Standard self-attention scales as $O(n^2)$ with respect to sequence length n , imposing severe constraints on both computational resources and memory capacity. This quadratic scaling creates a practical ceiling on context length, typically limiting transformers to sequences of 512 to 2,048 tokens. As the demand for longer context windows has grown across applications ranging from document analysis to code understanding, researchers have pursued diverse strategies to mitigate this fundamental limitation. The approaches fall into several broad categories: fixed sparse attention patterns that reduce the number of attended positions, linear approximations that reformulate the attention mechanism itself, state space models that replace attention with alternative sequence mixing operations, and hardware-aware optimizations that maintain quadratic complexity while dramatically improving wall-clock performance through algorithmic efficiency.

4.1 Taxonomy of Efficient Attention

[32] provided a comprehensive survey of efficient transformer architectures, establishing a taxonomy that has guided subsequent research. Their analysis identified that attention mechanisms can be modified along multiple dimensions, including the sparsity pattern imposed on attention weights, the computational approach used to approximate or compute attention, and the architectural paradigm employed for sequence modeling. Sparsity-based methods reduce the number of token pairs that participate in attention computation, either through fixed patterns determined by position or through learned patterns based on content. Approximation methods maintain dense attention but reduce computational complexity through mathematical reformulations, including kernel methods and low-rank factorizations. Alternative architectures abandon the attention mechanism entirely, replacing it with state space models, linear recurrences, or hybrid approaches that combine multiple paradigms.

4.2 Fixed Sparse Attention Patterns

4.2.1 Longformer

[9] introduced the Longformer architecture, which represents one of the most influential approaches to sparse attention through the combination of local windowed attention with task-motivated global attention. The local component employs sliding windows of fixed size w , typically set to 512 tokens, where each token attends to $w/2$ neighbors on either side. This windowed attention captures local dependencies and maintains fine-grained contextual information within the immediate vicinity of each token. The windowed approach can be extended with dilated windows, analogous to dilated convolutions in computer vision, enabling the model to capture longer-range patterns without incurring full quadratic cost.

The global attention component allows designated tokens to attend to all positions in the sequence and to receive attention from all other tokens. For classification tasks, the [CLS] token typically receives global attention, enabling it to aggregate information from the entire document. For question answering, both question tokens and special separator tokens may employ global attention to facilitate cross-document reasoning. This combination of local and global patterns achieves $O(n)$ complexity with respect to sequence length while maintaining expressiveness comparable to full attention.

Longformer demonstrated strong empirical results across multiple domains. On character-level language modeling benchmarks including text8 and enwik8, Longformer achieved state-of-the-art results, validating its ability to model long-range dependencies in sequential data. The architecture consistently outperformed RoBERTa on long document tasks, with particularly strong results on WikiHop question answering, achieving 65.10% F1 score, and TriviaQA, reaching 80.51

Beyond the core architecture, [9] introduced the Longformer-Encoder-Decoder (LED) variant for sequence-to-sequence tasks, extending the sparse attention paradigm to encoder-decoder architectures. This extension enables efficient processing of long-document summarization and translation tasks where both input and output sequences may be lengthy. The practical impact of Longformer has been substantial, with widespread adoption in applications requiring long-context understanding including legal document analysis, scientific paper processing, and genomic sequence modeling.

4.2.2 BigBird

[10] proposed BigBird, a sparse attention mechanism that combines three complementary components to achieve linear complexity while preserving theoretical properties of full attention. The first component consists of global tokens, a fixed set of g positions that attend to all other positions and receive attention from the entire sequence. These global tokens serve as information hubs, aggregating and distributing information across distant portions of the sequence. The second component employs local window attention, where each token attends to w neighboring positions, capturing fine-grained local context similar to Longformer’s windowed approach. The third and most distinc-

tive component introduces random attention connections, where each query attends to r randomly selected keys throughout the sequence.

The random attention component represents a key theoretical and practical contribution. Inspired by graph sparsification techniques, random connections provide probabilistic coverage of long-range dependencies without requiring the model to specify which distant positions are relevant a priori. This randomness introduces diversity in the attention pattern, enabling the discovery of unexpected dependencies that might be missed by purely local or global patterns. The combination of these three components yields an attention mechanism with $O(n)$ complexity while maintaining expressiveness.

BigBird’s theoretical contributions extend beyond empirical performance. [10] proved that BigBird sparse attention is a universal approximator of sequence functions, demonstrating that the sparse pattern can approximate any function that full attention could compute, given sufficient model capacity. Furthermore, they established that BigBird is Turing complete, showing that the architecture preserves the computational expressiveness of standard transformers despite the sparsity constraint. These theoretical guarantees distinguish BigBird from heuristic sparsity patterns and provide confidence that the sparse structure does not fundamentally limit model capabilities.

Empirically, BigBird handled sequences up to 8x longer than standard transformers could process on equivalent hardware, effectively managing sequences of 4,096 tokens and demonstrating viability for sequences extending to 8,000 tokens. The architecture achieved significant improvements on long-document question answering tasks including NaturalQuestions and HotpotQA, where the ability to reason across lengthy contexts proved crucial. Beyond natural language, BigBird demonstrated remarkable effectiveness on genomic sequence tasks, setting state-of-the-art results on DNA sequence modeling where sequences can extend to tens of thousands of base pairs. The successful application to genomics validated that sparse attention mechanisms generalize beyond language to other sequential domains with long-range dependencies.

The BigBird architecture encompasses multiple variants optimized for different use cases. BigBird-ITC employs internal transformer construction with sparse attention applied uniformly across layers, while BigBird-ETC uses extended transformer construction that varies the sparsity pattern across different layers of the network. These variants provide flexibility in trading off between computational efficiency and model expressiveness depending on task requirements.

4.3 Linear Attention Approximations

4.3.1 Performer

[33] introduced the Performer, which achieves linear-time attention through the FAVOR+ (Fast Attention Via positive Orthogonal Random features) algorithm. Unlike sparse attention methods that reduce the number of attended positions, Performer maintains dense attention but approximates the attention computation through kernel methods. The core insight is that softmax attention can be expressed as a kernel function, and kernel functions can be approximated using random feature

maps. By decomposing the attention matrix using positive orthogonal random features, Performer avoids the explicit computation of the $n \times n$ attention matrix.

The FAVOR+ algorithm provides several theoretical guarantees that distinguish it from earlier linear attention approximations. The approach yields unbiased or nearly-unbiased estimates of the true attention weights, ensuring that the expected value of the approximation matches the exact computation. The estimation exhibits low variance, providing consistent results across different random seeds and training runs. Uniform convergence properties ensure that the approximation quality improves predictably as the number of random features increases. These mathematical guarantees provide confidence in the approximation quality and enable principled tuning of the feature dimension to balance accuracy and efficiency.

The Performer achieves $O(n)$ time complexity and $O(n)$ space complexity with respect to sequence length, compared to $O(n^2)$ for both in standard attention. The approach maintains compatibility with standard transformer architectures, requiring minimal modification to existing codebases. Beyond softmax attention, FAVOR+ supports kernelizable attention mechanisms more broadly, enabling exploration of alternative attention formulations including polynomial kernels and custom kernel functions designed for specific domains.

Empirically, Performer demonstrated competitive performance across multiple modalities. On language modeling tasks, the architecture achieved results comparable to standard transformers while enabling processing of longer sequences within fixed memory budgets. For pixel-level image prediction in the style of ImageGPT, Performer successfully modeled high-resolution images as long sequences of pixels. The architecture proved particularly effective on protein sequence modeling, where the ability to process full-length proteins as single sequences enabled improved structure prediction. On standard benchmarks including the Long Range Arena suite, Performer achieved competitive results while maintaining the theoretical linear complexity guarantee.

The practical speedup achieved by Performer depends on sequence length, with benefits increasing as sequences grow longer. For moderately long sequences of approximately 1,000 tokens, Performer achieves 3-4x speedup compared to standard attention implementations. The speedup increases substantially for sequences of several thousand tokens, where memory constraints often prevent standard attention from running at all. The random feature approximation introduces a small quality degradation compared to exact attention, typically on the order of 1-2% on downstream metrics, representing a reasonable trade-off for applications where sequence length or computational efficiency are critical.

4.3.2 Linformer

[34] demonstrated that the self-attention mechanism can be approximated by low-rank matrices, enabling a different path to linear complexity. The key theoretical insight is that attention matrices in trained transformers often exhibit approximately low-rank structure, suggesting that much of the

information can be captured in a lower-dimensional subspace. Linformer exploits this structure by projecting the keys and values to a lower-dimensional space before computing attention, reducing the effective sequence length from n to k where $k \ll n$.

The projection strategy employs learned linear transformations that map the key and value matrices from dimension $n \times d$ to $k \times d$, where k is the projection dimension and d is the model dimension. By performing this projection, the subsequent attention computation operates on effective sequence length k rather than n , yielding complexity $O(nk)$ which is linear in n when k is treated as a constant. The projection matrices can be shared across different layers or attention heads to reduce parameters, or they can be layer-specific and head-specific to maximize expressiveness. Position-aware projections that incorporate positional information into the projection operation can further improve performance.

With typical projection dimensions of $k = 128$ or $k = 256$ for sequences of length $n = 512$, Linformer achieves substantial efficiency gains. The approach yields 1.5x faster inference time and enables 1.7x larger maximum batch sizes due to reduced memory consumption. These benefits scale favorably as sequences grow longer, with 10x memory reduction at sequence length 2,048 and 20x reduction at 4,096 tokens. Critically, Linformer maintains performance comparable to standard transformers on language modeling benchmarks, with minimal degradation in perplexity or downstream task accuracy.

The low-rank approximation approach provides interpretability benefits, as the learned projection matrices reveal which aspects of the sequence the model considers most important for attention computation. Analysis of trained Linformer models shows that projections often learn to emphasize content-bearing tokens while down-weighting function words and positional markers, suggesting that the approximation captures semantically meaningful structure. The approach proved effective across diverse applications including question answering on SQuAD, long document classification, and machine translation, demonstrating broad applicability of the low-rank attention paradigm.

4.4 State Space Models

4.4.1 Structured State Spaces (S4)

[35] introduced S4 (Structured State Space), representing a fundamental departure from attention-based architectures by modeling sequences through continuous-time state space equations. State space models describe dynamical systems through the equations $\frac{dx}{dt} = Ax + Bu$ and $y = Cx + Du$, where x is a latent state, u is the input, and y is the output. These equations, foundational in control theory and signal processing, capture how systems evolve over time while maintaining a compressed representation of history in the state vector.

The key innovation in S4 is the structured parameterization of the state transition matrix A through HiPPO (High-order Polynomial Projection Operators), which provide a principled approach to initializing the state dynamics. HiPPO matrices are designed to optimally compress history into a

finite-dimensional state representation by projecting the history onto a basis of orthogonal polynomials. This initialization enables the model to capture long-range dependencies from the start of training rather than requiring the model to discover appropriate time scales through gradient descent.

S4 achieves efficient computation through a dual representation that expresses the same model as either a convolution or a recurrence. During training, the convolutional representation enables parallel processing of entire sequences using FFT-based algorithms, achieving $O(n \log n)$ complexity. During inference, the recurrent representation processes tokens sequentially with constant memory, requiring only the maintenance of a fixed-size state vector regardless of sequence length. This duality provides the best of both paradigms: parallelizable training akin to transformers and efficient constant-memory inference akin to RNNs.

The structured parameterization of A proves critical for computational efficiency. By imposing structure through low-rank corrections to diagonal matrices, S4 enables stable diagonalization of the state matrix. This diagonalization reduces the state space computation to a Cauchy kernel, which admits closed-form evaluation with favorable numerical properties. The reduction to Cauchy kernels enables efficient and numerically stable computation even for sequences of tens of thousands of tokens, addressing a key weakness of earlier state space model approaches that suffered from numerical instability.

S4 achieved remarkable empirical results, particularly on tasks requiring long-range reasoning. The model became the first to solve the Long Range Arena Path-X task, which requires tracking dependencies across 16,384 positions, achieving 88% accuracy compared to 50% random baseline and near-zero performance from standard transformers. On Sequential CIFAR-10, where images are processed as sequences of 1,024 pixels, S4 achieved 91% accuracy without data augmentation, demonstrating effectiveness on long visual sequences. For speech classification tasks with audio sequences of length 16,000, S4 achieved 1.7% error rate, representing a 2x improvement over previous methods.

The architecture demonstrated 60x faster generation compared to standard transformers of equivalent size, enabled by the constant-memory recurrent inference mode. This efficiency advantage proved particularly valuable for applications requiring generation of long sequences, including audio synthesis and long-form text generation. S4’s success on raw audio generation tasks validated its ability to capture structure across multiple time scales, a critical requirement for modeling acoustic signals with both fine-grained timing and long-term musical or linguistic structure.

4.4.2 Mamba

[36] introduced Mamba, advancing state space models through selective state space mechanisms that address a key limitation of prior SSMs. While S4 and related models achieved impressive efficiency, they struggled on discrete, information-dense data like natural language where the ability

to selectively filter and route information proves crucial. Mamba introduces selectivity by making SSM parameters—specifically the discretization parameter Δ , input matrix B , and output matrix C —functions of the input content rather than fixed parameters.

This input-dependent parameterization enables content-based reasoning analogous to the query-key-value mechanism in attention. By allowing the model to dynamically adjust how information flows through the state based on token content, Mamba can selectively propagate relevant information and filter irrelevant content. This selectivity addresses the fundamental challenge that fixed-parameter SSMs face when processing language, where importance and relevance of tokens varies dramatically based on semantic content rather than following fixed patterns.

The selective mechanism is implemented through a hardware-efficient scan algorithm that computes state updates efficiently on GPUs. Following principles from FlashAttention, the implementation minimizes memory movement between GPU memory hierarchies, enabling the selective computation to run efficiently despite the additional complexity of input-dependent parameters. The architecture employs a simplified design compared to transformers, eliminating separate attention and MLP blocks in favor of a single Mamba block that combines state space dynamics with gating mechanisms.

Mamba achieves linear $O(n)$ time complexity while maintaining constant-space inference with no KV cache requirement. The throughput advantages are substantial: Mamba achieves 5x higher throughput than transformers of equivalent size, with throughput scaling linearly rather than quadratically as sequence length increases. The architecture handles sequences up to one million tokens with constant memory footprint, enabling applications previously infeasible with transformer-based models.

The empirical results validated Mamba’s approach across multiple modalities. For language modeling, Mamba-3B outperformed transformers of the same size and matched the performance of transformers twice its size, demonstrating that the selective state space mechanism provides sufficient expressiveness for language tasks despite abandoning attention entirely. On downstream language tasks including question answering, classification, and reasoning benchmarks, Mamba exhibited strong zero-shot and few-shot capabilities comparable to attention-based models. Beyond language, Mamba achieved state-of-the-art results on audio modeling tasks, competitive performance on genomic sequence analysis, and promising results on image understanding when adapted to vision tasks.

The impact of Mamba has been substantial, accumulating over 5,000 citations by 2024 and sparking extensive follow-up research. The architecture demonstrated that alternatives to attention can achieve competitive quality on the full range of tasks that have driven transformer development, while offering substantial efficiency advantages. The success of Mamba has renewed interest in recurrent architectures and state space models as viable alternatives to attention for sequence modeling at scale.

4.4.3 Hybrid Architectures: Jamba

[37] introduced Jamba, the first production-grade hybrid architecture that combines Transformer attention layers, Mamba state space layers, and Mixture-of-Experts (MoE) into a unified decoder architecture. The hybrid design emerged from the observation that attention and state space models offer complementary strengths: attention excels at capturing fine-grained dependencies and content-based routing, while state space models provide efficient long-range modeling and constant-memory inference. By interleaving these components, Jamba aims to leverage the advantages of both paradigms while mitigating their respective weaknesses.

The Jamba architecture employs a typical ratio of one Transformer layer to seven Mamba layers, creating repeating blocks that combine both architectural paradigms. The Transformer layers provide global attention capacity at strategic intervals, enabling the model to perform explicit content-based retrieval and reasoning. The Mamba layers handle the bulk of sequence processing with linear complexity, maintaining long-range state while avoiding the quadratic cost of full attention. Selected layers incorporate Mixture-of-Experts, where input is routed to a subset of expert networks, enabling parameter efficiency through conditional computation. With 52B total parameters but only 12B active parameters per token, the MoE component provides substantial model capacity while maintaining computational efficiency.

The hybrid approach yields impressive practical benefits. Jamba achieves 3x higher throughput than pure transformers of equivalent size, enabled by the predominance of efficient Mamba layers. The architecture supports a 256K token context window, among the largest available at the time of release, demonstrating that hybrid designs can scale to extreme context lengths. The combination of linear-complexity Mamba layers and selective attention allows the model to process long documents efficiently while maintaining strong performance on tasks requiring precise attention to detail.

Empirically, Jamba demonstrated quality competitive with larger pure-transformer models, validating that the hybrid architecture does not sacrifice capability for efficiency. The model proved particularly effective on long-context tasks including document question answering, code understanding, and retrieval-augmented generation scenarios. The production-ready implementation and Apache 2.0 open license facilitated adoption, with Jamba serving as a proof of concept that hybrid architectures can succeed in practical deployment.

The Jamba architecture suggests that future language models may increasingly employ heterogeneous layer types, selecting architectural components based on their strengths for specific computational roles. Rather than committing entirely to attention or alternatives, hybrid approaches enable fine-grained optimization of the efficiency-expressiveness trade-off across different layers of deep networks.

4.5 Linear-Time Alternatives

4.5.1 RWKV

[38] proposed RWKV (Receptance Weighted Key Value), which uniquely combines the training efficiency of transformers with the inference efficiency of RNNs through a dual formulation that can be expressed equivalently as either architecture. The RWKV mechanism decomposes attention-like computation into receptance (R), weight (W), key (K), and value (V) components, where receptance controls information reception, weight determines temporal importance, and key-value pairs carry information content. Unlike standard attention, RWKV employs channel-wise attention rather than position-wise attention, attending across feature dimensions rather than across sequence positions.

The dual formulation enables parallel computation during training, where the entire sequence is processed simultaneously in a transformer-like forward pass. During inference, the same computation can be reformulated as a recurrence, processing one token at a time while maintaining a constant-size hidden state. This duality provides parallelizable training with full GPU utilization while enabling constant-space, RNN-like inference without KV cache overhead. The approach achieves $O(n)$ time complexity during training with fully parallel computation, and $O(n)$ total complexity during generation with $O(1)$ space per step.

RWKV incorporates a learned time decay mechanism that determines how quickly information from previous positions fades. The exponential decay provides inductive bias toward more recent context while maintaining the ability to preserve information across long distances when relevant. This time decay mechanism serves a role analogous to positional encoding in transformers, providing temporal structure while remaining parameter-efficient.

The architecture was scaled to 14B parameters, representing the largest dense RNN trained to date and demonstrating that recurrent architectures can reach scales previously thought to require attention mechanisms. RWKV-14B achieved performance comparable to similarly-sized transformers on language modeling benchmarks while offering substantial inference efficiency advantages. The constant-memory inference proved particularly valuable for long-form generation tasks, enabling generation of arbitrarily long sequences without memory growth.

Beyond language, RWKV demonstrated versatility across multiple domains including computer vision, audio processing, and music generation. The architecture’s success validated that alternatives to both attention and state space models remain viable, and that RNN-style recurrence can compete with modern architectures when designed with appropriate mechanisms for information flow and temporal modeling. The community-driven development model, with open-source implementations and collaborative improvement, established RWKV as a grassroots alternative to corporate-developed architectures.

4.5.2 RetNet

[39] introduced RetNet (Retentive Network), which supports three distinct computation paradigms through a unified retention mechanism: parallel representation for training, recurrent representation

for inference, and chunkwise recurrent representation for efficient long-sequence processing. The retention mechanism replaces standard attention with an exponentially-decaying similarity function that enables all three formulations through mathematical equivalence.

The parallel representation expresses retention as a matrix operation similar to attention, enabling full GPU utilization during training. The computation processes all positions simultaneously, calculating weighted combinations of values based on exponentially-decayed position-dependent weights. This parallel form supports efficient backpropagation and gradient computation, enabling RetNet to match transformer training efficiency.

The recurrent representation reformulates the identical computation as an RNN-like recurrence, where the hidden state h_t evolves according to $h_t = \gamma h_{t-1} + k_t v_t^T$, with γ representing the decay factor. This formulation enables $O(1)$ complexity per token during inference, as each step requires only updating the fixed-size hidden state with the current token’s contribution. The recurrent formulation eliminates the KV cache entirely, as all history is compressed into the state vector.

The chunkwise recurrent representation divides long sequences into chunks, processes each chunk in parallel, and combines chunk outputs recurrently. Within each chunk, the parallel formulation enables efficient GPU computation. Across chunks, the recurrent formulation maintains a summary state with constant memory. This hybrid approach provides linear $O(n)$ complexity with favorable constants, enabling efficient processing of sequences that exceed GPU memory capacity for full parallel processing.

RetNet demonstrated impressive efficiency gains in practice. A 7B parameter RetNet model achieved 8.4x faster decoding than an equivalent transformer with KV cache when processing 8K token sequences. Memory consumption dropped by 70% compared to the transformer baseline, enabled by the absence of growing KV cache. The speedup increased for longer sequences, with even more dramatic improvements at 16K or 32K context lengths. Even compared to FlashAttention-optimized transformers, RetNet maintained 7x acceleration, demonstrating that the efficiency gains stem from algorithmic advantages rather than implementation details.

The retention mechanism proved more effective than other linear attention variants, outperforming RWKV, H3, and Hyena on language modeling benchmarks. The exponential decay formulation provides better position encoding than simple linear attention, as it naturally captures the intuition that nearby tokens are typically more relevant while maintaining the ability to preserve information when needed. On downstream tasks including question answering, summarization, and classification, RetNet achieved quality comparable to standard transformers while maintaining the substantial efficiency benefits.

The three computation paradigms provide flexibility for different deployment scenarios. The parallel mode enables efficient training on GPU clusters, the recurrent mode optimizes for streaming or real-time inference, and the chunkwise mode handles batch processing of long documents. This flexibility makes RetNet suitable for diverse production environments with varying computational constraints

and latency requirements.

4.6 Hardware-Aware Optimizations

4.6.1 FlashAttention

[11] introduced FlashAttention, representing a fundamentally different approach to efficient attention that operates orthogonally to sparsity and approximation methods. Rather than changing the attention mechanism itself, FlashAttention optimizes the implementation to account for GPU memory hierarchy, achieving dramatic speedups for exact attention computation with no approximation or quality loss. The core insight is that standard attention implementations are bottlenecked by memory bandwidth rather than arithmetic throughput, as they repeatedly read and write the attention matrix between slow high-bandwidth memory (HBM) and fast on-chip SRAM.

FlashAttention employs a tiling strategy that divides the attention computation into blocks small enough to fit entirely in fast SRAM. The algorithm loads blocks of queries, keys, and values from HBM to SRAM, computes attention within the block, and incrementally updates the output in HBM. By processing attention in tiles and never materializing the full $n \times n$ attention matrix, FlashAttention reduces memory traffic by orders of magnitude. The tiled computation requires careful handling of the softmax normalization to maintain numerical stability and exact equivalence with standard attention.

The backward pass employs a recomputation strategy, recalculating attention values during back-propagation rather than storing them. This approach trades computation for memory, an advantageous trade-off given that modern GPUs are often memory-bound rather than compute-bound. The recomputation maintains numerical stability through careful ordering of operations and use of log-space computations where appropriate. The resulting algorithm is provably IO-optimal for a range of SRAM sizes, meaning no algorithm can achieve better memory complexity while computing exact attention.

FlashAttention achieved substantial practical speedups across diverse scenarios. For BERT-large with 512-token sequences, the approach yielded 15% end-to-end training speedup. For GPT-2 with 1K sequences, speedup reached 3x, with even greater benefits for longer sequences. On long-range tasks with 4K tokens, FlashAttention achieved 2.4x speedup while maintaining identical model quality. The memory savings proved equally impressive: 5x reduction at sequence length 512, scaling to 10x at 2K tokens and 20x at 4K tokens. These memory savings enabled longer sequences on fixed hardware, effectively expanding the accessible context length.

The IO-aware approach proved broadly applicable, benefiting any attention computation regardless of the specific attention variant. FlashAttention can be applied to sparse attention patterns, multi-head attention, cross-attention in encoder-decoder models, and various forms of local or global attention. This generality led to rapid adoption across the field, with FlashAttention becoming the de facto implementation for attention in many production systems. The approach demonstrated

that algorithmic optimizations informed by hardware characteristics can yield practical benefits comparable to or exceeding mathematical approximations.

4.6.2 FlashAttention-2

[18] introduced FlashAttention-2, addressing remaining inefficiencies in the original FlashAttention algorithm. While FlashAttention achieved 25-40% of theoretical peak FLOPs utilization on A100 GPUs, analysis revealed that thread block distribution, shared memory communication, and non-matrix-multiply operations limited efficiency. FlashAttention-2 implemented three key optimizations: reducing non-GEMM FLOPs through algorithmic improvements, parallelizing computation across thread blocks even within a single attention head, and optimizing work distribution between warps to minimize shared memory communication.

The improvements yielded 2x speedup over FlashAttention and up to 9x speedup over standard PyTorch attention. FlashAttention-2 achieved 50-73% of theoretical maximum FLOPs per second on A100 GPUs, approaching the efficiency of highly optimized GEMM (matrix multiplication) operations. On A100 GPUs, the implementation reached 225 TFLOPs/s with 72% model FLOPs utilization, representing state-of-the-art efficiency for attention computation. The improved GPU occupancy through better parallelism meant that FlashAttention-2 scaled more effectively to different sequence lengths and batch sizes.

The practical impact extended beyond raw speed improvements. By nearly doubling attention efficiency, FlashAttention-2 enabled longer context windows within fixed time budgets, larger batch sizes within fixed memory budgets, and reduced training costs for large language models. The implementation maintained the exact computation guarantee, producing bitwise identical results to standard attention while operating dramatically faster. This combination of speed and exactness made FlashAttention-2 the preferred implementation for production systems where both efficiency and reproducibility matter.

Subsequent development produced FlashAttention-3, targeting newer H100 GPUs with additional optimizations for asynchronous memory operations and low-precision computation including FP8 support. The continued evolution of hardware-aware attention algorithms demonstrates that co-design of algorithms and hardware represents a sustainable path to efficiency gains, complementing mathematical improvements to attention mechanisms themselves.

4.7 Additional Efficient Mechanisms

Sparse Transformers. Prior to Longformer and BigBird, [40] introduced Sparse Transformers with factorized sparse attention patterns achieving $O(n\sqrt{n})$ complexity. The approach employed two-dimensional attention factorization, where separate attention heads attend to either rows or columns of a two-dimensional arrangement of tokens. This factorization proved particularly natural for image generation, where pixels are arranged in a spatial grid. Sparse Transformers set state-of-

the-art results on density modeling for Enwik8 and demonstrated strong performance on CIFAR-10 and ImageNet 64x64 generation, establishing early evidence that sparsity could maintain quality while improving efficiency.

Synthesizer. [41] questioned the necessity of token-token interactions in attention through the Synthesizer architecture, which learns synthetic attention weights without query-key dot products. The Dense Synthesizer variant learns position-dependent attention patterns through feed-forward networks, while the Random Synthesizer employs fixed random attention patterns. Surprisingly, these simplified mechanisms achieved performance comparable to standard transformers on multiple benchmarks, suggesting that positional attention patterns capture much of attention’s value. The Synthesizer achieved 60% speedup over dynamic convolutions and demonstrated that the dot-product attention formulation, while effective, is not the only viable approach to sequence mixing.

Grouped Query Attention. [2] introduced Grouped Query Attention (GQA), an intermediate approach between Multi-Head Attention (MHA) with separate key-value pairs per head and Multi-Query Attention (MQA) with a single shared key-value pair. GQA groups queries to share key-value heads, reducing KV cache size by 10-100x while maintaining quality closer to full MHA than MQA achieves. The approach enables 12x faster decoder inference through reduced memory traffic, providing a practical middle ground between quality and efficiency. GQA has seen widespread adoption in recent large language models including Llama 2 and Mistral, validating its effectiveness in production systems.

Attention with Linear Biases (ALiBi). [6] proposed Attention with Linear Biases (ALiBi), replacing positional embeddings with linear biases added to attention scores proportional to the distance between tokens. This simple modification enables remarkable extrapolation properties: models trained on sequences of length L can effectively process sequences of length $2L$ to $8L$ at test time. ALiBi maintains performance at 10,000+ token contexts when trained on far shorter sequences, providing a parameter-efficient alternative to learned positional embeddings. The approach has been adopted in models including MPT and BLOOM, demonstrating practical value for scenarios where test-time sequences may exceed training lengths.

Nyströmformer. [42] applied the Nyström method to approximate self-attention through sampling landmark tokens. By selecting m representative tokens and computing attention interactions only for this subset, the approach reconstructs approximate full attention from the sampled structure. Nyströmformer achieves $O(n)$ complexity while maintaining dense attention patterns, providing an alternative approximation strategy to kernel methods. The approach proved effective on Long Range Arena benchmarks while offering interpretability through the explicitly selected landmark tokens.

Hyena. [43] introduced Hyena, replacing attention with implicitly parameterized long convolutions and data-controlled gating. The architecture achieves subquadratic $O(n \log n)$ complexity through efficient FFT-based convolution while maintaining expressiveness through element-wise gating that depends on input content. Hyena demonstrated 20% compute reduction at sequence length 2K compared to transformers, with speedups increasing to 2x at 8K tokens and 100x at 64K tokens. The convolutional approach provides an alternative paradigm to both attention and state space models, validating that multiple architectural families can achieve competitive efficiency and quality.

4.8 Comparative Analysis

The landscape of efficient attention mechanisms reveals several clear trade-offs between computational complexity, memory requirements, implementation complexity, and model quality. Fixed sparse patterns including Longformer and BigBird achieve linear complexity through structured sparsity while maintaining exact computation on the sparse support. These methods offer strong interpretability and preserve the attention paradigm, but the fixed patterns may miss task-relevant dependencies that fall outside the sparse structure. The approaches integrate easily into existing transformer codebases and benefit from theoretical guarantees of expressiveness, particularly in the case of BigBird’s universal approximation results.

Linear approximations including Performer and Linformer reduce complexity through mathematical reformulation rather than sparsity. Performer’s kernel method provides unbiased approximation with theoretical convergence guarantees, while Linformer’s low-rank factorization exploits empirically observed structure in attention matrices. These methods achieve true $O(n)$ complexity and handle arbitrary-length sequences, but introduce approximation error that can manifest as small quality degradation on downstream tasks. The approximation error typically remains small, on the order of 1-2% on standard metrics, but may be more significant for tasks requiring precise attention to rare events or distant dependencies.

State space models including S4 and Mamba replace attention entirely with alternative sequence-mixing operations based on continuous-time dynamics. These architectures achieve linear or log-linear training complexity and constant-memory inference, providing the strongest efficiency guarantees particularly for very long sequences. The constant-memory inference eliminates KV cache overhead, enabling generation of arbitrarily long sequences without memory growth. However, state space models employ fundamentally different mechanisms from attention, requiring different training procedures and achieving different inductive biases. The ecosystem of pretrained models and fine-tuning recipes remains less mature than for transformers, though Mamba’s success has rapidly expanded available resources.

Linear-time alternatives including RWKV and RetNet provide hybrid approaches that combine transformer-like training with RNN-like inference. These architectures achieve $O(n)$ complexity with constant-space inference through dual formulations that express the same computation as either

parallel or recurrent. The approaches offer practical benefits for deployment scenarios where memory efficiency matters, while maintaining training efficiency through parallelization. The retention and receptance mechanisms provide different inductive biases than attention, which can be advantageous for certain tasks but may require adaptation of training procedures developed for transformers.

Hardware-aware optimizations including FlashAttention operate orthogonally to the above categories, making any attention implementation dramatically more efficient through IO-awareness. FlashAttention maintains exact attention computation with no approximation while achieving speedups of 2-9x and memory reductions of 5-20x depending on sequence length. The approach demonstrates that algorithm-hardware co-design can yield benefits comparable to mathematical approximation without sacrificing quality. The exact computation and drop-in compatibility have led to widespread adoption, with FlashAttention becoming standard in many transformer implementations.

Hybrid architectures including Jamba demonstrate that multiple paradigms can be combined within a single model, selecting architectural components based on their strengths for specific computational roles. By interleaving attention, state space, and mixture-of-experts layers, hybrid designs optimize the efficiency-expressiveness trade-off at a granular level. This direction suggests that future models may increasingly employ heterogeneous architectures rather than committing to a single mechanism, enabling fine-grained optimization based on layer-specific requirements.

4.9 Evaluation Metrics and Benchmarks

The Long Range Arena benchmark established by [44] provides standardized evaluation across multiple tasks requiring long-range reasoning, including ListOps for hierarchical structure, Text Classification for document-level understanding, Retrieval for similarity matching, Image Classification for sequential pixel processing, and Path-X for extreme-length path tracing. Performance on Long Range Arena has become standard for validating efficient attention mechanisms, enabling direct comparison across sparse patterns, approximations, and alternative architectures.

Beyond task-specific accuracy, efficient attention mechanisms must be evaluated on computational metrics including wall-clock training time, wall-clock inference latency, memory consumption during training, memory consumption during inference, throughput measured in tokens per second, and maximum sequence length achievable on fixed hardware. These metrics often trade off against each other—for instance, sparse patterns may increase computational FLOPs relative to their memory savings, while approximations may provide better throughput than sparse methods on modern hardware optimized for dense computation.

The practical value of efficient attention depends critically on deployment context. For offline batch processing of documents, total throughput matters most, favoring methods like FlashAttention that maximize GPU utilization. For interactive applications, per-token latency proves critical, favoring constant-memory inference methods like Mamba and RetNet. For edge deployment, total memory

footprint dominates, again favoring constant-memory approaches. For research and experimentation, implementation simplicity and compatibility with existing codebases becomes paramount, favoring drop-in replacements like Longformer and FlashAttention.

4.10 Future Directions

Several open research questions remain in the design of efficient attention mechanisms. The optimal mixing of attention and alternative architectures in hybrid models remains unclear, with design choices currently made through empirical exploration rather than principled understanding. Better position encoding schemes for linear attention methods could improve their effectiveness on tasks requiring precise positional reasoning. Understanding how efficiency methods affect scaling laws relative to standard transformers would inform decisions about when to employ efficient mechanisms versus simply scaling compute.

Training stability for state space models and linear methods at very large scale continues to present challenges, with numerical issues and optimization difficulties that do not affect standard transformers. Learned or adaptive sparsity patterns that adjust based on task and input could outperform fixed patterns while maintaining efficiency. Hardware co-design tailored to efficient attention mechanisms, rather than retrofitting algorithms to hardware designed for dense matrix multiplication, could unlock further performance gains.

The theoretical understanding of when approximations preserve task-relevant information remains incomplete. While universal approximation results provide existence guarantees, practical understanding of the relationship between approximation error and downstream task performance would guide design decisions. Multi-modal extensions that apply efficient attention to vision, audio, and video modalities require domain-specific adaptations of the core mechanisms.

The field continues rapid evolution, with new architectures and optimizations appearing regularly. The diversity of successful approaches—sparse patterns, kernel methods, state spaces, recurrences, and hardware optimizations—suggests that no single solution dominates across all contexts. Rather, the choice of efficient attention mechanism depends on the specific requirements of sequence length, available compute, deployment constraints, and task characteristics. As language models continue to scale and expand to longer contexts, efficient attention mechanisms will remain critical to enabling progress while managing computational costs.

5 Key-Value Cache Optimization

The Key-Value cache represents one of the most critical bottlenecks in large language model inference, fundamentally constraining the deployment of long-context models in production environments. During autoregressive generation, the KV cache stores attention keys and values for all previously processed tokens, enabling efficient inference by avoiding redundant computation of past token representations. However, this optimization comes at a substantial memory cost. The cache

size scales according to $2 \cdot n_{\text{layers}} \cdot n_{\text{heads}} \cdot d_{\text{head}} \cdot e \cdot b \cdot l$, where e represents element size in bytes, b denotes batch size, and l indicates sequence length. For a LLaMA 3 70B model processing a 128K context window, this formulation yields approximately 40 GB of memory for a single request, dwarfing the model’s parameter memory requirements.

The severity of this bottleneck extends beyond raw memory consumption. LLM inference operates in a fundamentally memory-bandwidth-bound regime rather than a compute-bound one. Generating each new token requires loading the entire KV cache from GPU memory, transforming what should be a computationally intensive operation into an I/O-dominated process. Empirical measurements of existing serving systems reveal utilization rates of only 20–38% for allocated KV cache memory, with 60–80% wasted through fragmentation and over-reservation to accommodate worst-case sequence lengths [45]. The inference efficiency plummets from approximately 70% during training to merely 10% during autoregressive generation, as the architecture shifts from compute-intensive parallel processing to memory-bandwidth-limited sequential token generation. This dramatic efficiency gap motivates a diverse ecosystem of optimization techniques spanning multiple dimensions: eviction policies that selectively discard tokens, quantization methods that reduce numerical precision, architectural modifications that fundamentally reduce cache requirements, and memory management systems that eliminate fragmentation waste.

5.1 KV Cache Eviction Policies

Eviction policies operate on the fundamental observation that not all cached tokens contribute equally to attention computation. By selectively retaining only the most influential tokens, these methods achieve substantial memory reductions while preserving model quality. The challenge lies in accurately identifying which tokens matter most, a determination that varies across attention heads, transformer layers, and generation contexts.

5.1.1 H₂O: Heavy-Hitter Oracle

[46] introduced H₂O (Heavy-Hitter Oracle), a pioneering eviction policy grounded in the empirical observation that attention values exhibit extreme concentration. Through systematic analysis across multiple model families, the authors demonstrated that a small subset of tokens—termed “heavy hitters”—receives disproportionate attention weight, contributing the majority of effective value during attention computation. These heavy hitters emerge naturally during pre-training, correlating strongly with token co-occurrence patterns in natural language. Crucially, their removal results in catastrophic performance degradation, whereas selectively retaining them preserves model quality even at aggressive compression ratios.

H₂O formulates token selection as a dynamic submodular optimization problem, balancing retention of two categories: recent tokens to maintain position-aware context, and heavy-hitter tokens to preserve semantic importance. The method provides theoretical guarantees on the optimality of its selection strategy, ensuring that the chosen subset maximizes expected attention value under

budget constraints. When evaluated on OPT-6.7B and OPT-30B models, H₂O achieves remarkable efficiency gains of up to 29× throughput improvement over DeepSpeed Zero-Inference and HuggingFace Accelerate, and up to 1.9× latency reduction compared to standard implementations. With only 20% of tokens retained as heavy hitters, the method maintains model quality across diverse benchmarks spanning question answering, summarization, and language understanding tasks. The technique operates as a plug-and-play solution requiring no model fine-tuning, though it incurs computational overhead during the prefill phase to identify heavy hitters through attention profiling.

5.1.2 ScissorHands and the Persistence of Importance

Building on similar intuitions about token importance, [47] proposed ScissorHands, which maintains the KV cache at a fixed budget through a different lens. The method rests on the “persistence of importance” hypothesis: pivotal tokens that substantially influenced attention at one generation step will continue to exert significant influence in future steps. This temporal stability of importance enables ScissorHands to make eviction decisions based on historical attention patterns rather than repeated profiling. The system stores pivotal tokens with higher retention probability, achieving up to 5× reduction in KV cache memory usage without requiring model fine-tuning or architectural modifications.

The effectiveness of ScissorHands extends further when combined with quantization techniques. Through a hybrid approach incorporating 4-bit quantization alongside selective token retention, the method achieves compression ratios up to 20× while maintaining competitive model quality. This combination demonstrates that eviction and quantization operate on complementary dimensions—eviction reduces the number of cached tokens, while quantization reduces the bits per token—enabling multiplicative rather than merely additive gains. The assumption of importance stability, however, introduces limitations. In scenarios where attention patterns shift dramatically during generation, such as multi-topic conversations or tasks requiring significant context switches, the persistence hypothesis may break down, resulting in suboptimal token selection.

5.1.3 StreamingLLM and Attention Sinks

[48] discovered a remarkable phenomenon that fundamentally reshapes our understanding of attention patterns in pre-trained language models. Their StreamingLLM framework revealed the existence of “attention sinks”—initial tokens that receive consistently high attention scores despite lacking semantic importance. This phenomenon emerges from the softmax operation’s constraint that attention weights must sum to one. When no token in the context provides a strong semantic match for a query, the model allocates excess attention weight to initial tokens, effectively using them as an attention “sink” or padding mechanism.

The practical implications of this discovery proved transformative for streaming applications. By retaining the KV states of merely four initial tokens alongside a sliding window of recent tokens, StreamingLLM enables LLMs to perform stable language modeling with effectively infinite sequence

lengths. Models including LLaMA-2, MPT, Falcon, and Pythia successfully process sequences up to 4 million tokens without fine-tuning, achieving speedups of up to $22.2\times$ compared to sliding window baselines that must recompute attention from scratch. The attention sink phenomenon emerges naturally during pre-training on sufficient data, becoming more pronounced in larger models and with extended training. Importantly, StreamingLLM operates as a training-free solution, requiring no position encoding modifications or interpolation techniques. The method has achieved widespread adoption, with integration into production systems including NVIDIA TensorRT-LLM and HuggingFace Transformers, demonstrating its practical utility for chatbots, document processing, and interactive systems requiring long-term context maintenance.

Subsequent theoretical work has illuminated the mechanisms underlying attention sink emergence. [49] provided systematic analysis of when and why sinks form during training, demonstrating that they emerge after sufficient exposure to diverse linguistic patterns and become stronger with increased model capacity. The phenomenon reflects a learned strategy for handling the softmax constraint in the absence of semantically relevant tokens, suggesting fundamental implications for attention mechanism design.

5.1.4 Adaptive Methods: FastGen, SnapKV, and PyramidKV

Recognition that different attention heads and layers exhibit fundamentally different patterns motivated the development of adaptive eviction strategies. [50] introduced FastGen, which employs targeted attention profiling during the prefill stage to discern the intrinsic structure of each attention module. Rather than applying a uniform eviction policy across all heads, FastGen adaptively selects from a portfolio of strategies: evicting long-range contexts for heads emphasizing local patterns, discarding non-special tokens for heads centered on special tokens like sentence boundaries, and employing standard full caching only for heads that broadly attend across all tokens. This adaptive approach achieves up to 55% latency reduction at 16K generation length and 44.9% pruned ratio on LLaMA 1-65B, with negligible generation quality loss compared to full caching. The method operates as a plug-and-play technique requiring no resource-intensive fine-tuning, profiling attention patterns once during prefill and applying the learned strategy throughout decoding.

[51] extended the concept of head-specific selection with SnapKV, which selects a fixed number of tokens per attention head by retaining those with highest estimated contribution to the attention distribution. The method approximates token importance by accumulating attention weights received from an observation window of recent input tokens, smoothed with a moving average to reduce variance from stochastic generation. Building on this foundation, [52] introduced PyramidKV, which incorporates a critical insight from analysis of layer-wise attention patterns: deeper layers require less KV cache than shallower layers. The method assigns layer-specific retention budgets with larger allocations to lower layers, which focus on local pattern recognition and require broader context, and smaller allocations to upper layers, which perform higher-level semantic operations on more abstract representations. PyramidKV preserves performance metrics comparable to full

caching using just 12% of the standard KV cache (2048 tokens) on LongBench evaluations, and maintains acceptable quality with only 0.7% of the full cache in extreme memory-constrained scenarios. The layer-aware budget allocation proves particularly effective for long-context tasks, where the pyramid structure naturally aligns with the information funneling that occurs across transformer depth.

5.1.5 Q-HITTER and Quantization-Aware Selection

A subtle but critical interaction between eviction and quantization motivated [53] to develop Q-HITTER, which reveals a fundamental limitation of combining H₂O-style selection with low-bit quantization. The problem manifests as a cascading error: H₂O identifies heavy hitters based on attention patterns computed with full-precision keys and values, but these selected tokens may possess activation distributions that are inherently difficult to quantize. High-magnitude outliers in heavy-hitter tokens cause quantization error to propagate through attention computation, corrupting the very patterns that motivated their selection. Furthermore, applying quantization before heavy-hitter identification distorts attention scores, causing selection to optimize for the wrong objective.

Q-HITTER addresses this coupling through dual-metric selection that jointly considers both attention importance and quantization friendliness. For each token, the method computes a composite score combining attention weight accumulation with a measure of how amenable the token’s activation distribution is to quantization. Tokens with smooth, well-behaved distributions score higher on quantization friendliness than those with extreme outliers, even if their attention importance is comparable. This quantization-aware selection achieves near-optimal quality at 4-bit precision and below, where H₂O experiences significant degradation. Performance evaluations demonstrate impressive gains: up to 20× memory reduction with 33× throughput improvement over HuggingFace Accelerate, 7× over DeepSpeed, 4× over FlexGen, and 1.3× over H₂O itself, highlighting the importance of considering interactions between complementary optimization techniques.

5.1.6 HashEvict and Locality-Sensitive Hashing

Traditional attention-based eviction methods face a computational dilemma: identifying important tokens requires computing attention scores, but the goal of eviction is to avoid such computation in the first place. [54] proposed HashEvict to resolve this circularity through locality-sensitive hashing (LSH). Rather than computing full attention to determine token importance, the method employs LSH to rapidly approximate similarity between query tokens and cached key tokens. Binary hash codes derived from random projections enable fast Hamming distance computation, identifying dissimilar tokens for eviction before attention computation begins. This pre-attention decision making eliminates computational overhead: tokens evicted via LSH never enter the attention computation, yielding cumulative savings across layers.

Evaluated on LLaMA 3 across question answering, retrieval, and free-response tasks, HashEvict

achieves 30–70% KV cache compression with minimal performance degradation. The prefill phase demonstrates particularly impressive speedups of $1.5\text{--}2\times$ compared to H₂O and ScissorHands, which must compute full attention before making eviction decisions. The method trades perfect similarity measurement for computational efficiency, relying on LSH’s theoretical guarantees that close vectors in high-dimensional space map to similar hash codes with high probability. This approximation proves sufficient for eviction decisions, where the goal is identifying tokens that are clearly unimportant rather than perfectly ranking all tokens by importance.

5.2 KV Cache Quantization

Quantization reduces the precision of stored key and value tensors, directly decreasing both memory footprint and memory bandwidth requirements. The fundamental challenge lies in maintaining attention accuracy despite reduced numerical precision, as quantization errors propagate through the attention mechanism and accumulate across decoding steps. Successful quantization requires careful consideration of activation distributions, outlier patterns, and the sensitivity of attention computation to numerical noise.

5.2.1 KIVI: Asymmetric 2-Bit Quantization

[55] developed KIVI, a tuning-free asymmetric 2-bit quantization method that achieves remarkable compression through a fundamental insight: keys and values exhibit different outlier patterns and thus benefit from different quantization strategies. Through systematic analysis of activation distributions across layers and attention heads, the authors discovered that key tensors exhibit high-magnitude outliers concentrated in specific channels, while value tensors display more uniform distributions with token-level rather than channel-level structure. This observation motivated an asymmetric quantization approach: keys receive per-channel quantization to handle channel-wise outliers, while values receive per-token quantization aligned with streaming inference patterns.

The asymmetric strategy enables KIVI to operate at aggressive 2-bit precision while maintaining model quality comparable to full precision inference. Evaluations on LLaMA, Falcon, and Mistral model families demonstrate $2.6\times$ reduction in peak memory usage (including model weights), enabling $4\times$ larger batch sizes for the same memory budget. Throughput improvements range from $2.35\times$ to $3.47\times$ on real LLM workloads, achieved through carefully optimized CUDA kernels that efficiently quantize and dequantize during inference. The per-token quantization strategy for values proves particularly beneficial for streaming inference, where new tokens append directly to the existing cache without requiring re-quantization of previous tokens. This streaming compatibility eliminates a major source of computational overhead in naive quantization approaches. KIVI operates as a plug-and-play solution requiring zero fine-tuning, making it immediately applicable to pre-trained models and enabling rapid deployment in production serving systems.

5.2.2 KVQuant and Extreme Context Lengths

Targeting extreme context lengths up to 10 million tokens, [56] introduced KVQuant, a comprehensive quantization framework incorporating four novel techniques that synergistically address the challenges of sub-4-bit quantization. First, per-channel key quantization adjusts quantization dimensions to match the inherent structure of key activation distributions, grouping elements along the channel dimension where outliers naturally concentrate. Second, pre-RoPE key quantization applies quantization before rotary positional embeddings, mitigating the distributional shift that RoPE introduces and enabling more accurate quantized representations. Third, non-uniform KV cache quantization derives per-layer sensitivity-weighted datatypes, recognizing that different layers tolerate quantization differently and allocating precision accordingly. Fourth, per-vector dense-and-sparse quantization explicitly isolates numerical outliers on a per-vector basis, preventing extreme values from skewing quantization ranges and degrading the representation of typical values.

This multi-pronged approach achieves less than 0.1 perplexity degradation with 3-bit quantization on WikiText-2 and C4 benchmarks, enabling previously impossible inference scenarios: LLaMA-7B processing a 1 million token context on a single A100-80GB GPU, and 10 million token contexts on an 8-GPU system with $8\times$ KV cache compression. Custom CUDA kernels optimized for low-precision matrix-vector multiplication deliver $1.7\times$ speedups at 4-bit precision compared to FP16 baseline operations. The non-uniform quantization scheme proves particularly important at extreme context lengths, where the cumulative effect of layer-wise quantization errors would otherwise cause catastrophic quality degradation. By carefully tuning per-layer precision based on sensitivity analysis, KVQuant maintains the delicate balance between memory efficiency and representational capacity required for million-token inference. The method has achieved integration into vLLM, one of the most widely deployed LLM serving frameworks, demonstrating its production readiness and practical utility.

5.2.3 GEAR: Hybrid Quantization with Low-Rank Correction

[57] proposed GEAR, recognizing that pure quantization approaches face fundamental limitations: at very low bit-widths, quantization alone cannot maintain accuracy across the diversity of activation patterns in KV cache. GEAR addresses this through a three-component hybrid approach. The majority of KV cache entries, which exhibit similar magnitude scales, receive ultra-low-precision 4-bit quantization. Quantization residuals—the errors introduced by quantization—are captured through low-rank matrix approximation via singular value decomposition, exploiting the observation that residual errors often lie in a low-dimensional subspace. Individual outlier errors that escape both quantization and low-rank approximation receive explicit sparse matrix correction, handling extreme values through a separate sparse representation that maintains high precision for outliers while applying aggressive compression to typical values.

This hybrid strategy achieves near-lossless 4-bit compression with up to $2.38\times$ throughput improvement and $2.29\times$ peak memory reduction. The method explicitly addresses a challenge unique to

autoregressive generation: errors compound at each decoding step, as the corrupted KV cache influences the generation of subsequent tokens, which in turn adds to the cache and propagates errors forward. By maintaining a low-rank correction term that captures systematic quantization biases, GEAR prevents the accumulation of correlated errors that would otherwise cause quality drift during long sequence generation. The sparse correction component handles the statistically rare but semantically critical outliers that often carry outsized importance for model behavior, ensuring that compression does not inadvertently discard the very information that determines generation quality.

5.2.4 KITTY and Information-Theoretic Quantization

Recent work by [58] approaches 2-bit quantization through the lens of information theory, proposing KITTY (K-means Information Theory-guided Token-wise quantization for accurate 2-bit KV cache). The method employs clustering algorithms to group tokens with similar activation patterns, then assigns quantization codebooks optimized for each cluster. Information-theoretic analysis guides the clustering process, ensuring that quantization preserves the mutual information between full-precision and quantized representations. This token-wise clustering approach adapts to the diversity of activation patterns across different contexts, allocating representational capacity where it provides maximal information preservation. Preliminary evaluations suggest improved accuracy compared to uniform 2-bit quantization at comparable computational overhead, though the method requires additional profiling to determine optimal cluster assignments.

5.3 Architectural Modifications

While eviction and quantization optimize within the constraints of standard transformer architecture, architectural modifications fundamentally reduce KV cache requirements by altering how attention operates. These changes typically require training or uptraining but deliver permanent efficiency gains that compound with other optimization techniques.

5.3.1 Multi-Query Attention

[59] introduced Multi-Query Attention (MQA) as a direct response to the memory bandwidth bottleneck in autoregressive decoding. Standard Multi-Head Attention maintains separate key and value projections for each attention head, requiring the model to load H copies of key-value tensors during inference, where H denotes the number of attention heads. Shazeer’s analysis identified this repeated loading of similar information as the primary performance limiter on modern accelerators. MQA modifies the architecture to share a single set of key and value projections across all query heads, while maintaining separate query projections to preserve representational diversity. This sharing dramatically reduces KV cache size proportional to the number of attention heads, typically yielding 8–16 $\times$ reduction for models with 8–16 attention heads.

The memory bandwidth savings translate directly to inference speedup, as each token generation requires loading the shared key-value state once rather than H times. Quality evaluations revealed

minimal accuracy degradation from this architectural change, particularly when models train with MQA from initialization. The technique enables substantially larger batch sizes within fixed memory budgets, improving throughput for serving scenarios. Early adopters including LLaMA-2 and Falcon demonstrated MQA’s effectiveness in production models, though subsequent work would reveal that the single shared key-value head sometimes introduces a quality-efficiency trade-off that later methods would seek to better balance.

5.3.2 Grouped-Query Attention

[2] proposed Grouped-Query Attention (GQA) as a generalization interpolating between standard Multi-Head Attention and Multi-Query Attention. GQA divides the H query heads into G groups, with each group sharing a single key-value head. This formulation recovers MHA when $G = H$ (each group contains one head) and MQA when $G = 1$ (all heads form a single group), while intermediate values of G explore the trade-off space between memory efficiency and representational capacity. The authors demonstrated that GQA captures much of MQA’s efficiency benefit while mitigating quality degradation, particularly important for larger models where the capacity reduction from single key-value head becomes more pronounced.

A critical contribution of the GQA work is the uptraining recipe: converting existing MHA checkpoints to GQA requires only 5% of original pre-training compute through a fine-tuning procedure that leverages the knowledge already captured in MHA weights. This low conversion cost enabled rapid adoption, as organizations could retrofit existing models rather than training from scratch. The quality-efficiency trade-off proves favorable across model scales, with GQA achieving near-MHA quality at speeds comparable to MQA. Following publication in May 2023, GQA achieved remarkably swift adoption in production models. Meta incorporated GQA in LLaMA 2 (July 2023) and LLaMA 3 (2024), while Mistral AI employed it in Mistral-7B (September 2023). By 2024, GQA had become the de facto standard for new model releases, demonstrating the practical importance of balanced KV cache reduction.

For concrete illustration, consider LLaMA 3 8B: with standard MHA, processing a long context requires approximately 280 GB of KV cache, triggering out-of-memory errors on most available GPUs. With GQA configured to use 4 groups, the same model requires approximately 70 GB of KV cache—a 75% reduction that enables inference on consumer hardware while maintaining model quality competitive with the MHA baseline. This dramatic improvement in memory requirements fundamentally expands the deployment scenarios for large language models, making long-context inference practical on single-GPU systems.

5.3.3 Cross-Layer KV Sharing

[60] introduced Cross-Layer Attention (CLA), extending the sharing concept beyond heads within a layer to sharing across adjacent layers. The method recognizes that transformer layers form a compositional hierarchy, with each layer performing increasingly abstract transformations of the input

representation. CLA computes key-value projections for only a subset of layers, allowing subsequent layers to reuse KV activations from previous layers rather than computing fresh projections. At both 1B and 3B model scales, applying CLA with a sharing factor of 2 (computing KV for every other layer) yields $2\times$ KV cache reduction while incurring less than 1% perplexity degradation for MQA models with typical head dimensions of 64 and 128. Crucially, CLA operates orthogonally to both MQA and GQA, enabling combination with either technique for multiplicative rather than additive gains.

[61] systematically investigated cross-layer sharing through their Layer-Condensed KV Cache (LCKV) framework, which uniformly divides transformer layers into groups of adjacent layers and pairs queries from all layers in each group with KVs from the group’s bottom layer. When reducing KV cache size by $2\times$, most LCKV configurations achieve higher throughput than standard transformers while maintaining competitive performance. The systematic study by [62] further demonstrated that cross-layer sharing enables $2\times$ throughput improvements across diverse model architectures and sizes, with careful analysis of which sharing patterns prove most effective for different model families and training regimes. The authors found that sharing works better in middle-to-deep portions of models, where adjacent layers produce increasingly similar representations, compared to early layers where representations change rapidly or final layers where output-specific transformations occur.

5.3.4 MiniCache and Depth-Dimension Compression

[63] developed MiniCache, exploiting the high similarity between KV states in adjacent layers at middle-to-deep portions of transformer models. Rather than discarding less important layers’ KV states entirely, MiniCache compresses across the depth dimension through magnitude-direction decomposition. The method disentangles each KV state into a magnitude component (the norm of the activation vector) and a direction component (the unit-normalized vector). For adjacent layers with highly similar directions, MiniCache interpolates the directional components while preserving the distinct magnitudes, achieving compression through the observation that directions change slowly across layers while magnitudes may vary. Token retention mechanisms preserve highly distinct state pairs unmerged, minimizing information loss for the minority of tokens where adjacent-layer representations diverge significantly.

On ShareGPT dialogues, LLaMA-2-7B with 4-bit MiniCache achieves $5.02\times$ compression, approximately $5\times$ throughput enhancement, and 41% memory footprint reduction compared to FP16 full cache baseline. The depth-dimension compression proves complementary to sequence-dimension techniques like token eviction, as they operate on orthogonal axes of the KV cache tensor. A system could potentially combine PyramidKV’s layer-wise token budgeting with MiniCache’s cross-layer compression, evicting unimportant tokens within each layer while merging similar representations across layers, though such combined approaches remain an open research direction.

5.4 Memory Management Innovations

Beyond reducing cache size, innovative memory management addresses fragmentation waste and enables efficient sharing of KV cache across requests. These systems-level optimizations operate orthogonally to compression techniques, providing multiplicative efficiency gains when combined.

5.4.1 PagedAttention

[45] introduced PagedAttention, drawing direct inspiration from virtual memory and paging in operating systems. Traditional attention implementations require contiguous memory for keys and values, but sequences have unpredictable lengths that necessitate either pre-allocating maximum-length buffers (wasting memory for short sequences) or dynamic allocation (causing fragmentation as sequences of different lengths finish at different times). PagedAttention resolves this through block-based allocation: the KV cache partitions into fixed-size blocks, each containing keys and values for a fixed number of tokens (typically 16). Contiguous logical blocks in a sequence map to potentially non-contiguous physical blocks through per-sequence block tables that translate logical to physical addresses, analogous to page tables in OS virtual memory.

This design achieves near-optimal memory utilization with under 4% waste compared to 60–80% in existing systems, as only the final block of each sequence contains unused slots. The block table mechanism enables sophisticated memory management strategies. Multiple sequences can share physical blocks containing identical content, critical for scenarios like beam search where hypotheses share common prefixes, or batched inference where multiple users submit requests with common prompt templates. Blocks deallocate immediately upon sequence completion and enter a free pool for allocation to new requests, eliminating the fragmentation that plagues traditional contiguous allocation schemes.

The vLLM serving system built on PagedAttention achieves 2–4 $\times$ throughput improvement compared to FasterTransformer and Orca while maintaining equivalent latency. The gains compound at larger batch sizes and longer sequences, precisely the operating regime where KV cache memory becomes most constrained. PagedAttention has achieved widespread adoption as the foundation for modern LLM serving systems, with deployment in major cloud providers and integration into open-source frameworks. The success of PagedAttention demonstrates that careful systems engineering, applying decades-old OS concepts to new domains, can deliver order-of-magnitude improvements even in heavily optimized systems.

5.4.2 vAttention: OS-Level Demand Paging

[64] proposed vAttention as an alternative approach to dynamic memory management, questioning whether application-level paging is necessary when operating systems already provide sophisticated demand paging mechanisms. PagedAttention requires custom attention kernels aware of block tables and capable of gathering from non-contiguous physical blocks. These custom kernels introduce

maintenance burden and potential performance overhead. vAttention instead leverages OS and GPU support for virtual memory: KV cache occupies contiguous virtual address space, but the operating system lazily allocates physical pages only when accessed, automatically evicts pages under memory pressure, and transparently handles the mapping between virtual and physical memory.

This design preserves the abstraction of contiguous memory for attention kernels, enabling use of highly optimized standard attention implementations without modification. The OS handles paging decisions based on access patterns, potentially capturing workload-specific locality better than application-level heuristics. Evaluated on LLaMA models, vAttention achieves $1.97\times$ faster token generation compared to vLLM’s PagedAttention, and $3.92\times$ faster prompt processing compared to PagedAttention’s FlashAttention variant. The method maintains near-zero memory waste comparable to PagedAttention while eliminating custom kernel complexity.

However, vAttention faces deployment constraints: not all GPU architectures provide unified virtual memory with demand paging support, and OS page sizes may not align optimally with attention computation granularity. These hardware dependencies limit vAttention’s applicability compared to PagedAttention’s broader compatibility. The tension between these two approaches reflects a broader question in systems design: whether domain-specific application-level management or general-purpose OS-level mechanisms better optimize for specialized workloads. Current evidence suggests both have roles, with PagedAttention dominating deployment due to wider hardware support, while vAttention demonstrates intriguing possibilities for future hardware-software co-design.

5.4.3 Prompt Caching

[65] addressed a different dimension of memory management: reuse of computation across requests sharing common prompt segments. Many production workloads exhibit significant prompt overlap: system messages repeat across all requests from an application, retrieval-augmented generation systems incorporate the same documents into multiple queries, and conversation systems maintain shared context across turns. Standard inference recomputes attention for these shared segments in every request, wasting both computation and memory bandwidth.

Prompt Cache introduces a schema-based approach where users explicitly mark reusable prompt modules through special tokens. During initial processing, the system computes and caches attention states (KV representations) for marked modules, associating them with unique identifiers. Subsequent requests referencing the same modules fetch cached states rather than recomputing, assembling a complete KV cache from a mix of cached and newly-computed components. This modular attention reuse delivers dramatic latency improvements: $8\text{--}60\times$ reduction in time-to-first-token depending on whether execution occurs on GPU ($8\times$) or CPU ($60\times$), with the larger CPU gains reflecting the higher cost of attention computation on less specialized hardware.

The technique proves particularly impactful for chatbot systems with shared system prompts, retrieval-augmented generation with repeated document contexts, multi-turn conversations reusing

history, and batch inference with common instructions. Major API providers including OpenAI and Anthropic have deployed prompt caching in production, offering reduced per-token pricing for cached content and thereby incentivizing users to structure prompts for reusability. The explicit schema requirement represents both a strength and limitation: users must consciously design for caching through proper prompt structure, but this explicitness ensures cache correctness and predictable behavior compared to automatic schemes that might incorrectly share states across semantically different contexts.

5.4.4 SqueezeAttention and Layer-Wise Budget Allocation

[66] introduced SqueezeAttention, optimizing KV cache allocation across two dimensions simultaneously: the sequence dimension (which tokens to keep) and the layer dimension (how much cache per layer). Traditional methods optimize only sequence dimension, applying uniform per-layer budgets. SqueezeAttention recognizes that transformer layers exhibit heterogeneous sensitivity to KV cache size, with some layers maintaining quality under aggressive compression while others degrade rapidly. The method measures per-layer importance through cosine similarity between input perturbations and output changes, identifying layers that merely pass through information (low importance) versus layers that substantially transform representations (high importance).

Based on importance scores, SqueezeAttention reallocates cache budgets from less sensitive to more sensitive layers. A layer that tolerates 70% token eviction with minimal degradation contributes its excess budget to a sensitive layer that requires 90% retention for quality preservation. This redistribution achieves 30–70% memory reduction with up to $2.2\times$ throughput improvement, outperforming uniform budget allocation that must choose between aggressive compression everywhere (degrading quality on sensitive layers) or conservative compression (wasting capacity on tolerant layers). The two-dimensional optimization proves particularly effective for long-context inference, where the cumulative effect of per-layer budget decisions determines overall system efficiency.

5.5 Integrated Systems and Future Directions

The diversity of KV cache optimization techniques raises important questions about integration and composition. [67] proposed entropy-guided dynamic budget allocation, using attention entropy as a signal for per-layer cache requirements. Layers with high attention entropy—indicating broad, diffuse attention across many tokens—receive larger budgets, while layers with concentrated attention patterns tolerate smaller caches. On Qwen3-4B, this approach achieves 4.18% memory reduction while preserving ROUGE scores, and on Mistral-0.1v-7B delivers 46.6% decoding time reduction. The entropy metric provides a theoretically grounded alternative to empirical importance measurement, though computing attention entropy introduces overhead that must be amortized across sufficient generation steps.

Research into combining complementary techniques remains nascent. Q-HITTER demonstrated the importance of considering quantization-eviction interactions, but comprehensive frameworks

that jointly optimize eviction, quantization, architectural choices, and memory management remain underdeveloped. [68] provided critical analysis of KV cache compression limitations, identifying scenarios where compression fails: out-of-distribution inputs that violate importance assumptions, reasoning-heavy tasks sensitive to information loss, and long-horizon dependencies that aggressive compression disrupts. The work by [69] specifically examined reasoning task sensitivity, finding that compression affects multi-step reasoning disproportionately compared to simple question answering, suggesting task-specific compression strategies may be necessary.

Long-context generalization methods like LM-Infinite [70] and InfLLM [71] explore alternative architectural approaches. LM-Infinite employs lambda-shaped attention masks and distance limits to achieve training-free generalization from 2K to 32K tokens with $7.5\times$ memory reduction and $2.7\times$ decoding speedup. InfLLM introduces memory-based long context handling, storing distant context in a separate memory structure with token-relevant lookup, enabling inference on sequences up to 1 million tokens without expanding the attention window. These methods suggest that fundamental rethinking of attention mechanisms, beyond optimization within standard transformer architecture, may unlock further efficiency gains.

The YOCO architecture [72] exemplifies radical architectural redesign. Rather than the standard self-attention decoder, YOCO employs a decoder-decoder architecture with a self-attention encoder followed by cross-attention decoder. The self-attention encoder processes tokens and produces a global KV cache, which all cross-attention decoder layers share, reducing memory scaling from $O(N \times L)$ for standard transformers to $O(N)$, where N denotes sequence length and L denotes layer count. This architectural change delivers approximately $4\times$ memory reduction, though it requires training from scratch rather than uptraining from existing checkpoints.

FlashAttention [11] and its successors optimize attention computation through IO-aware algorithms that minimize memory transfers between GPU memory hierarchy levels. While primarily targeting training efficiency, FlashAttention reduces inference memory footprint by up to $20\times$ and achieves $3\times$ speedup for sequences of 128–2K tokens, with continued efficiency gains up to 64K tokens. The IO-aware perspective proves complementary to KV cache compression: FlashAttention optimizes how we compute attention given a fixed cache size, while compression techniques reduce the cache size itself.

Industry deployment reveals the practical importance of KV cache optimization. NVIDIA’s technical documentation on CPU-GPU memory sharing for KV cache offload demonstrates hardware-software co-design approaches, leveraging NVLink and PCIe bandwidth to store portions of KV cache in CPU memory when GPU memory saturates, trading bandwidth for capacity. The LMCACHE system [73] addresses enterprise-scale serving across multiple nodes, incorporating fault tolerance, cost optimization, and cache coherence protocols for distributed KV cache. These systems-level concerns, often overlooked in research focused on single-model optimization, prove critical for production deployment at scale.

The rapid pace of innovation in KV cache optimization throughout 2023–2024 reflects both the critical importance of this bottleneck and the substantial headroom for further improvement. Multiple research directions remain open: unified frameworks combining complementary techniques, hardware-aware optimization tailored to specific accelerator architectures, dynamic adaptation adjusting compression strategies based on input characteristics, extreme context lengths pushing toward 100M+ tokens, and extension to multi-modal models where KV cache must accommodate visual tokens alongside text. The compression ratios of 2–20 $\times$ demonstrated by current techniques, often with minimal quality degradation, suggest that KV cache management will remain a productive research area as model sizes and context requirements continue their upward trajectory. As deployment scenarios diversify—from edge devices to data center clusters, from chatbots to complex reasoning systems—the portfolio of available optimization techniques grows increasingly valuable, enabling practitioners to select and combine methods appropriate for their specific constraints and objectives.

6 Context Compression Techniques

Context compression techniques address the fundamental challenge of efficiently managing long contexts in large language models while maintaining performance. As modern applications increasingly demand the processing of extensive documents, lengthy conversations, and complex reasoning chains, the need to compress contextual information without sacrificing semantic integrity has become paramount. The field has evolved from simple token removal strategies to sophisticated learned representations that capture semantic content in compressed form, reflecting diverse trade-offs between compression ratio, computational cost, and performance preservation.

The landscape of context compression can be understood through multiple complementary perspectives. At the highest level, methods divide into soft prompt compression, which employs learned continuous representations to encode contextual information into compact embeddings, and hard prompt compression, which explicitly removes or reorders tokens to reduce sequence length. Beyond this dichotomy, architectural approaches modify the underlying transformer structure to enable efficient memory management, while information-theoretic frameworks establish fundamental bounds on what compression can achieve. Recent comprehensive surveys [74] have identified additional perspectives including attention optimization, parameter-efficient fine-tuning, modality integration, and synthetic language approaches, each offering unique insights into compression mechanisms.

6.1 Soft Prompt Compression Through Learned Representations

Soft prompt compression methods transform contextual information into dense vector representations that serve as compact substitutes for lengthy text sequences. These approaches leverage the representational capacity of neural networks to encode semantic information in continuous embedding spaces, enabling significant compression while maintaining task-relevant information through learned transformations.

6.1.1 Gisting and Representational Bottlenecks

The gisting approach introduced by [75] represents a conceptually elegant solution to prompt compression through the introduction of representational bottlenecks in attention mechanisms. By modifying standard transformer attention masks to force models to condense input information into dedicated “gist” token activations, this method creates context-specific compression in a single forward pass. The approach requires minimal architectural modifications, introducing only new vocabulary token embeddings and attention mask adjustments, making it readily integrable with existing frameworks and systems.

Gisting achieves up to $26\times$ compression on LLaMA-7B and FLAN-T5-XXL models, yielding up to 40% FLOPs reductions and 4.2% wall time speedups with minimal quality loss on tasks amenable to high compression ratios. The method’s training efficiency stands as a particular advantage, as gist models can be trained with no additional cost beyond standard instruction fine-tuning through straightforward attention mask modifications. However, performance degrades rapidly when compressing longer contexts beyond hundreds of tokens, limiting applicability to contemporary long-context scenarios. The fundamental limitation stems from the fixed-capacity bottleneck, which cannot effectively scale to the diverse information densities present in extended sequences. Despite these constraints, gisting demonstrates that architectural modifications to attention mechanisms can enable compression without explicit token removal or summarization.

6.1.2 AutoCompressors and Summary Vector Generation

[76] advanced soft prompt compression through AutoCompressors, which adapt pre-trained language models into specialized compression models capable of transforming long contexts into summary vectors accessible as soft prompts. Unlike gisting’s attention-based bottleneck, AutoCompressors employ an unsupervised training objective where long documents are processed in segments, with summary vectors from all previous segments incorporated into language modeling of subsequent segments. This segmented approach enables processing of sequences up to 30,720 tokens during fine-tuning of OPT and Llama-2 models.

The AutoCompressor framework demonstrates that summary vectors serve as effective substitutes for plain-text demonstrations in in-context learning scenarios, simultaneously improving perplexity on long-context tasks while reducing inference costs. The method provides a simple and inexpensive solution to context window extension, requiring only standard fine-tuning procedures without architectural modifications or complex training objectives. By training on massive text data with language modeling objectives, AutoCompressors learn to extract and preserve task-relevant information in compact vector representations. The learned summary vectors maintain semantic information effectively, as evidenced by their ability to substitute for full-text demonstrations in few-shot learning scenarios. This capability suggests that the compression process captures abstract task-relevant patterns rather than merely memorizing surface-level token sequences.

The success of AutoCompressors highlights the potential of unsupervised learning for compression, where models discover compression strategies through exposure to diverse text without explicit supervision on what information to retain or discard. This approach contrasts with supervised methods that require labeled data indicating important content, instead relying on the implicit signal provided by language modeling objectives to guide compression decisions.

6.1.3 In-Context Autoencoder Architecture

[77] proposed the In-Context Autoencoder (ICAE), which frames context compression as an autoencoding problem with a learnable encoder and fixed decoder. The encoder, adapted from a language model using Low-Rank Adaptation (LoRA), compresses long contexts into a limited number of memory slots, while the decoder (the target LLM) conditions on these compressed representations for various downstream purposes. This architectural separation enables efficient training and deployment, as the encoder requires only approximately 1% additional parameters while the decoder remains frozen.

ICAE employs a two-stage training process that first pre-trains the encoder with both autoencoding and language modeling objectives on massive text data, ensuring that memory slots accurately represent original context. The second stage fine-tunes the system on instruction data to produce desirable responses to diverse prompts. This staged approach separates the learning of compression from the learning of task-specific behaviors, potentially improving both objectives through focused optimization. The method achieves $4\times$ context compression with improved inference latency and reduced GPU memory cost, demonstrating practical benefits for deployment scenarios.

The autoencoder framing provides theoretical grounding for compression through the lens of information bottlenecks and rate-distortion theory. By forcing information through a constrained number of memory slots, ICAE creates an explicit bottleneck that encourages retention of only essential information. The use of LoRA for encoder adaptation ensures parameter efficiency while maintaining model capacity for learning complex compression functions. The combination of autoencoding objectives (which encourage reconstruction fidelity) and language modeling objectives (which encourage semantic coherence) creates a multi-objective optimization that balances compression quality with downstream task performance.

6.2 Hard Prompt Compression Through Token Manipulation

Hard prompt compression methods directly modify prompt text through token removal, reordering, or paraphrasing, creating shorter sequences that preserve essential semantic content. These approaches offer high interpretability, as compressed prompts remain human-readable text, and require minimal or no additional training, enabling straightforward deployment with existing language models.

6.2.1 LLMLingua and Coarse-to-Fine Compression

[78] introduced LLMLingua, a coarse-to-fine prompt compression method that leverages small language models to identify and remove low-importance tokens. The approach employs perplexity from well-trained small models (GPT-2-small or LLaMA-7B) as a proxy for token importance, with higher perplexity indicating greater information content and lower perplexity suggesting redundancy or predictability. This perplexity-based ranking enables principled token selection without task-specific training.

The LLMLingua framework incorporates three key components that work synergistically to achieve high-quality compression. First, a budget controller maintains semantic integrity under high compression ratios by allocating compression budgets across prompt segments based on their estimated importance. Second, token-level iterative compression models interdependence between compressed contents, recognizing that token importance depends on which other tokens remain in the compressed prompt. Third, instruction tuning achieves distribution alignment between the small compression model and target LLM, ensuring that importance judgments transfer effectively across models with different training distributions.

The coarse-to-fine strategy operates hierarchically, first identifying important prompt segments at a document or paragraph level, then performing fine-grained token selection within retained segments. This multi-scale approach proves more effective than uniform token pruning, as it recognizes that information importance varies both within and between segments. LLMLingua achieves up to $20\times$ compression with minimal performance loss across diverse benchmarks including GSM8K for mathematical reasoning, BBH for challenging reasoning tasks, ShareGPT for conversational contexts, and Arxiv-March23 for summarization. The method’s state-of-the-art results demonstrate that carefully designed hard prompt compression can rival or exceed soft prompt methods while maintaining interpretability and requiring no architectural modifications.

6.2.2 LongLLMLingua for Extended Contexts

Building upon the LLMLingua foundation, [79] developed LongLLMLingua to address specific challenges in long-context scenarios, including higher computational cost, performance reduction due to the “lost in the middle” phenomenon, and position bias in transformer attention. The method extends the original coarse-to-fine compression with four key innovations tailored to long-context processing.

The question-aware coarse-to-fine compression mechanism conditions compression decisions on the specific query or task, recognizing that information importance is inherently task-dependent. This query-aware approach enables more aggressive compression of query-irrelevant content while preserving query-relevant information at higher fidelity. The document reordering mechanism strategically positions documents within the context window to mitigate position bias, placing high-importance documents at the beginning and end where transformer models demonstrate better

attention and retrieval. Dynamic compression ratios adapt to varying information densities across documents, applying aggressive compression to low-information content and conservative compression to information-dense segments. Finally, the subsequence recovery strategy improves the model’s perception of key information by maintaining important multi-token sequences intact rather than fragmenting them through token-level pruning.

LongLLMLingua achieves remarkable performance improvements, including up to 21.4% accuracy gains on NaturalQuestions with 4 $\times$ compression, 94% cost reduction on the LooGLE long-context benchmark, and 1.4 $\times$ to 2.6 $\times$ latency acceleration on 10,000-token prompts with 2 $\times$ to 6 $\times$ compression ratios. These results demonstrate that compression can not only reduce costs but actually improve task performance by reducing noise and mitigating position bias effects. The method’s effectiveness across chain-of-thought reasoning, long-context comprehension, and retrieval-augmented generation scenarios highlights its broad applicability to diverse long-context applications.

6.2.3 Selective Context and Redundancy Reduction

[80] proposed Selective Context, which identifies and prunes redundancy in input context to create more compact representations. Unlike LLMLingua’s perplexity-based approach, Selective Context focuses explicitly on redundancy detection, seeking to remove repeated or highly predictable information while preserving unique content. The method achieves 50% context cost reduction with 36% memory savings and 32% inference time reduction, while incurring minimal performance degradation of only 0.023 BERTscore and 0.038 faithfulness on evaluation metrics.

Selective Context demonstrates effectiveness across diverse document types including arXiv papers, news articles, and long conversations, and across tasks including summarization, question answering, and response generation. The method’s task-agnostic nature enables deployment without task-specific tuning, providing a general-purpose compression solution applicable to varied scenarios. The conservative compression ratios (typically 2 $\times$ to 4 $\times$) prioritize performance preservation over aggressive size reduction, positioning Selective Context as a safe, production-ready compression approach for applications where quality maintenance is paramount.

The redundancy-focused perspective offers complementary insights to perplexity-based methods. While low perplexity indicates predictability from a language modeling perspective, redundancy detection identifies repetition and overlap across context segments. Content may exhibit low perplexity due to grammatical regularity or common phrasing while still conveying unique semantic information, whereas redundant content explicitly repeats previously stated information. By targeting redundancy specifically, Selective Context preserves diverse information even when that information follows predictable linguistic patterns.

6.3 Advanced Hard Compression Techniques

Dynamic Token Pruning. Recent advances in token pruning have introduced dynamic selection mechanisms that adapt compression to generation context. LazyLLM, developed by Apple Machine Learning Research, enables language models to dynamically select different subsets of tokens from context in different generation steps, recognizing that token importance varies across the generation process. The method uses attention scores from prior transformer layers to measure token importance for predicting the next token, selectively computing key-value pairs for important tokens while deferring computation of remaining tokens to later steps when they may become relevant.

This lazy computation approach achieves efficient long-context processing by reducing computation while maintaining token availability, ensuring that all context tokens remain accessible when needed. Unlike static pruning methods that permanently remove tokens before generation, dynamic pruning adapts to the evolving information needs of generation, potentially accessing different context subsets for different generated tokens. This flexibility better aligns with the observation that different parts of long contexts become relevant at different points in generation, particularly for complex reasoning or multi-hop question answering.

Token Merging and Structured Pruning. Beyond individual token pruning, recent work has explored token merging strategies that combine similar tokens in the key-value cache to reduce distinct entries while maintaining semantic coverage. Clustering-based approaches group tokens by embedding similarity, replacing clusters with representative tokens or merged representations. This approach proves particularly effective when context contains substantial redundancy or repeated concepts expressed through varied phrasing.

Structured pruning extends token-level compression to higher-level units including attention heads and entire layers. Research on attention head pruning has revealed that only a small subset of attention heads prove important for most tasks, with important heads exhibiting interpretable functions such as attending to adjacent words or tracking specific syntactic relations. The vast majority of heads, especially in encoder self-attention, can be removed with minimal performance impact. Methods employing A* search for head selection have demonstrated the ability to eliminate redundant attention heads while maintaining strict accuracy guarantees through optimal head subset selection.

6.4 Architectural Memory Compression

6.4.1 Compressive Transformers

[81] introduced the Compressive Transformer, which extends the Transformer-XL memory mechanism by compressing old activations into compressed memory rather than discarding them when the attention window slides forward. This approach maintains short-term memory of recent activations in full detail while compacting older activations into lower-resolution compressed representations,

creating a two-tier memory system that balances detail and coverage.

The compression mechanism employs convolutional pooling to aggregate older hidden states into compact representations, with hyperparameters controlling the compression ratio and the boundary between detailed recent memory and compressed historical memory. Attention mechanisms span both recent and compressed memories, enabling the model to access both fine-grained recent context and coarse-grained historical context. This design proves particularly effective for tasks requiring long-range dependencies where distant context provides useful signal even in compressed form.

Compressive Transformers achieved state-of-the-art results on WikiText-103 (17.1 perplexity) and Enwik8 (0.97 bits per character) language modeling benchmarks, demonstrating that compressed memories can effectively substitute for full-detail long-range context. The approach generalizes beyond language, proving effective for high-frequency speech modeling and serving as a memory mechanism for reinforcement learning agents on visual reasoning tasks. The introduction of the PG-19 benchmark, derived from books to provide challenging long-range modeling tasks, has promoted research in long-sequence learning.

The architectural integration of compression into the model structure offers advantages over post-hoc compression methods, as the model learns to leverage compressed representations during training rather than adapting to externally imposed compression. This joint learning enables the development of attention patterns and feature representations optimized for compressed memory access. However, architectural approaches require training models from scratch or extensive fine-tuning to incorporate compression mechanisms, whereas post-hoc methods can be applied to existing pre-trained models without retraining.

6.4.2 Recurrent Context Compression

[82] proposed Recurrent Context Compression (RCC), which extends LLM context windows through iterative segment compression inspired by state space models and recurrent neural networks. For sequences exceeding the maximum context length, RCC divides input into fixed-length segments and compresses each segment sequentially, generating state vectors that capture essential information. The concatenation of state vectors from all segments provides compressed historical context accessible during inference.

The recurrent structure proves particularly effective for extremely long sequences, demonstrating up to $32\times$ compression on text reconstruction with BLEU4 scores near 0.95 and achieving nearly 100% accuracy on passkey retrieval tasks with sequences up to one million tokens. These results demonstrate that iterative compression can maintain information fidelity across extreme sequence lengths when compression operates hierarchically across segments rather than attempting to compress entire sequences in a single step.

RCC addresses a unique challenge in instruction-context compression interference, where compressing both task instructions and context into the same format reduces instruction clarity and degrades

model performance. The proposed instruction reconstruction strategy detects and recovers instruction quality while maintaining context compression, improving downstream task performance by separating instruction handling from context compression. This separation recognizes that instructions and context serve different roles, with instructions providing task specification requiring high fidelity and context providing information that may tolerate greater compression.

The recurrent compression architecture enables scalable processing of arbitrary-length sequences with linear memory growth through compression, contrasting with the quadratic memory requirements of standard transformer attention. The fixed-size state vectors provide constant memory overhead per segment regardless of segment length, enabling practical deployment on memory-constrained devices for extremely long sequences.

6.5 Key-Value Cache Compression

Key-value caching stores key and value matrices during inference to avoid recomputation, but cache memory grows linearly with sequence length, consuming substantial memory for long contexts. Each generated token requires storing key-value pairs for all previous tokens across all transformer layers and attention heads, leading to memory consumption that can exceed model parameter memory for long sequences.

6.5.1 Token Eviction Strategies

StreamingLLM addresses cache growth by maintaining key-value states for only the first few tokens (typically 4) combined with a sliding window of recent tokens. The initial tokens serve as “attention sinks” that anchor attention patterns, while recent tokens provide current generation context. This fixed-size cache enables processing of arbitrarily long sequences with constant memory, though at the cost of losing middle context information.

H2O (Heavy Hitter Oracle) dynamically identifies important tokens based on cumulative attention scores during generation, maintaining a two-part cache structure with budgets for recent tokens and “heavy hitter” tokens that receive high attention. This adaptive approach preserves tokens demonstrating high importance through empirical attention patterns while evicting tokens that receive minimal attention. The dynamic identification enables more effective compression than static rules, as importance emerges from actual model behavior rather than heuristic assumptions.

SnapKV compresses the key-value cache during the prefill stage using a small observation window at the prompt end to predict token importance. This early compression reduces cache size from the start of generation rather than waiting for cache growth to trigger eviction. The observation window principle recognizes that token importance for generation often becomes apparent from attention patterns in a limited context window, enabling predictive importance estimation without full-sequence processing.

6.5.2 Principled KV Compression

Recent work on Expected Attention [83] provides a principled framework for key-value compression by estimating KV pair importance through predicting how future queries will attend to them. The method analyzes the expected distribution of future queries and computes expected attention scores in closed form, enabling ranking of key-value pairs by predicted future importance. This forward-looking approach proves more effective than backward-looking methods based on historical attention, as it optimizes for future utility rather than past usage.

The training-free nature of Expected Attention enables deployment without model modification or fine-tuning, relying instead on mathematical analysis of attention mechanisms and activation properties. The closed-form computation provides efficiency advantages over simulation-based importance estimation, which would require sampling multiple future generation trajectories to estimate expected attention.

Model-directed compression approaches enable models to indicate which information to discard through learned importance scoring, potentially adapting compression to task requirements and input characteristics. Adaptive KV cache compression methods like ACC-RAG dynamically adjust compression rates based on input complexity, avoiding over-compression of simple queries and under-compression of complex queries. This complexity-aware adaptation recognizes that optimal compression ratios vary substantially across queries, with simple factual questions tolerating aggressive compression while complex multi-hop reasoning requires more conservative compression to preserve reasoning chains.

6.5.3 Compression Techniques and Limits

Key-value compression employs diverse techniques beyond token selection. Key quantization reduces the precision of key vectors while maintaining semantic information, achieving significant size reduction with minimal accuracy loss through careful bit-width selection and quantization-aware training. KIVI demonstrated that 2-bit asymmetric quantization of key-value caches enables longer context generation on consumer GPUs without quality degradation, making long-context inference accessible on resource-constrained hardware.

Value offloading moves some value vectors to slower storage (CPU memory or disk), fetching them when needed for attention computation. This approach trades memory for latency, enabling processing of longer contexts than GPU memory alone permits while incurring access latency for offloaded values. Selective offloading strategies prioritize keeping high-importance values in fast memory while offloading low-importance values, minimizing latency impact.

Recent theoretical work has begun exploring fundamental limits of key-value cache compression, investigating whether compression can continue indefinitely or whether phase transitions exist beyond which further compression causes catastrophic performance degradation. Early findings suggest that attention mechanisms have inherent selectivity enabling substantial compression for many tasks, but

information-theoretic bounds apply to compression limits for tasks requiring fine-grained context access.

6.6 Context Compression for Retrieval-Augmented Generation

Retrieval-augmented generation systems face unique compression challenges stemming from the need to process multiple retrieved documents with varying relevance, handle position bias effects in multi-document contexts, maintain factual grounding and citation capability, and balance information diversity with redundancy elimination.

6.6.1 Traditional Contextual Compression

Contextual compression in RAG iterates over retrieved documents, processing each through a language model to extract query-relevant information and returning only pertinent content. This approach removes irrelevant content and reduces noise, focusing on query-specific information and improving reasoning quality through noise reduction. The query-aware filtering proves more effective than query-agnostic compression, as relevance judgments depend fundamentally on information needs expressed in queries.

6.6.2 Extreme Compression for RAG

[84] introduced xRAG (Extreme Context Compression for RAG), which achieves one-token-per-document compression by reinterpreting document embeddings as modality features and fusing them with LLM representation space. This modality fusion framework treats dense retrieval embeddings as a retrieval modality analogous to vision or audio modalities in multi-modal models, integrating them directly into LLM representations without textual intermediaries.

The approach maps retrieval embeddings into the LLM’s representation space through learned projection layers, preserving semantic information while enabling the LLM to process compressed document representations. Each document compresses to a single token representation, enabling processing of dozens or hundreds of documents within limited context windows. xRAG achieves an average 10% improvement across six knowledge-intensive tasks, significantly surpassing previous compression methods while dramatically reducing context requirements.

The extreme compression ratio raises questions about information preservation, as a single token must encode the content of entire documents that may contain thousands of tokens. The success of this approach suggests that for retrieval-augmented scenarios where documents serve primarily to provide factual information to answer specific queries, the relevant information may be remarkably compressible. The retrieval embedding already captures semantic content optimized for relevance matching, and the projection into LLM space enables the model to leverage this semantic encoding directly.

6.6.3 Adaptive Compression for RAG

Adaptive Context Compression for RAG (ACC-RAG) addresses the limitation of fixed compression rates by dynamically adjusting compression based on query complexity. Simple queries may require only basic facts from retrieved documents, tolerating aggressive compression, while complex multi-hop reasoning queries require preserving reasoning chains and connections across documents, necessitating conservative compression.

ACC-RAG analyzes query complexity through features such as query length, entity count, syntactic complexity, and semantic ambiguity, then assigns compression ratios adaptively. Document-specific compression enables different rates for different retrieved documents based on relevance scores and information density, recognizing that top-ranked documents merit more careful preservation than marginally relevant documents. The adaptive approach achieves better accuracy on complex queries through conservative compression while maintaining faster inference on simple queries through aggressive compression, optimizing the performance-efficiency trade-off across varied query distributions.

6.6.4 RAG-Specific Compression Considerations

Multi-document handling in RAG requires careful consideration of document interactions and dependencies, as answers may require synthesizing information across multiple documents. Naive compression of documents independently may break cross-document references or eliminate complementary information distributed across documents. Document reordering strategies mitigate the “lost in the middle” phenomenon by positioning high-relevance documents at context extremes where transformer models demonstrate better attention and information access.

Factual grounding and citation preservation present challenges for aggressive compression, as models must trace information to source documents for fact-checking and attribution. Token-level compression may fragment citations or separate factual claims from their sources, reducing groundedness and increasing hallucination risk. Hierarchical compression approaches that maintain document-level structure while compressing within documents can preserve source attribution while achieving significant compression.

6.7 Semantic Preservation in Compression

Semantic preservation represents a central challenge in context compression, as the goal extends beyond mere size reduction to maintaining meaningful information content. Lossy compression accepts some information loss in exchange for high compression ratios, proving appropriate for lengthy contexts with substantial redundancy, while lossless compression preserves all information but achieves minimal compression ratios and remains rarely achievable with LLMs due to linguistic redundancy and context-dependent semantics.

6.7.1 Measurement and Evaluation

Embedding similarity provides a direct measure of semantic preservation through comparing embeddings of original and compressed text, with high cosine similarity indicating successful preservation of semantic content. BERTscore and similar metrics quantify semantic similarity at token and phrase levels, capturing fine-grained preservation quality. Downstream task performance offers ultimate validation, measuring whether compressed context supports task completion as effectively as full context across question answering, summarization, classification, and information extraction tasks.

Information retention metrics including ROUGE scores for summarization, F1 scores for factual content, and faithfulness measures quantify specific aspects of preservation quality. These metrics capture whether compression maintains key facts, preserves relationships between concepts, and supports required reasoning patterns. Multi-dimensional evaluation across diverse metrics provides comprehensive assessment of compression quality, as different applications prioritize different preservation aspects.

6.7.2 Advanced Preservation Methods

ChunkKV employs semantic chunks as compression units rather than individual tokens, grouping tokens into semantic units and retaining the most informative chunks while discarding less important tokens. This chunk-based granularity maintains semantic relationships better than token-level pruning, as meaningful units remain intact rather than fragmenting across pruning decisions. The approach demonstrates that compression unit selection significantly impacts preservation quality, with semantically-motivated units outperforming arbitrary tokenization boundaries.

Hybrid Context Compression (HyCo2) integrates global and local perspectives, balancing essential semantics captured in document-level summaries with critical details requiring fine-grained preservation. The hybrid strategy adapts compression to content type, applying different methods to different content segments based on information characteristics. This flexibility enables tailoring compression to content properties, applying aggressive summarization to background information while preserving technical details or factual claims at higher fidelity.

AdmTree employs adaptive semantic trees as a hierarchical compression framework, dynamically segmenting content based on information density with variable-length compression units. The semantic tree construction creates hierarchical representations capturing relationships between concepts at multiple granularities. This structured compression preserves both individual facts and their relationships, supporting reasoning that requires understanding connections between concepts.

6.7.3 Context-Aware Preservation Strategies

Hierarchical preservation categorizes information into primary (core facts and concepts compressed minimally), secondary (supporting details compressed moderately), and tertiary (background infor-

mation compressed aggressively) tiers. This tiered approach enables balanced compression that maintains essential information while aggressively compressing less critical content. The flexible trade-offs allow tuning preservation-compression ratios to application requirements, with fact-intensive applications prioritizing primary information preservation and summarization applications tolerating more aggressive compression.

Query-aware compression adjusts preservation based on query information needs, identifying query-relevant content for careful preservation while aggressively compressing query-irrelevant content. This task-specific optimization recognizes that information importance depends fundamentally on intended use, with the same document requiring different compression strategies for different queries. Gradient-based importance estimation quantifies content contribution to task loss, providing direct measurement of preservation necessity for downstream task performance.

6.8 Information-Theoretic Perspectives

Information theory provides fundamental limits on context compression through establishing connections between compression, entropy, and prediction. The Kolmogorov complexity framework reveals that optimal prediction of data sequences is intimately tied to most efficient compression, with the Kolmogorov structure function measuring optimal compressor performance under model-size constraints. This theoretical foundation illuminates compression limits and enables analysis of compression method optimality.

Compression Bounds and Effective Context. Finite-capacity models must compress information, with compression necessarily introducing errors on incompressible data. Information-theoretic constraints bound attainable accuracy on decidable tasks, with finite description length enforcing compression error and long-tail factual knowledge requiring prohibitive sample complexity. These fundamental limits constrain what any compression method can achieve, establishing performance ceilings independent of algorithmic sophistication.

Even with extremely long nominal context windows such as 128,000 tokens, geometric and computational effects compress contexts far below nominal size. Positional under-training causes effective compression because models receive limited training on extreme positions, leading to degraded performance at positions rarely encountered during training. Encoding attenuation effects reduce information density as sinusoidal or rotary positional encodings decay with distance, causing attention scores to decay for distant positions. Softmax crowding creates numerical bottlenecks through information crowding in attention computation, with score margin compression at long distances concentrating entropy in recent positions.

These effects jointly cause effective context to scale sub-linearly with nominal length, with gradient decay at rare positions, vanishing overlap in positional encodings, and logarithmic score-margin growth limiting utilization. The practical implication is that even without explicit compression, long contexts experience substantial implicit compression through these architectural and optimization

effects, suggesting that explicit compression methods that align with these implicit compression patterns may achieve better quality-ratio trade-offs.

Entropy Law and Compression-Performance Connections. Recent work has identified an “entropy law” connecting LLM performance with data compression ratio and first-epoch training loss, reflecting the information redundancy of datasets. Language models output probability distributions over next tokens conditioned on context, information that enables near-optimal compression via arithmetic coding. Shannon’s theorem guarantees optimality if probabilities match true distributions, establishing an elegant equivalence between language modeling and data compression where better language models yield better compressors.

This equivalence provides theoretical grounding for compression approaches, as language modeling objectives implicitly train models to identify and eliminate redundancy. The entropy of context bounds achievable compression ratio, with repetitive or predictable content enabling higher compression than information-dense technical content. Compression methods that leverage language model probabilities to guide compression decisions thus align with information-theoretic optimality, explaining the effectiveness of perplexity-based approaches like LLMLingua.

The rate-distortion framework formalizes trade-offs between compression ratio and distortion (information loss), establishing theoretical limits on useful compression ratios. With a frozen language model, hard limits exist for compression via token dropping or reordering, as any compression must discard information that the fixed model cannot reconstruct. Learned compression methods that adapt model representations to compressed inputs potentially achieve better rate-distortion trade-offs by enabling the model to leverage compression-specific features.

6.9 Summary and Comparative Analysis

Context compression techniques span a spectrum from aggressive methods achieving $20\times$ to $32\times$ compression through LLMLingua, Gisting, and Recurrent Context Compression, to conservative approaches achieving $2\times$ to $4\times$ compression through Selective Context and semantic compression methods, with intermediate methods such as ICAE and AutoCompressors balancing $4\times$ to $8\times$ compression with performance preservation. Training costs vary similarly, from zero additional cost for training-free methods like LLMLingua and Selective Context, through low costs for Gisting’s attention mask modifications and ICAE’s 1% parameter overhead with LoRA, to moderate costs for AutoCompressors’ fine-tuning on long sequences, and substantial costs for Compressive Transformers requiring architecture-specific training from scratch.

Application domains reveal method specialization, with RAG systems benefiting particularly from LongLLMLingua and xRAG’s document-handling capabilities, in-context learning scenarios favoring Gisting and AutoCompressors for demonstration compression, long document processing leveraging ICAE and Compressive Transformers’ architectural approaches, and general-purpose applications employing semantic compression and Selective Context for broad applicability. The diversity of

approaches reflects fundamental trade-offs between compression ratio (higher ratios reduce costs but risk information loss), computational cost (training-free methods enable immediate deployment while learned methods may achieve better quality), performance preservation (conservative compression maintains quality while aggressive compression maximizes efficiency), and interpretability (hard prompt methods maintain human readability while soft prompt methods achieve better compression through learned representations).

Recent comprehensive surveys [74] highlight emerging research directions including optimization of compression encoders through better architectural designs and more efficient information extraction, combining hard and soft methods to leverage complementary strengths, leveraging multimodality for cross-modal compression in image-text and audio-text scenarios, and advancing synthetic languages through more expressive learned tokens enabling universal compression languages with cross-model compatibility. These directions suggest that future compression methods will increasingly employ hybrid approaches that combine multiple compression strategies, adapt compression to task and content characteristics, and jointly optimize compression with downstream task performance.

The field’s maturation is evident in the transition from isolated compression techniques to integrated systems that combine multiple methods. Production systems increasingly employ compression pipelines that chain complementary approaches, such as initial hard compression to remove obvious redundancy followed by soft compression to encode remaining content, or query-aware compression to identify relevant content followed by semantic compression of retained information. This compositional approach enables achieving compression ratios and quality levels unattainable by individual methods, suggesting that future advances may derive as much from effective composition of existing techniques as from novel compression algorithms.

7 Memory-Augmented and Retrieval-Augmented Generation Architectures

Memory-augmented and retrieval-augmented generation architectures represent a fundamental paradigm shift in extending the effective context window of large language models beyond their architectural limitations. Rather than constraining information processing to the transformer’s fixed context window, these approaches leverage external memory systems, databases, and retrieval mechanisms to dynamically incorporate relevant information during generation. This section traces the evolution from foundational memory architectures through the development of retrieval-augmented generation, examining advanced variants that incorporate self-reflection and correction, and culminating in agentic systems that manage their own memory hierarchies.

7.1 Foundational Memory Architectures

The conceptual foundation for separating computation from memory storage in neural networks emerged from two parallel research directions in 2014: Neural Turing Machines and Memory Net-

works. Both architectures recognized that recurrent neural networks and LSTMs, which store information implicitly in hidden states, fundamentally limit the model’s ability to perform structured reasoning and maintain long-term dependencies.

7.1.1 Neural Turing Machines

[85] introduced Neural Turing Machines by coupling neural networks with differentiable external memory through attentional read and write operations. Drawing inspiration from classical Turing machines and Von Neumann architectures, NTMs combine the fuzzy pattern-matching capabilities of neural networks with the algorithmic power of programmable computers. The architecture consists of three core components: a continuous memory matrix serving as the analog of a Turing machine’s infinite tape, soft attention mechanisms providing differentiable read and write operations, and a controller recurrent neural network that manages memory operations.

The memory bank in an NTM is structured as a two-dimensional matrix of size $N \times M$, where N represents the number of memory locations and M denotes the dimension of each location. Content-based addressing enables the model to retrieve information through similarity search, computing attention weights via $\text{softmax}(\beta \cdot \cos_similarity(\text{key}, \text{memory}))$, where the sharpening parameter β controls the concentration of attention. Location-based addressing allows sequential memory traversal through circular shifts, improving generalization to sequences longer than those seen during training. Write operations decompose into two separate phases: an erase operation that performs element-wise multiplication of memory by $(1 - \text{erase_vector})$, followed by an add operation that performs element-wise addition, providing fine-grained control over memory updates.

Neural Turing Machines demonstrated the ability to infer simple algorithms including copying, sorting, and associative recall, achieving perfect recall on sequences up to 120 tokens during training and generalizing to sequences exceeding 1000 tokens at test time. The architecture’s interpretability, with memory contents and access patterns readily visualizable, established attention-based memory access as a foundational mechanism underlying modern retrieval-augmented systems. NTMs influenced subsequent developments in attention mechanisms, predating the Transformer architecture by three years and providing theoretical grounding for the connection between neural networks and classical computing paradigms.

7.1.2 Memory Networks

[86] introduced Memory Networks with explicit storage and reasoning components organized into four key modules: an input feature map for converting inputs to internal representations, a generalization module for retrieving relevant memories via attention, an output feature map for converting memory content to output format, and a response module for generating final outputs. The architecture treats memory as discrete slots storing variable-length information, with content-addressable retrieval enabling similarity-based access patterns. While effective on synthetic question answering tasks and capable of scaling to 14 million sentences, the original Memory Networks required

supervision on memory access during training, limiting their practical applicability.

[87] addressed this limitation with End-to-End Memory Networks, making the architecture fully differentiable through soft attention mechanisms. The key innovation involves computing differentiable attention weights via softmax, enabling backpropagation through the entire memory access process without explicit supervision. The architecture implements multi-hop reasoning through attention chains, where each hop refines the query based on previous memory access, allowing multiple computational steps per output symbol. This recurrent attention mechanism over large external memory extends the attention mechanisms of RNNsearch to multi-hop reasoning scenarios.

End-to-End Memory Networks achieved competitive performance with supervised Memory Networks on synthetic question answering benchmarks while requiring significantly reduced supervision. On language modeling tasks, the architecture achieved comparable performance to RNNs and LSTMs on Penn TreeBank and Text8 datasets, with perplexities of 81.3 versus 78.4 for LSTM baselines. The fully differentiable design enabled end-to-end training from input-output examples alone, eliminating the need for attention labels. These architectures directly influenced the development of attention mechanisms in Transformers, with the connection being evident in how self-attention generalizes the memory access mechanism: transformer layers can be viewed as memory network hops, while the query-key-value paradigm represents a generalization of content-addressable memory access.

7.2 Dense Passage Retrieval and Embedding Foundations

The transition from abstract memory mechanisms to practical retrieval systems required efficient methods for semantic search over large document collections. Traditional sparse retrieval methods like BM25 suffer from vocabulary mismatch problems, failing to capture semantic similarity beyond exact keyword matching. Dense retrieval methods address this limitation through learned embeddings that position semantically similar content nearby in a continuous vector space.

7.2.1 Dense Passage Retrieval

[88] established the dense retrieval foundation using dual BERT-based encoders for queries and passages, training the system through contrastive learning with positive and negative passage pairs. The dual-encoder architecture processes queries and documents independently, enabling pre-computation and efficient storage of passage embeddings in vector indexes like FAISS. The training process employs in-batch negatives, using other queries' positive passages as negatives within each batch, supplemented by hard negative mining from BM25 retrieval results to improve discrimination.

Dense Passage Retrieval achieved 9 to 19 percent absolute improvement in top-20 retrieval accuracy over Lucene-BM25 on open-domain question answering benchmarks. The architecture reduces semantic search time from 65 hours for BERT-based cross-encoder approaches to approximately 5 seconds for 10,000 sentences through pre-computed embeddings and approximate nearest neighbor

search. DPR established state-of-the-art results on Natural Questions, TriviaQA, and WebQuestions, becoming the backbone retrieval mechanism for most contemporary RAG systems. However, the approach faces challenges including semantic drift where subtle mismatches between query and passage encodings accumulate, vocabulary mismatch for rare concepts despite improved semantic understanding, and strong dependency on training data quality and hard negative mining.

7.2.2 Sentence-BERT and Efficient Embeddings

[89] addressed the computational impracticality of using standard BERT for semantic similarity at scale. Standard BERT requires feeding both sentences to the model in a cross-encoder fashion, resulting in $O(n^2)$ computational complexity for n sentences and approximately one second latency per sentence pair. Sentence-BERT introduces a siamese network architecture that processes sentences independently through shared BERT encoders, applying mean pooling over token embeddings to produce fixed-size sentence representations. The training employs triplet loss $L = \max(0, m + \text{sim}(\text{anchor}, \text{neg}) - \text{sim}(\text{anchor}, \text{pos}))$ or contrastive loss to learn meaningful embedding spaces.

SBERT reduces the 65-hour inference time for 10,000-sentence similarity computation to approximately 5 seconds, achieving roughly 1000-fold speedup while maintaining BERT-level accuracy with only 1 to 2 percent accuracy loss. The framework supports embeddings ranging from 384 dimensions for smaller models to 768 dimensions for base models, with the embedding space exhibiting desirable properties including high intra-class similarity, low inter-class similarity, and compositionality where averaging embeddings produces meaningful representations. SBERT has become the foundation for RAG embedding components, with over 10,000 pre-trained models available on Hugging Face spanning multiple languages and domains.

7.2.3 ColBERT and Late Interaction

[90] introduced late interaction architectures that preserve token-level granularity while maintaining computational efficiency. Unlike dense retrievers that create single embeddings per passage, losing rich token-level details in aggregation, ColBERT maintains per-token representations and computes similarity through late interaction after independent encoding. The similarity computation follows $\text{Score}(Q, P) = \sum_q \max_p (Q[q] \cdot P[p])$, where each query token finds its best match across all passage tokens through maximum cosine similarity.

ColBERT achieves two orders of magnitude speedup over full BERT-based retrieval while requiring four orders of magnitude fewer FLOPs per query compared to cross-encoders. The architecture maintains competitive effectiveness with existing BERT models and outperforms all non-BERT baselines, with 2 to 5 percent accuracy improvements over DPR on retrieval tasks. The token-level interaction provides interpretability, allowing examination of which query tokens matched which document regions. ColBERTv2 [91] further improved efficiency while maintaining effectiveness, demonstrating that late interaction enables end-to-end retrieval from large document collections

through vector-similarity indexes with minimal accuracy trade-offs.

7.3 Retrieval-Augmented Generation

The convergence of efficient dense retrieval with sequence-to-sequence generation established retrieval-augmented generation as a practical paradigm for knowledge-intensive natural language processing tasks.

7.3.1 The RAG Framework

[92] introduced the foundational RAG architecture, combining a pre-trained sequence-to-sequence transformer (parametric memory) with a dense vector index of Wikipedia (non-parametric memory). The architecture employs Dense Passage Retrieval to retrieve relevant passages for each input and conditions generation on these passages through two distinct formulations. RAG-Sequence uses the same retrieved passages across the entire generation process, maintaining consistent context throughout output production. RAG-Token allows different passages to be retrieved for each token, enabling more dynamic retrieval but increasing computational cost.

The training procedure employs end-to-end fine-tuning on knowledge-intensive tasks, using maximum likelihood training with marginalization over retrieved passages. The generator, typically BART or T5-based, computes $P(\text{output}|\text{input}, \text{retrieved_passages})$ by marginalizing over the top- k retrieved documents. During training, gradients can flow through the retriever, though in practice the retriever is often frozen for computational efficiency. The retrieval corpus consists of Wikipedia with approximately 100 million passages, with passages retrieved via DPR using BERT-based dual encoders.

RAG achieved state-of-the-art results on three open-domain question answering tasks including Natural Questions, TriviaQA, and WebQuestions. The system generates more specific, diverse, and factual language than parametric-only baselines, reducing hallucination through grounding in retrieved documents. RAG outperforms strong retrieval-only and generation-only baselines while achieving better generalization to unseen domains. The framework’s influence extends throughout the ecosystem, inspiring numerous variants including Self-RAG, CRAG, and GraphRAG, and establishing RAG as the foundational paradigm for knowledge-intensive NLP with over 10,000 citations as of 2024.

7.3.2 RETRO: Retrieval-Enhanced Pre-training

[93] introduced RETRO, demonstrating that retrieval can be integrated during pre-training rather than solely during fine-tuning or inference. RETRO retrieves from a database of 2 trillion tokens during both pre-training and inference, representing an order of magnitude more data than typical training corpora. The architecture employs a frozen BERT retriever for computational efficiency, a differentiable encoder for processing retrieved information, and chunked cross-attention that inte-

grates retrieved neighbors at the passage level while interleaving with regular self-attention at the document level.

The pre-training process divides input sequences into 64-token chunks, retrieving the top-2 nearest neighbor chunks from the massive database based on preceding token embeddings. The chunked cross-attention mechanism allows the model to attend to both local context within the current document and retrieved context from the database, effectively extending the model’s knowledge beyond what can be stored in parameters. RETRO employs FAISS indexes for fast nearest neighbor lookup, with retrieval operations adding 1 to 10 milliseconds per token during inference.

A 7.5 billion parameter RETRO model outperformed the 175 billion parameter Jurassic-1 on 10 out of 16 datasets and the 280 billion parameter Gopher on 9 out of 16 datasets, demonstrating that retrieval can substitute for approximately a 10-fold increase in parametric model size. The performance gain from retrieval proves comparable to increasing model parameters by 10-fold while requiring only 25-fold fewer parameters than GPT-3 to achieve comparable performance. RETRO demonstrates that retrieval scales with database size, with larger retrieval corpora yielding better performance, and that pre-trained transformers can be rapidly adapted with retrieval through the “RETROfitting” procedure. The architecture’s limitation includes the frozen retriever that cannot learn task-specific retrieval through gradients, fixed chunk sizes that may not align with semantic boundaries, and the computational overhead and latency introduced by retrieval operations.

7.3.3 REALM: Unsupervised Retriever Training

[94] introduced REALM, demonstrating that retrievers can be trained unsupervised using only the masked language modeling signal. Unlike RETRO’s frozen retriever, REALM trains the retriever end-to-end during pre-training through backpropagation. The architecture consists of a query encoder that processes input with masked positions, a document encoder that encodes passages from the corpus (both BERT-based), and a language representation model that predicts masked tokens conditioned on retrieved documents.

The key innovation involves using masked language modeling loss to provide supervision for the retriever. For each masked token position, the system encodes the surrounding context as a query, retrieves top- k documents from the corpus, and identifies the best document as the one containing the answer token. The MLM loss drives the retriever to select documents containing answers, enabling unsupervised retriever learning without explicit relevance labels. Training on Wikipedia with 13.3 million documents, REALM achieved 4 to 16 percent absolute improvement over previous state-of-the-art on Natural Questions, demonstrating superior performance on WebQuestions and TriviaQA as well.

REALM’s 300 million parameter model achieved competitive performance with the 11 billion parameter T5, demonstrating parameter efficiency through modular knowledge separation. The architecture provides interpretability through retrieved documents that explain predictions and enables

updatable knowledge without retraining by modifying the document corpus. The approach influenced subsequent work on retrieval-augmented pre-training and established that unsupervised signals can effectively train retrieval systems.

7.3.4 Fusion-in-Decoder

[95] introduced Fusion-in-Decoder (FiD), addressing the challenge of efficiently aggregating information from many retrieved passages. Traditional approaches either process passages independently for scoring or fuse them early with limited scalability. FiD encodes each retrieved passage independently with the query prepended, allowing embarrassingly parallel computation, then fuses all encoded passages in the decoder where cross-attention spans all passage representations simultaneously.

The encoding phase processes [Query][passage text] for each retrieved passage independently, producing contextualized representations. The decoder attends to the concatenation of all encoded passages, learning to synthesize information across multiple sources during generation. This architectural choice enables efficient scaling to 100 or more passages while maintaining high accuracy. FiD achieved state-of-the-art results on Natural Questions and strong performance on TriviaQA and WebQuestions, with accuracy improving as the number of passages increased from 1 to 100. The architecture’s simplicity, clean design, and flexibility with respect to retriever choice contributed to its widespread adoption.

7.4 kNN-Augmented and Memorizing Architectures

Complementary to retrieval from static document collections, kNN-augmented architectures enable models to leverage recent context through approximate nearest neighbor lookup over dynamically constructed memories.

7.4.1 Memorizing Transformers

[96] introduced memorizing transformers with approximate kNN lookup into non-differentiable external memory containing recent key-value pairs. Unlike retrieval-augmented approaches that search pre-indexed document collections, memorizing transformers store representations from the model’s own recent processing, enabling immediate knowledge acquisition at inference time without retraining. The memory structure stores key-value pairs from past inputs, with current hidden states serving as queries for approximate kNN retrieval via FAISS-based indexing. Retrieved values augment standard attention output through concatenation or averaging with linear projection to match dimensionality.

Memory size scales from 8,000 to 262,000 tokens, with performance steadily improving as memory increases. The architecture demonstrated improvements across diverse domains including the C4 dataset for generic web text, arXiv papers for mathematical content, PG-19 for long-form books,

GitHub repositories for code, and Isabelle proof assistant for formal theorems. In code generation and theorem proving, the system proved capable of utilizing newly defined functions and theorems during test time, demonstrating genuine test-time adaptation capabilities. The kNN lookup operates in $O(\log n)$ time with HNSW indexing, introducing minimal computational overhead compared to standard attention.

Memorizing transformers established that approximate kNN lookup suffices for performance improvements, that simple averaging of retrieved values proves effective without complex integration mechanisms, and that the approach can be added post-hoc to any pre-trained transformer without additional training. The approach inspired Extended Mind Transformers and related work on memory-augmented inference, demonstrating that test-time learning without weight updates provides practical benefits for domain-specific applications.

7.5 Advanced RAG Variants

The recognition that naive RAG systems retrieve indiscriminately and lack quality control mechanisms motivated the development of advanced variants incorporating adaptive retrieval, self-critique, and correction capabilities.

7.5.1 Self-RAG: Adaptive Retrieval with Self-Reflection

[97] introduced Self-RAG, which uses special reflection tokens to make adaptive retrieval decisions and self-critique both retrieved passages and generated outputs. Unlike traditional RAG that indiscriminately retrieves a fixed number of passages regardless of necessity, Self-RAG decides when and what to retrieve through learned token generation. The reflection token system includes retrieval tokens that decide whether retrieval is needed (**[Ret]** versus **[NoRet]**), relevance tokens that evaluate passage relevance (**[Rel]** versus **[Irrel]**), and critique tokens that assess response quality (**[Utility]** versus **[NonUtil]**).

The generation process begins with predicting whether retrieval is necessary. If the retrieval decision indicates retrieval, the system retrieves passages and evaluates the relevance of each through relevance token generation, keeping only passages marked as relevant. After generating a response, the system evaluates output quality through critique token generation. This adaptive approach enables the model to retrieve multiple times, skip retrieval entirely when parametric knowledge suffices, and filter irrelevant passages before generation.

Self-RAG models with 7 and 13 billion parameters outperformed ChatGPT on open-domain question answering tasks, achieved superior performance on reasoning and fact verification tasks, and demonstrated significant gains in factuality and citation accuracy for long-form generation. The architecture achieved state-of-the-art performance among both language models and retrieval-augmented models. The controllable generation capability allows adjustment of behavior through token manipulation, providing interpretability regarding retrieval decisions. The single-model design integrates

retrieval decisions within the main language model rather than requiring separate components.

7.5.2 Corrective RAG

[98] proposed CRAG, which addresses the reality that retrieval often fails by introducing a lightweight retrieval evaluator that assesses document quality before generation. The evaluator computes confidence scores for retrieved documents and triggers three distinct action types based on confidence levels. For correct retrievals with high confidence, CRAG applies a decompose-then-recompose technique that removes irrelevant information while retaining key knowledge. For incorrect retrievals with low confidence, the system discards faulty retrievals and triggers web search to obtain reliable information. For ambiguous cases with unclear confidence, CRAG combines refined retrieval with web search to ensure diverse and robust knowledge integration.

The confidence threshold strategy typically uses approximately 70 percent document relevance as the decision boundary. When fewer than 70 percent of retrieved documents prove relevant, the system declares retrieval failure and activates corrective mechanisms. The plug-and-play design allows CRAG to be inserted into any RAG-based framework, with Self-CRAG demonstrating integration by replacing retrieved items with processed knowledge, external knowledge, or combined knowledge based on confidence levels. Testing on four datasets spanning short-form and long-form generation demonstrated that CRAG significantly outperforms traditional RAG and Self-RAG across various benchmarks, reducing hallucinations through self-aware recognition of retrieval failures.

7.5.3 GraphRAG: Structured Retrieval

Microsoft’s GraphRAG framework introduces a structured, hierarchical approach to retrieval-augmented generation through knowledge graph extraction. Rather than treating documents as flat text chunks, GraphRAG uses language models to extract entities and relationships from raw text, constructs knowledge graphs capturing document structure, detects communities through hierarchical clustering, and generates summaries at multiple levels of the hierarchy. The graph machine learning component augments prompts at query time through graph traversal and relationship reasoning.

GraphRAG supports two distinct query modes tailored to different information needs. Local search, designed for specific entity queries, finds the entity in the graph, retrieves properties, fans out to neighboring nodes, and summarizes information from the local neighborhood. Global search, designed for broad holistic questions, traverses the community hierarchy, retrieves community summaries at appropriate levels, synthesizes across communities, and generates comprehensive answers spanning multiple information clusters.

The structured approach addresses baseline RAG’s struggle with aggregation queries such as “What are the top 5 themes?” by connecting information via relationships rather than semantic similarity alone. GraphRAG captures relationships beyond semantic similarity, uncovering hidden but crucial correlations. Empirical evaluations demonstrate up to 35 percent precision improvement over vector-

only retrieval, better context coverage, superior multi-hop reasoning, and enhanced interpretability through explicit relationship visibility. The framework has been deployed in Microsoft’s Discovery platform in Azure for scientific research applications, with LazyGraphRAG providing efficiency-optimized variants.

7.6 Dynamic and Active Retrieval Strategies

Recognition that retrieval timing and query formulation critically impact performance motivated research into dynamic retrieval strategies that adapt to generation needs.

7.6.1 FLARE: Forward-Looking Active Retrieval

[99] introduced FLARE for long-form generation, implementing predictive retrieval triggers based on generation confidence. Rather than retrieving once at the beginning or at fixed intervals, FLARE generates candidate sentences and evaluates confidence through token probability analysis. When confidence falls below a threshold, typically indicated by minimum token probability confidence = $\min(p(\text{token}_1), p(\text{token}_2), \dots, p(\text{token}_n))$, the system retrieves using the partial sentence as a query and regenerates with retrieved context.

The forward-looking mechanism predicts the next sentence, checks confidence through probability computation, and decides whether to retrieve based on the threshold comparison. If retrieval is triggered, the predicted sentence serves as the query for retrieval, enabling the system to retrieve information needed for content it is attempting to generate. After retrieval, the system regenerates the sentence with access to retrieved context, potentially producing more accurate content. This adaptive approach proves more efficient than fixed-interval retrieval while improving quality compared to no retrieval.

FLARE demonstrated superior performance on long-form question answering benchmarks including ALCE, ASQA, and QAMPARI. The system retrieves relevant information when needed, produces fewer hallucinations than parametric generation, and performs fewer retrievals than fixed-interval approaches. The approach’s limitations include latency from multiple retrieval operations, task-dependent confidence threshold selection, computational cost of regenerating text, and awkwardness in formulating retrieval queries from incomplete sentences.

7.6.2 HyDE: Hypothetical Document Embeddings

[100] addressed the semantic gap between short queries and long documents by generating hypothetical relevant documents for embedding. Rather than directly embedding a query like “What is DNA?”, HyDE uses a language model to generate a hypothetical answer, such as a multi-sentence explanation of DNA structure and function, embeds this generated document, and retrieves real documents similar to the hypothesis. The hypothetical document proves richer than the original query, better captures semantic intent, shares vocabulary with relevant documents, and positions the query naturally in document embedding space.

The zero-shot approach requires no fine-tuning, works with any query, and uses readily available instruction-following language models. The implementation generates hypotheses through prompts like “Write a document that would answer the following query” with temperature 0.7, embeds the hypothesis using standard dense retrievers, and retrieves documents through nearest neighbor search. Evaluation on TREC-DL, BEIR benchmarks, and Natural Questions demonstrated competitive zero-shot performance with fine-tuned dense retrievers, with combination of HyDE and fine-tuning outperforming DPR alone.

The approach’s simplicity, modularity with any embedder, and effectiveness without training data contributed to its adoption. Limitations include dependency on hypothesis generator quality, latency from hypothesis generation, language-specific effectiveness tied to instruction-following model availability, and potential for generator hallucinations to mislead retrieval.

7.7 Agentic Memory: MemGPT

The challenge of processing documents exceeding context window limits motivated operating system-inspired approaches to virtual context management.

7.7.1 Operating System Analogy

[101] introduced MemGPT, drawing inspiration from operating system memory hierarchies to create virtual context management for language models. Analogous to how operating systems provide the illusion of unlimited memory through data movement between fast RAM and slow disk storage, MemGPT provides the appearance of large memory through intelligent data movement between main context and external context. The main context, corresponding to the language model’s prompt window (typically 8,000 tokens), serves as fast memory with immediate access. External context, potentially unlimited in size, stores the complete document collection, conversation history, and summaries, serving as slow storage with on-demand retrieval.

The memory block structure organizes main context into persona (system context about agent identity and goals), human (summary of user characteristics and relationship), and scratch (temporary working memory for current task) blocks. These core memory blocks, totaling 1,000 to 2,000 tokens, reside permanently in main context for fast access with active agent management. External storage maintains document stores with raw documents, metadata, embeddings for semantic search, and chunk-based organization, alongside conversation logs with message history, timestamps, conversation chunk summaries, and relationship tracking.

7.7.2 Virtual Context Mechanism

The virtual context mechanism operates through the language model context window hosting persona, human, and scratch blocks, with active context for current interaction consuming the remaining space. External storage holds the document corpus (gigabytes or more), conversation history

(exceeding 100,000 tokens), and indexes enabling semantic search. The retrieval process activates when the agent detects information needs, performs semantic search over external context, retrieves relevant chunks, swaps content into main context through eviction of least important information, updates scratch and working memory, and continues generation with refreshed context.

Eviction policies range from simple least-recently-used approaches that remove oldest or least-used items, through importance-based methods that assign scores prioritizing relevant information retention, to adaptive strategies where the agent learns optimal eviction through experience. The interrupt-driven control flow tracks token usage, predicts context limit approach, triggers memory management interrupts transparently, saves current state, performs memory swaps, and resumes generation, mirroring operating system context switches.

7.7.3 Applications and Performance

Document analysis applications enable processing of documents 10 to 20 times larger than the context window by loading document chapters into external storage, retrieving relevant sections on-demand in response to queries, and maintaining consistent persona and analysis context throughout. Quality remains comparable to base models on shorter documents, with the system maintaining information across large document jumps, though retrieval failures can cause missed information. Multi-session conversational agents leverage MemGPT to maintain relationships across months-long interactions by storing full conversation history externally, maintaining persona and relationship memory in core blocks, retrieving relevant past conversations as needed, and evolving naturally across sessions. The system demonstrates stable performance with proper memory management and scales to arbitrary conversation history sizes, though some gradual personality drift occurs in very long sessions.

MemGPT has been deployed commercially under the name Letta, demonstrating practical viability. The approach suggests moving from purely technical context extension toward cognitive architectures that manage memory systematically, influenced subsequent work on hierarchical memory systems and agentic AI, and provides a template for building language model systems with persistent memory across interactions. Future directions include learning optimal retrieval policies, developing better compression and summarization of external context, personalizing memory strategy to individual users, reducing retrieval operation latency, and conducting formal analysis of memory management strategies.

7.8 Retrieval Infrastructure and Optimization

Effective retrieval-augmented generation depends critically on infrastructure components including embedding models, chunking strategies, and evaluation frameworks.

7.8.1 Chunking Strategy Impact

Document chunking strategy arguably represents the most important factor for retrieval-augmented generation performance. Fixed-size chunking segments text into equal-sized pieces by character, token, or word count, typically using 250 tokens (approximately 1,000 characters) as a starting point with overlap to maintain continuity across boundaries. This approach provides simplicity, predictability, and wide applicability but breaks semantic coherence at arbitrary boundaries.

Recursive character splitting, employed by approximately 80 percent of RAG applications, balances simplicity with structure awareness by chunking first by inherent separators (paragraphs, sections), then splitting further if chunks exceed size limits. This approach respects document structure while enforcing size constraints, making it the default choice for most implementations. Semantic chunking groups sentences by semantic similarity of embeddings, analyzing relatedness of consecutive sentences and creating chunks where topics shift, preserving semantic coherence at the cost of increased computation. Page-level chunking, tested by NVIDIA across 7 strategies and 5 datasets in 2024, achieved 0.648 accuracy with the lowest standard deviation (0.107), proving most effective for structured documents with clear page boundaries.

Late chunking feeds the entire document into a long-context embedding model to create token-level embeddings with full context understanding, then splits into chunks after embedding, preserving global context in chunk representations. Hierarchical chunking organizes content at multiple granularities, with broad sections and themes at the top layer and fine-grained details at lower layers, enabling multi-resolution retrieval. Language model-based agentic chunking, where language models determine splitting based on semantic meaning, considers paragraph types, section headings, and content structure, remaining experimental but promising for complex documents.

Query type optimization suggests that factoid queries benefit from 256 to 512 token chunks, while analytical queries require 1024 or more tokens. Best practices include testing multiple strategies for specific use cases, considering embedding model context window constraints (typically 512 to 8,192 tokens), and balancing chunk size against retrieval precision and recall trade-offs.

7.8.2 Embedding Model Selection

Modern embedding models span a diverse ecosystem with different strengths. Models like `multi-qa-mpnet-base-d` optimize for question-answering, `all-MiniLM-L6-v2` provides lightweight general-purpose embeddings, OpenAI's `text-embedding-ada-002` offers commercial state-of-the-art performance, and Cohere Embed v3 supports multilingual and multi-domain applications. Selection criteria include task-specific fine-tuning yielding better performance than general models, domain-specific models outperforming generic alternatives, multilingual models for cross-lingual applications, and dimension trade-offs balancing smaller dimensions (256) against larger dimensions (1024).

Hybrid retrieval strategies combine dense and sparse methods, with the query processed through both dense retrieval (SBERT or ColBERT) producing top-100 documents and sparse retrieval

(BM25) producing top-100 documents, followed by union or intersection operations and reranking to produce final top- k results. This combination captures both semantic understanding and keyword matching. Multi-stage retrieval pipelines employ dense retrieval for fast approximate candidate generation (retrieving 100 candidates), semantic reranking with ColBERT on top-100 to produce top-20, and cross-encoder reranking with BERT on top-20 to produce final top-10, balancing speed and accuracy across stages.

7.8.3 Evaluation Frameworks

Comprehensive evaluation of retrieval-augmented systems requires metrics at multiple levels. Component evaluation assesses retrievers through recall (finding relevant documents), precision (proportion of retrieved documents that are relevant), and ranking quality via mean reciprocal rank and normalized discounted cumulative gain. Generator evaluation examines fluency through grammaticality and coherence, relevance of answers to queries, factual correctness, and faithfulness to retrieved context.

Integration evaluation considers context relevance (do retrieved documents support the query), answer relevance (does the answer address the input query), answer faithfulness (is the answer grounded in retrieved context), and hallucination rate (percentage of answers with ungrounded claims). End-to-end evaluation examines downstream task accuracy, user satisfaction through human evaluation of helpfulness, robustness under adversarial inputs, and efficiency regarding latency and computational cost.

Major benchmarks include Natural Questions with 320,000 questions from Google search queries, TriviaQA with 950,000 question-answer pairs with Wikipedia and web evidence, the CRAG benchmark with 5,000+ questions across finance, medicine, law, science, and technology with eight question types, and RAGBench with 100,000+ examples for explainable RAG evaluation. The RGB robustness evaluation focuses on noise robustness (irrelevant documents in retrieval), negative rejection (rejecting non-answers), information integration (combining multiple sources), and counterfactual resistance (handling contradictions). RAGTruth provides 18,000+ annotated examples with response-level and span-level hallucination labels across question answering, summarization, and data-to-text tasks.

7.9 Pre-training with Retrieval

Integration of retrieval during pre-training rather than solely during fine-tuning represents a fundamental architectural choice with significant implications for model capabilities and knowledge management.

REALM’s unsupervised retriever training demonstrated that masked language modeling loss alone suffices to train effective retrievers without explicit relevance labels, with retrieval improving models’ ability to capture factual knowledge and enabling efficient knowledge updates through corpus

modification rather than retraining. The modular separation of parametric and non-parametric memory proved effective, with 300 million parameter models matching 11 billion parameter purely parametric models through retrieval augmentation.

RETRO’s integration of retrieval from 2 trillion tokens during pre-training achieved GPT-3 level performance with 25-fold fewer parameters, demonstrating that retrieval can substitute for massive parameter increases. The performance gains from retrieval proved comparable to 10-fold increases in model size, establishing retrieval as a parameter-efficient alternative to pure scaling. The frozen retriever design, while computationally efficient, limits task-specific adaptation, suggesting future work on learned retrieval mechanisms.

Fusion-in-Decoder’s late fusion strategy enables efficient processing of 100+ passages through independent encoding and decoder-level fusion, achieving state-of-the-art results on open-domain question answering while maintaining computational tractability. The architecture’s simplicity and clean design contributed to widespread adoption, demonstrating that fusion strategy matters as much as the information retrieved.

7.10 Summary

Memory-augmented and retrieval-augmented generation architectures have evolved from foundational work on Neural Turing Machines and Memory Networks, which established differentiable memory access and attention-based retrieval, through the development of efficient dense retrieval methods like Dense Passage Retrieval and Sentence-BERT, to the integration of retrieval with generation in frameworks like RAG, RETRO, and REALM. Advanced variants including Self-RAG, CRAG, and GraphRAG demonstrate increasing sophistication in when, what, and how to retrieve, incorporating self-reflection, correction, and structured knowledge representation. Dynamic retrieval strategies like FLARE and HyDE address the timing and formulation of retrieval queries, while agentic systems like MemGPT implement operating system-inspired memory hierarchies for virtual context management.

The progression reveals several key insights. First, effective context extension requires not merely larger windows but intelligent retrieval, organization, and integration of external information. Second, the separation of parametric and non-parametric memory enables parameter-efficient models that achieve performance comparable to much larger purely parametric systems. Third, adaptive retrieval that decides when and what to retrieve outperforms naive approaches that retrieve indiscriminately. Fourth, self-reflective and corrective mechanisms significantly improve retrieval quality and generation faithfulness. Fifth, structured representations like knowledge graphs enable more sophisticated reasoning than flat text chunks alone. Finally, systematic memory management inspired by operating systems provides a principled approach to handling contexts exceeding architectural limits.

As language models continue to scale and are deployed in increasingly demanding applications,

memory-augmented and retrieval-augmented architectures will remain essential for grounding generation in factual knowledge, enabling long-term memory across sessions, providing access to information beyond training cutoffs, supporting efficient updates to model knowledge, and enabling interpretable systems where retrieved evidence explains generations. The field continues to advance rapidly, with recent work exploring learned retrieval policies, improved compression of retrieved context, multimodal retrieval across text, images, audio, and video, cross-modal memory architectures, on-device RAG for privacy-sensitive applications, and unified training objectives for retrieval and generation. The convergence of these approaches with other context management strategies including sliding window attention, hierarchical memory, and prompt compression promises to yield systems capable of processing and reasoning over effectively unbounded contexts while maintaining computational tractability.

8 Hierarchical Human-Inspired Memory Architectures

This section constitutes the theoretical and empirical core of this survey, examining how principles from human cognitive science have informed the design of memory systems for large language models. The intellectual trajectory from foundational memory theories in experimental psychology, through computational neuroscience models of consolidation and replay, to modern LLM-based agent architectures reveals a striking convergence: the most promising approaches to context management draw explicitly on the hierarchical, multi-store, and consolidation-based architectures that characterize biological memory. We trace this lineage in detail, arguing that the field of LLM context management is, at its conceptual foundations, a branch of applied cognitive science. The section proceeds from classical memory theory through modern implementations, offering both theoretical depth and practical analysis of the systems that operationalize these ideas.

8.1 Foundations of Human Memory Theory

The design of memory systems for large language models does not occur in a theoretical vacuum. Over a century of research in experimental psychology has produced detailed models of how biological memory operates, and these models provide the conceptual vocabulary, architectural blueprints, and empirical benchmarks against which artificial memory systems are increasingly evaluated. This subsection reviews the foundational theories that have most directly influenced the design of LLM memory architectures, beginning with the multi-store model of [102] and proceeding through the working memory framework of [103], the capacity constraints identified by [104], serial position phenomena, the episodic-semantic distinction of [105], and the forgetting curve of [106].

8.1.1 The Atkinson-Shiffrin Multi-Store Model

The multi-store model proposed by [102] established the foundational architecture for understanding human memory as a system of distinct, interacting stores. This model, which has shaped memory research for over fifty years [107], posits three sequential stages through which information flows.

The first stage, sensory memory, functions as a large-capacity buffer that maintains raw sensory information for very brief durations, on the order of hundreds of milliseconds to approximately one second. Sensory memory acts as an input gate, holding a high-fidelity representation of the perceptual world before selective attention determines which information will be passed to the next stage. The second stage, short-term memory (STM), is a severely limited store that holds approximately 7 ± 2 chunks of information [104] for durations of roughly twenty to thirty seconds without active rehearsal. STM is maintained through control processes such as subvocal rehearsal, and its contents are readily accessible to conscious awareness. The third stage, long-term memory (LTM), is a very large capacity store with potentially lifetime duration, using primarily semantic encoding and susceptible to interference during retrieval rather than simple temporal decay.

Critically, and in a point that is often overlooked in simplified textbook treatments, Atkinson and Shiffrin placed considerable emphasis on the *control processes* that regulate the flow of information across these stores. Attention governs the selection of information from sensory memory into STM. Rehearsal maintains information in STM and facilitates its transfer to LTM. Search and retrieval processes access information stored in LTM, and coding and organizational strategies determine how information is encoded for long-term storage. This emphasis on dynamic processing, rather than merely static stores, is profoundly relevant to modern LLM memory management. The challenge confronting contemporary systems is not storage capacity per se—external databases can store essentially unlimited information—but rather the intelligent movement, selection, and retrieval of information across different storage tiers. The control processes that Atkinson and Shiffrin identified as central to human memory are precisely the mechanisms that LLM memory architectures must implement.

The model maps naturally onto the architecture of modern LLMs. Sensory memory corresponds to token embeddings, which provide a transient, high-dimensional representation of the raw input before any processing by the transformer layers. Short-term memory corresponds to the context window, which functions as a limited-capacity buffer holding the information currently available to the model’s attention mechanism. Long-term memory corresponds to the model’s parametric knowledge, stored in billions of weight parameters, together with any external databases or retrieval systems that provide persistent storage beyond the context window. The control processes find their analogue in the attention mechanism itself, which dynamically selects and weights information from across the context, and in the various memory management strategies—summarization, retrieval, eviction—that govern what information enters and exits the context window.

The historical impact of the Atkinson-Shiffrin model extends well beyond cognitive psychology. It generated extensive empirical research on memory stores and transitions, directly influenced the development of Baddeley’s working memory model, and has served as a conceptual foundation for the design of memory-augmented computational architectures from the earliest expert systems to the most recent LLM-based agents. The model’s limitations are also instructive: its assumption of strictly linear, sequential information flow oversimplifies the parallel processing capabilities of

biological memory, its treatment of STM as a unitary store was challenged by subsequent research demonstrating domain-specific storage, and its discrete “stores” concept has been questioned by neuroscience evidence suggesting more graded representations. These limitations, however, have motivated refinements rather than abandonment, and the core insight—that effective memory requires multiple stores with distinct temporal and capacity characteristics, coordinated by active control processes—remains as relevant to LLM design as it was to cognitive psychology.

8.1.2 Baddeley’s Working Memory Model

[103] refined and substantially extended the concept of short-term memory by proposing a multi-component *working memory* system. This model challenged the unitary STM view implicit in the Atkinson-Shiffrin framework, replacing it with a more nuanced architecture comprising specialized subsystems coordinated by a central executive. The original 1974 model specified three components, and a fourth was added by [108] to address limitations in the original formulation.

The Central Executive.. The central executive is a supervisory attentional control system responsible for managing cognitive resources, coordinating the subsidiary storage systems, and implementing flexible task switching and conflict resolution. It is domain-general and has severely limited capacity, estimated at approximately 1.5 to 2 chunks of actively maintained information. The central executive does not store information itself but rather directs the flow of information between the subsidiary systems and between working memory and long-term memory. It is responsible for the highest-level cognitive operations: selecting strategies, inhibiting irrelevant information, switching between tasks, and allocating processing resources. The underspecification of the central executive—precisely what mechanisms drive its coordination function—has been a persistent criticism of the model [109], but its conceptual role as an attentional controller remains essential to understanding how limited-capacity systems manage complex information processing.

The Phonological Loop.. The phonological loop is a specialized system for the storage and maintenance of verbal and auditory information. It comprises two subcomponents: a passive phonological store that holds acoustic memory traces for approximately two seconds before they decay, and an active articulatory rehearsal process that refreshes the contents of the store through subvocal repetition. The phonological loop has a capacity of approximately three to four seconds of speech, corresponding to roughly seven to nine words, and demonstrates characteristic phenomena such as the phonological similarity effect (confusions between similar-sounding items) and the word length effect (shorter words recalled better because more can be rehearsed in the two-second window). This component supports language learning, reading comprehension, and verbal problem-solving, and its specialization for sequential linguistic information makes it a particularly apt analogue for the token-level processing that characterizes LLMs.

The Visuospatial Sketchpad.. The visuospatial sketchpad is a system for maintaining and manipulating visual and spatial information. It supports mental rotation, visual problem-solving, navigation, and the maintenance of spatial layouts in working memory. Its capacity is approximately four to five items, and it can be selectively disrupted by concurrent visual tasks without affecting phonological processing, providing strong evidence for the domain-specific separation of working memory components. While LLMs are primarily text-based systems, the visuospatial sketchpad has analogues in the structured representations that LLMs maintain for spatial, relational, and graph-based information within their embedding spaces.

The Episodic Buffer.. Added by [108] to address the binding problem—how information from different domains and from long-term memory is integrated into coherent episodes—the episodic buffer is a limited-capacity store holding approximately four chunks of information. It integrates information across domains, binding verbal, visual, and spatial representations into coherent episodes. The episodic buffer serves as the interface between working memory and long-term memory, enabling temporal sequencing of events and episodic narrative construction. Baddeley described it as acting “as a buffer store, not only between the components of Working Memory, but also linking Working Memory to perception and Long-Term Memory.” This integrative function is critical: it explains how humans construct coherent representations of ongoing experience from diverse information sources, a challenge that LLMs face when they must synthesize information from different parts of a long context or from external memory systems.

Baddeley’s model provides a considerably richer framework for LLM design than the simpler Atkinson-Shiffrin model. The central executive maps onto the transformer’s self-attention mechanism, which selects and weights relevant information from across the context and coordinates the processing of different aspects of the input. The phonological loop parallels the token sequence buffer with positional encoding, which maintains sequential linguistic information in a format optimized for language processing. The visuospatial sketchpad corresponds to the multi-dimensional embedding representations that capture structured and relational information. The episodic buffer corresponds to the integrated token representations at the output of transformer layers, which capture multi-modal context bound into coherent representations. Most importantly, the model’s emphasis on *active manipulation* of information—not merely passive storage—aligns with the emerging understanding that LLMs must actively manage their context rather than simply accommodating larger inputs. The distinction between storage and processing, which is central to the working memory framework, suggests that context window expansion alone is insufficient; what matters is the quality of the control processes that operate over whatever context is available.

The robust empirical support for the working memory model across diverse experimental paradigms, including demonstrations of domain-specific interference patterns, correlations between working memory capacity and academic achievement, and neuroimaging evidence for component localization, lends confidence to its use as a design framework for artificial systems. The model’s limitations—particularly the underspecification of the central executive, the difficulty of measuring capacity con-

sistently across components, and the unclear distinction between refresh and rehearsal mechanisms—are precisely the areas where computational implementations can contribute to theoretical progress by requiring explicit specification of mechanisms that remain implicit in the psychological theory.

8.1.3 Miller’s Magical Number Seven and Chunking

[104] established one of the most enduring findings in cognitive psychology: that the capacity of immediate memory is limited to approximately 7 ± 2 “chunks” of information, where a chunk is defined as the largest meaningful unit recognized by the individual. This finding has profound implications for information processing because it places an absolute constraint on the amount of information that can be simultaneously maintained and manipulated in working memory. Modern research by [110] has suggested a lower bound of 4 ± 1 chunks for the active maintenance capacity of young adults, with Miller’s larger estimate potentially reflecting contributions from long-term memory and chunking strategies rather than pure working memory capacity.

The concept of chunking itself—the reorganization of information into larger meaningful units to overcome capacity limitations—is directly relevant to context compression techniques in LLMs. When a chess master perceives a board configuration as a single chunk rather than as thirty-two individual pieces, they dramatically increase the effective information content of their working memory without increasing its capacity in terms of chunks. Analogously, when LLM systems apply gisting, prompt compression, or summarization techniques, they are performing a computational version of chunking: reorganizing detailed information into more compact representations that preserve essential semantic content while reducing the number of tokens consumed in the context window. The effectiveness of these techniques depends on the quality of the chunking—just as expert chess players chunk meaningfully while novices cannot, the quality of LLM compression depends on whether the summarization preserves the information that will be relevant to downstream processing.

The empirical observation that LLMs exhibit working memory-like capacity limits that are surprisingly consistent with human performance—research has shown that GPT-3 and GPT-4 can effectively hold approximately four to seven distinct items in “active” consideration, with attention scores gradually concentrating on a small number of positions—suggests that the mathematical structure of attention may impose constraints that are functionally analogous to biological working memory capacity, even though the underlying mechanisms differ fundamentally. This convergence between biological and artificial capacity constraints is theoretically significant because it implies that working memory limitations may reflect computational principles rather than biological accidents, and that strategies for managing limited capacity in human cognition may transfer directly to the design of LLM memory management systems.

8.1.4 Serial Position Effects: Primacy, Recency, and the Lost-in-the-Middle Phenomenon

The serial position curve, first characterized systematically by [111] and elaborated by [112], demonstrates a robust pattern of differential recall based on an item’s position within a sequence. Items presented at the beginning of a list are recalled with enhanced accuracy (the primacy effect, attributed to greater rehearsal opportunity and consequent LTM encoding), and items presented at the end of a list are similarly advantaged (the recency effect, attributed to their continued availability in STM at the time of recall). Items in the middle of the list suffer the worst recall, producing the characteristic U-shaped serial position curve. The dissociation between primacy and recency effects—recency is eliminated by a brief period of distractor activity while primacy remains stable—provides strong evidence for the multi-store distinction between STM and LTM.

This phenomenon has a striking and empirically well-documented parallel in the behavior of large language models. [7] demonstrated that LLMs exhibit a U-shaped performance curve strikingly similar to the human serial position effect: information placed at the beginning and end of the context window is processed more effectively than information in the middle, a phenomenon they termed “lost in the middle.” The cognitive explanation for the human serial position effect—that boundary items receive preferential processing through distinct encoding mechanisms—suggests that the LLM analogue may arise from analogous computational properties of the attention mechanism. Positional embeddings, attention distribution patterns, and the structure of gradient flow through transformer layers may all contribute to preferential processing of boundary positions.

The parallel between human serial position effects and LLM position bias is more than a superficial analogy. In both systems, the effect reflects fundamental properties of how sequential information is processed in a limited-capacity system. The strategies that humans deploy to overcome serial position bias—such as rehearsal, elaborative encoding, and organizational strategies that reduce reliance on position—may inform computational strategies for mitigating the lost-in-the-middle effect in LLMs, such as strategic information placement, iterative re-reading, and architectural modifications that equalize attention across positions.

8.1.5 Tulving’s Episodic and Semantic Memory Distinction

[105] introduced one of the most influential distinctions in memory research: the separation of long-term memory into *episodic memory* for specific events grounded in time and place, and *semantic memory* for general knowledge, facts, and language. This distinction was further refined by [113], who associated each memory type with a distinct form of consciousness: episodic memory involves autonoetic consciousness (the ability to mentally re-experience past events, to “travel back in time”), semantic memory involves noetic consciousness (abstract knowing without re-experiencing), and procedural memory involves anoetic consciousness (non-conscious skilled performance).

The episodic-semantic distinction maps directly onto the architecture of LLM memory systems.

Parametric knowledge acquired during pre-training represents semantic memory: general truths, factual relationships, and linguistic patterns stored implicitly in the model’s weight parameters. This knowledge is accessed through the forward pass without any explicit retrieval mechanism, analogous to how humans access semantic knowledge without consciously recalling when or where they learned it. Context-specific information from particular interactions—conversation histories, user preferences, specific task details—represents episodic memory: particular events linked to temporal context and situational specifics.

Standard LLMs excel at semantic knowledge but struggle profoundly with episodic memory. They cannot, without external augmentation, remember specific past interactions, track the temporal context of information, or maintain spatiotemporal grounding across sessions. This limitation has been identified as the “missing piece” for long-term LLM agents [114]. The challenges are multifaceted: difficulty with information presented only once over long timescales, poor performance on sequence order recall tasks, interference from multiple related events stored in the same parametric space, and the fundamental problem that parametric knowledge is updated only through training, not through experience [115]. Solutions to these challenges—external episodic memory systems such as EM-LLM [116], explicit temporal tagging and event segmentation, and structured retrieval combining recency, relevance, and importance—all draw directly on Tulving’s theoretical framework, implementing computational versions of the episodic memory system that standard LLMs lack.

Modern cognitive science has increasingly viewed episodic and semantic memory as existing on a continuum rather than as rigidly discrete categories, with shared underlying neural processes and gradual transitions from episodic to semantic representations through consolidation. This perspective is equally relevant to LLM design: the most effective memory architectures may be those that support gradual transformation of specific episodic information into generalized semantic knowledge, mirroring the consolidation processes that operate in biological memory.

8.1.6 The Ebbinghaus Forgetting Curve

[106] conducted the first systematic experimental investigation of memory, establishing through heroic self-experimentation that memory retention follows an exponential decay function. The mathematical form of the forgetting curve is typically expressed as:

$$R = e^{-t/S} \tag{6}$$

where R represents retention (as a proportion of original learning), t is the time elapsed since learning, and S is a parameter representing memory strength that varies with the depth of encoding, the number of repetitions, and individual differences. The forgetting curve exhibits a characteristic shape: a steep initial decline, with approximately fifty to seventy percent of information lost within the first twenty-four hours without reinforcement, followed by a gradual leveling off at longer intervals. This pattern was replicated and confirmed by [117] using modern experimental methods, validating Ebbinghaus’s original findings more than a century after their initial publication.

The debate between exponential and power-law decay functions continues in the memory literature, with evidence suggesting that pure forgetting of homogeneous material follows exponential decay while the aggregate forgetting of heterogeneous material—where different items have different memory strengths—may produce power-law curves that are the superposition of multiple exponential functions. This distinction is relevant to LLM memory design because the information stored in artificial memory systems is invariably heterogeneous, suggesting that power-law decay models may be more appropriate than simple exponential decay for practical implementations.

Ebbinghaus also discovered the spacing effect—that distributed practice produces more durable learning than massed practice—and the related finding that retrieval practice (testing) produces stronger memory than passive review. These findings have been operationalized in spaced repetition algorithms that schedule reviews to maximize retention with minimum study time. The implications for LLM memory management are direct: information that is periodically retrieved and reinforced should be maintained more durably than information that is merely stored, and the scheduling of memory access and consolidation operations should follow principles analogous to spaced repetition.

The Ebbinghaus forgetting curve has been directly implemented in LLM memory systems. [118], in their MemoryBank system, implemented exponential decay of memory importance scores to determine which stored information should be retained versus allowed to fade, creating an artificial analogue of biological forgetting. This approach represents a departure from most LLM memory systems, which either retain everything indefinitely or use attention-based selection without temporal decay, and it highlights the insight that principled forgetting—not merely storage—is essential for scalable and effective memory management.

8.2 Memory Consolidation and Complementary Learning Systems

The transition of information from short-term to long-term storage is not a simple copying process but involves active transformation through consolidation. This subsection examines the neurobiological theories of memory consolidation that have most directly influenced the design of LLM memory architectures, focusing on the complementary learning systems framework, hippocampal replay, and sleep-dependent consolidation.

8.2.1 Systems Consolidation: From Hippocampus to Neocortex

Memory consolidation operates on two distinct timescales. Synaptic consolidation, occurring over minutes to hours, involves molecular cascades that strengthen synaptic connections at the cellular level through mechanisms including AMPA receptor insertion, protein synthesis, and calcium-dependent signaling. Systems consolidation, occurring over hours to years, involves a large-scale reorganization in which memories initially dependent on the hippocampus are gradually redistributed to neocortical networks, becoming increasingly hippocampus-independent over time. The empirical foundation for this distinction was established through lesion studies, most notably the case of patient H.M., whose selective hippocampal damage preserved remote memories while severely

impairing new episodic learning [119]. This dissociation demonstrated that the hippocampus is essential for the formation of new declarative memories but not for the storage of old ones, implying a process by which memories are transferred from hippocampal to neocortical substrates.

The distinction between synaptic and systems consolidation has a direct analogue in LLM memory design. Synaptic consolidation corresponds to the immediate processing that occurs when new information enters the context window—the rapid formation of representations through attention and feedforward computation. Systems consolidation corresponds to the longer-term processes by which information from the context window is selectively transferred to external memory stores, summarized, and integrated with existing knowledge. The absence of principled systems consolidation in standard LLMs—information either remains in the context window or is lost entirely when the window advances—represents a fundamental architectural limitation that motivates the development of explicit consolidation mechanisms in memory-augmented systems.

8.2.2 Complementary Learning Systems Theory

[120] proposed the complementary learning systems (CLS) framework to explain a puzzle that had long troubled connectionist modelers: why does the brain require two specialized learning systems rather than a single, unified network? The answer, McClelland and colleagues argued, lies in a fundamental tension between the requirements of rapid learning and the requirements of structured knowledge representation.

The hippocampal system implements rapid, one-shot learning through sparse, pattern-separated representations. Its encoding is pattern-specific and detail-preserving, allowing new experiences to be stored in a single exposure without disrupting existing memories. However, the hippocampus has limited capacity and stores memories that are relatively short-lived, on the order of days to weeks. The neocortical system, by contrast, implements gradual, interleaved learning through distributed, overlapping representations. It builds abstract, generalizable knowledge structures over many exposures, but it cannot rapidly incorporate new information without risking catastrophic interference with existing representations.

The consolidation process bridges these two systems. Memories are first rapidly encoded in the hippocampus, then gradually transferred to the neocortex through repeated reinstatement—a process in which hippocampal memory traces are replayed to the neocortex, allowing the neocortex to learn from these replayed experiences at a rate that avoids catastrophic interference. This interleaved replay ensures that new memories are integrated with existing knowledge structures rather than overwriting them.

The CLS framework provides a biological blueprint for LLM memory design that has been enormously influential. Current LLMs function primarily as neocortical systems: they build distributed knowledge representations through gradual training on large datasets, but they lack an equivalent of the hippocampal system that would allow rapid encoding of new episodic information without

disrupting existing parametric knowledge. Memory-augmented architectures that combine rapid external storage for episodic information with stable parametric knowledge for semantic information effectively implement a computational analogue of complementary learning systems. The external memory store (whether a vector database, a knowledge graph, or a structured memory bank) functions as the hippocampal system, providing rapid, pattern-separated storage for new information. The model’s parameters function as the neocortical system, providing stable, distributed storage for generalized knowledge. The retrieval and integration mechanisms that connect these two stores correspond to the consolidation processes that transfer information between hippocampus and neocortex.

Pattern Separation and Pattern Completion.. Two key computational principles from the CLS framework are directly applicable to LLM memory design. Pattern separation—the creation of distinct, non-overlapping representations for similar but different memories—is essential for preventing interference between stored memories. In the hippocampus, this is achieved through sparse coding in the dentate gyrus. In LLM memory systems, analogous mechanisms include the use of distinct embedding spaces for different memory types, the explicit separation of episodic and semantic memory stores, and the use of hash-based or graph-based indexing that maintains separation between similar but distinct memories. Pattern completion—the retrieval of a full memory from a partial cue—is the complementary process that enables effective memory retrieval. In the hippocampus, pattern completion operates through recurrent connections in CA3 that allow partial activation patterns to recover full stored patterns. In LLM memory systems, this corresponds to similarity-based retrieval from vector databases, where a query embedding is matched against stored memory embeddings to recover the full content of the most similar memories.

8.2.3 Hippocampal Replay and Sleep-Dependent Consolidation

Neural replay mechanisms support memory consolidation through the sequential reactivation of experience patterns during both waking rest and sleep [121, 122]. During waking rest periods, the hippocampus frequently reactivates recent experience patterns, a process that may serve multiple functions including consolidation, planning, and memory retrieval. During sleep, replay becomes particularly organized and effective, occurring within a framework of coordinated oscillatory activity that provides optimal conditions for hippocampal-cortical information transfer.

Sleep-dependent consolidation operates through several complementary mechanisms [123]. During non-rapid eye movement (NREM) sleep, particularly slow-wave sleep, three types of oscillatory activity coordinate memory consolidation. Slow oscillations at approximately 0.5 to 1 Hz coordinate large-scale information transfer between hippocampus and neocortex. Sleep spindles at 12 to 16 Hz, generated by thalamic circuits, mark periods of active memory replay and facilitate synaptic plasticity in cortical networks. Sharp-wave ripples at 140 to 200 Hz, generated in the hippocampus, represent the actual reactivation of specific memory traces. The temporal coupling of these oscillations—ripples nested within spindles, which are in turn nested within slow oscillations—

creates a hierarchical timing structure that orchestrates the precise sequence of events required for effective consolidation. Research has demonstrated that sleep enhances memory consolidation by twenty-five to forty percent compared to equivalent periods of wakefulness, and that targeted memory reactivation during sleep (presenting cues associated with specific memories) can selectively strengthen particular memories [124].

During rapid eye movement (REM) sleep, a distinct set of mechanisms operates. Theta-frequency oscillations at 4 to 8 Hz dominate hippocampal activity, and reduced norepinephrine levels enhance synaptic plasticity. REM sleep appears to play a complementary role to NREM sleep in consolidation, with particular importance for emotional memory processing, the integration of new memories into existing semantic networks, and the extraction of abstract patterns from specific experiences. The alternation of NREM and REM sleep stages across the night facilitates what [125] have termed “graceful integration”—the gradual incorporation of new information into existing knowledge structures without disrupting previously consolidated memories.

These biological mechanisms have no direct analogue in standard LLM training, which relies exclusively on online gradient-based learning from sequentially presented training examples. The absence of offline consolidation phases—analogue to sleep—is arguably a fundamental limitation of current LLM architectures. Standard LLMs have no mechanism for replaying and reorganizing recently acquired information, no process for integrating new knowledge with existing parametric representations without catastrophic interference, and no offline phase during which abstract patterns can be extracted from specific training experiences. This gap has motivated a growing body of research on sleep-inspired training procedures, discussed in detail below, that aim to introduce consolidation-like mechanisms into the training of artificial neural networks.

8.3 Classical Cognitive Architectures

Before the advent of large language models, the field of cognitive architecture had already established fundamental principles about how memory systems should be organized for intelligent behavior. Two architectures—SOAR and ACT-R—are particularly relevant because they formalized many of the memory distinctions that have been independently rediscovered in the design of LLM-based agents.

8.3.1 SOAR

The SOAR cognitive architecture [126] is a production system architecture that models intelligent behavior as the interaction between working memory and procedural knowledge encoded as production rules (if-then statements that match conditions in working memory to actions). SOAR represents working memory as relational graph structures composed of node-edge-value triples, providing a structured representation that captures relationships between entities rather than merely listing features. Procedural memory in SOAR stores the production rules that encode the agent’s behavioral repertoire, and the architecture includes mechanisms for learning new productions through experience (chunking) and for resolving impasses when no production matches the current working

memory state.

SOAR’s contribution to the understanding of memory in intelligent systems lies in its demonstration that effective behavior requires not merely storage but structured interaction between working memory and long-term procedural knowledge. The architecture’s focus on the dynamic interplay between current state (working memory) and accumulated expertise (procedural memory) anticipates the design of modern agent architectures, where the LLM’s context window serves as working memory and its parametric knowledge serves as a form of procedural-semantic memory. SOAR also demonstrated the importance of learning from experience—its chunking mechanism, which compiles frequently used reasoning sequences into compiled productions, parallels the reflection and abstraction mechanisms found in modern generative agent systems.

8.3.2 ACT-R

The ACT-R architecture [127] provides a more detailed cognitive model than SOAR, with specialized modules for different cognitive functions (visual, memory, motor, goal) coordinated by a central production system. ACT-R’s memory architecture distinguishes between declarative memory (facts and episodic knowledge stored as chunks with associated activation levels) and procedural memory (skills and behavioral routines stored as production rules). Declarative memory retrieval in ACT-R follows a rational analysis of memory: the probability of retrieving a memory chunk is a function of its base-level activation (reflecting recency and frequency of use) and its associative activation (reflecting the degree to which current context provides retrieval cues for the chunk).

The mathematical formulation of ACT-R’s memory retrieval—particularly its base-level learning equation, which models memory activation as a function of recency and frequency—provided an early computational specification of the principles that Ebbinghaus identified empirically. The architecture’s distinction between declarative and procedural memory directly parallels the semantic and procedural memory distinction in modern language agent frameworks such as CoALA. ACT-R’s integration of multiple specialized modules under central executive control also anticipates the modular architectures that characterize many contemporary LLM-based agent systems, where different modules handle different aspects of memory, retrieval, and action.

Lessons from Classical Architectures.. Both SOAR and ACT-R demonstrate a principle that has been rediscovered repeatedly in the design of LLM-based agents: effective intelligent behavior requires multiple memory systems with distinct access patterns, storage durations, and consolidation mechanisms. A single, undifferentiated memory store is insufficient for complex cognitive tasks. The structured interaction between working memory and long-term stores, the distinction between declarative and procedural knowledge, and the importance of learning mechanisms that refine and consolidate knowledge through experience—all principles established by classical cognitive architectures—are now being implemented in modern LLM memory systems, often with explicit acknowledgment of their cognitive science origins.

8.4 Modern LLM Memory Architectures Inspired by Human Cognition

The theoretical foundations reviewed above have directly inspired a generation of memory architectures for large language models and LLM-based agents. This subsection examines the most influential of these architectures in detail, analyzing how each one draws on cognitive science principles and evaluating its contributions and limitations.

8.4.1 CoALA: Cognitive Architectures for Language Agents

[128] proposed CoALA (Cognitive Architectures for Language Agents), the first comprehensive framework that systematically organizes the design space of language agents using concepts drawn from classical cognitive architectures. CoALA addresses a gap in the rapidly growing literature on LLM-based agents: while numerous systems had been proposed, no systematic framework existed to organize and compare them, to identify design tradeoffs, or to connect modern implementations to the decades of research on cognitive architectures that preceded them. CoALA provides this unifying lens by organizing language agents along three fundamental dimensions inspired by cognitive science.

Information Storage.. The first dimension concerns how agents store and organize information. CoALA distinguishes between working memory, which corresponds to the context window and holds the current state, recent observations, and active goals, and long-term memory, which encompasses three subtypes derived from cognitive psychology. Procedural memory stores skills and strategies encoded in model parameters through gradient descent, corresponding to the implicit knowledge that an LLM acquires during pre-training and fine-tuning. This includes learned patterns for tool use, reasoning strategies, and behavioral routines that are executed automatically during inference. Semantic memory stores facts and concepts in learned embeddings and, optionally, in external knowledge bases, corresponding to the explicit factual knowledge that can be retrieved through the forward pass or through external retrieval mechanisms. Episodic memory stores specific events and experiences in external memory banks, corresponding to the temporally grounded, context-specific memories that standard LLMs lack but that are essential for agents operating over extended interactions. The CoALA framework makes explicit a distinction that is often implicit in the design of LLM agents: the context window is working memory, not the entirety of memory, and effective agent design requires consideration of how information flows between working memory and the various forms of long-term storage.

Action Space.. The second dimension concerns the actions available to the agent, divided into external actions that affect the environment (API calls, tool use, environmental interaction) and internal actions that operate on the agent’s own representations (reasoning, memory retrieval, and learning). This distinction is important because it highlights that memory operations—storing, retrieving, and updating information—are themselves actions that the agent must decide to perform, not passive processes that occur automatically. The agent must decide when to store information in long-term memory, when to retrieve information from long-term memory into working memory,

and when to update or consolidate stored information. These decisions are analogous to the control processes that Atkinson and Shiffrin identified as central to human memory management.

Decision-Making.. The third dimension concerns how the agent decides which action to take, modeled as an interactive loop with planning and execution phases. The planning phase involves perception (observing the current state), goal activation (determining what to pursue), and deliberation (planning the next action). The execution phase involves performing the chosen action, observing the results, and updating working memory. This loop structure mirrors the perception-action cycle of classical cognitive architectures and provides a framework for understanding how memory operations are integrated into the agent’s overall behavior.

CoALA’s contribution extends beyond taxonomy. By retrospectively organizing over one hundred papers on language agents and mapping each to its framework, CoALA identifies systematic gaps in the literature. Most current systems, the analysis reveals, lack robust episodic memory and principled consolidation mechanisms. Procedural learning—the ability to acquire new skills and strategies through experience—is particularly underexplored. The framework also identifies promising but underexplored design combinations, such as agents with rich episodic memory combined with explicit planning capabilities, or agents with procedural learning mechanisms that can consolidate frequently used reasoning patterns into efficient routines. By connecting modern LLM agents to classical cognitive architectures such as ACT-R, SOAR, and CLARION, CoALA establishes a conceptual bridge that enables the transfer of insights from decades of cognitive science research to the design of contemporary systems.

8.4.2 Generative Agents: Memory Streams and Reflection

[129] developed generative agents—computational agents embedded in a simulated environment that exhibit believable human-like behavior through a hierarchical memory architecture that operationalizes episodic memory for language agents. This work represents a landmark demonstration that human-inspired memory systems can enable emergent complexity in artificial agents, and it has been enormously influential in subsequent research on agent memory design.

The Memory Stream.. The core data structure underlying generative agents is the memory stream: a chronologically ordered database of every observation, action, plan, and reflection that an agent experiences. Each entry in the memory stream is stored as a natural language description accompanied by a timestamp, an importance score (rated on a scale of 0 to 10 by the language model), and a vector embedding that enables similarity-based retrieval. The memory stream grows unboundedly over the course of the simulation, capturing the complete experiential history of the agent in a format that is both human-readable and computationally searchable. This design choice—storing memories as natural language rather than as structured data or compressed representations—reflects a commitment to interpretability and flexibility, allowing the same retrieval mechanisms to operate over observations, actions, and abstract reflections.

The Reflection Mechanism.. The reflection mechanism is the key innovation that transforms the memory stream from a passive log into an active knowledge construction system. Periodically, when the cumulative importance scores of recent unreflected memories exceed a threshold, the agent generates higher-level abstractions from its recent experiences. The reflection process involves querying the language model to identify patterns, draw conclusions, and form beliefs based on a collection of recent memories. The resulting reflections are themselves stored as entries in the memory stream, creating a hierarchical structure: raw observations at the leaf level, event-level summaries at intermediate levels, and abstract beliefs and goals at the highest level. Because reflections are stored alongside observations and can themselves be reflected upon, the system supports meta-reflection—the formation of increasingly abstract and general knowledge from specific experiences.

In practice, agents in the Smallville simulation reflected two to three times per simulated day. The reflection process closely parallels the consolidation mechanisms described in the CLS framework: specific episodic experiences are gradually abstracted into generalized semantic knowledge through a process of repeated review and integration. The analogy to sleep-dependent consolidation is also apparent: reflection represents an offline processing phase during which the agent steps back from active engagement with the environment to reorganize and abstract from its accumulated experience.

Retrieval Mechanism.. When planning actions, agents retrieve relevant memories from the memory stream using a weighted combination of three signals:

$$\text{score}(m) = \alpha \cdot \text{recency}(m) + \beta \cdot \text{importance}(m) + \gamma \cdot \text{relevance}(m, q) \quad (7)$$

where recency decays exponentially with time since the memory was last accessed, importance reflects the rated significance of the memory, and relevance is computed as the cosine similarity between the memory’s embedding and the current query embedding. This multi-signal retrieval mechanism draws on multiple principles from cognitive psychology: recency-weighted retrieval parallels the recency effect and the Ebbinghaus forgetting curve, importance-weighted retrieval parallels the preferential encoding of emotionally significant or goal-relevant information, and relevance-based retrieval parallels the associative, cue-dependent retrieval that characterizes human episodic memory.

Emergent Behaviors and Evaluation.. In a simulation of twenty-five agents in a virtual village (“Smallville”), powered by GPT-3.5-turbo, the full memory architecture enabled a remarkable range of emergent behaviors that were not explicitly programmed. Agents maintained consistent daily schedules influenced by their accumulated memories. Relationships developed naturally through repeated interactions, with agents greeting friends differently from strangers and conversations reflecting the history of previous interactions. Information spread through social networks in realistic patterns, with gossip propagating believably and at realistic velocities. Most strikingly, agents spon-

taneously coordinated group activities: a Valentine’s Day party was organized through a series of conversations among multiple agents, each independently deciding to attend based on their own memories and social relationships.

Human raters, blind to experimental condition, evaluated agent believability on scales of consistency, appropriateness, and realism. The full memory system (memory stream plus reflection) received an overall believability rating of 7.8 out of 10, compared to 5.2 without reflection and 3.1 without any memory stream. Random action agents received only 1.9. The 2.5-fold improvement from adding reflection alone demonstrates the critical importance of abstraction and consolidation mechanisms—raw episodic storage is necessary but insufficient for believable behavior; the consolidation of specific experiences into generalized knowledge is what enables coherent, adaptive, contextually appropriate behavior over extended periods.

Limitations and Future Directions.. Despite its success, the generative agents architecture has several important limitations. It lacks explicit forgetting mechanisms: the memory stream grows without bound, and no information is ever discarded, leading to increasing computational costs for retrieval as the simulation progresses. There is no sleep-like consolidation phase that would enable offline reorganization of memories. The reflection mechanism operates only on recent memories, potentially missing long-term patterns that span many simulated days. Hallucination is possible, with agents occasionally referencing events that did not occur. And the computational cost is substantial: with twenty-five agents each making multiple LLM calls per simulation step, a full simulation requires approximately twenty-five thousand LLM calls. These limitations define the research agenda for subsequent memory architectures, many of which address specific shortcomings of the generative agents approach.

8.4.3 MemGPT: Virtual Context Management as an Operating System

[101] introduced MemGPT, an architecture that draws inspiration not from cognitive psychology but from operating systems engineering, specifically from the concept of virtual memory. In computer operating systems, virtual memory creates the illusion of a large, uniform address space by intelligently managing the movement of data between fast but limited primary memory (RAM) and slow but capacious secondary storage (disk). MemGPT applies this principle to LLM context management, treating the context window as “main memory” and external storage as “disk,” with the LLM itself serving as the processor that issues instructions for data movement between tiers.

The key innovation of MemGPT is that the LLM agent manages its own memory through function calls. When the agent determines that it needs information not currently in the context window, it issues a retrieval function call to load relevant information from external storage. When the context window approaches capacity, the agent issues storage function calls to offload less immediately relevant information to external storage. An interrupt mechanism manages control flow when context limits are reached, pausing the current operation to perform memory management before resuming.

This self-managed approach contrasts with systems where memory management is handled by external controllers: in MemGPT, the LLM is both the processor and the memory manager, deciding what to page in and out of its own context.

MemGPT demonstrates two primary applications. For document analysis, it enables processing of documents that far exceed the context window by intelligently paging relevant sections into the context as needed. For multi-session chat, it maintains long-term memory across conversation sessions by storing and retrieving conversation summaries, user preferences, and factual information from persistent external storage. The result is an illusion of infinite memory through virtualization, analogous to how virtual memory creates the illusion of unlimited RAM.

While MemGPT’s inspiration is primarily computational rather than cognitive, the resulting architecture has clear parallels to hierarchical memory models from cognitive science. The context window functions as working memory with limited capacity and immediate accessibility. External storage functions as long-term memory with unlimited capacity but slower access requiring explicit retrieval operations. The memory management operations—deciding what to store, what to retrieve, and what to evict—correspond to the control processes that Atkinson and Shiffrin identified as central to human memory management. The system’s ability to create the illusion of continuous memory across sessions, despite the fundamentally stateless nature of each individual LLM inference, parallels the way that human memory creates the illusion of continuous experience despite the discontinuities introduced by sleep, distraction, and the limited capacity of working memory.

8.4.4 MemoryBank: Ebbinghaus-Inspired Forgetting

[118] directly implemented the Ebbinghaus forgetting curve in an LLM memory system, creating what is arguably the most explicit translation of classical memory theory into computational practice. MemoryBank stores past conversations, summarized events, and user portraits in a structured memory bank, with each memory assigned an importance score that decays exponentially over time according to a modified Ebbinghaus function:

$$\text{Importance}(m, t) = I_0 \cdot e^{-\lambda t} \quad (8)$$

where I_0 is the initial importance score, t is the time elapsed since last access, and λ is a decay rate controlling the speed of forgetting.

The system comprises three modules that work together to manage the memory lifecycle. The Writer module extracts important information from each dialogue turn—user preferences, emotional states, factual details, goals—and stores it with associated importance scores. The Retriever module finds relevant memories for the current dialogue context using a combination of semantic similarity, recency, and importance scoring. The Reader module integrates retrieved memories into the LLM’s input prompt, formatting them as natural language context that augments the model’s understanding of the user and the conversation history.

The critical innovation is the memory update mechanism. Memories that are not accessed undergo exponential decay: a memory created with importance 8.0 might decline to 7.1 after one day, 5.9 after three days, 3.2 after a week, and 0.9 after two weeks, at which point it falls below the retrieval threshold and effectively becomes inaccessible. However, when a memory is retrieved and used in dialogue, its importance is boosted, resetting the decay curve. This reinforcement mechanism mirrors the spacing effect: frequently relevant memories are strengthened through repeated retrieval, while irrelevant memories naturally fade. The result is an adaptive memory system in which the contents are shaped by relevance and usage patterns rather than by arbitrary retention or deletion rules.

MemoryBank was demonstrated through SiliconFriend, an AI companion chatbot that leverages the memory system to provide empathetic responses, personality understanding, and the impression of long-term companionship. Users interacting with SiliconFriend reported more personalized and contextually appropriate responses compared to systems without persistent memory. The explicit incorporation of a principled forgetting mechanism represents a significant departure from most LLM memory systems, which either retain everything indefinitely (leading to unbounded growth and retrieval degradation) or use ad hoc deletion strategies without theoretical motivation. MemoryBank demonstrates that forgetting is not merely a limitation to be overcome but an essential feature of effective memory management—a principle that Ebbinghaus established empirically and that modern information theory supports through the concept of optimal information retention under resource constraints.

8.4.5 Think-in-Memory: Storing Thoughts, Not Facts

[130] proposed Think-in-Memory (TiM), an architecture based on the observation that humans recall the results of previous thinking without re-reasoning from scratch, whereas LLMs repeatedly perform recall-reason cycles that produce biased and inconsistent results. When a human encounters a problem similar to one they have solved before, they recall their previous solution or conclusion directly, rather than re-deriving it from first principles. Standard LLMs, lacking persistent memory of their own reasoning, must re-reason every time, and this repeated reasoning is susceptible to inconsistency—the same question posed in different contexts may elicit different reasoning chains and different conclusions.

TiM addresses this problem by storing *post-thinking results*—the conclusions and insights generated by the model’s reasoning process—rather than raw input history. The architecture operates in two phases. In the pre-response phase, the system recalls relevant thoughts from memory before generating a response, providing the model with the results of its own previous reasoning as additional context. In the post-response phase, after generating a response, the system extracts the key thoughts and insights from the reasoning process and updates memory with these new thoughts. Memory operations include insertion of new thoughts, forgetting of outdated thoughts, and merging of similar thoughts based on similarity computed via Locality-Sensitive Hashing, which provides

efficient approximate nearest-neighbor search for memory management.

The insight underlying TiM—that storing processed thoughts is more efficient and effective than storing raw observations—has a direct analogue in cognitive science. Human long-term memory does not store verbatim records of experience but rather stores the gist, the meaning, and the conclusions derived from experience. The levels-of-processing framework [131] established that deeper semantic processing produces more durable memory traces than shallow perceptual processing, a finding that supports TiM’s approach of storing the products of deep reasoning rather than the raw inputs to reasoning.

8.4.6 Self-Controlled Memory

[132] introduced Self-Controlled Memory (SCM), which addresses the specific problem of noise accumulation when external memory is injected into the LLM context. When information from an external memory store is retrieved and inserted into the context window, it may introduce irrelevant or outdated information that degrades rather than improves the model’s performance. SCM addresses this through a memory controller that determines when and how to use stored information, implementing a form of metacognitive control over memory access.

SCM employs two memory types per processing segment. Activation memory stores long-term historical information that has been consolidated from previous processing segments, representing the accumulated knowledge relevant to the current task. Flash memory stores short-term, real-time information from the immediately preceding segment, representing the most recent context. The memory controller, implemented as a meta-level decision-making component, evaluates whether retrieved memories are relevant to the current processing context and controls their injection into the model’s input. This plug-and-play design requires no fine-tuning and handles ultra-long texts through selective memory injection, achieving seventy-eight percent retrieval recall compared to forty-five percent for baseline approaches on long-context tasks.

The distinction between activation memory and flash memory in SCM parallels the distinction between long-term and short-term memory in the Atkinson-Shiffrin model, while the memory controller’s function of evaluating and selectively admitting external information into the processing context parallels the attentional gating function of the central executive in Baddeley’s working memory model. The key contribution of SCM is its demonstration that uncontrolled memory injection can be counterproductive, and that intelligent memory management requires not only effective storage and retrieval but also effective filtering and selection at the point of integration.

8.4.7 RecallM: Graph-Based Temporal Memory

[133] introduced RecallM, which departs from the dominant paradigm of vector-database-based memory by using a neuro-symbolic graph database for relational modeling. The key insight motivating this approach is that vector databases, while effective for semantic similarity search, cannot

naturally represent relationships between concepts, temporal ordering of events, or the revision of beliefs over time. A graph database, by contrast, can explicitly represent entities, relationships, and temporal information, enabling more sophisticated reasoning about the structure and dynamics of stored knowledge.

RecallM’s key advantages include advanced relationship modeling between concepts, explicit temporal understanding that tracks when information was learned and how beliefs have changed over time, and one-shot belief updating that allows the system to revise prior beliefs based on new evidence. The one-shot belief updating capability is particularly significant: in a graph database, contradictory information can be resolved by updating relationship edges rather than by attempting to reconcile conflicting entries in a vector store. A hybrid version combining graph and vector databases (Hybrid-RecallM) leverages the complementary strengths of relational and semantic representations, using the graph database for structured knowledge and belief management and the vector database for flexible semantic retrieval.

RecallM’s approach has connections to the semantic network models of human memory that were influential in cognitive psychology during the 1960s and 1970s, particularly the spreading activation model of [134]. In these models, concepts are represented as nodes in a network, with relationships represented as edges, and memory retrieval operates through a process of spreading activation from cue nodes to target nodes. RecallM’s graph database implements a computational version of this spreading activation process, with explicit representation of the relational structure that underlies semantic knowledge.

8.4.8 RET-LLM: Triplet-Based Knowledge

[135] implemented a read-write memory system for LLMs based on semantic triplets (subject-predicate-object), grounded in Davidsonian event semantics. Davidsonian semantics represents events as logical entities with participants and properties, providing a formal framework for decomposing complex statements into structured, composable units. By storing knowledge as triplets, RET-LLM achieves four properties that distinguish it from opaque vector stores: scalability (the memory grows naturally with new information), aggregatability (related facts can be combined and summarized), updatability (existing knowledge can be revised when new information contradicts it), and interpretability (the stored knowledge is in human-readable form). A memory controller manages the lifecycle of stored knowledge, handling extraction of triplets from text, lookup of relevant triplets given a query, and generation of responses grounded in retrieved factual triplets. RET-LLM achieved superior performance on temporal question answering tasks, where the ability to track when facts were learned and how they have changed is essential for accurate responses.

8.4.9 RAISE: Dual-Component Memory for Conversational Agents

[136] enhanced conversational agents with a dual-component memory architecture that mirrors the human STM/LTM distinction. Dialogue-level memory maintains the current conversation history

and a scratchpad for intermediate reasoning, functioning as a form of working memory that supports ongoing processing. Turn-level memory stores query-response examples and task trajectories from previous interactions, functioning as a form of long-term episodic and procedural memory that enables the agent to draw on past experience when handling new situations. Fine-tuning the agent on domain-specific examples enhances the controllability and efficiency of memory operations, making the approach particularly effective in specialized applications such as real estate sales, customer service, and technical support, where the ability to recall and apply domain-specific knowledge from previous interactions is essential for effective performance.

8.5 Transformer Attention as Working Memory Analogue

The relationship between transformer attention and human working memory extends beyond loose analogy to reveal deep structural and functional parallels that illuminate both biological and artificial cognition.

8.5.1 Cue-Based Retrieval and Attention

Transformer attention implements a form of cue-based retrieval that closely parallels the associative retrieval mechanisms described in cognitive models of human memory. In the attention mechanism, queries serve as retrieval cues and keys serve as memory traces, with the attention weight between a query-key pair reflecting the degree of match between the cue and the stored trace. This parallels the content-addressed, associative retrieval that characterizes human memory, where memories are accessed through contextual cues rather than through fixed addresses. Research by [137] has explored these parallels in detail, asking whether attention provides a plausible cognitive model of human memory retrieval and what constraints this imposes on the nature of the underlying memory representations.

Attention weights have been shown to track relational complexity in ways that parallel the working memory demands of human cognitive tasks. When transformers process sentences with complex relational structures (e.g., nested relative clauses or multiple interacting entities), the pattern of attention weights reflects the complexity of the relational processing required, serving as a computational analogue of working memory load. Neuroimaging studies have identified structural correspondences between transformer components and brain regions: positional encoding representations align with visual cortex representations of spatial structure, while attention head activation patterns map onto the frontoparietal and default-mode networks that are associated with attentional control and memory retrieval in human neuroimaging studies [138].

8.5.2 TransformerFAM and Feedback Attention

[139] introduced TransformerFAM (Feedback Attention Memory), which explicitly models feedback loops within the attention mechanism to simulate working memory through iterative refinement. Standard transformer attention is a feedforward process: information flows from input to output

in a single pass through each layer. TransformerFAM adds recurrent feedback connections that allow the output of attention in one pass to influence the attention computation in subsequent passes, creating a form of iterative refinement that maintains and updates internal representations over multiple processing cycles. This recurrent processing within attention layers provides closer alignment with cognitive theories of active maintenance, where working memory contents are not passively stored but actively refreshed and updated through recurrent neural activity.

Hebbian Learning and Attention.. Research by [140] has demonstrated that short-term Hebbian learning—a form of synaptic plasticity in which connections between simultaneously active neurons are strengthened—can implement computations that are mathematically equivalent to transformer-like attention. This finding suggests that the computational principle underlying both biological and artificial attention may be fundamentally similar: both implement a form of content-addressed associative retrieval through temporary modifications of connection strengths. If this interpretation is correct, it provides a principled biological foundation for the use of attention as a working memory mechanism in artificial systems, and suggests that the capacity limitations observed in both human working memory and transformer attention may reflect deep computational constraints inherent in content-addressed associative retrieval.

Disanalogies.. Important disanalogies between transformer attention and human memory retrieval must also be acknowledged. Transformer attention requires the quadratic computation of all pairwise similarities between queries and keys, a process with no clear biological counterpart—human memory retrieval does not require exhaustive comparison of a retrieval cue against all stored traces. Human memory retrieval is content-addressed and probabilistic, with retrieval success depending on encoding conditions, retrieval cue specificity, and interference from competing memories, while transformer attention is deterministic and exhaustive within its context window. Human memory exhibits encoding specificity effects, generation effects, and testing effects that have no clear analogues in standard attention mechanisms. These disanalogies define opportunities for architectural innovation: incorporating stochastic retrieval, encoding-specificity constraints, or retrieval-practice effects into attention mechanisms could yield systems that more closely approximate the functional properties of human memory.

8.6 Episodic versus Semantic Memory in LLMs

The Tulving distinction between episodic and semantic memory [105, 113] has profound implications for LLM design that extend beyond the theoretical framework reviewed above. This subsection examines the specific challenges that LLMs face with episodic memory and the solutions that have been proposed to address them.

8.6.1 The Episodic Memory Deficit in Standard LLMs

Standard LLMs encode semantic memory effectively: the vast knowledge base acquired during pre-training captures general truths, factual relationships, and linguistic patterns in the distributed representations of the model’s parameters. However, they are fundamentally deficient in episodic memory. They cannot remember specific interactions without external augmentation. They cannot track the temporal context of information—when something was said, in what order events occurred, or how beliefs have changed over time. They cannot maintain spatiotemporal grounding across sessions, creating the illusion of a continuous agent identity only within the boundaries of a single context window [114, 115].

Recent empirical work has systematically characterized these deficits. State-of-the-art models including GPT-4, Claude, and Llama 3.1 struggle with episodic memory tasks involving multiple related events, complex spatiotemporal relationships, and person-specific memories [115]. When multiple related events are presented, models have difficulty tracking causal chains and maintaining consistent temporal ordering. When person-specific information is involved, models frequently confuse who said what, merging information from different sources into an undifferentiated semantic representation. These failures are not merely limitations of context window size: they reflect the absence of dedicated episodic memory mechanisms that would maintain the temporal, contextual, and source-specific information that distinguishes episodic from semantic memory.

8.6.2 Architectural Solutions for Episodic Memory

Several architectural approaches have been proposed to address the episodic memory deficit. External episodic memory systems, exemplified by EM-LLM [116] and the memory stream of Generative Agents [129], provide explicit storage for temporally grounded events with associated metadata (timestamps, source information, importance scores). These systems implement computational versions of the episodic memory system described by Tulving, with retrieval mechanisms that weight recency, relevance, and importance in ways that parallel the cue-dependent, context-sensitive retrieval that characterizes human episodic memory.

Explicit temporal tagging and event segmentation provide another approach. EM-LLM [116] uses Bayesian surprise combined with graph-theoretic boundary refinement to segment continuous experience into discrete events, paralleling the event segmentation processes that cognitive scientists have identified in human perception and memory. This segmentation enables temporally contiguous retrieval: when a relevant event is identified, surrounding events can also be retrieved, capturing the temporal context that is essential for episodic memory function. Structured retrieval mechanisms that combine multiple signals—recency, relevance, and importance—outperform any single retrieval signal, as demonstrated by both Generative Agents and EM-LLM. This multi-signal approach reflects the cognitive science finding that human memory retrieval is influenced by multiple factors simultaneously, including recency, emotional significance, and contextual similarity.

The integration of episodic and semantic memory represents the most ambitious research direction. The ideal architecture would support both the storage and retrieval of specific episodes (who said what, when, in what context) and the gradual consolidation of episodic information into semantic knowledge (general patterns, user preferences, domain expertise). This integration mirrors the biological process by which episodic memories are gradually consolidated into semantic knowledge through hippocampal-cortical replay, and it requires mechanisms for identifying when episodic information has been sufficiently consolidated to be represented more efficiently in semantic form.

8.7 Continual Learning, Catastrophic Forgetting, and Long-Term Memory

The challenge of maintaining stable long-term knowledge while acquiring new information connects directly to the CLS framework reviewed above. This subsection examines catastrophic forgetting as the computational manifestation of the interference problem that motivated the evolution of complementary learning systems in biological brains, and reviews the strategies that have been developed to mitigate it.

8.7.1 The Nature and Severity of Catastrophic Forgetting

When neural networks are trained on new tasks or data, they often catastrophically forget previously learned information [141]. This phenomenon, known as catastrophic forgetting or catastrophic interference, arises because gradient-based learning updates parameters to optimize performance on the current training objective, potentially overwriting the parameter configurations that were optimal for previous objectives. The mathematical basis of the problem is straightforward: the update rule $w_{\text{new}} = w_{\text{old}} - \eta \nabla L(w_{\text{old}})$ directly modifies the weights that encoded previous knowledge, and there is no intrinsic mechanism to preserve the information content of the original configuration while adapting to new requirements.

In large language models, catastrophic forgetting manifests through several interrelated mechanisms. Gradient interference in attention weights disrupts the carefully calibrated attention patterns that enable effective processing of previously learned tasks. Representational drift in intermediate layers shifts the internal representations away from configurations that were effective for prior knowledge. Loss landscape flattening around prior task minima reduces the model’s ability to maintain sharp, discriminative representations for old tasks while learning new ones. Empirical studies have documented the severity of these effects: [141] found that the severity of catastrophic forgetting increases with model size in the 1B to 7B parameter range, with fifteen to twenty-three percent of lower-layer attention heads showing severe disruption during continual fine-tuning. This scale-dependent effect is particularly concerning because it suggests that the problem will not be solved merely by scaling models to larger sizes.

The relationship between catastrophic forgetting and memory consolidation is fundamental. Catastrophic forgetting is, in essence, the computational manifestation of the problem that motivated the evolution of complementary learning systems in biological brains. LLMs lack the biological

mechanisms—separate fast-learning and slow-learning systems, sleep-dependent consolidation, interleaved replay—that prevent interference in human memory. This absence makes them vulnerable to forgetting in ways that humans are not, and addressing this vulnerability requires implementing computational analogues of the consolidation mechanisms that biological evolution developed to solve the same problem.

8.7.2 Parameter-Efficient Fine-Tuning and LoRA

Low-Rank Adaptation (LoRA) [142] represents one approach to reducing catastrophic forgetting by limiting the magnitude and scope of parameter changes during fine-tuning. LoRA freezes the pre-trained model weights and injects small, trainable low-rank matrices into the model’s layers. The parameter update is decomposed as $\Delta W = BA$, where B is an $r \times d$ matrix and A is a $d \times r$ matrix, with the rank r typically set to 8, 16, or 32. This decomposition reduces the number of trainable parameters by approximately 10,000-fold compared to full fine-tuning and reduces GPU memory requirements by approximately 3-fold, enabling fine-tuning of models with tens of billions of parameters on a single GPU.

However, recent research has revealed that LoRA alone does not prevent catastrophic forgetting. While LoRA reduces forgetting compared to full fine-tuning, it still suffers from significant knowledge degradation: empirical studies have shown that standard LoRA fine-tuning can result in seventy-one percent loss of original task knowledge, compared to eighty-nine percent for full fine-tuning. Advanced variants such as OPLoRA (Orthogonal Projection LoRA), which constrains updates to be orthogonal to the most important directions of the original weight matrices, and CL-LoRA (Continual Learning LoRA), which maintains shared adapter layers with task-specific gating mechanisms, have achieved substantially better results, reducing forgetting to approximately five to forty-five percent depending on the approach and evaluation conditions.

The relationship between LoRA and biological memory consolidation is instructive. LoRA acts as a “minimal perturbation” to parametric memory—a small, structured modification that preserves most of the original knowledge while allowing targeted adaptation. This parallels the biological principle that synaptic modifications during learning should be minimal and targeted, changing only the connections necessary for new learning while preserving the broader network structure. However, LoRA lacks the temporal dynamics of biological consolidation: it applies all modifications simultaneously rather than gradually integrating new knowledge through interleaved replay, and it has no mechanism for identifying and protecting the most important parameters for previously learned tasks.

8.7.3 Experience Replay

Experience replay maintains buffers of previous training examples and interleaves them with new training data, directly paralleling hippocampal replay in biological memory consolidation. The connection to [120] is explicit: replay prevents catastrophic interference by ensuring that the neocortical

(parametric) learning system is exposed to a mixture of old and new experiences, rather than being trained exclusively on new data that would overwrite old knowledge. Sparse memory fine-tuning techniques have demonstrated that storing and replaying only two percent of original training data during new task learning can reduce catastrophic forgetting from seventy-one percent (with LoRA alone) to eleven percent, while maintaining equivalent new task learning. This dramatic improvement with minimal additional storage supports the CLS prediction that interleaved replay is the most effective mechanism for preventing catastrophic interference.

8.7.4 Elastic Weight Consolidation

Elastic Weight Consolidation (EWC) [143] implements a regularization approach inspired by synaptic consolidation in biological neural networks. EWC computes the Fisher information matrix for each parameter after learning a task, identifying which parameters are most important for that task’s performance. During subsequent learning, a quadratic penalty prevents these important parameters from changing significantly:

$$L_{\text{total}}(w) = L_{\text{new}}(w) + \frac{\lambda}{2} \sum_i F_i (w_i - w_i^*)^2 \quad (9)$$

where w_i^* are the optimal parameters for the previous task, F_i is the Fisher information for parameter i , and λ controls the strength of the consolidation penalty. This approach is “elastic” in the sense that it allows some parameter movement (unlike rigid freezing) but penalizes movement in proportion to the parameter’s importance for previously learned tasks.

The biological analogue of EWC is synaptic tagging and capture, a mechanism by which recently active synapses are marked (tagged) for consolidation, making them resistant to modification during subsequent learning. Both EWC and synaptic tagging implement the same principle: not all connections are equally important, and consolidation should selectively protect the most critical ones while leaving others available for new learning. The limitation of EWC is that the Fisher information approximation becomes increasingly inaccurate as the number of sequential tasks grows, and the cumulative regularization can eventually prevent any further learning.

8.7.5 Dynamic Architectures

Dynamic architectures, such as progressive neural networks, address catastrophic forgetting by adding new capacity for new tasks without modifying existing parameters. New lateral connections allow the new capacity to access features learned by previous modules, enabling knowledge transfer without interference. This approach has a biological analogue in neurogenesis—the creation of new neurons in the hippocampus throughout life—which has been proposed as a mechanism for maintaining the ability to form new memories without disrupting existing ones. While dynamic architectures eliminate catastrophic forgetting entirely, they incur growing computational and memory costs as the number of tasks increases, making them impractical for systems that must learn

many sequential tasks.

8.8 Sleep-Inspired Consolidation for Artificial Neural Networks

The biological mechanisms of sleep-dependent consolidation reviewed above have inspired a nascent but potentially transformative research direction: the development of sleep-like offline processing phases for artificial neural networks.

8.8.1 NeuroDream: Explicit Dream Phases

[144] introduced NeuroDream, which adds explicit “dream phases” to neural network training. During dream phases, the model disconnects from input data and engages in internal simulations: it samples from learned latent representations, generates synthetic experiences using a learned dynamics model, and consolidates knowledge through Hebbian-like updates that strengthen important patterns without re-exposure to raw training data. The dream phase comprises four key components. A latent embedding space provides compressed representations of past experiences. A dynamics model learns temporal patterns and predicts state transitions. A dream generator samples from the latent space to produce synthetic experiences. A consolidation objective maximizes the diversity and quality of dream experiences while strengthening generalizable patterns.

The results of NeuroDream experiments demonstrate meaningful reductions in catastrophic forgetting—from forty percent forgetting without dream phases to fifteen percent with them—at a cost of approximately twenty percent additional training time. These results, while preliminary, suggest that offline consolidation phases can address the fundamental limitation of online-only learning in artificial systems. The mechanism parallels biological sleep consolidation in several respects: the disconnection from external input prevents interference between consolidation and new learning, the replay of stored representations provides the interleaved exposure that the CLS framework identifies as essential for preventing catastrophic interference, and the abstraction process that operates over replayed experiences extracts generalizable patterns from specific episodes.

8.8.2 Sleep Replay Consolidation

Research on sleep-like unsupervised replay has demonstrated that interleaving periods of offline replay between active learning phases can preserve task knowledge to a remarkable degree. In controlled experiments, networks trained on two sequential tasks showed dramatic forgetting of the first task without sleep replay (accuracy dropping from ninety-two percent to eighteen percent), but maintained near-original performance with sleep replay (eighty-six percent, compared to ninety-one percent for the original task and eighty-three percent for the new task). The sleep replay mechanism involves disconnecting from external data, reactivating stored memory traces through pattern replay, applying Hebbian consolidation to strengthen important connections, and reorganizing representations for improved abstraction. These results align with the biological finding that sleep

enhances memory consolidation by twenty-five to forty percent compared to equivalent periods of wakefulness.

8.8.3 Computational Models of Hippocampal-Neocortical Consolidation

[125] developed computational models that simulate the autonomous interactions between hippocampus and neocortex during sleep-dependent consolidation. These models demonstrate that offline learning through simulated replay can build structured representations via interleaved replay, with different simulated sleep stages serving complementary consolidation roles. NREM-like phases, characterized by slow oscillatory dynamics, support the transfer of specific episodic information from hippocampal to cortical representations. REM-like phases, characterized by theta-frequency dynamics, support the integration of transferred information into existing semantic structures and the extraction of abstract patterns. The alternation between these phases—mirroring the ultradian cycling of NREM and REM sleep in biological systems—produces more robust consolidation than either phase alone, suggesting that the temporal organization of consolidation processes is as important as the processes themselves.

These sleep-inspired approaches represent a research direction that could fundamentally change how LLMs are trained and deployed. If offline consolidation phases become standard in LLM training pipelines, they could address the fundamental limitation of catastrophic forgetting while enabling more human-like knowledge integration. The key challenge is computational: current LLMs require enormous resources for training, and adding offline consolidation phases would increase training costs. However, if consolidation phases reduce the total amount of training data required (by enabling more efficient knowledge integration) or improve the quality of the resulting models (by producing more robust, less interference-prone representations), the additional cost may be justified.

8.9 Infinite Context and Unbounded Memory Architectures

The desire to overcome the fundamental capacity limitations of the context window has motivated several architectures that aim to provide effectively unbounded context processing while maintaining computational feasibility.

8.9.1 Infini-Attention

[145] introduced Infini-Attention, which augments standard attention with a compressive memory layer that enables processing of contexts exceeding one million tokens with 114-fold less memory than standard attention. The architecture maintains a compressed memory state alongside the standard attention computation, allowing the model to retain information from the entire preceding context in a fixed-size compressed representation while performing full attention within a local window. The system demonstrated the ability to retrieve specific information (such as randomly inserted numbers) from million-token contexts and to summarize 500,000-token books effectively, performance levels that are impossible for standard attention architectures at these scales.

Infini-Attention’s compressive memory layer functions as a form of long-term memory that operates alongside the working memory of the local attention window. The compression process that maps detailed local representations into the fixed-size memory state parallels the consolidation processes that transform detailed episodic memories into more compact semantic representations in biological memory. The challenge is in the quality of compression: information that is compressed into the memory state may lose the fine-grained detail that would be necessary for certain retrieval tasks, analogous to the loss of episodic detail that occurs as biological memories are consolidated from hippocampal to cortical representations.

8.9.2 EM-LLM: Human-Inspired Episodic Memory for Infinite Context

[116] developed EM-LLM, which integrates human-inspired episodic memory and event cognition principles to enable unbounded context processing without fine-tuning. EM-LLM’s approach is grounded in the cognitive science of event segmentation, which has established that humans naturally segment continuous experience into discrete events at points of significant change in the ongoing activity.

EM-LLM implements event segmentation using a combination of Bayesian surprise (detecting points where the incoming information significantly deviates from predictions based on the preceding context) and graph-theoretic boundary refinement (adjusting event boundaries to maximize within-event coherence while minimizing between-event similarity). This produces a segmentation of the input text into coherent episodic units that can be individually stored, indexed, and retrieved. Retrieval operates in two stages: similarity-based retrieval identifies the most relevant events for a given query, and temporally contiguous retrieval expands the retrieved context to include events adjacent to the retrieved events, capturing temporal context.

EM-LLM demonstrated remarkable performance: it surpassed full-context models on most evaluation tasks and successfully retrieved relevant information across ten million tokens without any fine-tuning. The system’s ability to operate at this scale while maintaining retrieval quality demonstrates that human-inspired event segmentation and episodic memory mechanisms can provide practical solutions to the context scaling challenge. The event segmentation approach is particularly well-suited to the structure of many real-world LLM applications, where the input naturally decomposes into discrete episodes (conversation turns, document sections, task steps) that can be individually stored and retrieved.

8.9.3 Hierarchical Memory Transformer

The Hierarchical Memory Transformer (HMT), presented at NAACL 2025, implements a three-tier memory hierarchy within the transformer architecture itself. The architecture comprises a long-term memory tier that stores key-value embeddings from early input positions with sparse attention, a short-term working memory tier that maintains full self-attention within a sliding local window, and a current focus tier that generates predictions by attending across all memory tiers. This hierarchy

enables processing of sequences up to 32,000 tokens with 2.5 to 116-fold speedup compared to full attention, while maintaining generation quality comparable to full attention on short contexts and achieving superior quality on long contexts. The efficiency gains arise from replacing the quadratic cost of full attention with the combination of bounded quadratic cost within the local window and linear-cost sparse attention to the long-term memory tier.

8.9.4 Hierarchical Context Caching

Strata, a hierarchical context caching system for LLM serving, applies memory hierarchy principles at the infrastructure level. In production environments where LLM servers must maintain key-value caches for ongoing conversations, Strata implements a three-tier storage hierarchy: GPU VRAM (hot cache) for the most recent tokens with full precision and immediate access, CPU RAM (warm cache) for mid-range history in compressed form, and disk/NVMe storage (cold cache) for older context with high compression. A least-recently-used eviction policy promotes frequently accessed entries to faster tiers while demoting unused entries to slower, cheaper storage. This infrastructure-level memory hierarchy mirrors the cognitive memory hierarchy at a systems engineering level, demonstrating that the principles of hierarchical memory organization apply not only to the algorithms that process information but also to the systems that store and serve it.

8.10 Comparative Analysis: Human versus LLM Memory

A systematic comparison of human and LLM memory systems reveals both striking parallels and fundamental differences that inform the design of future architectures. The following analysis synthesizes findings from comparative studies [146, 147] with the theoretical frameworks reviewed above.

Property	Human Memory	LLM Memory
Sensory Memory	Large capacity; 100–500ms retention; acts as input gate	Token embeddings at API input; immediate processing without persistence
Working Memory	4–7 chunks; seconds to minutes; active manipulation	Context window (512–128K+ tokens); no inter-session persistence; quadratic attention cost
Long-Term Memory	Unlimited capacity; life-time; dynamic, reconstructive; susceptible to distortion	Parametric (fixed post-training) + non-parametric (external databases, RAG); static until re-training

Property	Human Memory	LLM Memory
Memory Formation	Encoding via attention and elaboration; hippocampal binding; consolidation over hours to years	Training (parametric) or direct storage in external memory (non-parametric); no consolidation process
Consolidation	Sleep-dependent; hippocampal replay; gradual neocortical integration; NREM/REM complementarity	Absent in standard LLMs; experimental in NeuroDream and sleep replay consolidation research
Forgetting	Exponential decay (Ebbinghaus); adaptive and importance-weighted; interference-based; retrieval failure	No intrinsic forgetting mechanism; catastrophic interference during fine-tuning; context window truncation
Episodic Memory	Temporally grounded events; auto-noetic consciousness; rich contextual detail; reconstructive	External systems (EM-LLM, Generative Agents); context-dependent; no phenomenal experience
Semantic Memory	General knowledge; noetic consciousness; conceptually stable across contexts	Parametric knowledge; static until re-training; context-dependent activation patterns
Retrieval	Cue-based; reconstructive; emotion-influenced; encoding specificity; testing effect	Attention-based (parametric) or similarity search (non-parametric); deterministic; prompt-sensitive
Statefulness	Continuous, persistent identity; autobiographical continuity	Stateless by default; each inference independent; memory systems approximate continuity
Selectivity	High; emotional significance modulates encoding via amygdala-hippocampal interaction	Low; all tokens receive equal computational resources; no intrinsic importance weighting

Property	Human Memory	LLM Memory
Metacognition	Calibrated judgments of learning; accurate confidence monitoring	Poorly calibrated; confidence-recall correlation near zero in empirical studies
Adaptability	Context-sensitive; continuous learning; personal biases shape recall	Static parametric knowledge; adaptable via external memory or fine-tuning
Knowledge Update	Continuous; synaptic plasticity; no catastrophic forgetting under normal conditions	Requires re-training or PEFT; catastrophic forgetting during continual learning
Computational Cost	Biological efficiency; parallel processing; approximately 20 watts	Quadratic attention complexity; high energy consumption; significant memory bottlenecks

Table 2: Comprehensive comparison of human memory and LLM memory systems, synthesizing findings across cognitive science and machine learning research.

8.10.1 Key Insights from the Comparison

Several insights emerge from this systematic comparison that have direct implications for LLM memory design.

The Statefulness Gap.. The most fundamental difference between human and LLM memory is statefulness. Human memory provides continuous, persistent identity across time—the sense of being the same person who went to sleep last night and woke up this morning. Standard LLMs are fundamentally stateless: each inference begins with a blank slate (or, at most, a context window filled with the current prompt), and there is no intrinsic mechanism for carrying information from one session to the next. Memory-augmented systems such as MemGPT and Generative Agents approximate statefulness by maintaining external stores that persist across sessions, but this approximation is qualitatively different from the continuous, embodied persistence of human identity. The statefulness gap is particularly significant for applications that require long-term agent deployment, where the agent must maintain consistent identity, accumulated knowledge, and relationship history across many interactions.

The Consolidation Gap.. Sleep-dependent consolidation is central to human memory but entirely absent in standard LLMs. The consequences of this gap are severe: without consolidation,

LLMs cannot gradually integrate new knowledge with existing representations, cannot extract abstract patterns from specific experiences, and cannot selectively strengthen important memories while allowing unimportant ones to fade. The experimental work on NeuroDream, sleep replay consolidation, and computational models of hippocampal-neocortical interaction represents the beginning of an effort to close this gap, but these approaches are far from practical deployment in large-scale LLM systems.

The Metacognition Gap.. Empirical research has demonstrated that LLMs exhibit near-zero correlation between their confidence judgments and their actual recall accuracy, in stark contrast to humans, who show strong correlations between judgments of learning and subsequent recall. This metacognitive deficit means that LLMs cannot reliably assess whether they “know” something or are confabulating, a limitation that has direct implications for the reliability of memory-augmented systems. If an LLM cannot distinguish between genuine memory retrieval and hallucinated information, the value of its memory system is fundamentally compromised.

The Selectivity Gap.. Human memory is highly selective, preferentially encoding emotionally significant, goal-relevant, and novel information through amygdala-hippocampal interactions. LLMs, by contrast, treat all information with approximately equal computational resources, lacking any intrinsic mechanism for importance-weighted encoding. Memory architectures that implement importance scoring (such as Generative Agents and MemoryBank) partially address this gap, but their importance ratings are assigned post hoc by the language model rather than being integrated into the encoding process itself.

Surprising Convergence.. Despite these profound differences, certain properties show surprising convergence. The effective working memory capacity of transformer attention (approximately four to seven actively considered items) is strikingly similar to the human working memory capacity measured by [110]. Both humans and LLMs exhibit serial position effects (primacy and recency). Both generate “false memories” (human memory distortion paralleling LLM hallucination). These convergences suggest that certain properties of memory systems may emerge from computational constraints that are shared between biological and artificial information processing, rather than being specific to either substrate.

8.11 Integrated Multi-Tier Memory Systems

Building on the individual architectures reviewed above, recent work has begun to develop integrated multi-tier memory systems that combine multiple cognitive-science-inspired principles into unified architectures.

8.11.1 Hierarchical Memory for Agents

Recent work on hierarchical memory for LLM agents has proposed four-layer memory structures that organize information by level of abstraction. A domain layer provides broad semantic categorization (work, personal, health). A category layer provides finer-grained sub-domain grouping within each domain. A memory trace layer stores abstracted summaries of key information. An episode layer preserves complete, detailed interaction records. This hierarchical organization enables top-down search strategies with logarithmic rather than linear complexity: a query about a work deadline can navigate from the “work” domain to the “projects” category to relevant memory traces without exhaustively scanning all stored memories.

This hierarchical organization mirrors the semantic network models of human memory that posit hierarchical categorical structures for organizing knowledge, and it implements a form of the chunking principle identified by [104]: by organizing memories into meaningful categories, the system effectively increases the amount of information that can be efficiently accessed despite limited processing resources.

8.11.2 Cognitive Workspace Theory Applied to LLMs

Inspired by Baars’s Global Workspace Theory of consciousness, the cognitive workspace approach treats the LLM’s context window as a limited-capacity workspace to which multiple cognitive modules compete for access. The workspace holds the current task goal, recent observations, retrieved relevant knowledge, and active reasoning traces, with a capacity of approximately 1,000 to 4,000 tokens. Multiple cognitive modules—reading, planning, memory, and execution modules—operate in parallel, with each module contributing information to the workspace and using workspace contents to guide its own processing. The workspace functions as a broadcast mechanism: information placed in the workspace becomes available to all modules simultaneously, enabling implicit coordination without explicit communication between modules.

This approach transcends traditional retrieval-augmented generation by treating memory not as a static store to be queried but as an active computational resource to be managed. The workspace metaphor shifts the focus from “what is stored” to “what is currently in the spotlight of attention,” a shift that aligns with the cognitive science understanding that working memory is not merely a passive buffer but an active processing system that shapes and transforms the information it contains.

8.11.3 Neuro-Symbolic Integration

The integration of symbolic knowledge representations (knowledge graphs, production rules, logical constraints) with neural language models represents a hybrid approach that leverages the complementary strengths of both paradigms. Symbolic representations provide interpretability, precise relational reasoning, and guaranteed consistency, while neural representations provide flexibility,

generalization, and natural language understanding. Hybrid architectures that combine a symbolic layer for structured knowledge and constraint enforcement with a neural layer for language understanding and flexible reasoning can achieve properties that neither layer provides alone: grounded generation that respects factual constraints, interpretable decision-making that can be audited and explained, and flexible reasoning that can handle novel situations not covered by explicit rules.

The connection to cognitive architecture is direct: the symbolic layer corresponds to the declarative and procedural memory systems of architectures like ACT-R and SOAR, while the neural layer corresponds to the pattern-recognition and association capabilities that connectionist models have long provided. The integration of these two layers—allowing symbolic knowledge to constrain neural generation while neural processing discovers new patterns to be represented symbolically—mirrors the interaction between explicit and implicit memory systems in human cognition.

8.12 Synthesis: Design Principles for Human-Inspired LLM Memory

The convergence of cognitive science and LLM research reviewed in this section yields a coherent set of design principles for the construction of effective, scalable, and human-like memory systems for large language models.

8.12.1 The Multi-Tier Imperative

Effective memory systems require, at minimum, three tiers of storage: a working memory tier (the context window) for immediate processing, an episodic memory tier (external storage with temporal and contextual metadata) for specific events and experiences, and a semantic memory tier (parametric knowledge and structured knowledge bases) for generalized facts and concepts. Single-tier designs—whether based on context window expansion, pure parametric knowledge, or undifferentiated external storage—are fundamentally limited because they cannot simultaneously satisfy the competing requirements of rapid access, temporal grounding, and stable, generalized knowledge representation. The biological evidence reviewed in this section strongly supports the multi-tier approach: evolution produced multiple memory systems with distinct temporal and representational properties precisely because no single system can meet all the requirements of intelligent behavior.

8.12.2 Selective Consolidation and Principled Forgetting

Not all information should persist indefinitely. Importance-weighted storage, as implemented by Generative Agents and MemoryBank, and temporal decay, as inspired by the Ebbinghaus forgetting curve, are essential mechanisms for preventing unbounded memory growth while preserving the information that is most likely to be relevant in the future. The principle of selective consolidation—that memory management should actively decide what to retain, what to compress, what to abstract, and what to forget—is supported by both the cognitive science of adaptive forgetting and the information-theoretic principle that optimal information retention under resource constraints requires selective rather than exhaustive storage. Forgetting is not a deficiency to be eliminated but

a feature to be designed, and effective LLM memory systems must implement principled forgetting mechanisms that balance the cost of storage and retrieval against the value of retained information.

8.12.3 Temporal Awareness

Timestamps, decay functions, and temporal ordering are critical for episodic memory function. Without temporal awareness, memory systems cannot distinguish between recent and remote information, cannot track the evolution of beliefs over time, and cannot implement the recency-weighted retrieval that is essential for contextually appropriate behavior. Systems such as RecallM, MemoryBank, and EM-LLM demonstrate the importance of temporal information for effective memory management, and the cognitive science evidence on the role of temporal context in human episodic memory strongly supports the integration of temporal awareness into LLM memory architectures.

8.12.4 Multi-Signal Retrieval

Combining recency, relevance, and importance in retrieval, as demonstrated by Generative Agents and EM-LLM, outperforms any single retrieval signal. This multi-signal approach reflects the cognitive science finding that human memory retrieval is influenced by multiple factors simultaneously, and it provides robustness against the failure of any individual signal. A memory that is old but highly important and relevant to the current query should be retrieved, just as a memory that is recent but unimportant and irrelevant should not be. Multi-signal retrieval enables this kind of flexible, context-sensitive access to stored information.

8.12.5 Dynamic Belief Revision

Memory systems must support the revision of stored knowledge when new information contradicts previous beliefs. Systems such as RecallM and RET-LLM demonstrate that knowledge representation formats that support explicit belief revision (graph databases, semantic triplets) are more effective for dynamic knowledge management than formats that only support accumulation (vector databases, unstructured text). The ability to update, revise, and correct stored knowledge is essential for agents operating in dynamic environments where facts change over time, and it parallels the human capacity for belief revision in the face of new evidence.

8.12.6 Reflection and Abstraction

Higher-order processing that generates summaries, insights, and abstractions from accumulated experience, as demonstrated by Generative Agents and Think-in-Memory, dramatically improves agent performance. Reflection transforms raw episodic data into actionable knowledge, reducing the volume of information that must be searched during retrieval while preserving and enhancing its informational value. The cognitive science analogue is the consolidation process that transforms specific episodic memories into generalized semantic knowledge, and the evidence reviewed in this

section suggests that this transformation is not merely a storage optimization but a fundamental cognitive operation that enables higher-level reasoning, planning, and decision-making.

8.12.7 Controlled Memory Integration

Memory controllers that evaluate the relevance of retrieved information before injecting it into the context, as implemented by SCM and MemGPT, are necessary for preventing noise accumulation during long-term operation. The injection of irrelevant or outdated information into the context window can degrade performance rather than improve it, and intelligent filtering at the point of integration is as important as effective storage and retrieval. This principle parallels the attentional gating function of the central executive in Baddeley’s working memory model, which selectively admits information to working memory based on its relevance to current processing goals.

8.12.8 Event Segmentation

Chunking continuous experience into meaningful episodes, as implemented by EM-LLM and motivated by CoALA, enables efficient retrieval by providing natural units of storage and search. Event segmentation mirrors the event perception processes that cognitive scientists have identified in human experience, where continuous sensory input is automatically segmented into discrete events at points of significant change. By storing memories as coherent episodic units rather than as undifferentiated streams of tokens, memory systems can retrieve contextually complete episodes that provide the temporal and situational context necessary for informed decision-making.

8.13 Future Directions and Open Problems

The synthesis of cognitive science and LLM memory design reviewed in this section points to several critical open problems and research directions that will shape the next generation of memory-augmented language systems.

Biological Plausibility.. The gap between biologically-inspired design principles and their computational implementations remains wide. While systems like MemoryBank implement simplified versions of the Ebbinghaus forgetting curve and Generative Agents implement a form of reflection that parallels consolidation, no existing system implements the full complexity of biological memory consolidation, including the coordinated oscillatory dynamics of sleep, the complementary roles of NREM and REM sleep, or the pattern separation and completion mechanisms of hippocampal circuits. Closing this gap will require closer collaboration between cognitive neuroscientists and machine learning researchers.

Scalable Consolidation.. If offline consolidation phases are to become practical for large language models, significant advances in computational efficiency will be required. Current sleep-inspired approaches add twenty to fifty percent to training time, which is prohibitive at the scale of

frontier LLMs. Developing consolidation mechanisms that operate efficiently within existing training pipelines, or that can be applied selectively to the most important information, is an essential research challenge.

Unified Memory Architectures.. No existing system seamlessly integrates parametric semantic memory, external episodic memory, and procedural learning within a single coherent framework. CoALA provides the conceptual vocabulary for such integration, but practical implementations remain fragmented across different systems with different storage formats, retrieval mechanisms, and update procedures. Developing unified architectures that support the full range of memory operations—encoding, consolidation, retrieval, updating, and forgetting—across multiple memory types is perhaps the most important challenge for the field.

Metacognitive Monitoring.. The near-zero correlation between LLM confidence and recall accuracy identified in empirical studies represents a fundamental limitation for memory-augmented systems. If an agent cannot assess whether a retrieved memory is accurate, the reliability of its memory-informed decisions is fundamentally compromised. Developing mechanisms for metacognitive monitoring—enabling LLMs to assess the reliability of their own memories and retrieval processes—is essential for trustworthy deployment of memory-augmented agents.

Episodic Richness.. Current external episodic memory systems store relatively impoverished representations of events compared to the rich, multimodal, emotionally tagged episodic memories that humans maintain. Capturing the full richness of episodic experience—including emotional valence, sensory detail, spatial context, and social dynamics—in a computationally tractable format remains an open challenge.

These principles and open problems suggest that the most effective context management systems will not merely extend context windows or add external storage. They will implement biologically-inspired hierarchical memory architectures with principled consolidation, retrieval, and forgetting mechanisms that draw on the deep insights that cognitive science has accumulated over more than a century of research on the nature and function of human memory.

9 Hallucination and Context Distance Effects

Hallucination—the generation of plausible yet incorrect or unverifiable information—represents one of the most persistent challenges in deploying large language models for real-world applications. This phenomenon is intimately connected to context management, as the manner in which models access, prioritize, and utilize information within their context windows fundamentally determines their propensity to generate factually accurate or hallucinated content. This section examines the intricate relationships between context distance, positional biases, retrieval mechanisms, and factual accuracy, with particular emphasis on how architectural limitations and information organization

strategies influence hallucination rates.

9.1 Conceptual Foundations and Taxonomies

The systematic study of hallucination in natural language generation has evolved considerably over the past several years. [148] established a foundational taxonomy that organized hallucination phenomena across diverse NLG tasks, including abstractive summarization, dialogue generation, question answering, data-to-text generation, machine translation, and vision-language tasks. This comprehensive survey provided the field with standardized terminology and evaluation frameworks, enabling more rigorous comparative analysis across different model architectures and application domains. Building upon this foundation, [149] introduced a refined taxonomy specifically tailored to large language models, identifying contributing factors across three critical stages: data collection and curation, model training procedures, and inference-time generation processes. This holistic perspective revealed that hallucinations arise not from a single failure mode but from complex interactions between training data quality, architectural design choices, and deployment configurations.

The distinction between intrinsic and extrinsic hallucination, introduced by [150] in their seminal work on abstractive summarization, has become central to understanding hallucination mechanisms. Intrinsic hallucination occurs when generated content directly contradicts information present in the source material—for instance, when a summary states that an event failed despite the source document describing its success. This form of hallucination represents a clear failure of faithfulness to the provided context. Extrinsic hallucination, by contrast, manifests when the model generates content that cannot be verified from the source material, introducing claims or details absent from the input. While extrinsic hallucinations may sometimes be factually accurate when verified against external knowledge sources, they nonetheless represent a departure from strict adherence to the provided context. Through large-scale human evaluation on the XSum dataset, [150] demonstrated that all neural abstractive summarization systems exhibit both hallucination types, with the balance between them varying by model architecture and training approach.

A critical conceptual distinction that has emerged from recent research separates source faithfulness from world factuality. Source faithfulness measures the consistency between generated output and the specific input context provided to the model, focusing on whether the generation accurately reflects the information present in retrieved passages, user documents, or conversation history. World factuality, conversely, assesses alignment with verifiable real-world knowledge independent of the immediate input context. These two dimensions can conflict in complex ways: a model might faithfully reproduce an error present in its source material, achieving high faithfulness but low factuality, or conversely generate factually correct information that was never mentioned in the source, achieving high factuality but low faithfulness. Recent empirical work has revealed that optimizing for one dimension can degrade performance on the other, with some interventions targeting factual accuracy causing severe declines in context-faithfulness by as much as sixty-nine percent. This tension necessitates careful consideration of application requirements when designing

hallucination mitigation strategies, as different tasks prioritize these dimensions differently.

9.2 Context Distance and Positional Bias Phenomena

The relationship between information position within context windows and model performance has emerged as a fundamental architectural limitation with profound implications for hallucination. [7] conducted systematic controlled experiments that revealed what they termed the “lost in the middle” phenomenon: when relevant information appears in the middle sections of long input contexts rather than at the beginning or end, model performance degrades substantially. Testing state-of-the-art models including GPT-3.5-Turbo, Claude-1.3, MPT-30B-Instruct, and LongChat-13B on multi-document question answering and key-value retrieval tasks, they observed a characteristic U-shaped performance curve. Performance peaks when critical information appears at context boundaries but drops precipitously for middle-positioned content, with degradations exceeding twenty percent in the most severe cases. Remarkably, they found that adding more context documents can actually hurt performance below closed-book baselines when critical information ends up in middle positions, fundamentally challenging the assumption that providing more context necessarily improves generation quality.

This positional bias manifests through distinct primacy and recency effects that mirror cognitive phenomena observed in human memory systems. Primacy effects cause models to disproportionately attend to and favor information presented early in the context window, while recency effects similarly privilege information appearing near the end. Empirical analysis across multiple model families reveals that these biases are systematic and quantifiable, with primacy effects influencing approximately ten percent of model decisions on average, though effect magnitudes vary considerably by task type and model architecture. ChatGPT and GPT-4 variants exhibit dominant primacy effects, GPT-J shows stronger recency bias, while GPT-3.5 displays both effects with primacy typically dominating. These patterns are not merely statistical artifacts but appear to be deeply embedded in model architectures through several mechanisms: positional encoding schemes that shape how models represent sequence position, attention mechanisms that systematically emphasize boundary tokens over middle positions, and the attention sink phenomenon whereby initial tokens serve as computational anchors that receive persistent elevated attention weights across all layers.

The interaction between context length and positional bias follows complex nonlinear patterns. In short contexts under one thousand tokens, position effects remain moderate as all content remains relatively accessible. However, as context length extends into the one thousand to ten thousand token range, position effects become pronounced and the U-shaped performance curve becomes clearly visible. For extremely long contexts exceeding ten thousand tokens, middle-content degradation intensifies further, with some evidence suggesting that recency effects may eventually dominate over primacy in these regimes. This context-length dependency has critical implications for retrieval-augmented generation systems, which must carefully consider not only which documents to retrieve but also how to order them to avoid placing critical evidence in regions subject to severe positional

bias.

Encoder-decoder architectures demonstrate somewhat greater robustness to positional effects compared to decoder-only models, particularly when evaluated on sequences shorter than their training maximum length. The bidirectional attention in the encoder component appears to provide some immunity to position bias. However, this advantage diminishes for sequences that exceed training lengths or approach maximum capacity, suggesting that architectural differences provide only partial mitigation rather than fundamental solutions. Decoder-only models, which have become dominant in recent large language model development, show particularly pronounced sensitivity to position, with U-shaped degradation curves appearing consistently across model families and scales. Even models explicitly designed and trained for long-context processing fail to eliminate these effects, indicating deep-seated limitations in how transformer-based architectures process sequential information.

9.3 Fidelity Gradients and Multi-Tier Memory Architectures

The lost-in-the-middle phenomenon necessitates a paradigm shift from conceptualizing context management as primarily about context capacity to understanding it as fundamentally about context fidelity. The ability to fit large amounts of information into a context window provides little practical benefit if the model cannot reliably access and utilize that information during generation. This insight has motivated exploration of multi-tier memory architectures inspired by human cognitive systems, which maintain distinct memory stores with different characteristics. Short-term memory corresponds to the direct context window with immediate but capacity-limited access. Long-term memory maps to external databases and vector stores that offer effectively unlimited capacity but require explicit retrieval operations. Working memory emerges from the combination of the active context window and strategically retrieved long-term memory elements, creating a dynamically managed information space that adapts to task requirements.

Intelligent decay and persistence policies play crucial roles in these multi-tier architectures. Time-based decay mechanisms reduce the salience of older information, ensuring that temporal factors influence retrieval ranking and that stale information does not crowd out fresh content. Event-driven forgetting removes or deprecates entries when newer, more relevant information becomes available, preventing the accumulation of obsolete facts that could lead to hallucinations based on outdated knowledge. Importance-based persistence provides countervailing forces that retain high-value or frequently accessed information despite age, ensuring that foundational knowledge remains available. Recency-weighted scoring in retrieval operations applies temporal decay factors after initial similarity matching, effectively boosting the ranking of temporally relevant content. These mechanisms collectively address the challenge that raw similarity matching may surface outdated information that scores highly on semantic relevance but poorly on temporal appropriateness.

The trade-offs inherent in multi-tier architectures must be carefully managed. Continuum Memory Architecture approaches that integrate multiple memory tiers demonstrate advantages on tasks

requiring persistence of information across long interactions, association of related concepts across temporal distance, and maintenance of context fidelity despite growing conversation or document length. However, these benefits come at the cost of increased system complexity, additional latency from retrieval operations, and the need for careful tuning of decay and persistence parameters. Intelligent summarization offers a potential compromise, compressing older context into dense representations that preserve essential information while reducing token consumption, though this introduces its own risks of information loss during the compression process that could lead to hallucinations when the model lacks access to details that were summarized away.

9.4 Evaluation Benchmarks and Measurement Frameworks

Rigorous evaluation of hallucination requires carefully designed benchmarks that isolate specific failure modes and provide quantitative metrics for comparison across systems. [151] introduced TruthfulQA, a benchmark comprising 817 adversarially-crafted questions across 38 diverse categories including health, law, finance, and politics. These questions specifically target common misconceptions and false beliefs likely to appear in training data, testing whether models learn to generate truthful responses or merely reproduce human falsehoods encountered during pre-training. The benchmark employs both single-correct-answer and multiple-correct-answer formats, with a fine-tuned GPT-Judge model that predicts human truth evaluations with ninety to ninety-six percent accuracy. Initial evaluation revealed concerning inverse scaling relationships: larger models generally performed worse than smaller models, with the best-performing model achieving only fifty-eight percent truthfulness compared to ninety-four percent human performance. This inverse scaling suggests that naive scaling increases the capacity to memorize and reproduce falsehoods from training data without proportionately improving truthfulness.

[152] developed HaluEval, a large-scale benchmark comprising thirty-five thousand examples spanning general user queries and three task-specific domains: question answering, knowledge-grounded dialogue, and text summarization. Their two-step sampling-then-filtering framework leveraged ChatGPT to generate diverse responses followed by human annotation to identify hallucinated content, creating realistic distributions of hallucinations that reflect actual model behavior rather than synthetic error patterns. Evaluation across this benchmark revealed that ChatGPT generates hallucinated content in approximately nineteen and a half percent of responses on specific topics, with substantial variation across domains. Critically, the research demonstrated that providing external knowledge and eliciting explicit reasoning steps significantly improved models' ability to recognize hallucinations in their own outputs, suggesting that hallucination detection capabilities exist within models but require appropriate prompting to activate.

[153] introduced FActScore, a fine-grained atomic evaluation framework for assessing factual precision in long-form text generation. Rather than providing binary judgments of entire generations, FActScore decomposes generated text into atomic facts—individual claims about entities that can be independently verified—and computes the percentage of these atomic facts supported by a re-

liable knowledge source, defaulting to Wikipedia but supporting custom knowledge bases. The four-step pipeline comprising atomic fact generation, evidence retrieval, fact validation, and score computation achieves high agreement with human evaluators while enabling automated assessment at scale. Evaluation of ChatGPT on biography generation tasks revealed only fifty-eight percent factual precision, indicating that nearly half of the atomic facts in generated biographies could not be verified or were demonstrably false. This granular approach provides actionable feedback for model improvement by identifying specific types of facts where hallucinations concentrate.

[154] contributed ALCE (Automatic LLMs' Citation Evaluation), the first comprehensive benchmark for evaluating citation generation in retrieval-augmented systems. Comprising three datasets—ASQA for aspect-based summary question answering, QAMPARI for multi-entity relational questions, and ELI5 for long-form explanations—ALCE evaluates systems across three complementary dimensions: fluency of generated text, correctness of factual claims, and quality of citations linking claims to source passages. The evaluation framework assesses citation precision (whether cited passages actually support claims), citation recall (whether all claims receive appropriate citations), and citation accuracy (whether passage content aligns with associated claims). Empirical evaluation revealed that even the best-performing models lack complete citation support for fifty percent of generated content on the ELI5 dataset, highlighting substantial gaps between generation fluency and proper source attribution. The strong correlation between automated ALCE metrics and human judgments enables efficient large-scale evaluation while maintaining alignment with human quality assessments.

9.5 Mitigation Strategies and Their Limitations

Various mitigation strategies have been proposed to reduce hallucinations, each offering distinct benefits and confronting specific limitations. [155] introduced self-consistency as a decoding strategy that replaces greedy decoding in chain-of-thought prompting. The core insight is that complex reasoning problems typically admit multiple distinct reasoning paths that converge on the same correct answer. Rather than following a single reasoning chain, self-consistency samples diverse reasoning paths through temperature-based sampling, then selects the most consistent answer by marginalization over paths. Empirical evaluation demonstrated substantial improvements: seventeen point nine percent gains on GSM8K mathematical reasoning, eleven percent on SVAMP, twelve point two percent on AQUA, six point four percent on StrategyQA, and three point nine percent on ARC-challenge. The magnitude of improvement correlates with task difficulty, with harder reasoning tasks showing larger benefits. However, self-consistency requires K independent samples for each query, multiplying inference costs proportionally. For typical deployments with K ranging from five to forty samples, this represents a substantial computational burden that must be weighed against accuracy improvements.

[156] proposed Chain-of-Verification (CoVe), a four-step process designed to leverage models' ability to detect errors in their own outputs. The process begins by drafting an initial response without

constraints, then plans verification questions to fact-check specific claims in that draft, answers those verification questions independently without reference to the draft to avoid confirmation bias, and finally generates a verified response incorporating only claims that withstood verification. Empirical evaluation on closed-book question answering demonstrated a twenty-three percent improvement in F1 score, with hallucination reductions of fifty to seventy percent across list-based questions, multi-span QA, and long-form generation tasks. The independence of verification from the original draft proves critical, as it prevents the model from unconsciously confirming its initial beliefs. However, CoVe at minimum doubles inference cost due to the multiple generation passes required, and verification quality depends on the model’s own knowledge, creating potential circular dependencies where a model with flawed knowledge generates both flawed drafts and flawed verifications.

Retrieval-augmented generation has emerged as perhaps the most widely adopted hallucination mitigation strategy, grounding generation in external evidence retrieved from document collections or knowledge bases. The integration of real-time information from authoritative sources constrains generation to claims supportable by retrieved evidence, reducing the model’s reliance on potentially flawed or outdated pre-trained knowledge. Dense passage retrieval methods enable efficient semantic search over large corpora, while hybrid approaches combining dense retrieval with traditional sparse methods like BM25 improve coverage. Multi-evidence approaches such as MEGA-RAG in public health applications retrieve information from multiple sources including dense vector stores, keyword-based indices, and structured knowledge graphs, then apply cross-encoder reranking to prioritize semantically relevant passages. Despite these sophisticated approaches, hallucinations persist in retrieval-augmented systems, with legal AI tools exhibiting hallucination rates of seventeen to thirty-three percent even with retrieval grounding. Several factors contribute to continued hallucinations: retrieved passages may contain conflicting information that models struggle to reconcile, relevant information may be positioned in middle sections subject to lost-in-the-middle effects, retrieved content may itself be inaccurate or outdated, and models may blend retrieved information with unverifiable claims from pre-trained knowledge.

The limitations observed across mitigation strategies reveal fundamental challenges. Self-consistency cannot improve beyond the maximum accuracy achievable by any individual reasoning path, failing entirely when all sampled paths lead to incorrect conclusions. Chain-of-Verification remains bounded by the model’s own knowledge and cannot detect hallucinations in domains where the model lacks accurate information to perform verification. Retrieval-augmented generation depends critically on retrieval quality and struggles when relevant information is absent from the corpus, when queries fail to retrieve pertinent passages, or when models preferentially attend to pre-trained knowledge over retrieved evidence. These limitations suggest that no single approach suffices; effective hallucination mitigation likely requires integrated strategies that combine retrieval grounding with verification mechanisms and architectural innovations addressing positional biases.

9.6 Synthesis and Future Directions

Hallucination in large language models emerges from the complex interplay of architectural constraints, training data characteristics, and inference-time context management. The lost-in-the-middle phenomenon and associated primacy-recency biases reveal fundamental limitations in how transformer architectures access information across long sequences, creating systematic fidelity gradients that render middle-positioned content substantially less influential than boundary information. These positional effects persist across model scales and architectures, suggesting deep-seated rather than superficial limitations. The distinction between faithfulness and factuality highlights that hallucination encompasses multiple distinct failure modes requiring different mitigation approaches: faithfulness failures indicate problems tracking and utilizing provided context, while factuality failures reflect gaps or errors in pre-trained knowledge.

Current evaluation frameworks provide complementary perspectives on hallucination. TruthfulQA reveals knowledge gaps and misconceptions learned from training data, HaluEval quantifies hallucination rates across diverse tasks, FActScore enables fine-grained factual precision measurement in long-form generation, and ALCE assesses the critical dimension of source attribution. Together, these benchmarks expose the multifaceted nature of hallucination and the inadequacy of any single metric to capture all aspects of the problem. The persistent gaps between human and model performance on these benchmarks—with models typically achieving fifty to sixty percent on dimensions where humans reach ninety percent or higher—underscore the magnitude of remaining challenges.

Mitigation strategies demonstrate varying degrees of effectiveness across different hallucination types and application contexts. Self-consistency and chain-of-verification offer inference-time improvements without requiring retraining, making them attractive for rapid deployment, but impose substantial computational costs and remain bounded by underlying model capabilities. Retrieval-augmented generation provides the most substantial hallucination reductions by grounding generation in authoritative external sources, yet hallucinations persist at non-negligible rates even with retrieval grounding, particularly when retrieved evidence contains conflicts, noise, or gaps. The interaction between retrieval-augmented generation and positional biases creates additional challenges, as multi-document retrieval must carefully manage document ordering to avoid critical evidence landing in middle positions subject to severe access penalties.

Future progress will likely require several complementary directions. Architectural innovations that achieve position-invariant attention or explicitly address middle-context degradation could fundamentally improve context fidelity. Multi-tier memory systems with sophisticated decay and persistence policies may better approximate human-like information management, maintaining fidelity across longer interactions. Enhanced retrieval methods that incorporate temporal factors, source reliability assessments, and multi-hop reasoning capabilities could improve evidence quality and reduce reliance on flawed sources. Verification mechanisms integrated directly into the generation process rather than applied post-hoc might catch hallucinations before they propagate through reasoning chains. Domain-specific grounding strategies adapted to particular application areas

could address the reality that hallucination patterns, acceptable error rates, and optimal mitigation approaches vary substantially between casual conversation, legal analysis, medical diagnosis, and other use cases. Most fundamentally, the field must continue developing evaluation frameworks that respect the faithfulness-factuality distinction, measure hallucinations at appropriate granularities for different applications, and enable systematic comparison of mitigation strategies across diverse contexts.

The persistent nature of hallucinations despite context expansion and sophisticated mitigation strategies indicates that this challenge will require sustained research attention. While current large language models demonstrate remarkable capabilities on many tasks, their propensity to generate plausible but incorrect information at rates far exceeding human error rates constrains deployment in high-stakes applications where factual accuracy is paramount. Addressing hallucination comprehensively will require not only technical innovations in architectures and training procedures but also careful consideration of how context is managed, how information is positioned within that context, and how generation processes are constrained to respect evidence while maintaining fluency and utility.

10 Multi-Turn Conversation Management

Multi-turn conversation management represents one of the most pressing challenges in contemporary large language model deployment. Recent empirical research has revealed a sobering reality: top open- and closed-weight models exhibit an average performance drop of 39% when evaluated in multi-turn conversational scenarios compared to single-turn tasks [157]. This degradation affects frontier models including GPT-4, Claude, Gemini, and LLaMA variants across six distinct generation tasks, indicating a fundamental architectural challenge rather than an implementation artifact. The phenomenon occurs regardless of context window size, persists across different training data compositions, and manifests consistently across diverse model architectures. Understanding the mechanisms underlying this performance degradation and developing robust mitigation strategies has emerged as a critical research priority for deploying conversational AI systems at scale.

The multi-turn conversation problem manifests through several interconnected challenges. First, models exhibit a tendency to make premature assumptions in early turns, attempting to generate final solutions too quickly rather than engaging in deliberate multi-step reasoning [157]. Second, when conversational agents take incorrect turns or make erroneous assumptions, they frequently fail to recover, instead compounding errors through subsequent interactions. Third, information positioned in the middle of extended context windows becomes substantially less accessible than content at the beginning or end, creating a U-shaped retrieval effectiveness pattern [7]. Fourth, maintaining consistency across dozens or hundreds of turns proves exceptionally difficult, with models frequently contradicting earlier statements or losing track of established facts. These challenges are exacerbated by computational constraints, as each additional turn increases token consumption quadratically in standard transformer architectures, leading to escalating costs and latency in production systems.

10.1 Truncation and Sliding Window Strategies

Truncation represents the most straightforward approach to managing context window constraints in multi-turn conversations, but naive implementation risks catastrophic information loss. The fundamental challenge lies in determining which portions of conversation history to preserve and which to discard when accumulated context exceeds model capacity. Early approaches employed simple strategies such as keeping only the most recent N messages or truncating from the beginning of the conversation, but these methods frequently resulted in accuracy degradation and increased hallucination rates as critical context disappeared from the model’s view [158].

Classical sliding window attention mechanisms reduce computational complexity from quadratic $O(n^2)$ to linear $O(n \times w)$, where n represents sequence length and w denotes window size. In this paradigm, each token attends only to a fixed-size neighboring window rather than the entire sequence. While computationally efficient, standard sliding window approaches lose information beyond the attention window entirely, preventing tokens outside the window from contributing to prediction. Empirical evidence suggests that despite theoretical support for long sequences, models struggle to effectively utilize information beyond approximately 1,500 words in practice, even when such information falls within the nominal context window [6].

Recent advances have substantially improved upon classical sliding window techniques through architectural innovations. Sliding Window Attention Training, commonly known as SWAT, replaces the traditional softmax attention mechanism with a sigmoid function while utilizing balanced positional encodings that combine Attention with Linear Biases and Rotary Position Embedding [159]. This modification addresses the attention sink phenomenon, where attention distributions become overly concentrated on specific tokens regardless of relevance, and improves the model’s ability to compress and utilize information distributed across the context window. Experimental evaluations demonstrate that SWAT-trained models achieve superior performance on long-context tasks compared to models using standard sliding window attention.

Sliding Window Attention Adaptation, abbreviated as SWAA, offers a plug-and-play toolkit that combines five complementary strategies to optimize long-context inference [160]. The first strategy applies sliding window attention exclusively during the prefilling phase while maintaining full attention during generation. The second preserves sink tokens, which empirical research has identified as critical anchor points in attention distributions. The third interleaves full attention layers with sliding window layers, balancing computational efficiency with global context integration. The fourth incorporates chain-of-thought prompting to encourage explicit reasoning. The fifth employs parameter-efficient fine-tuning to adapt models for specific deployment contexts. When combined, these strategies achieve speedups ranging from 30% to 100% for long-context inference tasks while maintaining acceptable quality levels, though some degradation remains inevitable.

Intelligent truncation strategies distinguish between essential and optional content within the conversation context. Must-have content typically includes system prompts that establish the assis-

tant’s behavior and capabilities, the current user message that requires a response, and critical state information that contextualizes the ongoing interaction. Optional content encompasses conversation history, which provides helpful context but can be selectively included based on available capacity. Token budgeting frameworks allocate specific portions of the context window to each category: commonly 5-10% for system prompts, 15-25% for output generation, and the remainder for conversation history and retrieved information [161]. When context capacity approaches saturation, systems employing intelligent truncation preferentially preserve must-have content while progressively compressing or removing optional elements.

Hybrid approaches that interleave full attention with sliding window layers represent a promising direction for balancing efficiency with context retention. Rather than applying uniform attention patterns across all layers, hybrid architectures designate certain layers as full attention while others employ sliding windows. This stratification enables models to capture both local dependencies through windowed attention and global patterns through periodic full attention, potentially offering superior trade-offs between computational cost and context integration compared to uniform approaches.

10.2 Conversation Summarization

Recursive summarization stimulates large language models to memorize dialogue contexts through a multi-stage compression process [162]. The methodology first prompts the model to generate a concise memory representation of a small dialogue context, typically encompassing 3-5 conversational turns. Subsequently, the system recursively produces new memory by combining the previous memory representation with following context segments. This hierarchical approach enables at least fivefold compression of conversation history while maintaining semantic coherence, as validated through evaluation on the ICLR 2025 benchmark datasets. The generated summaries prove significantly shorter than full dialogues, enabling systems to handle extremely long contexts spanning multiple dialogue sessions without exhausting available context windows.

Microsoft’s production implementation appends short, dynamically updated summaries immediately before the latest user message while removing older message exchanges from the active context [163]. The system maintains the original system prompt plus recent assistant-user message pairs in full detail, typically preserving the last 5-10 turns verbatim. As conversations extend beyond this window, older segments undergo summarization, with the resulting compressed representations replacing the original messages. This approach prevents unbounded context growth while preserving recent exchanges where fine-grained details matter most for generating contextually appropriate responses. Empirical deployment data indicates that users experience minimal degradation in conversational quality when summary-based context management activates, suggesting that the approach successfully balances compression efficiency with information preservation.

Mixture-of-experts architectures for dialogue summarization employ role-oriented routing mechanisms to select appropriate expert modules for different summarization subtasks [164]. The system

analyzes conversation structure and content to determine which specialized experts should process particular dialogue segments, routing task-oriented conversations to experts trained on transactional interactions while directing open-domain discussions to experts optimized for social dialogue. Following expert selection, a fusion generation stage locates salient information across expert outputs and produces a finalized summary that integrates insights from multiple specialists. This approach, presented at ACL 2024, demonstrates superior performance compared to monolithic summarization models, particularly for conversations that span multiple domains or exhibit complex turn-taking patterns.

Smart memory systems implementing hierarchical summarization strategies achieve token cost reductions ranging from 80% to 90% while maintaining or in some cases improving response quality relative to basic chat history management [165]. Evaluation benchmarks from Mem0 demonstrate 26% improvement in response quality when comparing advanced memory architectures against baseline approaches that simply append recent conversation history to prompts. The performance gains stem from several factors: summarization filters noise and tangential discussions that can distract the model, compressed representations reduce the cognitive load of processing extensive context, and hierarchical organization enables more efficient retrieval of task-relevant information from conversation history.

Hierarchical memory architectures organize conversation information across multiple compression levels. The H-MEM framework implements a four-layer structure comprising Domain, Category, Memory Trace, and Episode layers [166]. Each layer applies progressively higher compression ratios while maintaining position indices that enable retrieval of detailed information when needed. The Domain layer captures broad conversation topics, the Category layer organizes related information within domains, the Memory Trace layer maintains compressed representations of specific interaction sequences, and the Episode layer preserves individual conversational events in greater detail. This stratification enables systems to navigate conversations at multiple granularities, retrieving high-level summaries for context-setting while accessing detailed episode information when specific facts or decisions require reference.

The SGMEM framework structures conversational memory as a sentence graph, where nodes represent individual sentences or short utterances and edges capture explicit semantic associations between conversation elements [167]. This graph organization addresses memory fragmentation challenges that arise in long-term conversational agents by modeling relationships between non-adjacent statements. When users reference earlier topics or make connections across distant conversation segments, the graph structure enables efficient traversal to relevant information without scanning entire conversation histories. Semantic retrieval algorithms leverage the graph topology to reconstruct contextual understanding even when relevant information spans many intermediate turns.

Practical implementation considerations for summarization-based systems include trigger conditions that determine when compression should occur, summary quality assessment to validate informa-

tion preservation, integration patterns that combine full messages with summaries, and monitoring frameworks that track retention effectiveness [168]. Trigger conditions may include token count thresholds such as activating summarization when conversation history reaches 70% of available capacity, time-based triggers that compress after fixed numbers of turns, importance-based triggers that respond to conversation phase transitions, or hybrid approaches combining multiple signals. Summary quality can be assessed through specialized models trained for consistency evaluation, by preserving key entities and commitments explicitly, through maintaining temporal ordering in compressed representations, and by including confidence scores that indicate summary completeness. Integration patterns typically mix full verbatim messages for recent exchanges with summaries for older interactions, use summaries as context for subsequent generation, maintain summary histories for multi-level retrieval, and track summary chains to support reproducibility and debugging.

10.3 Dialogue State Tracking

Dialogue state tracking has evolved from traditional slot-value representations to natural language descriptions generated by large language models, substantially enhancing both expressiveness and interpretability [169]. Classical dialogue state tracking systems maintained predefined slot structures for each supported domain, such as restaurant reservations with slots for cuisine type, location, price range, party size, and date-time. Each slot contained a value extracted from user utterances or marked as null if unfilled. While structured and computationally efficient, this representation struggled with out-of-vocabulary concepts, complex user intents, and nuanced preferences that did not map cleanly to predefined categories.

Modern natural language dialogue state tracking leverages large language models to directly generate state descriptions in free text rather than constrained slot-value pairs [169]. This approach enables systems to capture subtleties such as "a romantic restaurant suitable for a proposal" or "vegetarian options but not strictly vegan" that resist encoding in traditional schemas. The interpretability benefits prove particularly valuable for debugging and system monitoring, as natural language states can be directly inspected by developers and users without specialized schema knowledge. Evaluation on standard benchmarks including MultiWOZ 2.1 and Taskmaster-1 demonstrates that language model-based dialogue state tracking achieves competitive or superior performance compared to specialized architectures while offering substantially greater flexibility.

The MultiWOZ benchmark, comprising over 10,000 fully-labeled human-human written conversations spanning multiple domains, has emerged as the standard evaluation framework for dialogue state tracking research [170]. The dataset represents an order of magnitude expansion over previous task-oriented corpora, providing sufficient scale and diversity to support robust model evaluation. Subsequent versions including MultiWOZ 2.2 and 2.4 incorporate annotation corrections that address inconsistencies and errors in the original release, improving the reliability of state tracking evaluation. Recent extensions such as Multi-User MultiWOZ introduce three-party interactions involving two users and one agent, capturing the complexity of group decision-making dialogues,

while ToolDial published at ICLR 2025 integrates tool use and external API calls within multi-turn conversations.

State-of-the-art approaches to dialogue state tracking combine natural language understanding with structured reasoning. The CoDial system achieves leading performance on the STAR dataset through explicit modeling of conversational context and dialogue acts, enabling more accurate state updates across turn boundaries [171]. UniPPN demonstrates exceptional results on MultiWOZ by unifying dialogue policy and natural language generation within a single framework that maintains consistent state representations throughout conversations [172]. Both systems incorporate mechanisms for error detection and correction, addressing the critical challenge of identifying when the dialogue state tracker has misunderstood user intent and enabling graceful recovery rather than error propagation.

Recent research focuses on enabling reliable estimation of dialogue agent errors and guiding language model decoders to generate more accurate actions based on dialogue state [173]. Error detection mechanisms analyze confidence scores in state updates, monitor for contradictions between user utterances and extracted state, and track dialogue flow patterns that indicate potential misunderstandings. When errors are detected, recovery strategies include generating clarification questions to resolve ambiguity, offering explicit state confirmations for user verification, backtracking to previous valid states, and providing multiple interpretation options when uncertainty persists. These capabilities prove essential for production deployment, where conversational failures can severely impact user satisfaction and task completion rates.

Chain-of-thought explanation for dialogue state tracking enhances both accuracy and interpretability by prompting models to articulate their reasoning process when updating state [174]. Rather than directly outputting state updates, systems generate intermediate reasoning steps that explain which portions of user utterances motivated particular state changes, how new information integrates with existing state, and why certain interpretations were selected over alternatives. This explicit reasoning serves multiple purposes: it improves state tracking accuracy by encouraging systematic analysis, enables debugging by making model decisions transparent, facilitates user trust through explainability, and provides training signal for improving model reasoning capabilities.

10.4 User Modeling and Personalization

Personalization in multi-turn conversations requires dynamic user profiling mechanisms that evolve through ongoing interaction. Proactive questioning enables systems to construct detailed user profiles by actively soliciting information about preferences, constraints, and goals [175]. Memory extraction from conversation history infers user traits and preferences from implicit signals such as topic selection, linguistic style, response patterns, and decision criteria. Implicit behavioral metrics including session duration, response latency, sentiment indicators, language complexity, and typing speed provide additional signals about user engagement and comprehension. Explicit feedback mechanisms allow users to directly specify preferences and correct misunderstandings, providing

high-confidence training signals for personalization models.

The PERSONAMEM benchmark evaluates 15 state-of-the-art models on their ability to internalize user traits from conversation history, track evolving user profiles across sessions, and generate appropriately personalized responses [175]. The benchmark comprises over 180 simulated user profiles with up to 60 sessions per user spanning 15 real-world tasks requiring personalization. Evaluation reveals that frontier models including GPT-4o, GPT-4.5, Gemini 2.0, DeepSeek-R1, and LLaMA-4 struggle with user awareness, particularly when knowledge acquired in one context must generalize to novel scenarios. Performance gaps widen substantially when models must apply learned user preferences to tasks that differ from those encountered during profile construction, indicating fundamental challenges in cross-domain personalization transfer.

The PersonalLLM dataset published at ICLR 2025 provides prompts, responses, and personality annotations specifically designed for training and evaluating personalization algorithms [176]. Unlike previous personalization benchmarks that focus on single-turn preference matching, PersonalLLM emphasizes multi-turn consistency, preference evolution tracking, and the ability to handle contradictory user statements that require nuanced reconciliation. The dataset includes annotations for temporal dynamics such as preference shifts over time, contextual preferences that vary by situation, and stable traits that persist across contexts, enabling research on dynamic user modeling.

Parameter-efficient personalization approaches prove essential for practical deployment, as maintaining separate fine-tuned models for individual users becomes computationally prohibitive at scale. Low-Rank Adaptation and quantized variants such as QLoRA enable personalization through small adapter modules that modify base model behavior without full parameter updates [177]. These adapters can be cached and loaded on demand, supporting large user populations while maintaining reasonable memory footprints. Alternative approaches employ retrieval-augmented generation to inject user-specific context from vector databases, combining the benefits of explicit memory with the flexibility of retrieval. Hybrid architectures that integrate parameter-efficient fine-tuning with retrieval demonstrate promising results, using adapters to capture high-level user characteristics while retrieving specific historical interactions for detailed context.

Curiosity-driven reinforcement learning enhances personalized multi-turn dialogue by incorporating intrinsic rewards that encourage models to actively learn about users [178]. Rather than optimizing solely for immediate response quality, curiosity-augmented systems balance exploitation of current user knowledge with exploration to improve user modeling accuracy. This approach leads to behaviors such as asking clarifying questions even when not strictly necessary for the immediate task, proactively offering personalized suggestions to gauge user reactions, and experimenting with different interaction styles to identify user preferences. Empirical evaluation demonstrates that curiosity mechanisms improve long-term user satisfaction and engagement compared to purely reactive personalization approaches.

Privacy compliance represents a critical constraint on personalization system design. The European

Data Protection Board guidance issued in 2024 establishes that AI memory systems must align with General Data Protection Regulation principles including lawful basis for processing, data minimization, storage limitation, and purpose limitation [179]. Design-level compliance requirements for enterprise deployments include explicit user consent for memory collection, granular controls for editing stored memories, complete deletion capabilities that extend to backups and logs, and mechanisms for temporarily disabling memory features. Users must retain visibility into what information systems have remembered and the ability to correct inaccuracies. These requirements shape architectural decisions, often necessitating separation between anonymized behavioral patterns that inform general system improvements and personally-identifiable user profiles that enable individualized experiences.

10.5 Multi-Agent Shared Context

Multi-agent frameworks have achieved 72% enterprise adoption in AI projects by 2026, representing dramatic growth from 23% adoption in 2024 [180]. This rapid proliferation reflects recognition that complex tasks often benefit from decomposition across specialized agents with distinct capabilities and knowledge domains. Three dominant frameworks have emerged, each offering fundamentally different approaches to managing shared context in collaborative agent systems: AutoGen employs conversational collaboration with per-agent context management, CrewAI implements layered memory with explicit role-based organization, and LangGraph provides dual in-thread and cross-thread memory with graph-based workflow orchestration.

AutoGen’s architecture centers on multi-agent conversations as the primary collaboration mechanism [181]. Each conversable agent maintains short-term context through a context variables object that stores recent interaction history at the agent level. The framework does not provide built-in persistent memory, instead relying on external integrations when long-term storage becomes necessary. Conversation sharding distributes context across multiple concurrent dialogues, enabling scalability to large agent populations while avoiding context centralization bottlenecks. This design philosophy prioritizes flexibility and composability, allowing developers to implement domain-specific memory strategies rather than prescribing universal solutions. The message-passing simplicity proves particularly effective for dialogue-based tasks where natural conversational flow aligns well with the underlying architecture.

CrewAI implements a comprehensive multi-tier memory system with built-in support for different memory types and timescales [182]. Short-term memory utilizes ChromaDB vector stores to maintain recent task context with efficient similarity-based retrieval. Recent task results persist in SQLite storage, enabling agents to reference completed work without recomputation. Long-term memory occupies a separate SQLite table indexed by task descriptions, supporting retrieval of relevant historical experiences when encountering similar tasks. Entity memory maintains vector embeddings for important entities encountered across tasks, facilitating consistent entity recognition and attribute tracking. This layered architecture aligns with role-based organizational models where each

agent maintains specialized expertise relevant to its designated function. The framework proves particularly effective for hierarchical team structures with clear role divisions and sequential task workflows.

LangGraph provides a dual memory system distinguishing between in-thread memory for single tasks or conversations and cross-thread memory for data that persists across sessions [183]. The MemorySaver component saves complete task flows linked to specific thread identifiers, enabling resumption of interrupted workflows and supporting branching execution paths. Long-term storage backends can be implemented through InMemoryStore for development and testing or database backends for production deployments requiring persistence and scalability. The graph-based workflow approach enables explicit representation of complex state transitions and dependencies, making LangGraph particularly suitable for scenarios where workflow visibility and control prove critical. Conditional edges in the graph structure support dynamic context routing based on intermediate results, enabling adaptive information flow that responds to task evolution.

Context sharing patterns in multi-agent systems adopt several common architectural approaches. Context propagation flows linearly through agent pipelines, with each agent reading the current state, performing its designated operations, updating the state, and passing the modified context to subsequent agents. The framework ensures consistency through transaction-like semantics or explicit coordination protocols. Context branching enables parallel agent execution, where multiple agents process shared input state concurrently and results are subsequently aggregated through voting, averaging, or hierarchical arbitration. Conflict resolution strategies become essential when parallel agents produce contradictory state updates. Context specialization provides each agent access to a filtered view of the shared state, reducing cognitive load and preventing information overload while requiring explicit scoping rules to determine visibility boundaries. Context caching computes expensive shared representations once and enables multiple agents to reference the cached version, substantially reducing redundant computation in scenarios where many agents require similar contextual information.

Real-world challenges in multi-agent context management include context explosion as agent populations grow and shared state accumulates contributions from multiple sources, consistency maintenance across distributed agent views of evolving state, privacy enforcement to control context visibility between agents with different authorization levels, performance optimization for efficiently passing and caching large context structures, debugging complexity in tracking how context evolves through distributed agent interactions, and serialization requirements for storing and loading complex context structures across process boundaries and system restarts [184].

10.6 Commercial Memory Systems

All major conversational AI platforms deployed persistent memory features by mid-2025, representing a fundamental shift from stateless interaction models to context-aware collaboration. OpenAI's ChatGPT implements automatic context inclusion, transparently incorporating details from previ-

ous conversations at the start of every new interaction [185]. The system does not expose explicit tool calls for memory access; instead, memory integration occurs invisibly to users while providing control mechanisms for editing, deleting, and disabling stored memories. ChatGPT employs a four-layer memory architecture comprising session metadata that adapts to environmental factors such as time of day and device, explicit facts layer containing user-requested memories like "I'm vegetarian" or "My name is John," lightweight summary layer with compressed representations of recent conversation patterns, and sliding window layer that maintains current conversation context in full detail [186]. This multi-tier approach trades detailed historical context for speed and efficiency, with lightweight summaries pre-computed offline and injected directly into prompts rather than requiring complex retrieval operations.

Claude memory, rolled out to all paid users in October 2025, implements memory access through visible tool calls that provide transparency into when and how context is retrieved [187]. This design philosophy prioritizes user understanding of system behavior, enabling users to observe precisely what information the assistant accesses and when such access occurs. Project-based memory architecture maintains separate memory banks for each project, ensuring context separation between distinct workflows such as product launch planning versus client consulting work. Users maintain complete visibility into stored memories and can edit or delete entries at will. A distinctive capability enables importing saved memory data from other accounts or sessions, including context exported from ChatGPT, facilitating migration and continuity across platforms. The system transforms Claude from a stateless assistant that begins each session *tabula rasa* into a context-aware collaborator that accumulates understanding through repeated interactions.

Claude's context management capabilities extend beyond basic memory through context editing and memory tools designed for agent applications [188]. Context editing automatically removes stale tool calls and results from previous turns, preventing context pollution from outdated information while preserving conversation flow. This capability extends how long agents can run before context window exhaustion becomes limiting. Evaluation on extended 100-turn web search tasks demonstrates that context editing alone improves performance by 29% over baseline approaches. The memory tool provides a file-based system for storing information outside the context window, enabling agents to build knowledge bases that persist across sessions. Combined with context editing, the memory tool achieves 39% performance improvement over baseline while reducing token consumption by 84%, offering substantial efficiency gains for long-running agent applications.

Gemini 1.5 Pro introduced a production context window of 2 million tokens in February 2024, with research testing successfully extending to 10 million tokens [189]. The model architecture was purpose-built for long context rather than retrofitted, enabling superior in-context learning capabilities and more effective utilization of extended context compared to models that achieved large windows through position encoding modifications alone. Needle-in-a-Haystack evaluation demonstrates 99% retrieval accuracy at 1 million tokens, with near-perfect retrieval persisting beyond 99% up to 10 million tokens in research settings. The 2 million token capacity enables processing 1 hour

of video, 11 hours of audio, over 30,000 lines of code, or approximately 700,000 words of text in a single request. Context caching reduces costs approximately fourfold for cached content in Gemini Flash, with users paying hourly storage fees for maintained caches rather than repeatedly processing identical content.

The evolution from 4,000 tokens at ChatGPT’s launch in late 2022 to millions of tokens by 2025 represents a dramatic expansion of practical context capacity, though evidence suggests that raw window size alone does not solve multi-turn conversation challenges. Models with million-token windows still exhibit the 39% multi-turn performance degradation, indicating that context quantity cannot substitute for architectural improvements in information integration and consistency maintenance [157]. Industry standard context windows reached 128,000 tokens by 2025, with cutting-edge research models such as LLaMA-4 demonstrating 10 million token capacity. Gemini 1.5 Pro’s 2 million token production window represents the largest mainstream deployed context as of early 2026, though practical utilization often remains substantially below theoretical limits due to cost, latency, and quality trade-offs.

10.7 Context Engineering

Context engineering has emerged as a critical production discipline, treating the context window as a scarce resource requiring careful budgeting and optimization. The quadratic computational cost scaling inherent in transformer architectures means that processing a sequence of length n requires $O(n^2)$ operations for self-attention, with memory consumption growing proportionally [1]. For production systems serving thousands of concurrent users, inefficient context utilization compounds costs exponentially while degrading latency. Token-based pricing models employed by commercial API providers translate longer contexts directly to higher per-request costs, creating strong economic incentives for optimization.

Fixed budget approaches allocate specific token counts to distinct components of the request: system prompts typically consume 5-10% of available capacity establishing assistant behavior and capabilities, user conversation history receives 30-50% depending on conversation phase, retrieval results from external knowledge sources occupy 20-40% when retrieval-augmented generation is employed, and output generation reserves 15-25% ensuring sufficient space for comprehensive responses [190]. These allocations must sum to less than the maximum context window, with safety margins maintained to prevent truncation during generation. Fixed budgets provide predictable behavior and simplified reasoning about system constraints but lack flexibility to adapt to varying task requirements.

Dynamic budgeting adjusts allocations based on conversation phase and task characteristics. During early turns when conversation history remains minimal, systems can allocate additional capacity to retrieval results or output generation. As conversations extend and history accumulates, allocations shift to preserve recent exchanges while compressing or removing older content. Task complexity signals derived from user queries guide allocation decisions: factual question-answering may increase

retrieval allocation while reducing output budget, whereas creative generation tasks might reverse these priorities. Multi-turn reasoning tasks that build on previous exchanges prioritize conversation history preservation at the expense of retrieval capacity [191].

Information position effects substantially influence model performance, with content placement mattering as much as content quantity. Empirical analysis reveals a U-shaped retrieval effectiveness pattern: the first 20% of context exhibits approximately 85% effectiveness, the middle 60% drops to approximately 40% effectiveness, and the last 10% recovers to approximately 80% effectiveness [7]. This "lost in the middle" phenomenon has important implications for context engineering. Critical information should be positioned near the beginning or end of the context window rather than buried in the middle. Anthropic's official documentation for Claude explicitly recommends placing long documents of 20,000 tokens or more near the top of prompts, as this positioning yields significant empirically-verified performance improvements across all model variants [192].

Monitoring tools track token consumption patterns across request components, identify optimization opportunities where high token expenditure produces limited value, prevent budget overruns that cause truncation or request failures, and manage rate limits imposed by API providers on requests per minute and tokens per minute [193]. Production observability typically instruments total token consumption per request, breakdown by component such as system prompt, history, retrieval, and completion, utilization percentage relative to available capacity, cache hit rates for frequently-accessed context, and latency contributions from context processing versus generation. Dashboards visualize these metrics across user populations, enabling identification of pathological cases where inefficient context management degrades experience or inflates costs.

Practical ceiling on context size emerges from cost-performance trade-offs even when technical capacity exists. While models support 128,000 tokens or more, production deployments frequently maintain working context below 16,000 to 32,000 tokens to balance response quality, latency, and cost. Beyond these thresholds, the probability of utilizing all available context decreases, rendering marginal token inclusion increasingly wasteful. Optimal utilization typically falls between 60-75% of available capacity: below 60% suggests inefficient underutilization, while above 85% correlates with 23% performance degradation as models struggle with information overload [194].

10.8 Summary

Multi-turn conversation management remains a critical bottleneck for large language model deployment, with frontier models exhibiting average performance degradation of 39% in multi-turn scenarios compared to single-turn evaluation. This degradation appears independent of context window size, training data quality, and model architecture, suggesting fundamental challenges in how transformer-based models integrate information distributed across extended conversational sequences. The convergence of truncation strategies, conversation summarization, dialogue state tracking, user personalization, multi-agent coordination, and context engineering provides a comprehensive toolkit for mitigating these challenges, though no silver bullet has emerged.

Sliding window approaches and hybrid attention mechanisms offer computational efficiency while preserving some global context integration, with recent innovations such as SWAT and SWAA demonstrating 30-100% speedups with acceptable quality trade-offs. Recursive and hierarchical summarization techniques achieve 5-20x compression ratios, with advanced methods such as LLM-Lingua maintaining core capabilities even at aggressive compression levels. Dialogue state tracking has evolved from rigid slot-value schemas to flexible natural language representations that capture nuanced user intents. Personalization benchmarks reveal that current models struggle with cross-domain generalization, indicating substantial room for improvement in user modeling capabilities.

Multi-agent frameworks have achieved widespread enterprise adoption through distinct approaches to shared context management: AutoGen’s conversational simplicity, CrewAI’s layered memory architecture, and LangGraph’s graph-based workflow orchestration each prove effective for different collaboration patterns. Commercial platforms have converged on persistent memory as a baseline feature while diverging in implementation details: ChatGPT’s invisible integration versus Claude’s transparent tool calls versus Gemini’s massive context windows reflect different philosophical stances on the memory-context trade-off. Context engineering has emerged as an essential discipline, with empirical research demonstrating that strategic information placement and dynamic budget allocation yield performance improvements comparable to or exceeding those from raw context expansion.

Fundamental challenges remain unresolved. The 39% multi-turn performance drop persists even in models with million-token windows, suggesting that architectural innovations beyond context expansion are necessary. Maintaining consistency across hundreds of turns proves exceptionally difficult, with models frequently contradicting earlier statements or losing track of established context. Computational costs scale quadratically with context length, creating economic barriers to unbounded context accumulation even when technical capacity exists. Privacy regulations impose design-level constraints on memory persistence and personalization, requiring careful balance between capability and compliance. The field continues rapid evolution, with weekly advances in compression techniques, memory architectures, agent coordination protocols, and evaluation methodologies shaping the trajectory toward more capable and reliable multi-turn conversational systems.

11 Evaluation and Benchmarks

The evaluation of long-context language models presents unique challenges that extend far beyond traditional language model assessment. As context windows have expanded from thousands to millions of tokens, researchers have discovered that traditional evaluation metrics and methodologies prove insufficient for capturing the full spectrum of capabilities required for effective long-context understanding. This transformation has necessitated the development of specialized benchmarks, novel evaluation paradigms, and more sophisticated metrics that can accurately characterize model performance across extended contexts. The evolution of evaluation methodologies reveals a critical insight: context window size and effective context utilization represent fundamentally different capabilities, with most models utilizing only a fraction of their claimed capacity.

11.1 Foundational Evaluation Paradigms

11.1.1 Needle-in-a-Haystack Testing

The Needle-in-a-Haystack test emerged as a foundational methodology for assessing long-context retrieval capabilities. Introduced by [195], the test embeds specific, targeted information within a larger body of text and evaluates the model’s ability to retrieve this information across varying context lengths and positional depths. The methodology draws from Paul Graham essays as haystack content in its original implementation, with the needle consisting of a specific statement such as “The best thing to do in San Francisco is eat a sandwich and sit in Dolores Park on a sunny day.” Models are then queried to retrieve this information using only the provided context, with systematic testing across different document lengths ranging from one thousand tokens to the model’s maximum context window, and varying needle positions from zero to one hundred percent through the document.

The test’s simplicity represents both its primary advantage and its fundamental limitation. While NIAH provides rapid screening capability for basic retrieval functionality, research has demonstrated that near-perfect NIAH scores do not predict downstream performance on real-world tasks. [196] observed that models achieving nearly perfect NIAH accuracy exhibit large performance drops on more complex tasks requiring multi-hop reasoning or aggregation. The original results on GPT-4 revealed performance degradation beginning at sixty-four thousand tokens, with sharp declines beyond one hundred thousand tokens. Furthermore, models demonstrate strong primacy and recency biases, with substantially degraded performance on information positioned in the middle of long contexts.

The community has developed several extensions to address NIAH’s limitations. Multi-needle variants embed multiple facts requiring simultaneous retrieval, evaluating how models handle potential conflicts or need to synthesize information from multiple needles. Multimodal extensions test retrieval across image and text modalities, as explored in recent work on multimodal long-context models. Research on conflicting needles in haystacks examines how models prioritize information when contradictory statements appear at different positions, revealing systematic biases in information prioritization. Despite these extensions, the fundamental critique remains: NIAH tests only simple point-fact retrieval rather than the complex reasoning and synthesis capabilities required for practical long-context applications.

11.1.2 Evolution Beyond Simple Retrieval

The recognition of NIAH’s limitations spurred development of more comprehensive evaluation frameworks. Researchers identified that real-world long-context applications demand capabilities far exceeding simple retrieval, including multi-hop reasoning chains that follow dependencies across thousands of tokens, aggregation of information from multiple disparate sources, question answering requiring synthesis and inference, and the maintenance of coherence across extremely long

documents. These requirements necessitated benchmarks that could systematically evaluate each capability dimension while maintaining reproducibility and interpretability.

11.2 Comprehensive Multi-Task Benchmarks

11.2.1 RULER: Assessing Real Context Size

RULER, developed by [196], represents a significant advance beyond NIAH by introducing multi-dimensional task complexity while maintaining synthetic control. The benchmark addresses a critical question: what is the true, effective context size of models claiming extended windows? RULER extends NIAH testing to encompass four primary task categories: retrieval tasks including multiple needle variants with varying quantities and types, multi-hop tracing that requires following chains of reasoning across extended contexts, aggregation tasks demanding information synthesis from multiple sources, and question answering that necessitates comprehensive understanding and reasoning.

The benchmark’s configurability enables precise control over sequence length and task complexity, allowing researchers to systematically probe model capabilities across different difficulty levels. Evaluation of seventeen long-context language models across thirteen tasks revealed critical findings that challenged industry claims about context capabilities. Despite achieving near-perfect NIAH scores, most models exhibited substantial performance degradation as context length increased. Perhaps most significantly, only approximately half of models claiming thirty-two thousand token contexts maintained satisfactory performance at that length, exposing a substantial gap between marketing claims and actual capabilities.

Performance patterns varied substantially across task categories. Retrieval tasks demonstrated relatively stable performance, though still degrading with length. Multi-hop tracing showed sharp degradation, suggesting fundamental limitations in maintaining reasoning chains across extended contexts. Aggregation tasks revealed even steeper declines, indicating particular difficulty in synthesizing information from multiple disparate sources. These findings suggested that context window expansion alone, without architectural innovations addressing reasoning across extended sequences, provides limited practical benefit.

Recent extensions have broadened RULER’s scope. The multilingual RULER benchmark extends evaluation to over ten languages, testing whether long-context capabilities transfer across linguistic boundaries. Early results suggest language-specific effects, with tokenization efficiency and grammatical structure influencing effective context utilization. This multilingual perspective highlights that context window specifications must account for language-specific characteristics, as the same semantic content may require substantially different token counts across languages.

11.2.2 LongBench: Bilingual Multi-Task Evaluation

LongBench, introduced by [197], provides the first bilingual, multi-task benchmark for long-context understanding, addressing critical gaps in standardized evaluation across diverse domains and languages. The benchmark encompasses twenty-one datasets across six task categories in both English and Chinese, with average lengths of approximately six thousand seven hundred words in English and thirteen thousand characters in Chinese. This bilingual design enables direct comparison of model capabilities across linguistic contexts and reveals language-specific strengths and weaknesses.

The six task categories cover the breadth of long-context applications. Single-document question answering evaluates information retrieval within individual long documents, testing whether models can locate and extract specific facts across thousands of tokens. Multi-document question answering requires synthesis across multiple documents, evaluating cross-document reasoning capabilities essential for research and analysis tasks. Summarization tasks assess the ability to condense long-form content while preserving essential information and maintaining coherence. Few-shot learning tasks test in-context learning from extended examples, evaluating whether models can leverage long demonstration sequences. Synthetic tasks provide controlled evaluation of specific capabilities with ground truth validation. Code completion tasks evaluate long-context understanding in programming contexts, where function definitions and variable references may span thousands of tokens.

Evaluation across multiple commercial and open-source models revealed consistent patterns. Commercial models, particularly GPT-3.5-Turbo with a sixteen thousand token window, outperformed open-source alternatives, though all models struggled significantly on longer contexts. Position embedding techniques emerged as critical determinants of long-context capability, with scaled position embeddings and position interpolation proving more effective than extrapolation approaches. Fine-tuning on longer sequences produced substantial improvements, suggesting that exposure to extended contexts during training enables better utilization of architectural capacity. However, context compression techniques showed limited ceiling effects, with retrieval-based methods helping weaker models but strong native long-context models outperforming retrieval-augmented approaches.

LongBench v2, released in twenty twenty-five, extends evaluation to five hundred three challenging multiple-choice questions with contexts ranging from eight thousand to two million words. This extension addresses the observation that models had begun approaching saturation on the original benchmark, necessitating more challenging evaluation to differentiate capabilities. The extended version emphasizes questions requiring genuine understanding rather than simple pattern matching, with adversarial design to prevent shortcut solutions.

11.2.3 L-Eval: Standardizing Long-Context Evaluation

L-Eval, developed by [198], institutes standardized evaluation practices for long-context language models, earning ACL twenty twenty-four Outstanding Paper designation for its methodological contributions. The benchmark features twenty sub-tasks spanning five hundred eight long documents

with over two thousand human-labeled query-response pairs, providing comprehensive coverage across diverse domains. Input lengths range from three thousand to two hundred thousand tokens, enabling systematic evaluation of length-dependent effects.

The benchmark organizes tasks into two primary groups: closed-ended tasks focusing on reasoning and understanding, typically with verifiable correct answers, and open-ended tasks including summarization and generation, requiring more subjective evaluation. This distinction proves critical for metric selection, as closed-ended tasks enable reliable automatic evaluation while open-ended tasks benefit from more sophisticated assessment approaches.

L-Eval’s most significant contribution lies in demonstrating that traditional n-gram metrics correlate poorly with human judgment on long-context tasks. Metrics such as BLEU and ROUGE, standard in machine translation and summarization evaluation, fail to capture the nuances of long-context understanding. The benchmark advocates for Length-Instruction-Enhanced evaluation methodology, which explicitly incorporates length requirements into evaluation prompts, and LLM-as-judge evaluation, using capable language models to assess output quality. These approaches show substantially better correlation with human preferences compared to surface-level n-gram matching.

Comprehensive evaluation of four commercial and twelve open-source models revealed consistent performance gaps. Commercial models outperformed open-source alternatives by approximately fourteen percentage points on average, with the gap widening on more complex reasoning tasks. Performance degraded clearly with document length, with effects stronger for complex reasoning than simple extraction. Domain-specific effects proved substantial, with legal and economics domains showing lower performance than news or technology domains, likely reflecting training data distributions and domain-specific vocabulary challenges.

11.3 Specialized Domain Benchmarks

11.3.1 InfiniteBench: Beyond One Hundred Thousand Tokens

InfiniteBench, introduced by [199], pushes evaluation boundaries to contexts exceeding one hundred thousand tokens, representing the first benchmark with average document length surpassing this threshold. The benchmark features twelve unique tasks across five domains: retrieval tasks for finding specific information in very long documents, query-focused summarization requiring creation of targeted summaries, key-value retrieval testing precise information location capability, open-domain question answering demanding synthesis and reasoning, and specialized reasoning tasks testing logical inference over extreme lengths.

The benchmark’s design principle emphasizes long-dependency focus. Tasks are explicitly constructed such that simply retrieving a limited number of passages proves insufficient for success; models must demonstrate understanding of long-range dependencies and meaningful relationships embedded throughout extended contexts. This design prevents shortcut solutions and ensures that high performance genuinely reflects long-context capability rather than clever pattern matching on

local information.

Experimental results on major commercial and open-source models revealed that existing long-context capabilities require significant advancement for effective one hundred thousand plus token processing. Performance degradation accelerates substantially at extreme lengths, with no model approaching human-level capability at two hundred thousand tokens. Domain variation proved striking, with natural language tasks showing relatively higher performance compared to code and mathematics domains. The gap between claimed context windows and effective context utilization grew increasingly apparent, with most models showing severe performance drops well before reaching their advertised limits.

11.3.2 BABILong: Reasoning Across Extreme Lengths

BABILong, developed by [200], tests reasoning capabilities across extremely long documents extending to ten million tokens. Published in the NeurIPS twenty twenty-four Datasets and Benchmarks track, the benchmark evaluates reasoning over distributed facts using twenty diverse reasoning tasks including fact retrieval, fact chaining following logical dependencies, simple induction learning patterns from examples, deduction and logical inference, counting and aggregation, list and set manipulation, multi-hop reasoning requiring complex inference chains, temporal reasoning with time-aware fact relationships, and comparative reasoning across entities and attributes.

The benchmark’s unique extensibility enables scaling to arbitrary lengths, positioning it for evaluation of future architectures claiming even longer context windows. Pre-built splits extend to ten million tokens, representing orders of magnitude beyond current practical deployment contexts but enabling stress-testing of architectural limits. Tasks are based on bAbI dataset principles with facts embedded in PG19 background text, ensuring realistic document characteristics while maintaining ground truth for precise evaluation.

The benchmark’s major findings prove sobering for long-context research. Popular large language models effectively utilize only ten to twenty percent of their available context, ignoring the vast majority of provided information. Performance declines sharply with increased reasoning complexity, with simple retrieval achieving acceptable accuracy but complex multi-hop reasoning showing failure rates exceeding ninety-five percent at million-token scales. Retrieval-augmented generation methods achieve modest sixty percent accuracy on single-fact question answering but plateau without further improvement as context length increases, suggesting fundamental limitations in current RAG approaches.

Architectural comparisons revealed recurrent memory transformers, after substantial fine-tuning, achieving the best performance and processing up to fifty million tokens. However, this capability requires significant computational resources and task-specific adaptation, representing far from an out-of-the-box solution. The findings challenge the narrative that context window expansion alone solves long-context understanding, suggesting fundamental architectural limitations in how current

models process and reason over extended sequences.

11.3.3 SCROLLS and ZeroSCROLLS: Natural Long Text Understanding

The SCROLLS benchmark suite, introduced by [201], provides standardized comparison over naturally long texts, emphasizing documents that are intrinsically long rather than artificially extended. The consortium effort from Tel Aviv University, Meta AI, IBM Research, and the Allen Institute for AI aggregates seven diverse tasks including GovReport for government document summarization, SummScreenFD for screenplay summarization, QMSum for meeting transcript question answering, Qasper for research paper question answering, NarrativeQA for book-length narrative understanding, QuALITY for story comprehension, and ContractNLI for legal document inference.

These tasks span diverse domains and document characteristics, from technical government reports to narrative fiction, from academic papers to legal contracts. Document lengths range from thousands to tens of thousands of tokens, with sources reflecting genuine long-form content creation rather than synthetic concatenation. This naturalistic approach ensures evaluation reflects real-world document characteristics including complex formatting, domain-specific terminology, and the coherence patterns of human-authored long-form content.

ZeroSCROLLS, introduced by [202], extends SCROLLS to zero-shot evaluation, eliminating training data to focus on pure generalization capability. The benchmark adapts six SCROLLS tasks to zero-shot settings and introduces four new datasets emphasizing novel information fusion challenges: review aggregation requiring synthesis of sentiment across multiple reviews, scoping tasks identifying which statements apply to specific clauses in complex documents, conditional acceptance testing understanding of conditional logic, and information ordering requiring reconstruction of shuffled document structure.

Evaluation results revealed substantial performance gaps. GPT-4 achieved the highest average score at approximately forty-two points, with Claude at thirty-nine and ChatGPT trailing further behind. However, all models showed substantial room for improvement, particularly on aggregation tasks where naive baselines proved surprisingly competitive. The zero-shot challenge proved especially difficult for specialized tasks requiring novel information synthesis patterns, suggesting that current models rely heavily on task-specific patterns learned during training rather than developing generalizable long-context reasoning capabilities.

11.4 Holistic Evaluation Frameworks

11.4.1 HELMET: Application-Centric Assessment

HELMET, introduced by [203], provides holistic evaluation across seven diverse, application-centric categories, addressing the critical gap between synthetic benchmark performance and real-world utility. The benchmark evaluates retrieval-augmented generation using real retrieval passages, recall measuring information retention across context, long-document question answering requiring

comprehensive understanding, passage re-ranking for relevance assessment, citation testing proper source attribution, summarization evaluating information condensation, and in-context learning assessing few-shot adaptation capability.

The benchmark’s design emphasizes controllable context lengths from eight thousand to one hundred twenty-eight thousand tokens with straightforward extensibility, model-based evaluation for reliable metrics avoiding pitfalls of n-gram matching, few-shot prompting enabling robust assessment of base models without instruction tuning requirements, and comprehensive coverage through evaluation of fifty-nine long-context language models spanning commercial and open-source variants across multiple size classes.

Critical findings from HELMET evaluation challenged prevailing assumptions about long-context capabilities. Synthetic tasks, particularly NIAH, proved unreliable predictors of downstream performance, with models achieving perfect NIAH scores showing wide variation on real applications. The seven evaluation categories exhibited distinct performance trends with low inter-correlations, demonstrating that capability in one domain does not predict capability in others. Correlations between categories ranged from approximately zero point three five to zero point four five, indicating that long-context understanding is not a unitary capability but rather a collection of specialized skills.

Open-source models lagged significantly behind closed-source alternatives on full-context reasoning tasks, with the performance gap widening substantially as context length increased. While open-source models achieved competitive performance on simpler retrieval tasks, complex reasoning and synthesis proved particularly challenging. The benchmark’s findings suggest that RAG tasks provide the most reliable signal for rapid model development iteration, better predicting downstream performance than other task categories while requiring less computational expense for evaluation.

11.5 Evaluation Metrics and Methodologies

11.5.1 Perplexity and Its Limitations

Perplexity has traditionally served as a primary metric for language model evaluation, defined as the exponentiated average negative log-likelihood of a sequence. The metric provides an interpretable measure of prediction confidence, remains model-agnostic in application, requires no human annotations, and can be computed efficiently during training and evaluation. However, research has revealed fundamental limitations when applying perplexity to long-context assessment.

The core problem lies in token averaging. Traditional perplexity averages performance across all tokens in a sequence, failing to distinguish between key tokens critical for understanding and filler tokens providing little information. In long contexts, this averaging obscures the model’s actual capability on information-bearing content. Research demonstrates near-zero correlation between standard perplexity and long-context task performance, with models showing good perplexity but poor task accuracy and vice versa. The metric reflects local information modeling rather than

long-range dependency understanding, making it fundamentally unsuited for evaluating extended context capabilities.

11.5.2 LongPPL: A Context-Aware Alternative

Addressing perplexity’s limitations, researchers have developed LongPPL, a metric specifically designed for long-context evaluation through long-short context contrastive methods. The approach identifies key tokens by comparing model predictions with long versus short context, with tokens showing large prediction differences classified as key tokens benefiting from extended context. LongPPL then calculates perplexity focusing exclusively on these key tokens rather than averaging across all positions.

This targeted approach achieves remarkable correlation with long-context benchmarks, with Pearson correlation coefficients reaching negative zero point nine six compared to near zero for standard perplexity. The metric demonstrates strong predictive power across diverse benchmarks including LongBench tasks showing correlations of zero point eight five to zero point nine five, InfiniteBench with correlations of zero point seven nine to zero point nine two, and RULER achieving zero point eight two to zero point nine four correlation. These results establish LongPPL as a reliable proxy metric enabling efficient screening of model capabilities before expensive full benchmark evaluation.

The development of LongPPL inspired corresponding training methodology. LongCE applies key-token weighting to training loss, prioritizing learning on information-bearing tokens. Models trained with LongCE loss demonstrate consistent improvements across diverse benchmarks, with gains of two to five percent on LongBench, three to seven percent on LongBench v2, and task-specific improvements reaching ten to fifteen percent on particularly challenging evaluations.

11.5.3 Memory-Specific Evaluation Metrics

The emergence of explicit memory systems in long-context architectures necessitates specialized evaluation metrics beyond standard language modeling assessment. MemBench, introduced in twenty twenty-four, provides comprehensive memory evaluation from multiple perspectives including effectiveness measuring retrieval accuracy, efficiency assessing computational cost, capacity evaluating maximum storable information, and interaction scenarios testing both participation and observation modes.

Memory evaluation distinguishes between factual memory for specific information retention and reflective memory for understanding and summarization capability. Temporal dynamics prove critical, with recall accuracy varying substantially based on information insertion position and temporal distance. Metrics include recall accuracy across varying temporal distances, hallucination rate in memory retrieval assessing false positive generation, memory system latency and token usage measuring efficiency, retrieval relevance and precision evaluating quality, and user satisfaction with memory-augmented responses capturing practical utility.

LoCoMo, evaluating very long-term conversational memory, demonstrated that long-context models improve memory performance by twenty-two to sixty-six percent over baseline approaches but still lag human performance by fifty-six percent on average. Temporal reasoning proves particularly challenging, with a seventy-three percent gap to human capability. These findings suggest fundamental limitations in current memory mechanisms, with substantial room for improvement in how models maintain and reason over extended interaction histories.

11.6 Open Problems and Future Directions

11.6.1 Scaling Laws for Long Context

Understanding how model performance scales with context length remains an active research challenge with profound implications for architecture design and training methodology. Current models exhibit non-linear performance degradation, with cliff behaviors and phase transitions rather than smooth decline as context length increases. The optimal allocation of training data between short and long contexts remains unclear, with fine-tuning on extended sequences sometimes reducing short-context performance, suggesting fundamental trade-offs that current training approaches fail to address.

Scaling laws appear architecture-dependent, with different position encoding schemes including RoPE, ALiBi, and others exhibiting distinct degradation patterns. No unified framework exists for predicting how architectural choices influence context length capabilities, hampering principled architecture design. Empirical observations from RULER suggest that only fifty percent of models maintain acceptable performance at their claimed context length, while BABILong demonstrates effective utilization rates of merely ten to twenty percent, indicating vast gaps between theoretical capacity and practical capability.

11.6.2 Unified Memory Architectures

The integration of multiple memory types including short-term context windows, working memory for ongoing computation, long-term persistent storage, and semantic abstracted knowledge remains an open challenge. Current approaches treat these memory types independently, lacking principled frameworks for resource allocation and information flow between memory layers. Memory-computation trade-offs prove complex, with larger working memory requiring more computation, long-term storage introducing retrieval latency, and compression causing information loss. No optimization framework exists for balancing these trade-offs based on task requirements and computational constraints.

Query-memory alignment presents particular challenges. Information storage occurs without knowledge of future query distributions, necessitating either generic storage that may not match specific queries or compression based on assumed query patterns that may prove incorrect. Adaptive storage strategies that learn to prioritize information based on likely future queries remain largely

unexplored. Memory coherence across layers adds further complexity, with information potentially becoming outdated, conflicts emerging between memory sources, and temporal coherence requiring tracking how facts change over time.

11.6.3 Fundamental Limits of Unbounded Context

Whether truly unbounded context processing is achievable remains an open question with both theoretical and practical dimensions. Information-theoretic analysis suggests fundamental capacity constraints, with finite parameter models necessarily limited in representable long-range dependencies. The relationship between model size and maximum effective context length lacks theoretical characterization, hampering principled scaling decisions. Attention mechanism limitations prove particularly acute, with quadratic complexity in context length, observed diagonal attention patterns on long sequences indicating concentration on local context, and positional encoding limitations as sequences extend beyond training distributions.

Empirical evidence suggests fundamental limits may exist. The observation that models utilize only ten to twenty percent of provided context, combined with lost-in-the-middle phenomena showing sixty-three percent accuracy degradation for middle-positioned information, indicates that uniform attention distribution over extended sequences may be fundamentally infeasible. Alternative architectures including Infini-Attention with compressed key-value caches, Megalodon using linear-complexity exponential moving average attention, state-space models like Mamba providing alternatives to attention, and recurrent transformers emphasizing memory retrieval show promise but face their own limitations and trade-offs.

11.6.4 Evaluation Framework Unification

The proliferation of over ten major benchmarks with inconsistent methodologies creates challenges for comparative assessment and capability characterization. Different benchmarks employ varying task definitions, document sources, context length ranges, and evaluation metrics, making cross-benchmark comparison difficult. No consensus exists on minimal sufficient task sets for characterizing long-context capability, nor on appropriate metric selection for different task types.

Benchmark gaming increasingly threatens evaluation validity. Models trained on test sets or similar synthetic data distributions may achieve artificially high scores that do not reflect genuine capability. Context window claims often lack validation through comprehensive benchmark evaluation, with marketing assertions exceeding demonstrated performance by substantial margins. The community requires dynamic benchmarks that evolve alongside model capabilities, preventing overfitting to fixed evaluation suites while maintaining comparability across time.

11.6.5 Real-World Grounding and Domain Transfer

Synthetic benchmarks, while providing controlled evaluation environments, increasingly diverge from actual deployment scenarios. Domain shift proves substantial, with models trained and eval-

uated on general domains showing unpredictable performance on specialized applications in law, medicine, finance, and other expert domains. Real documents exhibit characteristics absent from synthetic constructs, including complex formatting, embedded tables and figures, domain-specific terminology, and the coherence patterns of human-authored content. Multimodal documents combining text, images, charts, and structured data require evaluation approaches beyond current text-only benchmarks.

Interactive evaluation remains underdeveloped relative to its practical importance. Benchmarks predominantly employ single-turn question answering, while real applications involve multi-turn conversations with memory across turns, iterative refinement of queries and responses, and integration of user feedback. Static evaluation fails to capture adaptive capabilities essential for practical deployment, including how models learn from interactions, adapt to user preferences, and maintain consistency across extended engagements.

11.6.6 Computational Efficiency of Evaluation

The computational expense of long-context evaluation creates significant barriers to research progress and model development iteration. Evaluation at one hundred thousand plus token contexts requires substantial time and resources, with full benchmark suites potentially requiring weeks of computation even on high-end hardware. This cost prohibits rapid iteration essential for model development, limiting evaluation to well-resourced organizations and slowing progress across the research community.

Surrogate metrics including LongPPL show promise for efficient screening, enabling rapid capability assessment before expensive full evaluation. However, the extent to which surrogate metrics can replace task-specific evaluation remains unclear. Stratified sampling approaches that evaluate on task subsets while maintaining statistical validity could reduce costs but risk missing important capability variations. Transfer learning between benchmarks, using performance on one suite to predict results on another, requires better understanding of correlation structure across evaluation paradigms.

11.6.7 Five Fundamental Limitations

Recent theoretical and empirical analysis has identified five intrinsic challenges for language models operating over unbounded contexts that may prove fundamental rather than merely implementation-dependent. First, hallucination emerges as an intrinsic property of learning systems operating over unbounded, open-ended query spaces with necessarily incomplete training data. No amount of training can eliminate hallucination in the limit of unbounded context, as the model must generalize beyond observed distributions. Second, context compression inherently causes information loss when reducing extended sequences to fixed representations, with no lossless compression scheme for arbitrary natural language. Third, reasoning degradation occurs as inference chains extend across tokens, with error accumulation and attention dilution causing progressively worse performance on

multi-hop reasoning. Fourth, retrieval fragility manifests when irrelevant information dominates long contexts, with models struggling to identify and focus on relevant subsets. Fifth, multimodal misalignment challenges coherence maintenance across modalities in extended contexts, particularly as image-text relationships must be tracked over thousands of tokens.

These limitations suggest that unbounded context processing may require fundamentally different paradigms rather than incremental improvements to current architectures. Hierarchical processing with multiple levels of abstraction, explicit symbolic reasoning combined with neural pattern recognition, and memory systems with explicit structure and access patterns represent potential directions, though each introduces new challenges and trade-offs.

The evolution of long-context evaluation reflects the field’s maturation from simple claims about context window size to sophisticated understanding of effective utilization, task-specific capabilities, and fundamental limitations. While context windows have expanded dramatically, evaluation reveals that effective context utilization lags far behind architectural capacity. Current models demonstrate remarkable capabilities on specific tasks but show brittleness and inconsistency across the full range of long-context applications. Future progress requires coordinated advances in architecture design informed by theoretical understanding of fundamental limits, training methodologies that more effectively teach long-range reasoning and information integration, evaluation frameworks that accurately predict real-world performance while remaining computationally tractable, and memory systems that efficiently maintain and access relevant information across extended interactions.

The disconnect between synthetic benchmark performance and practical utility remains perhaps the most pressing challenge, with models achieving high scores on controlled evaluations while failing on real deployment scenarios. Bridging this gap requires evaluation approaches that better reflect actual use cases, more comprehensive assessment across diverse task types and domains, and honest characterization of both capabilities and limitations. As the field moves toward even longer contexts and more sophisticated applications, evaluation methodology must continue evolving to provide reliable signal about genuine capability rather than performance on increasingly gameable static benchmarks. The future of long-context language models depends as much on improved evaluation as on architectural innovation, with better measurement essential for guiding development toward truly useful long-context understanding.

12 Discussion: Synthesis and Future Directions

The preceding sections have surveyed a remarkably diverse ecosystem of techniques for managing context in large language models, spanning architectural innovations, algorithmic optimizations, memory-augmented systems, and human-inspired hierarchical memory architectures. This diversity reflects both the multifaceted nature of the context management challenge and the absence of a single dominant solution. As we synthesize insights across these approaches, several overarching themes emerge that illuminate the current state of the field and suggest productive directions for

future research.

The first theme concerns the fundamental tension between context capacity and context utilization. While architectural innovations have extended nominal context windows from thousands to millions of tokens, empirical evaluation consistently reveals that effective utilization lags far behind theoretical capacity. The lost-in-the-middle phenomenon, whereby information positioned in central portions of long contexts becomes substantially less accessible than boundary-positioned content, persists across model scales and architectures. Evaluation on benchmarks such as BABILong demonstrates that models utilize only ten to twenty percent of their available context, effectively ignoring the vast majority of provided information. This gap between capacity and utilization suggests that raw context expansion, while necessary, is insufficient for robust long-context understanding. Future progress requires not merely larger windows but architectural modifications that enable uniform access to information regardless of position, training procedures that explicitly address positional biases, and memory systems that selectively maintain high-fidelity representations of important information while compressing or abstracting less critical content.

The second theme addresses the interplay between efficiency and expressiveness. Quadratic attention complexity creates an exponential barrier to context scaling, motivating extensive research into efficient alternatives including sparse attention patterns, linear attention approximations, state space models, and hardware-aware optimizations. Each approach navigates distinct trade-offs. Sparse attention methods such as Longformer and BigBird achieve linear complexity through structured sparsity while maintaining exact computation on the sparse support, but fixed patterns may miss task-relevant dependencies falling outside predefined structures. Linear approximations including Performer and Linformer reduce complexity through mathematical reformulation but introduce approximation error that compounds across layers. State space models such as S4 and Mamba abandon attention entirely in favor of recurrence-based alternatives, achieving impressive efficiency but requiring different training procedures and exhibiting different inductive biases. FlashAttention demonstrates that exact attention can be dramatically accelerated through memory hierarchy optimization without changing the underlying computation, achieving two to nine times speedup through algorithmic restructuring alone. The coexistence of these diverse approaches suggests that optimal solutions may be context-dependent, with different applications benefiting from different efficiency-expressiveness trade-offs. Hybrid architectures such as Jamba, which interleave transformer layers with Mamba layers and mixture-of-experts, exemplify the emerging recognition that heterogeneous designs selecting components based on computational role may outperform monolithic architectures committed to a single mechanism.

The third theme concerns the integration of parametric and non-parametric memory. Retrieval-augmented generation has emerged as a dominant paradigm for grounding generation in factual knowledge, enabling models to access information beyond their training cutoff and providing explicit source attribution for generated claims. The evolution from foundational RAG through advanced variants such as Self-RAG, CRAG, and GraphRAG demonstrates increasing sophistication

in when, what, and how to retrieve, incorporating self-reflection mechanisms that decide whether retrieval is necessary, correction procedures that detect and remediate retrieval failures, and structured knowledge representations that enable multi-hop reasoning across graph-organized information. However, retrieval-augmented systems remain vulnerable to position effects, as retrieved passages positioned in middle contexts suffer from the same accessibility degradation observed in standard long-context processing. Furthermore, the separation between retrieval and generation introduces latency and complexity, motivating research into tighter integration such as RETRO’s retrieval during pre-training and Fusion-in-Decoder’s late fusion of multiple passages. The convergence of retrieval-augmented approaches with prompt compression techniques such as LLMingua and context management systems such as MemGPT suggests a future where memory systems fluidly combine parametric knowledge in model weights, working memory in the context window, and long-term memory in external stores, with intelligent mechanisms managing information flow between these tiers.

The fourth theme addresses the productive application of cognitive science to computational memory systems. The hierarchical human-inspired memory architectures demonstrate that principles from experimental psychology and cognitive neuroscience provide valuable design guidance for artificial memory systems. The Atkinson-Shiffrin multi-store model, distinguishing sensory, short-term, and long-term memory with distinct characteristics and capacities, maps naturally onto systems with immediate context windows, working memory for active computation, and external long-term storage. Baddeley’s working memory framework, decomposing working memory into phonological loop, visuospatial sketchpad, and central executive components, informs agent architectures with specialized buffers for different information types. Tulving’s distinction between episodic memory for specific experiences and semantic memory for general knowledge motivates systems that maintain both event logs and abstracted summaries. Ebbinghaus’s forgetting curve and the spacing effect inform decay mechanisms that naturally prioritize recently accessed and frequently retrieved information. McClelland’s complementary learning systems theory, positing rapid episodic encoding in hippocampus and slow statistical learning in neocortex, parallels systems that combine fast few-shot in-context learning with slow parameter updates through fine-tuning.

The Generative Agents framework exemplifies these principles through its three-component memory system combining perception, reflection, and planning. MemoryBank advances this architecture through dynamic memory updating, knowledge retrieval, and memory consolidation via summarization. The Cognitive Architectures for Language Agents framework provides a general specification for memory-augmented agents, distinguishing working memory from long-term memory and defining interfaces for selective retention and consolidation. The Self-Controlled Memory framework introduces self-awareness and metacognitive control over memory operations, enabling agents to decide what to remember and when to forget based on task demands. These systems demonstrate that cognitively-inspired architectures can overcome limitations of purely engineering-driven approaches, producing more robust, interpretable, and human-like memory systems.

The fifth theme concerns the persistent challenge of hallucination and its intimate connection to context management. Hallucinations arise from multiple interacting factors including incomplete or biased training data, insufficient capacity to represent all factual knowledge, and context management failures that prevent access to relevant information during generation. The distinction between faithfulness to provided context and factual accuracy with respect to world knowledge proves critical, as interventions targeting one dimension may degrade the other. The lost-in-the-middle effect directly contributes to hallucination through making middle-positioned information inaccessible, causing models to generate content inconsistent with provided evidence. Mitigation strategies including self-consistency, chain-of-verification, and retrieval grounding demonstrate varying degrees of effectiveness, but hallucinations persist at non-negligible rates even in sophisticated systems combining multiple mitigation approaches. Future progress requires not only continued refinement of mitigation techniques but also architectural innovations addressing the root causes, including position-invariant attention mechanisms, improved position encoding schemes, and memory systems with explicit source tracking enabling verification of generated claims.

The sixth theme addresses multi-turn conversational coherence, which presents unique challenges beyond single-document processing. Conversations evolve incrementally over multiple turns and sessions, requiring coherent entity state tracking, coreference resolution, cross-session continuity, and management of progressive context growth that inevitably exceeds available windows. The 39 percent performance degradation observed in multi-turn scenarios compared to single-turn evaluation, persisting across frontier models regardless of context window size, indicates fundamental limitations in how current architectures integrate information across extended conversational sequences. Approaches from conversation summarization through hierarchical memory to dialogue state tracking each address specific aspects of this challenge, but no comprehensive solution has emerged. The convergence of techniques including intelligent truncation, recursive summarization, vectorized memory retrieval, and psychology-informed importance weighting suggests that robust multi-turn systems will require orchestrating multiple complementary mechanisms.

The seventh theme concerns evaluation methodology and the critical gaps between synthetic benchmarks and real-world performance. While benchmarks such as Needle-in-a-Haystack, RULER, LongBench, InfiniteBench, HELMET, BABILong, and SCROLLS provide valuable standardized evaluation, research increasingly reveals that strong performance on synthetic tasks fails to predict success on practical applications. Models achieving near-perfect scores on simple retrieval exhibit large performance drops on tasks requiring multi-hop reasoning, aggregation, or synthesis. The low correlation between different evaluation categories demonstrates that long-context understanding is not a unitary capability but a collection of specialized skills with limited transfer. Furthermore, evaluation predominantly employs single-turn question answering while real applications involve multi-turn interaction, adaptive behavior, and integration of user feedback. The development of more sophisticated metrics such as LongPPL, which focuses on key tokens showing long-short context differences, represents progress toward evaluation that better predicts downstream performance.

Looking forward, several research directions appear particularly promising for advancing the state of long-context language models. First, unified memory architectures integrating parametric knowledge, working memory, and long-term storage with principled mechanisms for information flow, resource allocation, and coherence maintenance across memory tiers could provide more robust and efficient context management than current approaches treating these components independently. Second, architecture-aware training procedures that explicitly address positional biases, teach uniform context access, and develop robust mechanisms for information integration across extended sequences may improve effective utilization toward theoretical capacity. Third, dynamic adaptation mechanisms that adjust compression ratios, retrieval strategies, and attention patterns based on task demands, input characteristics, and available computational resources could enable systems to navigate efficiency-quality trade-offs more intelligently. Fourth, neurosymbolic approaches combining neural pattern recognition with explicit symbolic reasoning and structured knowledge representation may address fundamental limitations in multi-hop reasoning and causal inference that persist despite scaling. Fifth, hardware-software co-design optimizing memory hierarchies, communication patterns, and computational kernels specifically for long-context workloads could unlock substantial efficiency gains beyond what general-purpose hardware permits.

The rapid pace of innovation in context management throughout 2023 through 2026, with weekly advances in compression techniques, memory architectures, attention mechanisms, and evaluation methodologies, reflects both the critical importance of this research area and the substantial remaining challenges. Context windows have expanded from thousands to millions of tokens, enabling applications from book-length document analysis to multi-session conversations spanning months. Yet effective utilization of these extended windows remains limited, with models struggling to maintain coherent reasoning across extended contexts, exhibiting systematic positional biases, and often ignoring the majority of provided information. Closing the gap between capacity and utilization, between synthetic benchmarks and practical performance, and between engineering optimization and cognitively-grounded memory systems constitutes the central challenge for the field moving forward.

13 Conclusion

This survey has examined the multifaceted challenge of context management in large language models through ten interconnected perspectives, tracing a progression from foundational architectural constraints through diverse mitigation strategies to cognitively-inspired memory systems. The transformer’s self-attention mechanism, while enabling the remarkable scaling and capabilities of modern language models, imposes a quadratic computational complexity that fundamentally limits context length. This constraint has motivated an ecosystem of techniques spanning architectural modifications to reduce complexity, algorithmic optimizations to improve efficiency, memory-augmented systems to extend effective capacity, and human-inspired hierarchical architectures to manage information across multiple timescales and granularities.

The central insight emerging from this survey is that effective context management requires coordinated advances across multiple dimensions rather than optimization of any single component in isolation. Architectural efficiency techniques including sparse attention, state space models, and hardware-aware implementations address computational bottlenecks but do not resolve challenges of effective utilization and information integration. Memory-augmented and retrieval-augmented systems ground generation in external knowledge and enable access to information beyond parametric capacity but remain vulnerable to position effects and retrieval failures. Context compression methods achieve substantial size reduction but face inevitable information loss and must balance compression ratio against task performance. Human-inspired memory architectures provide cognitively-grounded design principles for hierarchical organization, selective consolidation, and intelligent forgetting but require substantial engineering to implement effectively and scale to production deployment.

The persistent gap between nominal context capacity and effective utilization, evidenced by phenomena such as lost-in-the-middle effects and the observation that models utilize only ten to twenty percent of available context, indicates that raw window expansion alone is insufficient. Future progress requires architectural innovations enabling position-invariant attention, training procedures explicitly addressing positional biases, memory systems with explicit structure and access patterns, and evaluation frameworks that accurately predict real-world performance rather than synthetic benchmark scores. The convergence of insights from computer architecture, algorithm design, cognitive science, and empirical machine learning provides the foundation for this continued progress.

As language models continue their trajectory toward even longer contexts, more sophisticated reasoning, and broader deployment in critical applications, the importance of principled context management will only intensify. The techniques surveyed in this work demonstrate that diverse perspectives and approaches contribute essential insights. The future of long-context language understanding lies not in any single technique but in the intelligent orchestration of complementary mechanisms, drawing inspiration from both computational efficiency and human cognition to build systems capable of coherent, scalable, and robust reasoning over effectively unbounded information streams.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [2] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” *arXiv preprint arXiv:2305.13245*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.13245>

- [3] P. Shaw, J. Uszkoreit, and A. Vaswani, “Self-attention with relative position representations,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2018, pp. 464–468. [Online]. Available: <https://aclanthology.org/N18-2074/>
- [4] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, “Transformer-XL: Attentive language models beyond a fixed-length context,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 2978–2988. [Online]. Available: <https://aclanthology.org/P19-1285/>
- [5] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024. [Online]. Available: <https://doi.org/10.1016/j.neucom.2023.127063>
- [6] O. Press, N. A. Smith, and M. Lewis, “Train short, test long: Attention with linear biases enables input length extrapolation,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=R8sQPpGCv0>
- [7] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. [Online]. Available: https://doi.org/10.1162/tacl_a_00638
- [8] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=NG7sS51zVF>
- [9] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.05150>
- [10] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, “Big bird: Transformers for longer sequences,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: <https://arxiv.org/abs/2007.14062>
- [11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [12] S. Chen, S. Wong, L. Chen, and Y. Tian, “Extending context window of large language models via positional interpolation,” *arXiv preprint arXiv:2306.15595*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.15595>
- [13] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, “YaRN: Efficient context window extension

- of large language models,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=wHBfxhZu1u>
- [14] Y. Ding, L. L. Zhang, C. Zhang, Y. Xu, N. Shang, J. Xu, F. Yang, and M. Yang, “LongRoPE: Extending LLM context window beyond 2 million tokens,” in *International Conference on Machine Learning*, 2024. [Online]. Available: <https://proceedings.mlr.press/v235/ding24am.html>
- [15] J. Yu *et al.*, “ComRoPE: Scalable and robust rotary position embedding,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, arXiv:2506.03737. [Online]. Available: <https://arxiv.org/abs/2506.03737>
- [16] Z. Zhang *et al.*, “Exploring context window of large language models via decomposed positional vectors,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 9975–9991, distributional perspective on positional encoding. [Online]. Available: <https://arxiv.org/abs/2408.01570>
- [17] H. Liu, M. Zaharia, and P. Abbeel, “Ring attention with blockwise transformers for near-infinite context,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.01889>
- [18] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” *arXiv preprint arXiv:2307.08691*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [19] A. Mohtashami and M. Jaggi, “Landmark attention: Random-access infinite context length for transformers,” in *International Conference on Learning Representations*, 2024, arXiv:2305.16300. [Online]. Available: <https://arxiv.org/abs/2305.16300>
- [20] A. Bulatov, Y. Kuratov, and M. S. Burtsev, “Recurrent memory transformer,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 11 079–11 091. [Online]. Available: <https://arxiv.org/abs/2203.16634>
- [21] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, and F. Wei, “Augmenting language models with long-term memory,” *Advances in Neural Information Processing Systems*, vol. 36, 2023, arXiv:2306.07174. [Online]. Available: <https://arxiv.org/abs/2306.07174>
- [22] H. Jin, X. Han, J. Yang, Z. Jiang, Z. Liu, C.-Y. Chang, H. Chen, and X. Ren, “LLM maybe LongLM: Self-extend LLM context window without tuning,” *arXiv preprint arXiv:2401.01325*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.01325>
- [23] Y. Bai, X. Lv, J. Zhang, Y. He, J. Qi, L. Hou, J. Tang, Y. Dong, and J. Li, “LongAlign: A recipe for long context alignment of large language models,” *arXiv preprint arXiv:2401.18058*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.18058>
- [24] Xnhyacinth, J. Sun, B. Zhou, Y. Xiong *et al.*, “E²-LLM: Efficient and extreme length extension

- of large language models,” *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, arXiv:2401.06951. [Online]. Available: <https://arxiv.org/abs/2401.06951>
- [25] G. Chen, X. Li, Z. Meng, S. Liang, and L. Bing, “CLEX: Continuous length extrapolation for large language models,” in *International Conference on Learning Representations*, 2024, arXiv:2310.16450. [Online]. Available: <https://arxiv.org/abs/2310.16450>
- [26] H. Li, Y. Liu, J. Zhang, B. Min *et al.*, “SkyLadder: Better and faster pretraining via context window scheduling,” *arXiv preprint arXiv:2503.15450*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.15450>
- [27] Y. Zhang, T. Wu, P. Ding *et al.*, “Core context aware attention for long context language modeling,” *arXiv preprint arXiv:2412.12465*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.12465>
- [28] W. Jiang, S. Zhou, S. Wu, C. Wang, X. Qi, and Y. Lu, “TokenSelect: Efficient long-context inference and length generalization for LLMs via dynamic token selection,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 22 089–22 104, eMNLP 2025. [Online]. Available: <https://arxiv.org/abs/2502.14098>
- [29] J. Chang, X. Yu, H. Deng, Z. Wu, and X. Huang, “Extending LLM context window with adaptive grouped attention,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 489–501, aCL 2025. [Online]. Available: <https://arxiv.org/abs/2502.10182>
- [30] J. Kim, M. Park, J. Lee, and D. Kim, “Extending context window in large language models with segmented base adjustment for rotary position embeddings,” in *Applied Sciences*, vol. 14, no. 7. MDPI, 2024, p. 3076. [Online]. Available: <https://arxiv.org/abs/2402.08657>
- [31] Y. Chen, T. Zhang, and X. Liu, “Bayesian attention mechanism: A probabilistic framework for positional encoding and context length extrapolation,” *arXiv preprint arXiv:2505.22842*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.22842>
- [32] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, “Efficient transformers: A survey,” *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–28, 2022, arXiv:2009.06732. [Online]. Available: <https://arxiv.org/abs/2009.06732>
- [33] K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller, “Rethinking attention with performers,” in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=Ua6zuk0WRH>
- [34] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, “Linformer: Self-attention with linear complexity,” *arXiv preprint arXiv:2006.04768*, 2020. [Online]. Available: <https://arxiv.org/abs/2006.04768>

- [35] A. Gu, K. Goel, and C. Ré, “Efficiently modeling long sequences with structured state spaces,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=uYLFoz1vlAC>
- [36] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.00752>
- [37] O. Lieber, B. Lenz, H. Bata, G. Cohen, J. Osin, I. Dalmedigos, E. Safahi, S. Meirom, Y. Belinkov, A. Shashua, and Y. Shoham, “Jamba: A hybrid transformer-mamba language model,” *arXiv preprint arXiv:2403.19887*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.19887>
- [38] B. Peng, E. Alcaide, Q. Anthony *et al.*, “RWKV: Reinventing RNNs for the transformer era,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. [Online]. Available: <https://doi.org/10.18653/v1/2023.findings-emnlp.936>
- [39] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei, “Retentive network: A successor to transformer for large language models,” *arXiv preprint arXiv:2307.08621*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.08621>
- [40] R. Child, S. Gray, A. Radford, and I. Sutskever, “Generating long sequences with sparse transformers,” *arXiv preprint arXiv:1904.10509*, 2019. [Online]. Available: <https://arxiv.org/abs/1904.10509>
- [41] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, “Synthesizer: Rethinking self-attention in transformer models,” in *International Conference on Machine Learning (ICML)*, 2021, arXiv:2005.00743. [Online]. Available: <https://arxiv.org/abs/2005.00743>
- [42] Y. Xiong, Z. Zeng, R. Chakraborty, M. Tan, G. Fung, Y. Li, and V. Singh, “Nyströmformer: A nyström-based algorithm for approximating self-attention,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 16, 2021, pp. 14 138–14 148, arXiv:2102.03902. [Online]. Available: <https://arxiv.org/abs/2102.03902>
- [43] M. Poli, S. Massaroli, E. Nguyen, D. Y. Fu, T. Dao, S. Baccus, Y. Bengio, S. Ermon, and C. Ré, “Hyena hierarchy: Towards larger convolutional language models,” in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2302.10866. [Online]. Available: <https://arxiv.org/abs/2302.10866>
- [44] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, and D. Metzler, “Long range arena: A benchmark for efficient transformers,” in *International Conference on Learning Representations (ICLR)*, 2021, arXiv:2011.04006. [Online]. Available: <https://arxiv.org/abs/2011.04006>
- [45] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with

- PagedAttention,” in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [46] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H₂O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.14048>
- [47] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, “Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.17118>
- [48] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2309.17453>
- [49] X. Wu, H. Li, Y. Liu, J. Wang, Y. Yang, T. Wei, L. Chen, D. Qin, J. Song, Y. Gao, W. Huang *et al.*, “When attention sink emerges in language models: An empirical view,” in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: <https://arxiv.org/abs/2410.08560>
- [50] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, “Model tells you what to discard: Adaptive KV cache compression for LLMs,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=uNrFpDPMYo>
- [51] Y. Li *et al.*, “SnapKV: LLM knows what you are looking for before generation,” *arXiv preprint arXiv:2404.14469*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.14469>
- [52] Z. Cai *et al.*, “PyramidKV: Dynamic KV cache compression based on pyramidal information funneling,” *arXiv preprint arXiv:2406.02069*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.02069>
- [53] Z. Kang, J. Qiu, J. Sun, M. Wang, J. Chen, J. Jiang, W. Wang, Q. Cheng, and W. Lin, “Q-HITTER: A better token oracle for efficient LLM inference via sparse-quantized KV cache,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 6, 2024. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/file/bbb7506579431a85861a05fff048d3e1-Paper-Conference.pdf
- [54] L. Liu, H. Li, Y. Hu, C. Zhuang, K. Cheng, J. Zhou, and N. Xia, “HashEvict: A pre-attention KV cache eviction strategy using locality-sensitive hashing,” *arXiv preprint arXiv:2412.16187*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.16187>
- [55] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A tuning-

- free asymmetric 2bit quantization for KV cache,” in *International Conference on Machine Learning*, 2024. [Online]. Available: <https://proceedings.mlr.press/v235/liu24bz.html>
- [56] C. Hooper, S. Kim, H. Mohammadi, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/028fcbcf85435d39a40c4d61b42c99a4-Abstract-Conference.html
- [57] M. Kang, X. Zhang, C. Lian, J. Yang, Z. Lyu, and J. Feng, “GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM,” *arXiv preprint arXiv:2403.05527*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.05527>
- [58] J. Yuan, Z. Liu, Q. Jin, Z. Chen, Z. Wang, and X. Wang, “KITTY: Accurate and efficient 2-bit KV cache quantization with information theory-guided token-wise clustering,” *arXiv preprint arXiv:2511.18643*, 2024. [Online]. Available: <https://arxiv.org/abs/2511.18643>
- [59] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” *arXiv preprint arXiv:1911.02150*, 2019. [Online]. Available: <https://arxiv.org/abs/1911.02150>
- [60] W. Brandon, M. Mishra, A. Nrusimha, R. Panda, and J. R. Kelly, “Reducing transformer key-value cache size with cross-layer attention,” *arXiv preprint arXiv:2405.12981*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.12981>
- [61] H. Yang, Z. Yue, Z. Wang, Y. Huang, R. Wu, Y. Zhu, J. Zhang, Y. Xian, and Y. Pan, “Layer-condensed KV cache for efficient inference of large language models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 11 380–11 391. [Online]. Available: <https://arxiv.org/abs/2405.10637>
- [62] K. Tu, Z. Li, Z. Wu, J. Cheng, Q. Cao, and X. Zhang, “A systematic study of cross-layer KV sharing for efficient LLM inference,” in *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025, pp. 577–588. [Online]. Available: <https://arxiv.org/abs/2410.14442>
- [63] Z. Jia *et al.*, “MiniCache: KV cache compression in depth dimension for large language models,” in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/fd0705710bf01b88a60a3d479ea341d9-Abstract-Conference.html
- [64] R. Liu, Y. Liu, Z. Chen, Y. Fang, H. Yuan, H. Xue, Y. Zhang, O. Ruwase, A. Li, and X. Xie, “vAttention: Dynamic memory management for serving LLMs without PagedAttention,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, vol. 1, 2025, pp. 1092–1107. [Online]. Available: <https://arxiv.org/abs/2405.04437>

- [65] I. Gao, M. Cai, E. Hsu, Y. S. Lu, A. Bharadwaj, P. Cheng, W.-L. Chiang, Y. Xue, I. Stoica, and J. E. Gonzalez, “Prompt cache: Modular attention reuse for low-latency inference,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 6, 2024, pp. 636–653. [Online]. Available: <https://arxiv.org/abs/2311.04934>
- [66] Z. Li, D. He, C. Cheng, Q. Liu, X. Xu, W. Qian, D. Zhu, H. Wang, T. Liu, J. Zhou, P. Song, Z. Luan, D. Yuan, and D. Qian, “SqueezeAttention: 2D management of KV-cache in LLM inference via layer-wise optimal budget,” in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: <https://arxiv.org/abs/2404.04793>
- [67] J. Kang, J. Park, H. Kim, S. Shin, D. Lee, H. Park, and E. Ko, “Entropy-guided KV caching for efficient LLM inference,” *Mathematics*, vol. 13, no. 15, p. 2366, 2024. [Online]. Available: <https://doi.org/10.3390/math13152366>
- [68] P. S. Paul, S. Das, N. Sanket, A. Deshmukh, and C. Kanakakis, “The pitfalls of compressing the KV cache,” *arXiv preprint arXiv:2510.00231*, 2024. [Online]. Available: <https://arxiv.org/abs/2510.00231>
- [69] Y. Wang, Z. Yang, X. Liu, Z. Zhang, X. Yang, C. Chen, Y. Liu, and X. Shen, “Hold onto that thought: Assessing KV cache compression on reasoning,” *arXiv preprint arXiv:2512.12008*, 2024. [Online]. Available: <https://arxiv.org/abs/2512.12008>
- [70] C. Han, Q. Wang, W. Xiong, Y. Chen, H. Ji, and S. Wang, “LM-Infinite: Zero-shot extreme length generalization for large language models,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, pp. 3884–3902. [Online]. Available: <https://arxiv.org/abs/2308.16137>
- [71] C. Zhang, Z. Zhang, X. Zhao, Y. Wang, Y. Cao, Y. An, Z. Liu, M. Xu, X. Nie, J.-R. Wen *et al.*, “InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.04617>
- [72] Y. Sun, L. Dong, B. Patra, S. Ma, S. Huang, A. Benhaim, V. Chaudhary, X. Song, and F. Wei, “YOCO: You only cache once for decoder-decoder architectures,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.05254>
- [73] Y. Liu, Y. Huang, Y. Wang, X. Wang, X. Cao *et al.*, “LMCache: An efficient KV cache layer for enterprise-scale LLM inference,” *Technical Report*, 2024. [Online]. Available: https://lmcache.ai/tech_report.pdf
- [74] Z. Li, Y. Wang, Y. Fu, D. Wei, B. Xu, X. Huang, K. Ma, Q. Zhang, and E. Chen, “Prompt compression for large language models: A survey,” in *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics: Human*

- Language Technologies*. Association for Computational Linguistics, 2025, pp. 7022–7042. [Online]. Available: <https://aclanthology.org/2025.naacl-long.368/>
- [75] J. Mu, X. L. Li, and N. D. Goodman, “Learning to compress prompts with gist tokens,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2304.08467>
- [76] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, “Adapting language models to compress contexts,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.232/>
- [77] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei, “In-context autoencoder for context compression in a large language model,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=uREj4ZuGJE>
- [78] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.825/>
- [79] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. [Online]. Available: <https://aclanthology.org/2024.acl-long.454/>
- [80] Y. Li, B. Dong, F. Guerin, and C. Lin, “Compressing context to enhance inference efficiency of large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.391/>
- [81] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap, “Compressive transformers for long-range sequence modelling,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=SylKikSYDH>
- [82] C. Ge, C. Song, J. Shen, Y. Deng, X. Zhou, and X. Yan, “Recurrent context compression: Efficiently expanding the context window of LLM,” *arXiv preprint arXiv:2406.06110*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.06110>
- [83] Z. Zhang, K. Yang, J. Zhang, T. Zhang, Y. Wang, and Z. Li, “Expected attention: KV cache compression by estimating attention,” *arXiv preprint arXiv:2510.00636*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.00636>
- [84] X. Cheng, D. Luo, X. Chen, L. Zhao, Y. Ruan, and M. Zhou, “xRAG: Extreme context compression for retrieval-augmented generation with one token,” in *Advances*

- in *Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: <https://arxiv.org/abs/2405.13792>
- [85] A. Graves, G. Wayne, and I. Danihelka, “Neural turing machines,” *arXiv preprint arXiv:1410.5401*, 2014. [Online]. Available: <https://arxiv.org/abs/1410.5401>
- [86] J. Weston, S. Chopra, and A. Bordes, “Memory networks,” *arXiv preprint arXiv:1410.3916*, 2015. [Online]. Available: <https://arxiv.org/abs/1410.3916>
- [87] S. Sukhbaatar, A. Szlam, J. Weston, and R. Fergus, “End-to-end memory networks,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/hash/8fb21ee7a2207526da55a679f0332de2-Abstract.html
- [88] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>
- [89] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992. [Online]. Available: <https://doi.org/10.18653/v1/D19-1410>
- [90] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–48. [Online]. Available: <https://doi.org/10.1145/3397271.3401075>
- [91] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, “ColBERTv2: Effective and efficient retrieval via lightweight late interaction,” *arXiv preprint arXiv:2112.01488*, 2022. [Online]. Available: <https://arxiv.org/abs/2112.01488>
- [92] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
- [93] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark *et al.*, “Improving language models by retrieving from trillions of tokens,” in *International Conference on Machine Learning*, 2022. [Online]. Available: <https://proceedings.mlr.press/v162/borgeaud22a.html>
- [94] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” in *Proceedings of the 37th International*

- Conference on Machine Learning (ICML)*, 2020, pp. 3929–3938. [Online]. Available: <https://proceedings.mlr.press/v119/guu20a.html>
- [95] G. Izacard and É. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proceedings of the 2021 Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2021, pp. 874–880. [Online]. Available: <https://aclanthology.org/2021.eacl-main.74/>
- [96] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy, “Memorizing transformers,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=TrjbxzRcnf->
- [97] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=hSyW5go0v8>
- [98] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv preprint arXiv:2401.15884*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.15884>
- [99] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, “Active retrieval augmented generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 7969–7992. [Online]. Available: <https://openreview.net/forum?id=fSUs78pqwtV>
- [100] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” in *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022, pp. 1762–1777. [Online]. Available: <https://aclanthology.org/2023.acl-long.99/>
- [101] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.08560>
- [102] R. C. Atkinson and R. M. Shiffrin, “Human memory: A proposed system and its control processes,” in *The Psychology of Learning and Motivation*. Academic Press, 1968, vol. 2, pp. 89–195, dOI: 10.1016/S0079-7421(08)60422-3. [Online]. Available: <https://doi.org/10.18653/v1/D19-1410>
- [103] A. D. Baddeley and G. Hitch, “Working memory,” in *The Psychology of Learning and Motivation*. Academic Press, 1974, vol. 8, pp. 47–89, dOI: 10.1016/S0079-7421(08)60452-1. [Online]. Available: [https://doi.org/10.1016/S0079-7421\(08\)60452-1](https://doi.org/10.1016/S0079-7421(08)60452-1)
- [104] G. A. Miller, “The magical number seven, plus or minus two: Some limits on our capacity for processing information,” *Psychological Review*, vol. 63, no. 2, pp. 81–97, 1956. [Online]. Available: <https://doi.org/10.1037/h0043158>

- [105] E. Tulving, “Episodic and semantic memory,” in *Organization of Memory*. Academic Press, 1972, pp. 381–403. [Online]. Available: <https://psycnet.apa.org/record/1973-08477-004>
- [106] H. Ebbinghaus, *Memory: A Contribution to Experimental Psychology*. Dover, 1885. [Online]. Available: <https://psychclassics.yorku.ca/Ebbinghaus/index.htm>
- [107] I. Neath and A. M. Surprenant, “50 years of research sparked by Atkinson and Shiffrin (1968),” *Memory & Cognition*, vol. 47, pp. 561–574, 2019. [Online]. Available: <https://doi.org/10.3758/s13421-019-00896-7>
- [108] A. Baddeley, “The episodic buffer: A new component of working memory?” *Trends in Cognitive Sciences*, vol. 4, no. 11, pp. 417–423, 2000. [Online]. Available: [https://doi.org/10.1016/S1364-6613\(00\)01538-2](https://doi.org/10.1016/S1364-6613(00)01538-2)
- [109] —, “Working memory: Looking back and looking forward,” *Nature Reviews Neuroscience*, vol. 4, no. 10, pp. 829–839, 2003. [Online]. Available: <https://doi.org/10.1038/nrn1201>
- [110] N. Cowan, “The magical number 4 in short-term memory: A reconsideration of mental storage capacity,” *Behavioral and Brain Sciences*, vol. 24, no. 1, pp. 87–114, 2001. [Online]. Available: <https://doi.org/10.1017/S0140525X01003922>
- [111] B. B. Murdock, “The serial position effect of free recall,” *Journal of Experimental Psychology*, vol. 64, no. 5, pp. 482–488, 1962. [Online]. Available: <https://doi.org/10.1037/h0045106>
- [112] M. Glanzer and A. R. Cunitz, “Two storage mechanisms in free recall,” *Journal of Verbal Learning and Verbal Behavior*, vol. 5, no. 4, pp. 351–360, 1966. [Online]. Available: [https://doi.org/10.1016/S0079-7421\(08\)60422-3](https://doi.org/10.1016/S0079-7421(08)60422-3)
- [113] E. Tulving, “Memory and consciousness,” *Canadian Psychology/Psychologie Canadienne*, vol. 26, no. 1, pp. 1–12, 1985. [Online]. Available: <https://doi.org/10.1037/h0080017>
- [114] Y. Sun *et al.*, “Position: Episodic memory is the missing piece for long-term LLM agents,” *arXiv preprint arXiv:2502.06975*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.06975>
- [115] Y. Ning *et al.*, “Towards large language models with human-like episodic memory,” *Trends in Cognitive Sciences*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.01349>
- [116] Z. Hu *et al.*, “EM-LLM: Human-inspired episodic memory for infinite context LLMs,” *arXiv preprint arXiv:2407.09450*, 2025. [Online]. Available: <https://arxiv.org/abs/2407.09450>
- [117] J. M. Murre and J. Dros, “Replication and analysis of Ebbinghaus’ forgetting curve,” *PLoS ONE*, vol. 10, no. 7, p. e0120644, 2015. [Online]. Available: <https://doi.org/10.3758/s13421-015-0519-7>
- [118] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “MemoryBank: Enhancing large language models with long-term memory,” in *Proceedings of the AAAI Conference on Artificial*

- Intelligence*, vol. 38, no. 17, 2024. [Online]. Available: <https://doi.org/10.1609/aaai.v38i17.29946>
- [119] L. R. Squire and S. Zola-Morgan, “The medial temporal lobe memory system,” *Science*, vol. 253, no. 5026, pp. 1380–1386, 1991. [Online]. Available: <https://doi.org/10.1126/science.1896849>
- [120] J. L. McClelland, B. L. McNaughton, and R. C. O’Reilly, “Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory,” *Psychological Review*, vol. 102, no. 3, pp. 419–457, 1995. [Online]. Available: <https://doi.org/10.1037/0033-295X.102.3.419>
- [121] M. F. Carr, S. P. Jadhav, and L. M. Frank, “Hippocampal replay in the awake state: A potential substrate for memory consolidation and retrieval,” *Nature Neuroscience*, vol. 14, no. 2, pp. 147–153, 2011. [Online]. Available: <https://doi.org/10.1038/nn.2732>
- [122] D. J. Foster, “Replay comes of age,” *Annual Review of Neuroscience*, vol. 40, pp. 581–602, 2017. [Online]. Available: <https://doi.org/10.1016/j.neuron.2017.10.009>
- [123] S. Diekelmann and J. Born, “The memory function of sleep,” *Nature Reviews Neuroscience*, vol. 11, no. 2, pp. 114–126, 2010. [Online]. Available: <https://doi.org/10.1038/nrn2762>
- [124] B. Rasch and J. Born, “About sleep’s role in memory,” *Physiological Reviews*, vol. 93, no. 2, pp. 681–766, 2013. [Online]. Available: <https://doi.org/10.1152/physrev.00032.2012>
- [125] A. González-Rueda *et al.*, “A model of autonomous interactions between hippocampus and neocortex driving sleep-dependent memory consolidation,” *Proceedings of the National Academy of Sciences*, vol. 119, no. 44, p. e2123432119, 2022. [Online]. Available: <https://doi.org/10.1073/pnas.2123432119>
- [126] J. E. Laird, *The Soar Cognitive Architecture*. MIT Press, 2012. [Online]. Available: https://web.eecs.umich.edu/~soar/sitemaker/docs/pubs/Laird_Lebiere_Rosenbloom_2017_CogArch_chapter.pdf
- [127] J. R. Anderson, D. Bothell, M. D. Byrne, S. Douglass, C. Lebiere, and Y. Qin, “An integrated theory of the mind,” *Psychological Review*, vol. 111, no. 4, pp. 1036–1060, 2004. [Online]. Available: <https://doi.org/10.1037/0033-295X.111.4.1036>
- [128] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, “Cognitive architectures for language agents,” *Transactions on Machine Learning Research*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.04170>
- [129] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. [Online]. Available: <https://doi.org/10.1145/3586183.3606763>

- [130] L. Liu, X. Yang *et al.*, “Think-in-memory: Recalling and post-thinking enable LLMs with long-term memory,” *arXiv preprint arXiv:2311.08719*, 2023. [Online]. Available: <https://arxiv.org/abs/2311.08719>
- [131] F. I. M. Craik and R. S. Lockhart, “Levels of processing: A framework for memory research,” *Journal of Verbal Learning and Verbal Behavior*, vol. 11, no. 6, pp. 671–684, 1972. [Online]. Available: [https://doi.org/10.1016/S0022-5371\(72\)80001-X](https://doi.org/10.1016/S0022-5371(72)80001-X)
- [132] X. Liang, B. Wang *et al.*, “Enhancing large language model with self-controlled memory framework,” *arXiv preprint arXiv:2304.13343*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.13343>
- [133] B. Kynoch, H. Latapie *et al.*, “RecallM: An adaptable memory mechanism with temporal understanding for large language models,” *arXiv preprint arXiv:2307.02738*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.02738>
- [134] A. M. Collins and E. F. Loftus, “A spreading-activation theory of semantic processing,” *Psychological Review*, vol. 82, no. 6, pp. 407–428, 1975. [Online]. Available: [https://doi.org/10.1016/S0079-7421\(08\)60528-8](https://doi.org/10.1016/S0079-7421(08)60528-8)
- [135] A. Modarressi *et al.*, “RET-LLM: Towards a general read-write memory for large language models,” *arXiv preprint arXiv:2305.14322*, 2024. [Online]. Available: <https://arxiv.org/abs/2305.14322>
- [136] N. Liu *et al.*, “From LLM to conversational agent: A memory enhanced architecture with fine-tuning of large language models,” *arXiv preprint arXiv:2401.02777*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.02777>
- [137] R. Ryskin *et al.*, “If attention serves as a cognitive model of human memory retrieval, what is the plausible memory representation?” 2025, arXiv:2502.11469. [Online]. Available: <https://arxiv.org/abs/2502.11469>
- [138] A. Nikolaev *et al.*, “Aligning transformer circuit mechanisms to neural representations in relational reasoning,” *bioRxiv*, 2025. [Online]. Available: <https://www.biorxiv.org/content/10.1101/2025.10.29.685457v1>
- [139] K. Irie *et al.*, “TransformerFAM: Feedback attention is working memory,” *arXiv preprint arXiv:2404.09173*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.09173>
- [140] L. Kozachkov *et al.*, “Short-term Hebbian learning can implement transformer-like attention,” *PNAS Nexus*, 2022. [Online]. Available: <https://doi.org/10.1371/journal.pcbi.1010682>
- [141] Y. Luo *et al.*, “An empirical study of catastrophic forgetting in large language models during continual fine-tuning,” *arXiv preprint arXiv:2308.08747*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.08747>

- [142] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [143] J. Kirkpatrick, R. Pascanu, N. Rabinowitz *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017. [Online]. Available: <https://doi.org/10.1073/pnas.1611835114>
- [144] B. T. Tutuncuoglu, “NeuroDream: A sleep-inspired memory consolidation framework for artificial neural networks,” *SSRN Working Paper*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.03616>
- [145] T. Munkhdalai *et al.*, “Leave no context behind: Efficient infinite context transformers with Infini-attention,” *arXiv preprint arXiv:2404.07143*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.07143>
- [146] N. Cowan *et al.*, “Judgments of learning distinguish humans from large language models in predicting memory,” *Nature Scientific Reports*, vol. 15, p. 6753, 2025. [Online]. Available: <https://doi.org/10.1038/s41598-025-22290-x>
- [147] A. B. Arrieta *et al.*, “Large language models and cognitive science: A comprehensive review of similarities, differences, and challenges,” 2024, arXiv:2409.02387. [Online]. Available: <https://arxiv.org/abs/2409.02387>
- [148] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. [Online]. Available: <https://arxiv.org/abs/2309.05922>
- [149] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *arXiv preprint arXiv:2311.05232*, 2023. [Online]. Available: <https://arxiv.org/abs/2311.05232>
- [150] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On faithfulness and factuality in abstractive summarization,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 1906–1919. [Online]. Available: <https://aclanthology.org/2020.acl-main.173/>
- [151] S. Lin, J. Hilton, and O. Evans, “TruthfulQA: Measuring how models mimic human falsehoods,” *arXiv preprint arXiv:2109.07958*, 2022. [Online]. Available: <https://arxiv.org/abs/2109.07958>
- [152] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, “HaluEval: A large-scale hallucination evaluation benchmark for large language models,” in *Proceedings of the 2023*

- Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 6449–6464. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.397/>
- [153] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, “Factscore: Fine-grained atomic evaluation of factual precision in long form text generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/b5372f7731afd53dde24cfae07ad49a4-Abstract-Conference.html
- [154] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling large language models to generate text with citations,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://aclanthology.org/2023.acl-long.557/>
- [155] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=1PL1NIMMrw>
- [156] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, “Chain-of-verification reduces hallucination in large language models,” *arXiv preprint arXiv:2309.11495*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.11495>
- [157] Y. Wang, Z. Liu, D. Chen *et al.*, “A survey on multi-turn interaction capabilities of large language models,” *arXiv preprint arXiv:2501.09959*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.09959>
- [158] C. Greyling, “Chatgpt apis and managing conversation context memory,” Medium, 2024. [Online]. Available: <https://cobusgreyling.medium.com/chatgpt-apis-managing-conversation-context-memory-8b100dfe544a>
- [159] Y. Chen, L. Wang, and M. Zhang, “Sliding window attention training for efficient large language models,” *arXiv preprint arXiv:2502.18845*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.18845>
- [160] H. Zhang, J. Li, and Y. Wu, “Swaa: Sliding window attention adaptation for efficient long-context inference,” *arXiv preprint arXiv:2512.10411*, 2024. [Online]. Available: <https://arxiv.org/abs/2512.10411>
- [161] M. Al Salbough, “Strategies and techniques for managing the size of the context window when using llm,” Medium, 2024. [Online]. Available: <https://mohdmus99.medium.com/strategies-and-techniques-for-managing-the-size-of-the-context-window-when-using-llm-large-3c2dbc5dcc3a>
- [162] Q. Wu, L. Wang, Y. Zhou *et al.*, “Recursively summarizing enables long-term dialogue memory in large language models,” in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2308.15022. [Online]. Available: <https://arxiv.org/abs/2308.15022>

- [163] Microsoft Surface Duo Team, “Infinite chat with history summarization,” Microsoft DevBlogs, 2024. [Online]. Available: <https://devblogs.microsoft.com/surface-duo/android-openai-chatgpt-18/>
- [164] Y. Wang, X. Chen, and Z. Liu, “Dialogue summarization with mixture of experts based on large language models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 1234–1245. [Online]. Available: <https://aclanthology.org/2024.acl-long.385/>
- [165] Mem0.ai Team, “Llm chat history summarization guide october 2025,” Mem0.ai Blog, 2025. [Online]. Available: <https://mem0.ai/blog/llm-chat-history-summarization-guide-2025>
- [166] J. Wang, Y. Liu, and H. Chen, “H-mem: Hierarchical memory for high-efficiency multi-turn dialogue,” *arXiv preprint arXiv:2507.22925*, 2024. [Online]. Available: <https://arxiv.org/abs/2507.22925>
- [167] C. Liu, W. Zhang, and T. Wang, “Sgmemo: Sentence graph memory for long-term conversational agents,” *arXiv preprint arXiv:2509.21212*, 2024. [Online]. Available: <https://arxiv.org/abs/2509.21212>
- [168] A. Ibrahim, “Don’t let your ai agent forget: Smarter strategies for summarizing message history,” Medium, 2025. [Online]. Available: <https://techwithibrahim.medium.com/dont-let-your-ai-agent-forget-smarter-strategies-for-summarizing-message-history-a2d5284539f1>
- [169] S. Feng, H. Wan, C. Gunasekara *et al.*, “Interpretable and robust dialogue state tracking via natural language,” *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 789–805, 2023. [Online]. Available: <https://arxiv.org/abs/2503.08857>
- [170] P. Budzianowski, T.-H. Wen, B.-H. Tseng *et al.*, “Multiwoz – a large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 5016–5026. [Online]. Available: <https://doi.org/10.18653/v1/D18-1547>
- [171] S. Kim, J. Park, and D. Lee, “Codial: Collaborative dialogue state tracking with chain-of-thought reasoning,” *arXiv preprint arXiv:2403.11847*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.11847>
- [172] L. He, Y. Chen, and Z. Wang, “Unipppn: Unified policy and natural language generation for task-oriented dialogue,” *arXiv preprint arXiv:2312.09876*, 2024, doi: [10.18653/v1/D18-1547](https://doi.org/10.18653/v1/D18-1547). [Online]. Available: <https://arxiv.org/abs/2312.09876>
- [173] X. Zhao, J. Liu, and M. Chen, “Reliable error estimation in task-oriented dialogue systems,” *arXiv preprint arXiv:2501.03421*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.03421>
- [174] Y. Li, H. Wang, and L. Zhang, “Chain of thought explanation for dialogue state tracking,” *arXiv preprint arXiv:2403.04656*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.04656>

- [175] Y. Zhang, C. Liu, and X. Wang, “Know me, respond to me: Benchmarking llms for dynamic user profiling and personalized responses at scale,” *arXiv preprint arXiv:2504.14225*, 2024. [Online]. Available: <https://arxiv.org/abs/2504.14225>
- [176] L. Wang, X. Chen, and H. Li, “Personallm: Learning user preferences for personalized language generation,” 2025. [Online]. Available: <https://arxiv.org/abs/2601.01885>
- [177] Together AI Team, “Fine-tuning llms for multi-turn conversations: A technical deep dive,” Together.ai Blog, 2024. [Online]. Available: <https://www.together.ai/blog/fine-tuning-llms-for-multi-turn-conversations-a-technical-deep-dive>
- [178] W. Zhang, Y. Liu, and H. Chen, “Enhancing personalized multi-turn dialogue with curiosity reward,” *arXiv preprint arXiv:2504.03206*, 2024. [Online]. Available: <https://arxiv.org/abs/2504.03206>
- [179] European Data Protection Board, “Guidance on ai memory systems and gdpr compliance,” EDPB Official Guidelines, 2024. [Online]. Available: https://www.edpb.europa.eu/our-work-tools/our-documents/opinion-board-art-64/opinion-282024-certain-data-protection-aspects_en
- [180] Amplework Software, “Top multi-agent tools compared: Langgraph, autogen, crewai,” Amplework Blog, 2024. [Online]. Available: <https://www.amplework.ai/blog/langgraph-vs-autogen-vs-crewai-multi-agent-framework/>
- [181] Q. Wu, G. Bansal, J. Zhang *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.08155>
- [182] DataCamp Team, “Crewai vs langgraph vs autogen: Choosing the right multi-agent ai framework,” DataCamp Tutorial, 2024. [Online]. Available: <https://www.datacamp.com/tutorial/crewai-vs-langgraph-vs-autogen>
- [183] LangChain Team, “Langgraph documentation: Memory and state management,” LangChain Documentation, 2024. [Online]. Available: <https://docs.langchain.com/langgraph>
- [184] foxgem, “Ai agent memory: A comparative analysis of langgraph, crewai, and autogen,” DEV Community, 2024. [Online]. Available: <https://dev.to/foxgem/ai-agent-memory-a-comparative-analysis-of-langgraph-crewai-and-autogen-31dp>
- [185] OpenAI, “Memory and new controls for chatgpt,” OpenAI Blog, 2024. [Online]. Available: <https://openai.org/index/memory-and-new-controls-for-chatgpt/>
- [186] C. Greyling, “How chatgpt 5.1 manages context, memory, and long conversations,” Skywork AI Blog, 2024. [Online]. Available: <https://skywork.ai/blog/ai-agent/chatgpt-5-1-context-memory-long-conversations/>

- [187] Anthropic, “Claude memory: Personalized ai that remembers,” Anthropic Blog, 2025, rolled out September 2025. [Online]. Available: <https://www.anthropic.com/news/memory>
- [188] —, “Managing context on the claude developer platform,” Anthropic Blog, 2024. [Online]. Available: <https://www.anthropic.com/news/context-management>
- [189] Google DeepMind, “Introducing gemini 1.5, google’s next-generation ai model,” Google Blog, 2024. [Online]. Available: <https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/>
- [190] Factory.ai Team, “Context window budgeting in production llm systems,” Factory.ai News, 2024. [Online]. Available: <https://factory.ai/news/compressing-context>
- [191] T. Kim, W. Su, S. Lee, C. Yin, J. Kim, J. Li, and P. Zhao, “Token-budget-aware llm reasoning,” *arXiv preprint arXiv:2412.18547*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.18547>
- [192] Anthropic, “Long context prompting tips,” Claude Documentation, 2024. [Online]. Available: <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/long-context-tips>
- [193] Maxim AI Team, “Context window management: Strategies for long-context ai agents and chatbots,” GetMaxim.ai, 2024. [Online]. Available: <https://www.getmaxim.ai/articles/context-window-management-strategies-for-long-context-ai-agents-and-chatbots/>
- [194] Anthropic, “Context window budgeting best practices,” Claude Platform Documentation, 2024. [Online]. Available: <https://platform.claude.com/docs/en/build-with-claude/context-windows>
- [195] G. Kamradt, “LLMTest needle in a haystack,” *GitHub repository*, 2023. [Online]. Available: https://github.com/gkamradt/LLMTest_NeedleInAHaystack
- [196] C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, “RULER: What’s the real context size of your long-context language models?” *arXiv preprint arXiv:2404.06654*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.06654>
- [197] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “LongBench: A bilingual, multitask benchmark for long context understanding,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 2619–2636. [Online]. Available: <https://aclanthology.org/2024.acl-long.70/>
- [198] C. An, S. Gong, M. Zhong, M. Wang, J. Zhao, G. Pang, C.-Y. Lin, and J. Zeng, “L-eval: Instituting standardized evaluation for long context language models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 14 324–14 351. [Online]. Available: <https://aclanthology.org/2024.acl-long.33/>
- [199] X. Zhang, X. Lin, S. Diao, and T. Zhang, “ ∞ Bench: Extending long context

- evaluation beyond 100K tokens,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 15 262–15 285. [Online]. Available: <https://aclanthology.org/2024.acl-long.814/>
- [200] Y. Kuratov, A. Bulatov, P. Anokhin, I. Rodkin, D. Sorokin, A. Sorokin, and M. Burtsev, “BABIIlong: Testing the limits of LLMs with long context reasoning-in-a-haystack,” in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=o5vL6oZOMj>
- [201] U. Shaham, E. Segal, M. Ivgi, A. Efrat, O. Yoran, A. Haviv, A. Gupta, W. Xiong, M. Geva, J. Berant, and O. Levy, “SCROLLS: Standardized compaRison over long language sequences,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 12 007–12 021. [Online]. Available: <https://aclanthology.org/2022.emnlp-main.823/>
- [202] U. Shaham, M. Ivgi, A. Efrat, J. Berant, and O. Levy, “ZeroSCROLLS: A zero-shot benchmark for long text understanding,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 7977–7989. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.536/>
- [203] H. Yen, T. Gao, J. Y. Koh, W. Shi, C. Zhou, T. Pfister, and D. Chen, “HELMET: How to evaluate long-context language models effectively and thoroughly,” *arXiv preprint arXiv:2410.02694*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.02694>